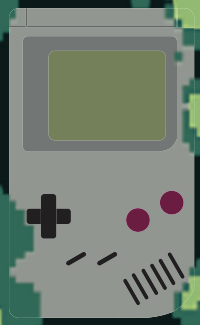
EARLY ACCESS



GAME BOY CODING ADVENTURE





MAXIMILIEN DAGOIS

no starch press®

NO STARCH PRESS EARLY ACCESS PROGRAM: FEEDBACK WELCOME!

Welcome to the Early Access edition of the as yet unpublished *Game Boy Coding Adventure* by Maximilien Dagois! As a prepublication title, this book may be incomplete and some chapters may not have been proofread.

Our goal is always to make the best books possible, and we look forward to hearing your thoughts. If you have any comments or questions, email us at **earlyaccess@nostarch.com**. If you have specific feedback for us, please include the page number, book title, and edition date in your note, and we'll be sure to review it. We appreciate your help and support!

We'll email you as new chapters become available. In the meantime, enjoy!

GAME BOY CODING ADVENTURE

Maximilien Dagois

Early Access edition, 5/1/2025

Copyright © 2025 by Maximilien Dagois.

ISBN-13: 978-1-7185-0390-8 (print)
ISBN-13: 978-1-7185-0391-5 (ebook)

Publisher: William Pollock
Managing Editor: Jill Franklin
Production Editor: Jennifer Kepler
Developmental Editor: Annie Choi
Cover Design: Calamari Studios
Interior Design: Octopod Studios
Technical Reviewer: Sy Brand
Copyeditor: Rachel Head

No Starch Press and the No Starch Press iron logo are registered trademarks of No Starch Press, Inc. Other product and company names mentioned herein may be the trademarks of their respective owners. Rather than use a trademark symbol with every occurrence of a trademarked name, we are using the names only in an editorial fashion and to the benefit of the trademark owner, with no intention of infringement of the trademark.

All rights reserved. No part of this work may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying, recording, or by any information storage or retrieval system, without the prior written permission of the copyright owner and the publisher.

The information in this book is distributed on an "As Is" basis, without warranty. While every precaution has been taken in the preparation of this work, neither the author nor No Starch Press, Inc. shall have any liability to any person or entity with respect to any loss or damage caused or alleged to be caused directly or indirectly by the information contained in it.

About the Author

Maximilien Dagois has been programming games for more than 15 years. He has designed and developed graphics engines on games like *Final Fantasy XIV*, *Dragon Quest Builders*, and *Pokémon Ranger*, and he has shipped a dozen games on PC, mobile, DS, PS3, PS Vita, PS4, and Stadia.

Maximilien now works as an engineer at Google. He lives with his wife and two kids in Tokyo, and you are most likely to find him playing on a good old gaming system.

Follow him on X at *https://x.com/mdagois* or Gumroad at *https://mdagois.gumroad.com* to get the latest news about his retro console programming books.

About the Technical Reviewer

Sy Brand is Microsoft's C++ Developer Advocate and has more than a decade of experience in developer tooling. They graduated from the University of St. Andrews with a first-class degree in computer science, specializing in compiler implementation. Outside the world of technology, they are a published poet, maker of experimental films and music, activist, and parent.

BRIEF CONTENTS

Foreword xix
Acknowledgments xxiii
Introduction xxv

PART I: GETTING STARTED

Chapter 1: Game Boy Fundamentals 3
Chapter 2: Assembly Language Fundamentals 11
Chapter 3: The Game Boy Toolchain 41
Chapter 4: Introducing the Sample Template 57

PART II: ESSENTIAL DMG PROGRAMMING

Chapter 5: Graphics Overview 69
Chapter 6: Graphics Asset Management 81
Chapter 7: High-Level Features 93
Chapter 8: The Joypad 113
Chapter 9: Audio 137

PART III: ADVANCED GAME BOY PROGRAMMING

Chapter 10: The Counter and Timer 185
Chapter 11: Serial Communication 193
Chapter 12: Advanced Graphics 215
Chapter 13: Memory Bank Controllers 235
Chapter 14: Interrupts 247
Chapter 15: Programming and Debugging Tricks 255

PART IV: BEYOND THE GAME BOY

Chapter 16: Game Boy Color 271
Chapter 17: Game Boy Printer 305

Chapter 18: Super Game Boy 325
Chapter 19: Going Further 393

Appendix A: Instructions 397
Appendix B: Opcodes 403
Index 413

CONTENTS IN DETAIL

FOREWORD **xix**

ACKNOWLEDGMENTS **xxiii**

INTRODUCTION **xxv**

About This Book xxvi
What You'll Learn xxvii
Who This Book Is For xxviii
Required Tools xxviii
Toolchain xxviii
Emulator xxviii
Code Samples xxix
Text Editor xxix
Recommended Tools xxix
Optional Tools xxix
Game Boy xxix
Flashcart xxx
About This Book xxx

PART I
GETTING STARTED

1
GAME BOY FUNDAMENTALS **3**

External Features 3
Internal Components 5
Cartridge Components 6
Console Components 7
Memory Map 9
Game Boy Programs 10
Summary 10

2
ASSEMBLY LANGUAGE FUNDAMENTALS **11**

What Is Assembly? 12

The Central Processing Unit 13
Registers 14
Flags 14
Special-Purpose Registers 15
Number Formats 16
Assembly Language Syntax 16
Labels 17
Instructions 17
Comments 19
Instruction Categories 20
Loads 20
Jumps 21
Functions 25
Arithmetic Operations 26
Bitwise Operations 28
Bit Operations 28
Shifts 29
Rotations 31
Stack Operations 32
Miscellaneous 34
Beyond Instructions 35
Sections 35
Local Labels 36
Macros 37
Constants 39
Summary 40

3
THE GAME BOY TOOLCHAIN **41**

Writing a Game Boy Program 41
Building a ROM 43
Assembly 43
Linkage 44
Fixing 44
Running a ROM 45
Using the Debugger 46
Code View 46
Register View 48
Data View 48
Stack View 48
Debugging Shortcuts 49
Debugging Tips 50
Inspecting a ROM 51
ROM Memory Map 53
Summary 55

4
INTRODUCING THE SAMPLE TEMPLATE **57**

The Four Files of the Sample Template 57
hardware.rgbinc 58
main.rgbasm 59
sample.rgbasm 60
utils.rgbinc 61
Compiling the Template 62
The Map File 62
The Symbol File 63
Running the Template 65
Summary 66

PART II
ESSENTIAL DMG PROGRAMMING

5
GRAPHICS OVERVIEW **69**

The LCD, Vblank, and Frame 70
High-Level Features 71
Elements of Graphics Data 74
Colors 74
Palettes 74
Tiles 74
Tilemaps 77
Tile Addressing Modes 77
Putting It All Together 78
Summary 79

6
GRAPHICS ASSET MANAGEMENT **81**

Converting Assets 82
Using the RGBDS Toolchain 82
Using a Custom Tool 83
Exporting Text Files 84
Adding Assets to the ROM 85
Copying Assets to VRAM 87
Sample Code 89
Summary 91

7
HIGH-LEVEL FEATURES **93**

Background 93

Palette 94
LCD Parameters 94
Scrolling and Wrapping 96
Frame Pacing 98
Understanding the Synchronization Issue 98
Fixing the Pacing 99
Summing Up the Frame Pacing Mechanics 100
Window 101
Comparing the Window and the Background 101
Moving the Window 102
Using the Window 104
Sprites 106
Palettes 106
LCD Parameters 106
Object Attribute Memory 107
Limitations 108
A Sprite Sample 109
Summary 112

8
THE JOYPAD **113**

Input Polling 113
Declaring Constants 117
Scrolling the Background 118
Handling Input 118
Checking WRAM Usage 120
Initializing the WRAM 122
Updating the Display and Input 122
Press and Release 125
Expanding the Input Structure 125
Updating the Input Structure 125
Adding Test Macros 127
Fixing Window Toggling 127
Going Further 127
The Joypad Interrupt 128
Declaring the Variables 129
Initializing the Variables 129
Updating the Sample 130
Defining the Interrupt Function 133
Summary 134

9
AUDIO **137**

Getting Started 137
Control Registers 138

Sound Sample 139
WRAM Initialization 140
The Update Function 140
Channel 1 142
Control Flags 142
Parameters 143
Functions 155
Continuous Sound 162
Tips for Using Channel 1 164
Channel 2 165
Channel 3 166
Control Flags 167
Parameters 167
Custom Waveforms 168
Duty Cycle 175
Volume Control 175
Tips for Using Channel 3 176
Channel 4 177
The Polynomial Counter 178
Sound Effect Examples 180
Summary 182

PART III
ADVANCED GAME BOY PROGRAMMING

10
THE COUNTER AND TIMER 185

Using the Counter 186
Using the Timer 188
The Built-in Counter Connection 189
A Timer Sample 190
Tips for Using the Timer 192
Summary 192

11
SERIAL COMMUNICATION 193

Communication Devices 193
Using Real Hardware 194
Using BGB 195
From the UI 195
From the Command Line 197
Transfer 198
Registers and Interrupt 198
A Basic Transfer 199

Tips on Transfers ... 201
A Simple Protocol ... 202
Synchronization Phase ... 202
Transfer Phase ... 205
Chaining Transfers ... 210
Summary ... 214

12
ADVANCED GRAPHICS 215

LCD Details ... 216
Modes ... 216
Event Tracking ... 217
Graphics Tricks ... 218
Parallax Scrolling ... 219
Palette Fades ... 224
Direct Memory Access ... 226
Copying to OAM Without DMA ... 227
Leveraging DMA to Copy to OAM ... 229
Displaying More Than 40 Sprites ... 232
Summary ... 233

13
MEMORY BANK CONTROLLERS 235

Fundamentals ... 235
The Evolution of Cartridges ... 236
Bank Switching ... 236
ROM with MBC Enabled ... 237
MBC5 ... 239
Characteristics ... 239
ROM Bank Switching ... 240
Saves with ERAM ... 243
Summary ... 246

14
INTERRUPTS 247

Executing Interrupts ... 247
Understanding Interrupt Priorities ... 248
Nesting Interrupts ... 248
Triggering Interrupts Manually ... 249
Handling the Vblank Interrupt Differently ... 250
Writing Interrupts ... 251
Repurposing Reserved Addresses ... 251
Keeping Registers Clean ... 251
More About halt ... 252

Game Boy Coding Adventure (Early Access) © 2025 by Maximilien Dagois

IME Influence 252
Multiple Interrupts and Vblank 252
Summary 253

15
PROGRAMMING AND DEBUGGING TRICKS 255

Jump Tables 255
The Classic Method 256
The Jump Table Method 257
More Debugging Features 260
Source Code Breakpoints 260
Logging 261
Faulty Jumps 263
Moving the Stack to WRAM 265
Implementing a Linear-Feedback Shift Register 265
Summary 267

PART IV
BEYOND THE GAME BOY

16
GAME BOY COLOR 271

CGB Graphics 272
Setting Palettes 272
Leveraging the VRAM Banks 277
Using the Sprite Functionalities 284
Managing Priority 287
Copying Data to VRAM with DMA 289
CPU Double Speed Mode 295
Additional Upgrades 297
WRAM Banks 298
Serial Transfer Rates 298
Infrared 299
Compatibility Mode 300
Detecting the Hardware Model 300
Testing in BGB 301
Running on Different Hardware 302
Summary 304

17
GAME BOY PRINTER 305

Testing Printer Programs 306
Using Real Hardware 306

Simulating the Printer 307
Serial Protocol 307
Packet Types 308
Init 309
Print 309
Data 310
Break 311
Status 312
Printer Flags 312
Print Sequence 314
Sending an Image to the Printer 314
Packet Definition 316
State Tracking 318
Packet Transfer 318
Error Check 322
Summary 323

18
SUPER GAME BOY **325**

SGB Overview 326
Commands 327
Format 327
Payload 328
Serial Protocol 331
Types 333
Graphics 335
Color Palettes 335
Screen Tiles 336
Border Graphics 336
Asset Converters 338
Building and Testing SGB Programs 339
Enabling Enhanced Features 339
Testing on BGB 340
Testing on Real Hardware 341
Hardware Detection 341
Sample Foundations 344
Defining the Command Headers 344
Implementing the Wait Mechanics 346
Sending Commands 347
Initialization Procedure 350
Adding Color to the Screen 354
The Base Color Sample 355
Alternative Attribute Commands 358
Palette Commands with LCD Payload 364
Attribute Commands with LCD Payload 367
Combination of LCD Payload Commands 370

Game Boy Coding Adventure (Early Access) © 2025 by Maximilien Dagois

Setting a Custom Border 372
Foundation 372
Glitch Workaround 376
More Border Samples 378
Unlocking Multiplayer Options 378
Playing Preloaded Sounds 381
Controlling the SGB Settings 387
Sending Data for the SNES 389
Summary 391

19
GOING FURTHER **393**

A
INSTRUCTIONS **397**

B
OPCODES **403**

INDEX **413**

Game Boy Coding Adventure (Early Access) © 2025 by Maximilien Dagois

FOREWORD

I was surprised and delighted to be asked to write the foreword for Maximilien's book about the Game Boy. This small game console had a huge impact on my life and career, as it brought me to Japan at the young age of 18 to meet the developers at Nintendo itself. I had a lot of fun developing on it and pushing it to its limits.

My story with the Game Boy really began with the Sinclair ZX Spectrum, a cheap programmable home computer that was popular in Europe during the 1980s. The CPU was a Z80, and that's where the connection arises. When I was about 11 years old, I bought something called a "machine code assembler," thinking that it would simply automatically speed up my rather badly written BASIC programs. In fact, it required me to learn how to program in Z80. Thus began my love of assembly programming, and especially Z80, with all its quirks and unique ways to optimize code. This book could be just as important to you, helping you understand the fun and raw power of low-level machine code programming.

Fast-forward a few years, and I found myself leaving school at the age of 17 because I'd managed to find a job at Argonaut Software. At the time, Argonaut was the leading game developer in the UK specializing in 3D games, an area I was really interested in. About five months into that job, I started to think Z80 was behind me, as all the new machines used either 16-bit or 32-bit CPUs. Then, Jez San, my then 20-something boss and the founder of Argonaut (originally a programmer himself), walked up to my desk. In his hand was the brand-new, just-released Nintendo Game Boy. He said, "Let's hack this to do 3D," and the gauntlet was thrown down. Jez had remembered that, while most of the other programmers at Argonaut knew 6502, I was a bit of a whiz at Z80.

The only problem was, we weren't official developers, and we had no access to any information whatsoever about the Game Boy apart from the basic specs and that the CPU was a slightly stripped-down Z80 variant that ran even more slowly than the ZX Spectrum. We really could have used this book! Of course, to balance the slower CPU speed, the display hardware had a bunch of features, but we had no idea what they were at that point. So, we took a guess that it would be similar to the Nintendo Entertainment System (NES) console, a much older system that we had a bit more information about, and jumped in.

Within a month of hacking around in the dark, I had a fun little 3D shooting game demo I called *Eclipse* running on this hacked-together system with wires sticking out of it, and we burned an EPROM. Jez then trotted off to the Consumer Electronics Show to see if anyone was interested in it. It turned out Nintendo had never even considered 3D as a possibility and wanted to fly us out the very next week to speak to the creators themselves.

Wet behind the ears, aged 18, I arrived in Japan, and one of the first people I met was Gunpei Yokoi, the "inventor" of the Game Boy. He took a shine to us and liked the demo.

I ended up working alongside many Game Boy luminaries, including Hip Tanaka, famed Nintendo musician and engineer, Totaka (also known as Totakeke), whose first assignment was as the musician for our project, and Sakamoto of Metroid fame, who was assigned to be the game's director. It was a great little team.

Gunpei Yokoi was a great guy with a wicked sense of humor, and even though it was a band of rebels in his group who secretly believed they could pull off something like the Game Boy and still have good enough battery life, he quickly understood what they were doing, got behind them fully, and made the Game Boy a huge success. He also gave me a candy expense budget (I was 19 by then) so I could try out all the strange Japanese confectionery in the supermarket near the Nintendo offices. The sugar rush fueled my Z80 programming, and I ended up pushing the hardware so hard the game couldn't run on first edition Game Boys anymore. It appeared there was a bug in the hardware, and some of the extreme stuff I was doing would cause random glitched data to be written to screen RAM. Only about 100,000 first edition Game Boys had been sold, but I figured all of them probably had that LCD glue issue where vertical strips of the screen would become nonfunctional. I had to do something to get around it, so I ended up writing a routine that slowly scanned the screen for data that was out of place and rewrote any glitches it found. This allowed *X*, as the game ended up being called, to ship, and on May 29, 1992, it was unleashed in Japan.

Reading through Maximilien's book, I got an adrenaline rush as I remembered all the fun and excitement of this era of my life, and it made me feel the urge to poke around inside the Game Boy again after all these decades. I'm sure the enthusiasm he exudes throughout the book will easily spill over a little to most readers and give them the urge to dip their toes in, if only to `LD A, (HLI)` a few times in a row.

This book is full of information about all the intricate ways you can program the various systems in the Game Boy, and it teaches you about everything from the ground up, which means novices and experienced people alike should be able to get a lot out of reading it. I discovered there was still a lot I didn't know! There are code samples covering many different areas, and the book goes into detail on the later Game Boys too, such as the Game Boy Color. I hope everyone gets as much enjoyment (and nostalgia) as I did by reading it and learning all about this lovely little handheld system.

Dylan Cuthbert
Founder and CEO of Q-Games and
co-founder of BitSummit

Game Boy Coding Adventure (Early Access) © 2025 by Maximilien Dagois

ACKNOWLEDGMENTS

I would like to thank the following people for promoting the book and providing feedback on the material and samples: Dave VanEe, Francis Stokes, ISSOtm, Masayoshi Sugimoto, Matt Gilgenbach, Paul Tolède, and Ulf Wetzker. Their invaluable help significantly increased the quality, accessibility, and visibility of the content.

Game Boy Coding Adventure (Early Access) © 2025 by Maximilien Dagois

INTRODUCTION

The Game Boy is an iconic handheld console and a remarkable piece of gaming history. Whether you've got one or not, you can program the Game Boy. This book teaches you how.

Why should you be interested in this in the first place? Programming the Game Boy is challenging but very rewarding because it's a unique entry point to learning assembly language, which provides a deep understanding of how computers work at the hardware level and can significantly enhance your programming skills in higher-level languages. Game Boy programming uses a simplified version of assembly, making it perfect for newcomers.

My goal is to give the reader an organized, hands-on experience covering everything there is to know about the Game Boy hardware, from the very basics up to advanced topics. Along the way, you'll learn about important assembly concepts like registers, machine code, and flags. And best of all, the language is tied to real hardware, though you won't need any hardware to follow this book.

Whether you want to create games or applications, are interested in ROM hacking, or are looking for a fun way to improve your programming skills, this book has something for you.

About This Book

This book is a comprehensive tutorial on programming assembly using the Game Boy as the vehicle. It covers all the features of the hardware and explains how to control them using programs written in assembly. All the concepts are illustrated with code samples.

Here, the term *Game Boy* means all the variations of the handheld console that was first released back in 1989. This includes five devices: the original Game Boy, Game Boy Pocket, Game Boy Light, Super Game Boy, and Game Boy Color. Figure 1 shows the first three of these devices.



Figure 1: The Game Boy, Game Boy Pocket, and Game Boy Light

From a programmer's perspective, these three devices are identical. They differ only in features that are beyond the reach of the programmer, such as their form factor and the color shades of their liquid crystal displays (LCDs). In this book, the Game Boy, Game Boy Pocket, and Game Boy Light (released only in Japan) are collectively referred to as the *Dot Matrix Game (DMG)*, which was the internal code name of the original Game Boy.

The fourth device, the Super Game Boy, shown in Figure 2, is an adapter designed to allow Game Boy games to be played on the Super Nintendo Entertainment System (SNES), released in Japan under the name Super Famicom.



Figure 2: The Super Game Boy 1 and 2 (Super Famicom) and Super Game Boy 1 (Super NES)

The Super Game Boy is basically a DMG inside a SNES cartridge. However, it is possible to leverage a limited subset of the SNES hardware to enhance DMG games. There are two versions of the Super Game Boy: the

Super Game Boy 1 and the Super Game Boy 2 (released only in Japan). When it comes to programming, both versions exhibit almost the same functionalities, so they are both referred to as *SGB* in this book.

Figure 3 shows the last device, the Game Boy Color, which is an upgrade over the DMG.



Figure 3: Game Boy Color

For the most part, the Game Boy Color behaves and programs very much like the DMG. However, it does offer a nice set of new functionalities to the programmer, the most distinctive being an extended color palette. The Game Boy Color is referred to as *CGB* in this book, following the model number (GCB-001), but you may also see it referred to by the acronym GBC.

Table 1 summarizes the hardware variations that this book covers, along with the code names used to refer to them.

Table 1: Game Boy Hardware Variations

Code name	Devices	Description
DMG	Game Boy, Game Boy Pocket, Game Boy Light	The original hardware with the four shades of green or gray LCD
SGB	Super Game Boy	A peripheral designed to play Game Boy games on the SNES
CGB	Game Boy Color	An upgrade of the DMG that features new functionalities

The Game Boy Advance, Game Boy Advance SP, and Game Boy Player are based on a completely different hardware architecture. They are not covered in this book.

What You'll Learn

The goal of this book is *not* to take you through the process of making a complete game. Instead, it aims to give you a strong understanding of the Game Boy hardware and the programming techniques that can be used to leverage it properly. That said, you will find some fully functional games

in the companion GitHub repository for the book, at *https://github.com/mdagois/gca/tree/main/games*.

Once you have a good grasp of the material this book covers, I recommend checking out the games available there to get an idea of how you can apply what you've learned.

Who This Book Is For

This book does not assume any prior knowledge of assembly language or the Game Boy hardware. Thorough explanations of all the code samples are provided. The content will be challenging for those with no prior programming experience, but anyone with basic programming skills should be able to successfully follow along.

You should have experience writing programs using fundamental concepts such as variables, loops, conditionals, and functions. Which programming language you're familiar with doesn't matter.

Required Tools

To follow along with the examples in this book, you'll need a few tools. They are all available for free online, and I'll assume you have them installed and ready to use before you begin.

Toolchain

A *toolchain* is a set of tools that turns source code and assets into an executable. There are several toolchains available for the Game Boy, but for assembly language, the de facto standard is RGBDS, which you can find at *https://rgbds.gbdev.io*. The book presents the most important options of this toolchain; comprehensive RGBDS documentation is available at *https://rgbds.gbdev.io/docs/master*.

Emulator

An *emulator* is a program that behaves like a hardware device and can run programs. Emulators are a great substitute for real hardware: they do not cost anything, they are very convenient to use, and they come with handy tools such as debuggers and memory viewers.

This book uses BGB (*https://bgb.bircd.org*). BGB is a native Windows program, but it also works on Linux and macOS through Wine (a version of BGB that you can run on these platforms is available on WineHQ at *https://appdb.winehq.org/objectManager.php?sClass=application&iId=11138*).

If BGB is unavailable on your system, a good alternative is Emulicious (*https://emulicious.net*).

Code Samples

This book uses code listings to illustrate important concepts, but for the sake of brevity, it provides only relevant parts of the code. You can access the complete code examples in the GitHub repository at *https://github.com/mdagois/gca*.

Text Editor

You'll need a raw text editor to edit assembly source code. Assembly syntax is relatively simple, so just about any editor will do. For reference, I used gVim with assembly syntax highlighting to edit the code in this book (see *rgbasm.vim* at *https://www.vim.org/scripts/script.php?script_id=819*).

Recommended Tools

The following tools are not strictly necessary, but they make a programmer's life easier.

Pan Docs

While not really a tool per se, Pan Docs is a fantastic Game Boy technical reference. It provides comprehensive information about the console and the cartridge hardware. You can find Pan Docs at *https://gbdev.io/pandocs/*.

Image Editor

Any nontrivial program displays graphics on the screen. Almost all the samples in this book use graphics. While it's possible to define graphics assets directly inside the assembly code, it's not a very practical approach for producing even moderately complex graphics. An image editor is a better solution.

As RGBDS provides a tool to handle Portable Network Graphics (PNG) images, I recommend getting an editor that can export images to the PNG format. The assets used in this book were edited with the commercial editors Aseprite (*https://www.aseprite.org*) and Pyxel Edit (*https://pyxeledit.com*), but you can also use GIMP (*https://www.gimp.org*), which is free and open source.

Optional Tools

The following tools require some monetary investment and are completely optional. This book doesn't assume you have any of them. They are presented here for information only.

Game Boy

You don't need to own a Game Boy to use this book. Even though the Game Boy is an inexpensive device, it's getting pretty old now, and depending on your location, it might be hard to find one in good

condition. Using an emulator is a good alternative and is much more convenient during development.

That said, if you want to run programs on real hardware, you'll need a Game Boy. A Game Boy Color is probably the best choice for this book. It can run both DMG and CGB programs and is missing only the features from the Super Game Boy, which would require a SNES anyway. A Game Boy Advance is also a good alternative, as it contains a CGB. The best model would be the Game Boy Advance SP AGS-101, because it features a backlit LCD.

You'll also need a Game Boy Printer if you'd like to get actual printouts from the samples in Chapter 17-1.

Flashcart

The Game Boy does not have any out-of-the-box method to load and play custom executables, so if you want to run programs on real hardware you'll need a flashcart. A *flashcart* is a cartridge that contains flash memory, typically a microSD card, into which you can copy programs. Figure 4 shows the EverDrive-GB X7 and EverDrive-GB X3 flashcarts used to test all the code in this book.



Figure 4: EverDrive-GB X7 and X3 flashcarts

EverDrive offers several models of high-quality flashcarts for the Game Boy, and even the cheapest one is very capable. You can find one at *https://krikzz.com*.

About This Book

This book consists of four parts. The first three parts cover the DMG exclusively. This is the common denominator to all the hardware variations of the Game Boy, so it's important to master all of its aspects before moving on to other hardware models. Parts I, II, and III also contain many samples written in assembly that introduce fundamental programming skills, such as loops and conditionals. You'll want to ensure you're comfortable with these before moving on to more advanced techniques, like implementing communication protocols. The fourth part of the book covers DMG variations and peripherals. In these chapters, you'll build on the assembly knowledge you acquired in the first three parts of the book to get control over new pieces of hardware.

Here's a quick synopsis of what you'll find in each part and chapter of the book.

Part I: Getting Started covers all the information you need to get you started programming the DMG and using this book.

Chapter 1: Game Boy Fundamentals Establishes the terminology used throughout the book, presents both the external and internal components of the DMG, and introduces the fundamental concept of a memory map. Equipped with this knowledge, you'll be ready to explore each programming topic in detail.

Chapter 2: Assembly Language Fundamentals Offers a comprehensive tour of the programming language and introduces all the instructions, as well as the most common constructs used in assembly. Everything we'll do in this book will require some amount of assembly programming. By the end of this chapter, you'll be able to follow the code examples, which are all written in assembly.

Chapter 3: The Game Boy Toolchain Covers all the steps necessary to create and test DMG programs. We'll start by writing the very first sample of this book. Then, we'll turn the sample code into an executable program with the RGBDS toolchain. Finally, we'll run and test our first compiled program in the BGB emulator.

Chapter 4: Introducing the Sample Template Describes the simple code template that is the foundation for all the samples in this book. It also introduces two important techniques to debug DMG programs: the map file and the symbols file. By the end of this chapter, you'll have a strong understanding of the structure of the samples, and you'll be able to efficiently inspect and debug programs in BGB.

Part II: Game Boy Programming Essentials introduces the essential pragmatic knowledge required to write interesting DMG programs.

Chapter 5: Graphics Overview Provides a theoretical introduction to graphics, one of the most exciting DMG topics. Most, if not all, programs will use some form of graphics. This chapter will give you a strong grasp of all the fundamental concepts, such as tiles, tilemaps, and sprites.

Chapter 6: Graphics Asset Management Covers the basics of managing graphics assets, from converting image files to making graphics data accessible from your programs. This is the first pragmatic step to getting graphics into your programs.

Chapter 7: High-Level Features Puts all the theory from the two previous chapters into practice. Through a series of samples, you'll learn all the details of the background, the window, and the sprites. We'll also cover frame pacing, a fundamental concept for getting smooth animations.

Chapter 8: The Joypad Shows you how to read the status of the joypad so that you can write programs that react to user input.

Chapter 9: Audio Provides a comprehensive introduction to producing sound with the DMG. Audio is a complex and fascinating topic, so this chapter covers a lot of ground. Samples are provided for each functionality of the DMG audio channels, and by the end of the chapter, you'll have a practical understanding of all the facets of audio programming on the DMG.

Part III: Advanced Game Boy Programming explores advanced topics related to programming the DMG, building on what was covered in the previous chapters.

Chapter 10: The Counter and Timer Introduces two features of the DMG central processing unit (CPU) related to timing: the counter and the timer. The counter tracks the clock ticks of the CPU, and we'll use it to implement a cheap random number generator. The timer supports the precise scheduling of events at given time intervals; it is used extensively in subsequent chapters.

Chapter 11: Serial Communication Covers serial communication, a feature for sending and receiving data via the DMG serial port. You'll learn the basics for making your programs communicate with external devices, such as another DMG or a printer. You'll also get introduced to advanced communication protocols that could be used to implement a multiplayer game.

Chapter 12: Advanced Graphics Discusses advanced graphics topics and techniques, such as parallax scrolling and direct memory access. This content is meant for programmers who want to take full advantage of what the hardware has to offer.

Chapter 13: Memory Bank Controllers Shows how to leverage memory bank controllers to dramatically increase the amount of memory available to your programs. With more memory, you'll be able to implement more complex algorithms and use more graphical and audio assets. Memory bank controllers also enable data persistence, which is essential to implement game saves.

Chapter 14: Interrupts Provides additional information on interrupts. You'll learn about the relative priority of each interrupt and the level of control you can exert on their execution order. This chapter also provides some writing tips for interrupts, to make sure you avoid the most common pitfalls and bugs.

Chapter 15: Programming and Debugging Tricks Explores programming techniques and useful debugging tricks that might come in handy when developing complex programs. As you gain more skills developing larger programs, you might want to revisit this chapter.

Part IV: Beyond the Game Boy covers DMG variations and peripherals. The chapters in this part can be read in any order.

Chapter 16: Game Boy Color Introduces the CGB, which is a major hardware upgrade over the DMG. The CGB brings a slew of additional features, including colored graphics and a CPU running at double speed. This chapter covers the theory and pragmatic aspects of these features.

Chapter 17: Game Boy Printer Discusses the Game Boy Printer, which is a peripheral that lets programs print graphics to thermal paper. This chapter provides all the information you need to control and use the printer from your programs.

Chapter 18: Super Game Boy Introduces the SGB, a DMG with a plethora of additional features, from colored graphics to support for multiple joypads. The chapter starts by explaining how to control the SGB through commands sent via a serial protocol, then gives a complete introduction to all the new features of the SGB and illustrates them with code samples.

Chapter 19: Going Further Concludes our exploration of Game Boy programming and points you to potential next steps. You'll also find a handy table here that links all the features of the Game Boy to their respective chapters, in case you need a reminder of where a particular subject was discussed.

At the end of the book are two appendixes that will come in handy when you're writing your own programs.

Appendix A Lists all the assembly instructions, which will be useful when you're searching for the most efficient and effective instructions for your code.

Appendix B Lists all the instruction opcodes, which will be useful when you're debugging or writing tools.

By the time you finish reading this book, you will have acquired enough technical skills and knowledge to create your own Game Boy programs. Let's get started!

Game Boy Coding Adventure (Early Access) © 2025 by Maximilien Dagois

PART I

GETTING STARTED

Game Boy Coding Adventure (Early Access) © 2025 by Maximilien Dagois

1

GAME BOY FUNDAMENTALS

This chapter will introduce you to the original Game Boy, referred to as the *Dot Matrix Game (DMG)* in this book. This name, a reference to the display technology, is reflected in the original device's model number, DMG-01.

We'll begin with an overview of the external features of the Game Boy, followed by a review of the internal components. These components will be covered in more detail in the following chapters, so it's important to understand their role in the high-level system architecture. We'll also cover the fundamental concept of the memory map, which you need to understand to interact with the DMG. Finally, we'll broadly define what a DMG program is and your role as a programmer.

External Features

The Game Boy system consists of two parts: the console and the cartridge. The *cartridge* is a physical container for a program. The *console* is the machine that runs programs from cartridges, where all the actual hard work is done.

Figure 1-1 shows the front of the console, where all the external components are visible.

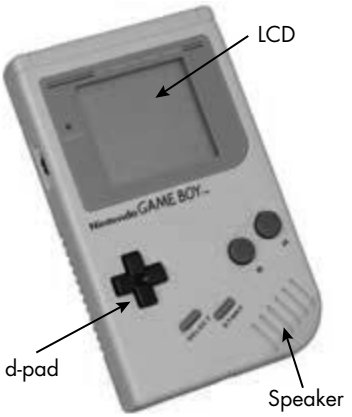


Figure 1-1: The front of the console

The *LCD*, which stands for *liquid crystal display*, displays the graphics. The *d-pad*, or directional pad, is a flat, four-way control button that restricts how many buttons you can press simultaneously. For example, you can use it to press UP and LEFT at the same time, but not UP and DOWN.

The SELECT, START, A, and B buttons are regular push buttons you can control independently of each other. Together with the d-pad, these four game buttons form the *joypad*, the sole input device of the DMG. Finally, the rectangular holes house the speaker.

Figure 1-2 shows the back of the console.



Figure 1-2: The back of the console

You insert the cartridge into the slot on the back of the console. Figure 1-3 shows a blank cartridge.



Figure 1-3: A blank cartridge

Typical commercial cartridges would feature some game art. Once powered on, the console runs the program stored in the memory of the inserted cartridge.

Internal Components

The Game Boy has both active and passive components inside it. The *active components* are responsible for doing the heavy lifting, like computation and rendering. The *passive components* are areas of memory holding data and controlling the DMG's behavior. Figure 1-4 is a high-level overview of these components.

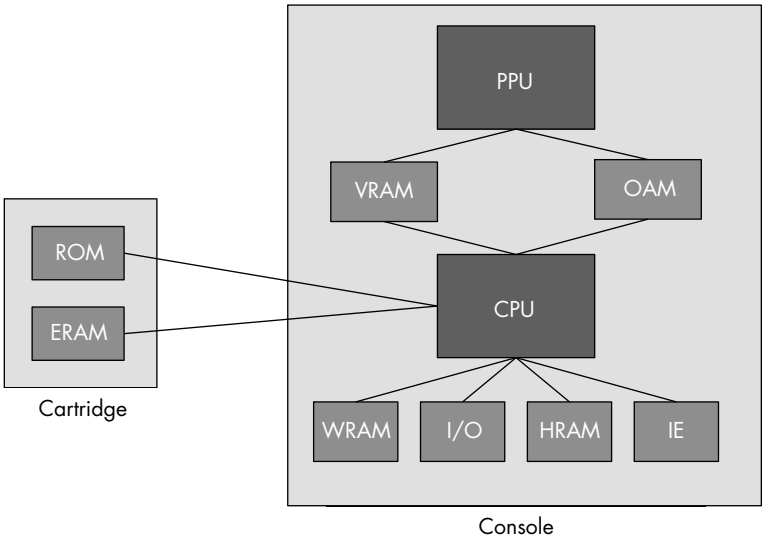


Figure 1-4: The internal components of the DMG

The central processing unit (CPU) and picture processing unit (PPU) in this diagram are active components, and the rest are passive. In the following sections, I'll cover each of these components in turn.

Cartridge Components

Every cartridge contains a *ROM*, or *read-only memory*. This is a type of physical memory that holds all the instructions and assets of a program. As the name suggests, the CPU can only read from the ROM; it cannot be written to.

Some cartridges have another passive component called the *external random-access memory (ERAM)*, an optional piece of volatile memory in which programs can store arbitrary data. ERAM typically stores game saves, in which case it needs a power source so that its content is not lost even when the DMG is turned off. In DMG cartridges, that power source is usually a small button battery.

The ROM and ERAM are the main cartridge components you need to care about, but cartridges may contain additional hardware inside them. Figure 1-5 shows open cartridges for three DMG games: *Tetris*, *Super Mario Land*, and *The Legend of Zelda: Link's Awakening*.



Figure 1-5: Differences in cartridge hardware for Tetris, Super Mario Land, *and* The Legend of Zelda: Link's Awakening

These games provide excellent examples of how cartridge hardware can differ. For example, *Tetris* has only a small ROM of 32KiB, which is the default size for DMG ROMs. *Super Mario Land* requires more graphics assets than a ROM of that size can contain, so it uses a bigger ROM and features a *memory bank controller (MBC)* to unlock the access to the extra memory of the extended ROM.

Zelda has an ERAM to support game saves. The battery is visible in the upper-right corner; it's there to keep the ERAM powered at all times. There's also a small RAM protector in the upper-left corner of the cartridge, which controls the voltage going from the battery to the ERAM to preserve the game save data.

Console Components

The CPU is the first active component at the heart of the console. It has access to all the passive components. The CPU can also perform operations on data, such as incrementing a number or applying a bitwise operator. Its main job is to fetch and execute the instructions inside the ROM.

The PPU, the second active component, is responsible for displaying graphics on the LCD. The PPU is not directly accessible, which means it cannot execute custom instructions you give. Instead, data is put into the *video random-access memory (VRAM)*, which stores graphics data, and the *object attribute memory (OAM)*, which stores attributes of the sprites drawn on the screen. Then, a bunch of LCD-related I/O registers are set to control the PPU behavior.

The *work random-access memory (WRAM)* is a piece of memory where you can temporarily store data. Its purpose is similar to that of the RAM on a regular PC. For example, you can use it to store a puzzle board state or the number of lives of a player.

The *high random-access memory (HRAM)* is similar to the WRAM in that it can hold temporary data. However, the DMG features special instructions to access the HRAM. These instructions take less space in the ROM and execute faster than those that would target WRAM. The catch is that the HRAM is very small, with only 127 bytes versus 8,192 bytes for the WRAM. For this reason, it's better to use the HRAM for small subsets of data accessed very often, or as a scratchpad memory.

The *input/output (I/O)* registers are used to communicate with the console's internal devices. These include the LCD, the joypad, the timers, the serial interface, and the speaker.

The *interrupt enable (IE)* component is a single 8-bit register used to enable interrupts. Interrupts correspond to special events that require prompt attention from the programmer. There are five types of interrupt, each with its own special purpose; you'll learn about these later in the book.

Figure 1-6 shows the motherboard of the DMG.

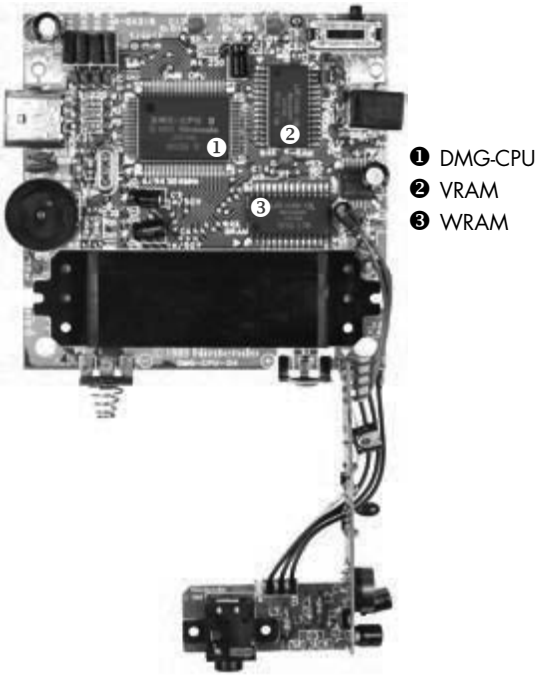


Figure 1-6: The DMG motherboard

The central component, labeled DMG-CPU on the chip itself, has a misleading name. It actually contains the CPU, the PPU, the OAM, the I/O registers, and the HRAM. The VRAM and WRAM are outside the CPU. Interestingly, both VRAM and WRAM use the same type of 8KiB RAM chip made by Sharp; this differs from modern consoles, where the system memory and video memory have very distinct characteristics.

The DMG has a second board, sometimes referred to as the LCD board, shown in Figure 1-7.



Figure 1-7: The LCD board

The LCD board has far fewer electronics on it. It's not visible here, but the LCD is attached to the back of this board. A white ribbon cable at the top links the LCD board to the motherboard.

Memory Map

The layout of all the possible memory addresses in the DMG is called the *memory map*. The DMG has a 16-bit address bus, so there are $2^{16} = 65,536$ available addresses. Each address is in the hexadecimal range `0000`–`FFFF` and points to a single byte.

All the passive components shown in Figure 1-4 are accessible through the memory map. Table 1-1 shows the starting and ending addresses for each one.

Table 1-1: The DMG Memory Map

Start	End	Size (bytes)	Name	Location	Description
`0000`	`7FFF`	32,768	ROM	Cartridge	Stores instructions and assets
`8000`	`9FFF`	8,192	VRAM	Console	Stores graphics data for the PPU
`A000`	`BFFF`	8,192	ERAM	Cartridge	Mostly used for game saves
`C000`	`DFFF`	8,192	WRAM	Console	Temporary data storage
`E000`	`FDFF`	7,680	—	—	Not used
`FE00`	`FE9F`	160	OAM	Console	Stores attributes of the sprites
`FEA0`	`FEFF`	96	—	—	Not used
`FF00`	`FF7F`	128	I/O	Console	Control registers for internal devices
`FF80`	`FFFE`	127	HRAM	Console	Fast temporary data storage
`FFFF`	`FFFF`	1	IE	Console	Interrupt enabling register

Using the passive components is a matter of reading and writing bytes of data from and to the right addresses in the memory map. There is nothing else to interact with: no disk, no library, no driver, and no operating system. It all starts and ends in the memory map.

Game Boy Programs

A *DMG program* consists of instructions and assets that are compiled into a ROM image. That image is transferred into a physical ROM inside a cartridge. When the cartridge is inserted in a DMG and that DMG is powered on, the physical ROM becomes part of the memory map.

The CPU then starts fetching and executing instructions from the ROM. This results in data being moved around the memory map. This data flow determines the whole behavior of the DMG, from what happens when the buttons are pressed to what is displayed on the screen.

NOTE *The term* ROM *is often used to refer to both the physical ROM and the ROM image, because the only purpose of the physical ROM is to contain the ROM image. This book uses the term in this way too, as this abuse of language causes no harm.*

Your role as a DMG programmer is to write the instructions and prepare the assets that go into a ROM. Producing a ROM is the end result of the programming process.

Summary

You should now have a very high-level understanding of the DMG components. Those components may differ widely at the hardware level, ranging from simple pieces of memory to complex peripherals. But from a programmer's perspective, they are all accessed and controlled through address ranges in the memory map. You'll work with all of these components and the memory map repeatedly throughout the course of this book. For reference, an exhaustive view of the memory map is available in Chapter 19.

The next chapter focuses on the CPU, which is responsible for running DMG programs, and introduces assembly language, which is used to write those programs.

2

ASSEMBLY LANGUAGE FUNDAMENTALS

This chapter introduces the fundamentals of assembly language. We'll begin by exploring what assembly is and why we use it to program the Game Boy. Then, we'll look at the DMG CPU, including its registers and flags, which you'll use throughout this book.

We'll cover the basics of assembly language syntax, such as labels, instructions, and comments, and then we'll go through all the instructions available on the DMG so that you understand how each works and how to implement high-level constructs like loops and `if...else` statements. Coverage of these instructions forms the bulk of this chapter, and you can come back to this section later if you need to review specific instructions. An instruction reference is also available in Appendix A.

Finally, you'll learn about concepts like sections and macros, which are not part of assembly language itself but apply in many modern toolchains and can make assembly programming easier. By the end of this chapter, you'll have a solid understanding of how assembly works and should be able to follow all the samples in this book.

We'll cover a lot of ground here, and it's okay if you don't understand everything right away. As you work through the samples, you'll revisit these concepts in context repeatedly, which will help you absorb the material. For now, focus on getting a general grasp of the assembly language.

What Is Assembly?

The native language of a processor is *machine code*. On the DMG, machine code consists of a series of bytes that instructs the processor to perform specific operations. Listing 2-1 shows an example.

```
3E 00
EA 00 C0
3C
18 FA
```

Listing 2-1: Machine code instructions

This snippet of machine code sets a register to zero, then enters an infinite loop that increments that register in each iteration.

While it is technically possible to create programs directly in machine code, the code is difficult to write and read. This is why programming languages were developed. Assembly is one of the earliest programming languages, and although it's not as widely used today as it was when the DMG was released, it remains relevant.

Assembly has a very close relationship with machine code, but instead of using byte sequences, it uses human-readable instructions. For example, Listing 2-2 shows the assembly code equivalent to the previous machine code instructions.

```
ld a, 0              ; 3E 00
.loop ld [$C000], a  ; EA 00 C0
inc a                ; 3C
jr .loop             ; 18 FA
```

Listing 2-2: The assembly code equivalent of Listing 2-1

Each line of this assembly code corresponds to an instruction in machine code, but it's more readable than the machine code, even for someone unfamiliar with the language syntax.

Nowadays, very few programs are written in assembly. Programmers usually opt for higher-level languages, such as C++, Java, or Rust. There are three main reasons for that: the growing sophistication of computer architectures, the complexity of modern assembly, and the need for portability.

Assembly was widely used in the early days of computing, but since then computer architectures have become much more sophisticated. With the addition of various systems meant to optimize program execution time, such as layers of caches, instruction pipelines, and out-of-order execution, it has become difficult for programmers to fully understand the nooks and crannies of an architecture. This has led to more abstractions over the hardware,

often in the form of high-level language libraries, so that programmers can focus on software architecture rather than the details of carrying out each operation through the processor.

The instruction set of modern assembly has also grown over time, to accommodate the growing capabilities of processors. As a result, choosing the right instruction from the vast number available has become more difficult. Compilers, which convert higher-level code into machine code, have surpassed humans in optimizing code, because they can evaluate an incredible number of parameters to select the best machine code instruction.

Finally, many programs today need to run on several architectures, each with its own set of machine code instructions. Compilers simplify this process, because they can output machine code for very different architectures from the same high-level language code.

However, the situation is very different when it comes to the DMG. Its architecture is simple enough that you can expect to master most of its features within a reasonable time frame. The DMG's behavior when running a program is highly predictable, because there's no memory cache, instruction pipeline, or branch prediction. Only one program runs at a time, with no operating system layer.

Accordingly, the assembly language is also accessible. The number of instructions is small enough for a programmer to memorize all of them. And because the DMG is such a constrained environment, with limited memory and computing power, handwritten code can still beat compiler-generated code.

Portability is also rarely a concern when programming the DMG. The hardware is so specific that only a fraction of the data or algorithms can be reused on another machine. To fully unlock the console's capabilities, programmers must leverage every dedicated features the DMG has to offer. Assembly allows precise control over the hardware, making it an ideal choice for the kind of low-level, bare-metal programming we'll do in this book.

Finally, programming in assembly has unique benefits that high-level languages don't provide. Because it's so close to the hardware, programming in assembly helps you gain a deeper understanding of computer architecture and build intuition about how code gets executed. Its minimal building blocks to write programs will also grow your problem-solving skills, making you a better programmer even when you return to high-level languages.

The Central Processing Unit

The CPU is responsible for executing the instructions from the ROM. An understanding of its basic characteristics is necessary before we dive into the assembly language itself. Here, I'll introduce you to the registers available in the DMG CPU and the flags used to track the outcome of different operations.

Registers

The DMG CPU has eight 8-bit registers: `a`, `b`, `c`, `d`, `e`, `f`, `h`, and `l`. *Registers* are small named areas of memory that the CPU can access quickly and easily, and they are used in almost every instruction.

We can combine some pairs of 8-bit registers to form four 16-bit registers: `af`, `bc`, `de`, and `hl`. These 16-bit registers share the same storage as the 8-bit registers. This means that modifying the values of any of them will change the values of the corresponding 8-bit registers, and vice versa.

The `a` register, also called the *accumulator*, is the most commonly used register, because most 8-bit arithmetic and bitwise operations can be performed only on `a`.

The `b`, `c`, `d`, `e`, `h`, and `l` registers are less versatile than `a` and are mostly used to hold data temporarily while working with `a`. There is one specific load instruction that is usable only with `c`, but otherwise, these registers are all equivalent. The `f` register, also called the *flag register* or *status register*, is not used directly.

Registers `bc`, `de`, and `hl` are commonly used for holding 16-bit values and performing indirect addressing, which is explained in "Loads" on page 20. The `hl` register is more versatile, as it can also be used for 16-bit addition and other specialized operations. You can think of `hl` as the 16-bit counterpart of `a`.

In addition to these, the CPU features two other 16-bit registers with specific purposes, which we'll look at shortly: `pc` and `sp`.

Flags

A *flag* is a specific bit in the `f` register that will be set or cleared (that is, set to zero or one) depending on the result of the last instruction that affected it. Flags are important for making decisions and controlling the execution flow in assembly programs.

Not every instruction affects every flag. Unaffected flags will stay in their current state until an instruction that affects them executes. See Appendix A for more details about which flags each instruction affects.

The `f` register contains four flags: `Z`, `N`, `H`, and `C`. Also known as the *zero flag*, `Z` indicates whether the result of an operation was zero. For example, if you perform an addition and the result is zero, `Z` will be set. If the result is not zero, `Z` will be cleared. You can use `Z` with conditional instructions to implement the equivalent of `if...else` statements and loops.

The *carry flag*, `C`, indicates a carry or borrow in an addition or subtraction. For example, assuming the 8-bit register `a` contains 200 and you add 60 to it, `a` will overflow and cycle back to 4, because the highest value that an unsigned 8-bit register can store is 255. As a result, `C` will be set. Like `Z`, you can use `C` with conditional instructions.

The `N` and `H` flags are related to *binary-coded decimal (BCD)*, a system where each decimal digit (0-9) is represented by a four-bit binary code, allowing for easy conversion between decimal and binary systems. Sometimes referred to as the *negative flag*, `N` is the add/sub flag, indicating whether the operation

was a subtraction. The *half carry flag*, `H`, acts as a carry for the low nibble during an arithmetic operation. A *nibble* is a group of 4 bits. Each byte consists of a low nibble (bits 0 to 3) and a high nibble (bits 4 to 7). The `N` and `H` flags are leveraged by only a single instruction, `daa`, used to convert the result of an arithmetic operation on two BCD numbers back to BCD.

While BCD is useful for representing decimal numbers, I won't cover it in this book. Unless you want to make a calculator or a financial application, it's not that relevant for DMG programming. To learn more about BCD, visit *https://en.wikipedia.org/wiki/Binary-coded_decimal*.

Special-Purpose Registers

As mentioned previously, the CPU has two additional 16-bit special-purpose registers: the program counter, `pc`, and the stack pointer, `sp`. These registers don't have 8-bit counterparts.

The program counter (`pc`) points to the next instruction that the CPU will execute. For example, let's say `pc` is equal to 1000 and the instruction at address `1000` is 3 bytes long. The CPU will fetch and execute the instruction at address `1000` next. After that instruction executes, `pc` will update to 1003 (1000 + 3 bytes of the instruction). The CPU will then fetch and execute the instruction at address `1003`, updating the `pc` register again to point to the next instruction, and so on.

The stack pointer (`sp`) points to the top of the stack. The *stack* is a special area of memory that can save data temporarily, but its main use is for function calls. When a function is called, the address of the instruction that should be executed once the function finishes execution is stored on the stack. This address is called the *return address*. The stack on the DMG is a *push-down stack*. Pushing a value onto the stack (or calling a function) will decrement `sp`, while popping a value from the stack (or returning from a function) increments `sp`. At the start of a program, `sp` is always initialized to point to the top address of the HRAM (`$FFFE`). However, the stack can be relocated to anywhere else (for example, in the WRAM) by changing the value of `sp`.

Table 2-1 summarizes all the CPU registers.

Table 2-1: The DMG CPU Register Set

16-bit	High 8-bit	Low 8-bit	Function
`af`	`a`	—	`a`: accumulator, `f`: flags (`ZNHC----`)
`bc`	`b`	`c`	Generic 8/16-bit
`de`	`d`	`e`	Generic 8/16-bit
`hl`	`h`	`l`	Generic 8/16-bit, arithmetic operations
`sp`	—	—	Stack pointer
`pc`	—	—	Program counter

In this book, register names are lowercase, while flag names are uppercase. This convention is especially important to distinguish between the register `c` and carry flag `C`.

Number Formats

There are several number formats in assembly language. This book uses three of them heavily: hexadecimal, binary, and decimal. The format used depends largely on what the number represents. You can expect all memory addresses to be expressed in hexadecimal. Flags will often be in binary, and sometimes in hexadecimal. Ordinary values, like loop iteration counts, will be in decimal.

Hexadecimal numbers have the `$` prefix (`$0100`), binary numbers have the `%` prefix (`%00000011`), and decimal numbers do not have any prefix (11). For example, `$37` is the hexadecimal equivalent of the binary number `%00110111` and the decimal number 55.

Assembly Language Syntax

A statement in assembly language consists of three different elements. The syntax is as follows:

```
[label:] [instruction] [; comment]
```

Each element is optional; a statement may contain one, two, or three elements. To be syntactically valid, it must satisfy the following rules:

- There can be only one element of each type.
- If more than one element is present, the order matters. The label comes first, then the instruction, and finally the comment.

Listing 2-3 shows a few examples of syntactically valid statements.

```
hello: ld a, b  ; Loads the value of register b into a

bye:  ; The "bye" label

ld a, b

; This is the end of the syntax examples.
```

Listing 2-3: Syntax examples

Assembly is typically written with one statement per line. You can have instances of each element on a single line, but never two elements of the same type. Let's talk a bit more about each type of element.

Labels

Labels are symbols followed by a colon (`:`). Each label must be defined only once. When the toolchain processes your assembly code, it will assign an address value to each label based on where the first instruction after the label will be in the ROM. You can use labels as instruction operands, wherever an address (16-bit value) would be used. Listing 2-4 shows some examples.

```
label_1:
    ld a, 10
    add a, b
label_2:
label_3:
```

Listing 2-4: Label examples

Let's assume the instructions in Listing 2-4 are at address `$1000` in the ROM. Then, `label_1` would be equal to `$1000`. Because the `ld` instruction takes 2 bytes and the `add` instruction takes 1 byte, `label_2` would be equal to `$1003` (`$1000` + 2 + 1). Finally, because there are no instructions between `label_2` and `label_3`, `label_3` would also be `$1003`.

Labels are just constructs to make the programmer's life easier. They do not have any impact on the ROM per se.

Instructions

While programming in assembly might seem intimidating at first, DMG assembly is simpler than any modern CPU assembly. Modern assembly languages may have hundreds, if not thousands, of instructions, combined with complex hardware architectures. They are meant to be used by compilers rather than humans. On the other hand, DMG assembly has only 107 instructions, and most DMG games were programmed exclusively in assembly.

You might think that 107 instructions is a lot to remember. But the good news is that many of these instructions are extremely similar and can be accessed using a single symbolic name, or *mnemonic*. For example, the DMG has 22 different instructions for loading data into memory, differing only in the kinds of data and memory they operate on. However, we don't need to remember all of those. Instead, we can use a single name for these instructions: `ld`. Based on how we use that mnemonic, the toolchain will generate the correct instruction in the generated ROM.

In DMG assembly, there are only 44 mnemonics, about a quarter of which we use frequently. Another quarter correspond to very specific cases, and the remaining half are used only occasionally.

An instruction consists of a mnemonic followed by up to two operands separated by commas. Listing 2-5 shows a few examples.

```
; no operand
halt
stop

; 1 operand
rrc a
inc b

; 2 operands
ld a, b
add a, 57
jp z, $1000
```

Listing 2-5: Instruction examples

There are only three patterns, and assembly uses very few symbols compared to higher-level languages such as C. This makes the statement syntax relatively easy to remember.

Next, let's go over two very important concepts related to instructions: byte cost and cycle cost.

Byte Cost

Each time you write an instruction, it will consume part of the ROM space. That's the *byte cost* of the instruction.

The toolchain translates instructions to machine code. The machine code for an instruction consists of an *operation code (opcode)*, followed by some data depending on the operands of the original instruction. For example, `inc a`, an instruction to increment the `a` register, translates to opcode `3C` without any data. This opcode tells the CPU to increment `a`, so no additional data is necessary. On the other hand, `ld a, 10`, an instruction to load the value 10 into `a`, translates to `3E 0A`. The opcode `3E` tells the CPU to load a literal into `a`. That literal here is 10. It follows the opcode in the machine code as the hexadecimal value `0A`.

There is a strong correlation between the parameters of an instruction and its byte cost, which makes the metric quite intuitive: The more specific the instruction, the fewer bytes it takes. Usually, using registers as parameters increases specificity. On the other hand, literals and addresses decrease specificity. Using instructions with smaller byte costs saves ROM space, which means that we can fit more code and resources into the ROM.

See Appendix B for a reference table with all the opcodes.

Cycle Cost

The *execution cost* of an instruction is the amount of time it takes to execute. A natural way to express the execution cost is to use *T-states*, also known as *clock ticks*. All instructions take a multiple of 4 T-states to execute. That is, they take either 4, 8, 12, 16, 20, or 24 clock ticks to complete. For example, `nop` takes 4 T-states and `ld a, $FF` takes 8 T-states.

In this book, I'll refer to the execution cost as *cycle cost*, which is expressed in machine cycles (*M-cycles*). An M-cycle is equal to 4 T-states, so instructions take from 1 to 6 M-cycles to complete. To go back to the previous example, `nop` would take 1 M-cycle and `ld a, $FF` would take 2 M-cycles to execute.

The numbers are smaller than when using T-states and the precision is the same, so M-cycles are arguably more convenient to use. That said, whether you use T-states or M-cycles is a matter of preference; ultimately, what really matters are the trade-offs between byte and execution costs, more than the actual numbers involved. M-cycles are commonly referred to as just *cycles*, and that's the term I'll use from now on.

The DMG CPU is a custom Sharp LR35902 core running at 4.194304 MHz (2^{22} Hz), or 4,194,304 T-states per second. But because we'll use cycles, we can consider the CPU as actually running at 1.048576 MHz (2^{20} Hz). The theoretical maximum number of instructions the CPU can execute per second (if all instructions were 1 cycle long) is therefore 1,048,576. Of course, we're unlikely to ever reach that theoretical maximum because many instructions take more than 1 cycle to execute, but it helps illustrate the scale of the CPU's potential.

Comments

Anything that appears after a semicolon (`;`) in an instruction is a comment, and the toolchain ignores it. This is equivalent to `//` in Java or C, `#` in Python, and `--` in Lua.

Comments are important for the programmer's sanity. Assembly code is very explicit about the CPU operations to be performed. Instructions are precise and narrowly scoped, and even moderately complex algorithms will take many instructions. The code's intent will also be far less obvious than it would be in a higher-level programming language like C. For this reason, it's good practice to comment packs of lines that fulfill a given goal. For example, you could add the comment `; a = a * 7` in front of five instructions that result in multiplying the `a` register by 7.

Instruction Categories

Instructions fall into several categories, as summarized in Table 2-2.

Table 2-2: Instruction Categories

Category	Mnemonics
Loads	`ld`, `ldh`
Jumps	`jp`, `jr`
Functions	`call`, `ret`, `reti`, `rst`
Arithmetic operations	`adc`, `add`, `dec`, `inc`, `cp`, `sbc`, `sub`
Bitwise operations	`and`, `or`, `xor`
Bit operations	`bit`, `res`, `set`, `swap`
Shifts and rotations	`rl`, `rla`, `rlc`, `rlca`, `rr`, `rra`, `rrc`, `rrca`, `sla`, `sra`, `srl`
Stack operations	`pop`, `push`
Miscellaneous	`ccf`, `cpl`, `daa`, `di`, `ei`, `halt`, `nop`, `scf`, `stop`

Let's go through each of these in turn.

Loads

Loads are the most common instructions. Their parameters differ and some of the loads have side effects, but overall, they all follow the same pattern: They take two operands, a destination and a source, and copy the value of the source into the destination. Listing 2-6 shows a few examples.

```
ld c, 57       ; loads value 57 into c
ld d, b        ; loads the value of b into d
ld a, [$1234]  ; loads the value at address $1234 into a
ld [hl], a     ; loads the value of a into the address inside hl
```

Listing 2-6: Load instruction examples

Notice the use of square brackets in the last two instructions. The brackets indicate that we're providing an address instead of a value. The concept is very similar to that of a pointer in C. For example, `ld a, $12` means "load the value `$12` into `a`," while `ld a, [$1234]` means "load the value at address `$1234` into `a`." Likewise, `ld hl, $1234` means "load the value `$1234` into `hl`," while `ld [hl], a` means "load the value of `a` into the address contained in `hl`." Using a 16-bit register as an address between brackets is called *indirect addressing*.

Note that many load instructions are available only for `a` or `hl`. For example, loading from a hardcoded address is possible only with `a`. Likewise, some forms of indirect addressing are possible only through `hl`, not through `bc` or `de`.

Of the 22 variations of loads, 4 are `ldh` instructions. `ldh` works like `ld`, but it can load only to and from an address in the range `$FF00–$FFFF`, which includes the I/O registers, HRAM, and the IE component. The advantage of `ldh` is that it is both smaller and faster than an `ld` instruction.

Listing 2-7 shows examples of both types of load operation.

```
ld [$FF80], a   ; 3 bytes, 4 cycles
ldh [$FF80], a  ; 2 bytes, 3 cycles
ldh [c], a      ; 1 byte,  2 cycles (assuming c holds $80)
```

Listing 2-7: Comparing `ld` and `ldh`

These instructions all have equivalent outcomes (that is, the value of the `a` register is copied to address `$FF80`), but their costs are different.

As mentioned earlier, some load instructions have side effects. There are only four such instructions, and they all involve `hl` and `a`. Instead of using `hl`, you use `hli` to increment or `hld` to decrement `hl` after the load instruction is executed. Note that `hli` and `hld` are not registers. The `i` and `d` suffixes are just syntactic sugar; the load instructions act on `hl`.

Listing 2-8 shows a standard load operation followed by an increment.

```
ld [hl], a  ; 1 byte, 2 cycles
inc hl      ; 1 byte, 2 cycles
```

Listing 2-8: Loading and incrementing `hl` without side effects

This takes a total of 2 bytes and 4 cycles.

Listing 2-9 has the same effect, but it's more efficient.

```
ld [hli], a  ; 1 byte, 2 cycles
```

Listing 2-9: Loading and incrementing `hl` using side effects

When using `hli`, the `hl` increment basically comes for free. The same benefit applies when using `hld` to decrement the value in `hl`.

NOTE *You might come across the instructions `ldi` and `ldd`. These are equivalent to using `ld` with `[hli]` and `[hld]`, respectively. For example, `ldd a, [hl]` is the same as `ld a, [hld]`. There are no particular benefits to using one syntax over the other; it's a matter of personal preference.*

Jumps

You can use jumps as a way to change the value of the `pc` register. They help you take control of the instruction execution flow and are essential building blocks for conditionals and loops.

Conditional vs. Unconditional

There are two types of jumps: unconditional and conditional. An *unconditional jump* takes only a single operand, the address to jump to, and always updates the value of `pc` with that address. A *conditional jump* takes two operands, a condition and an address, and updates the value of `pc` only when the condition is met. When the condition is not met, the jump instruction has no effect on `pc`. Table 2-3 shows the four possible jump conditions.

Table 2-3: Jump Conditions

Condition	Description
z	The zero flag is set.
nz	The zero flag is not set.
c	The carry flag is set.
nc	The carry flag is not set.

Conditions are also used in instructions other than jumps, and they behave the exact same way in every circumstance. Once you understand conditions for jumps, you'll understand them for all other instructions as well.

Listing 2-10 shows examples of each type.

```
jp $1000        ; always jumps to $1000
jp nz, $1000    ; jumps to $1000 only if the zero flag is not set
jp c, location  ; jumps to the "location" label only if the carry flag is set
```

Listing 2-10: Unconditional and conditional jump instructions

The first instruction jumps unconditionally to address `$1000`. The second jumps to address `$1000` only if the zero flag is not set. Otherwise, the jump is ignored. The last instruction jumps to the address pointed to by the label `location` only if the carry flag is set.

Let's look at a few examples that demonstrate how to build `if...else` and loop constructs with jumps. I'll provide the assembly and the equivalent code in C-like syntax.

Listing 2-11 shows how to build an `if` statement in assembly.

```
add a, b
jp z, zero
    ; do something when "a + b != 0"
zero:
```

Listing 2-11: Writing an `if` statement in assembly

The code uses a single jump. A label, `zero`, is used to track the jump destination address, so that we can let the toolchain compute it for us.

Listing 2-12 shows the equivalent in C for reference.

```
a = a + b;
if(a != 0)
{
    // do something when "a + b != 0"
}
```

Listing 2-12: An `if` construct in C

Listing 2-13 shows an `if...else` construct.

```
    add a, b
❶ jp nz, not_zero
      ; do something when "a + b == 0"
   ❷ jp zero_check_end
❸ not_zero:
      ; do something when "a + b != 0"
    zero_check_end:
```

Listing 2-13: Writing an `if...else` statement in assembly

This time, two jumps are necessary. The first jump ❶ is conditional. If the condition is not met, then the code directly following the jump instruction is executed. This corresponds to the `if` part. If the condition is met, then the code jumps to the destination address ❸ and executes the code there. That's the `else` part. The second jump ❷ is inside the `if` part. It is unconditional and is essential to skip the `else` part after the `if` part executes. Listing 2-14 shows the equivalent in C for reference.

```
a = a + b;
if(a == 0)
{
    // do something when "a + b == 0"
}
else
{
    // do something when "a + b != 0"
}
```

Listing 2-14: An `if...else` construct in C

Listing 2-15 shows a basic loop.

```
❶ ld c, 8
  loop:
      ; do something while c is between 8 and 1
   ❷ dec c
   ❸ jp nz, loop
  ; c is 0 when the loop is exited
```

Listing 2-15: Writing a loop in assembly

Only one jump instruction is necessary. That jump sets `pc` back to the start of the loop as long as a particular condition is met. In this code snippet, as long as the `c` register is not zero, the loop keeps on going.

Notice the pattern of the loop. `c` is initialized before the loop begins ❶, then decremented in each iteration of the loop ❷. The side effect of this decrement is to set the zero flag if `c` is zero. Right after that, the conditional jump ❸ is executed and determines whether the loop ends or continues.

Most loops follow a very similar pattern: an initialization phase, an operation that affects a flag, and a conditional jump leveraging the flag. Listing 2-16 shows the equivalent in C for reference.

```
c = 8;
while(c != 0)
{
    // do something while c is between 8 and 1
    --c;
}
```

Listing 2-16: A loop construct in C

`if...else` statements and loops are fundamental building blocks for any program. They will appear repeatedly throughout the code samples in this book.

Absolute vs. Relative

In all the previous examples, we've used the `jp` jump instruction. But there are actually two instructions for jumps: `jp` and `jr`.

The `jp` instruction is an *absolute* jump. It takes a 16-bit address, which is used as it is in the machine code. The `jr` instruction is a *relative* jump. It also takes a 16-bit address, but the toolchain converts that address to an 8-bit signed offset, which is used to jump relatively from the address right after the jump instruction.

Listing 2-17 illustrates the use of `jr`.

```
jr some_label
ld a, b
some_label:
; jr will jump here
```

Listing 2-17: A relative jump

This is extremely similar to an absolute jump. The advantage of using `jr` over `jp` is that it saves both a byte and a cycle, so it's smaller and faster. The disadvantage is that you can only jump up to 128 bytes backward or 127 bytes forward with `jr`, as the offset is encoded within a single byte. In comparison, a `jp` can jump to any address in the memory map.

This restriction is why I started by introducing `jp`. In practice, however, it's unlikely to cause you problems, so I generally recommend using `jr` instead (Listings 2-13 and 2-15, for example, could be rewritten to use this instruction). The toolchain will complain if the offset is too large to fit into a byte. In that case, you can try the hard way and adjust your code until you can use `jr`, or you can simply use `jp` when `jr` can't do the job.

I also recommend always using labels. They work with both `jp` and `jr` and will free you from hardcoding addresses, which should rarely be necessary.

Functions

Like most languages, assembly has functions. They are supported by two instructions: `call` and `ret`.

Listing 2-18 shows a simple example of a function call.

```
❶ my_function:
      ld a, 1
   ❷ ret

  ; some code is in here

❸ call my_function
  ; the return address is here; that's where "ret" will jump back to
```

Listing 2-18: Using `call` and `ret`

The `call` and `ret` instructions work in tandem.

In this example, the `call` ❸ jumps to the `my_function` label ❶, which is where the function's instructions are located. But before the jump, it pushes the function return address onto the stack. That address is right after the `call` instruction and is equal to `pc` + 3 (the byte cost of `call`).

The function body is then executed until the `ret` instruction ❷ is encountered. The `ret` pops the return address that `call` pushed onto the stack. It then jumps to that address so that the execution can resume after the function call.

There are conditional versions of `call` and `ret` that work exactly like conditional jumps. For example, `call c, my_function` would call the function only if the carry flag is set. Likewise, `ret z` would return from a function only if the zero flag is set.

In addition to `call` and `ret`, there is also a special return instruction, `reti`, which is basically equivalent to `ret` but with the additional side effect of enabling interrupts. We'll skip that for now because it is only used in the context of interrupts, which are covered later in this book.

Finally, there's the less intuitive `rst` instruction, which behaves exactly like `call` but allows you to use only eight addresses as parameters: `$00`, `$08`, `$10`, `$18`, `$20`, `$28`, `$30`, and `$38`. These addresses, known as *restart vectors*, are located at the very beginning of the ROM. The `rst` instruction is more efficient than `call` (you gain 2 bytes and 2 cycles), but it comes with that huge restriction on where your function code can be located. It was likely included in the DMG's instruction set because contemporary hardware (calculators and 8-bit computers) with similar CPU designs were using `rst` to do specific operations, like resetting the hardware or printing characters on screen. The DMG does not attach any meaning to the `rst` addresses, so you can use them as you wish, or not use them at all.

Arithmetic Operations

The DMG supports 8-bit and 16-bit arithmetic operations. This section covers only the 8-bit operations, which are usually more flexible than their 16-bit counterparts. Once you understand these, it should be easy enough to use the 16-bit operations, which are described in Appendix A.

Addition and Subtraction

Addition is covered by two instructions: `add` and `adc`. The `add` instruction adds a number to the register `a`. For example, `add a, b` adds `b` to `a`. The `adc` instruction does the same thing, but it also adds the value of the carry flag, which is either 0 or 1. For example, `adc a, d` adds `d + C` to `a`. When the carry flag is not set, `add` and `adc` are equivalent. Listing 2-19 illustrates the various cases.

```
ld a, 10
add a, 20
; a contains 30

; assuming the carry flag is not set (that is, C = 0)
ld a, 10
adc a, 20
; a contains 30

; assuming the carry flag is set (that is, C = 1)
ld a, 10
adc a, 20
; a contains 31
```

Listing 2-19: Addition examples

Because `a` is an 8-bit register, you can perform only 8-bit additions on it. If you need to use numbers with more bits, `adc` will come in handy.

For example, Listing 2-20 shows one way to perform an addition on two 24-bit integers. The code assumes that the two 24-bit numbers are located at addresses `$1000` and `$2000`, and the result is stored at address `$3000`.

```
; first load the least significant bytes (at $1000 and $2000) and "add" them
ld a, [$1000]
ld b, a
ld a, [$2000]
add a, b
ld [$3000], a

; then "adc" the second bytes (at $1001 and $2001)
ld a, [$1001]
ld b, a
ld a, [$2001]
adc a, b
ld [$3001], a
```

```
; finally "adc" the most significant bytes (at $1002 and $2002)
ld a, [$1002]
ld b, a
ld a, [$2002]
adc a, b
ld [$3002], a

; addresses $3000-$3002 contain the result of the 24-bit addition
```

Listing 2-20: An example of 24-bit addition

Note that the code takes advantage of the fact that `ld` does not change the carry flag. You can have as many `ld` instructions as you want between `add` and `adc`; the carry flag won't be affected.

Unsurprisingly, subtraction is very similar to addition. The two relevant instructions are `sub` and `sbc`. You could perform a 24-bit subtraction using similar code to that in Listing 2-20, replacing `add` with `sub` and `adc` with `sbc`.

Comparison

The `cp` instruction is used for comparison. It performs a subtraction, just like `sub` would, but it throws the result away instead of storing it into `a`. Its only observable effects are on the flags in the status register, `f`.

Listing 2-21 shows an example of using the `cp` instruction.

```
cp a, 128
jr nz, not_equal
    ; do something when a == 128
not_equal:
```

Listing 2-21: An example comparison

It uses the zero flag to determine whether `a` is equal to 128. If `a` contains 128, then the result of the subtraction performed by `cp` will be zero, which will raise the zero flag. A conditional jump can then be used to check the flag and execute any relevant piece of code. Again, `a` remains totally unchanged after `cp`. Only the flags are affected.

Table 2-4 lists what the status register's flags would allow us to conclude about `a` after the `cp` in Listing 2-21.

Table 2-4: The Meaning of the `cp` Flags

Condition	Description	Meaning
z	Z is set.	a == 128
nz	Z is not set.	a != 128
c	C is set.	a < 128
nc	C is not set.	a >= 128

Using `cp` and the zero and carry flags it's possible to fully compare two values, just like in any higher-level language such as C.

Increment and Decrement

The simplest arithmetic operations are incrementing and decrementing. The `inc` and `dec` instructions increment or decrement (respectively) the value stored at the location supplied as an argument. Unlike other instructions, they work on most registers, instead of being limited to `a`.

Bitwise Operations

Bitwise operations (`and`, `or`, and `xor`) work similarly to 8-bit arithmetic operations. Like the arithmetic instructions, the bitwise instructions operate only on `a`, but there are no 16-bit versions.

There are three patterns for bitwise operations: register, indirect addressing, and literal. Listing 2-22 shows examples of the three patterns for `and`.

```
; pattern 1 - register
ld a, $0C
ld b, $F8
and a, b

; pattern 2 - indirect addressing
ld a, $0C
ld [hl], $F8
and a, [hl]

; pattern 3 - literal
ld a, $0C
and a, $F8
```

Listing 2-22: Examples of using `and`

In each case, after the `and` instruction is executed, `a` will hold `$08`. The `or` and `xor` (exclusive or) instructions can be used in the same ways but will yield different results.

Bit Operations

Bit operations are useful to track Boolean (true/false) status. The `bit` instruction tests if a given bit is set in an 8-bit integer. Bits are numbered from 0 to 7, with 0 being the least significant bit. The `bit` instruction will set the `Z` flag when the tested bit is zero. You can then test the flag with a condition (`z` or `nz`).

The `set` and `res` instructions set and clear a single bit in an 8-bit integer, respectively. They work similarly to `bit`.

The last instruction, `swap`, will transpose the two nibbles inside an 8-bit integer.

Listing 2-23 shows examples of their use.

```
ld a, %00001000
bit 0, a
; Z = 1

ld a, %00001000
bit 3, a
; Z = 0

ld a, %00000000
set 3, a
; a = %00001000

ld a, %10001000
res 3, a
; a = %10000000

ld b, $D4
swap b
; b = $4D
```

Listing 2-23: Bit operation examples

Bit operations are frequently used in assembly, so it's good to keep them in mind.

Shifts

Shifts and rotations are quite similar and often used together. First, let's see how shifts work.

A *shift* moves all the bits of an 8-bit register one position to the left or to the right. One bit falls off and goes into the carry, and one bit comes in. Its value depends on the shift instruction used. There is one left shift instruction, `sla`, and two right shift instructions, `sra` and `srl`. All three shift operations are shown in Figure 2-1.

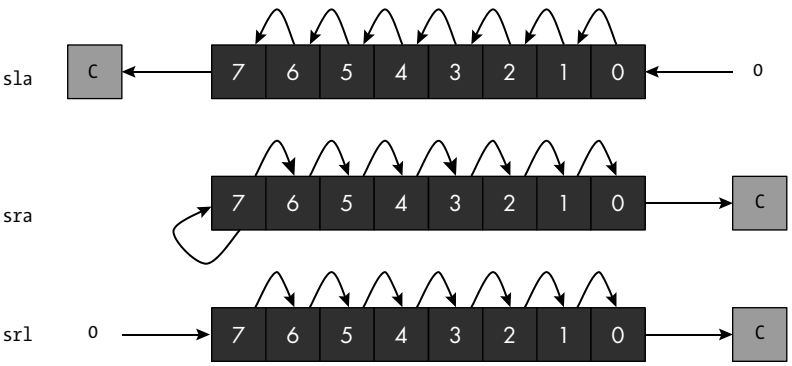


Figure 2-1: The three shift operations

The arrows point to where each bit goes after the shift. The exiting bit always goes to the carry. The `sla` and `srl` shifts fill the missing bit with a zero, while `sra` reuses the value of bit 7.

Listing 2-24 shows some examples of shift instructions in assembly.

```
; arithmetic left shift
ld b, %00110101
sla b
; b = %01101010, C = 0

; arithmetic right shift
ld c, %11100100
sra c
; c = %11110010, C = 0

; logical right shift
ld d, %11100100
srl d
; d = %01110010, C = 0
```

Listing 2-24: Shift operation examples

Shifts are handy because a left shift is equivalent to multiplying by 2, while a right shift is equivalent to dividing by 2. As the DMG CPU does not support generic multiplications, it's common practice to leverage shifts for this purpose. Multiplying by any power of 2 can be implemented as a series of shifts, and it's possible to reduce other multiplications to a combination of shifts and additions. Listing 2-25 shows an example.

```
ld b, 30  ; b = 30   ; b is 30 at the start

ld a, 0   ; a = 0
add a, b  ; a = 30   ; a holds b * 1

sla b     ; b = 60
add a, b  ; a = 90   ; a holds b * 3

sla b     ; b = 120
add a, b  ; a = 210  ; a holds b * 7

ld b, a   ; b = 210  ; b is multiplied by 7
```

Listing 2-25: Multiplying by 7

Here, the register `b` is multiplied by 7 using shifts and additions. The idea is to replace `7 * b` by `4 * b + 2 * b + 1 * b`.

Another solution would be to compute `8 * b - 1 * b`. The code would be more concise, but for any value over 32, any register holding `8 * b` would overflow.

Note that there is a reason for having two right shift operations: to support signed and unsigned division. In signed numbers represented using the two's complement method, bit 7 determines the sign. So, when using shifts for signed division, it is necessary to keep that bit at the same value it was before the shift. Otherwise, the sign might change due to a division, which is obviously the wrong behavior. In these cases, you can use `sra`; for unsigned division, `srl` does the job.

Rotations

Rotations are very similar to shifts. The difference is that the bit that falls off one end is put back on the other end. Four of the eight rotation instructions (`rlc`, `rlca`, `rrc`, and `rrca`) do just that. The other four instructions (`rl`, `rla`, `rr`, and `rra`) use the carry as a temporary buffer for the bit that falls off. The bit that comes in has the value of the carry before the rotation instruction is executed. In a way, it's equivalent to a rotation on a 9-bit value.

The `rl` and `rlc` instructions are illustrated in Figure 2-2.

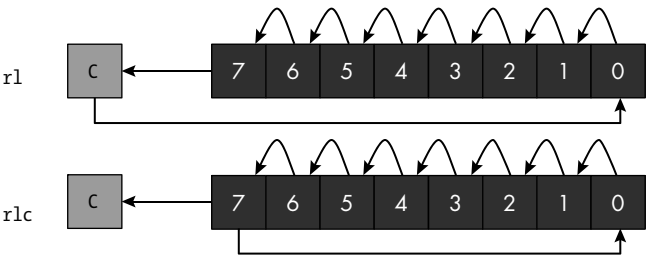


Figure 2-2: The two rotation operations

The `rl` instruction goes through the carry, while `rlc` rotates inside the register only (though the carry is influenced as well). Listing 2-26 shows examples of these two operations in assembly code.

```
xor a            ; clear the carry
ld a, %10000000  ; a = %10000000, C = 0
rl a             ; a = %00000000, C = 1

xor a            ; clear the carry
ld a, %10000000  ; a = %10000000, C = 0
rlc a            ; a = %00000001, C = 1
```

Listing 2-26: Using the `rl` and `rlc` instructions

The other rotation instructions work in a similar fashion. The `rla` and `rlca` instructions work like `rl` and `rlc`, respectively, but only with register `a`. Consequently, they have lower byte and cycle costs.

The `rr`, `rrc`, `rra`, and `rrca` instructions work like `rl`, `rlc`, `rla`, and `rlca`, respectively, but the rotation goes in the right-hand instead of the left-hand direction.

It can be difficult to memorize the behavior of all the rotation instructions and their side effects, so feel free to refer back to this section or to Appendix A to find the instruction you need.

Stack Operations

There are two stack operations: `push` and `pop`. The `push` operation puts the value of any 16-bit register onto the stack, while `pop` pulls the last pushed value into any 16-bit register. If several registers are pushed onto the stack, they will pop in reverse order.

As discussed earlier in this chapter, the stack on the DMG is a push-down stack. The `sp` register always contains the address of the top of the stack, where the next pushed value will be stored. It is updated after each `push` and `pop` to reflect the fact that a new 16-bit value has been added to or removed from the stack. It is decremented by 2 on a `push` and incremented by 2 on a `pop`.

Listing 2-27 shows a simple `push`/`pop` sequence.

```
; assuming sp = $FFFE

ld a, 10
add a, -2  ; a = 8, Z = 0

push af    ; sp = $FFFC

add a, -8  ; a = 0, Z = 1 as a became zero

pop af     ; a = 8, Z = 0, sp = $FFFE
```

Listing 2-27: Using push and pop

Right after the `pop`, `af` gets back its value from before the `push`. The `push` operation will basically save the value of a register, while `pop` will restore it. The `push` and `pop` operations work in tandem, and unless you are doing a sophisticated stack manipulation, there should be a `pop` for every `push` in your code.

Note that stack overflows and stack underflows are not reported by the DMG. It is the responsibility of the programmer to make sure they do not occur. For example, let's assume the stack is in HRAM. Too many `push`es would start overwriting data at the bottom of the HRAM. Too many `pop`s would wrap up to the beginning of the ROM, which is not writable. Both behaviors will very likely create critical bugs, so they must be avoided.

Besides `push` and `pop`, there are four other instructions that can affect the `sp` register directly: `add`, `dec`, `inc`, and `ld`. They can be used for various custom stack manipulations.

For illustrative purposes only, Listing 2-28 shows code snippets that push `b`, `c`, and `d` onto the stack without using `push`.

```
; assuming sp = $FFFE

add sp, -3    ; reserve 3 bytes on the stack (i.e., decrement sp by 3)

; sp = $FFFB

ld hl, sp+0   ; position hl to point to the first byte reserved on the stack

; hl = $FFFB

; push b
ld a, b
ld [hli], a   ; [$FFFB] = b

; hl = $FFFC

; push c
ld a, c
ld [hli], a   ; [$FFFC] = c

; hl = $FFFD

; push d
ld [hl], d    ; [$FFFD] = d
```

Listing 2-28: Stack manipulation without using push

First, 3 bytes are reserved on the stack by subtracting 3 from the stack pointer. Then, load operations combined with `hli` are used to fill the reserved space with the values of `b`, `c`, and `d`.

Listing 2-29 shows how to pop the registers back off the stack without using `pop`.

```
; sp = $FFFB

ld hl, sp+0   ; position hl to point to the first byte reserved on the stack

; hl = $FFFB

; pop b
ld a, [hli]
ld b, a

; hl = $FFFC

; pop c
ld a, [hli]
ld c, a
```

```
; hl = $FFFD

; pop d
ld d, [hl]

add sp, 3     ; release 3 bytes from the stack (i.e., increment sp by 3)

; back to sp = $FFFE
```

Listing 2-29: Stack manipulation without using pop

This code performs the exact opposite operations of Listing 2-28, demonstrating that you are basically free to use the stack as you wish. Be careful, though, as there are no safety nets.

Here are some points to remember when working with the stack:

- Every `push` must be matched with a `pop`.
- The `pop`s are performed in reverse order of the `push`es.
- There won't be any stack overflow or underflow checks; your memory will just get corrupted if you `push` or `pop` too many times.
- Any instruction related to functions, like `call` or `ret`, implicitly changes `sp`.
- Manipulating the stack without using `push` and `pop` is an advanced technique.

Miscellaneous

This section covers the remaining instructions, which do not fit into any particular category.

The `ccf` and `scf` instructions affect the carry flag. The `ccf` instruction will invert the carry flag, while `scf` will set it to one. There is no instruction to clear the carry flag to zero, but you can use another instruction that has this effect, like `xor a`.

The `cpl` instruction inverts the bits in the `a` register. For example, `%10110101` would become `%01001010`.

The `ei` and `di` instructions enable and disable interrupts, respectively. They modify a DMG internal flag, the *interrupt master enable (IME)* flag, which is accessible only through them. We'll learn more about interrupts starting in Chapter 7.

The `nop` instruction is a no-operation. It basically tells the CPU to do nothing for one cycle. It comes in handy when you need to control the timing of other instructions precisely.

The `daa` instruction converts `a` into BCD format. That format is not used in this book, but Listing 2-30 shows an example of how this instruction might be used for reference.

```
ld a, $19  ; a = $19, valid BCD
add a, 1   ; a = $1A, invalid BCD
daa        ; a = $20, valid BCD
```

Listing 2-30: An example of daa

BCD allows only values from 0 to 9 in each nibble of an 8-bit number. Here, we load `a` with `$19` (hexadecimal), but it truly represents the decimal number 19. Then, we add 1 to `a`. The natural result is `$1A`, but that's not a valid BCD number. What we need is `$20`, which stands for the decimal number 20 in BCD. So, we use `daa` after the addition to adjust `a` back to BCD.

The last two instructions in this group are `halt` and `stop`. The `halt` instruction puts the DMG in *low power mode*, which halts the CPU until an interrupt occurs. This helps save some battery life, and it's also handy to handle graphics-related pacing issues. I'll cover this in Chapter 7.

The `stop` instruction puts the DMG in *very low power mode*, which stops the CPU until a joypad interrupt (a button press or reset) occurs. In theory, it provides a way to reduce power consumption to a minimum in the event that the DMG user forgets to turn the power off. However, if not used properly, it can break the LCD. I'm not aware of any commercial games that use it, and as its usefulness is debatable, I don't recommend that you use it either.

Beyond Instructions

While technically any program could be written only with statements (see "Assembly Language Syntax" on page 16), the reality is that it would be very inconvenient. So, assembly language contains lots of additional features to make the programmer's life easier.

This chapter covers the most important of those features: sections, local labels, macros, and constants. Other features will be explained as we encounter them in the book.

Note that you can find a complete description of the DMG assembly language in the RGBDS manual at *https://rgbds.gbdev.io/docs/master*. It's good practice to revisit the manual from time to time as you get used to programming the DMG. You will most likely find new tricks every time.

Sections

Sections provide a way to place code and assets inside the ROM. There should be at least one section in any program. For example, we could create a section that starts at address `$1000` in the ROM. If we were to put 256 bytes' (`$100` bytes in hexadecimal) worth of code in that section, the code would span the ROM area `$1000–$10FF`.

Listing 2-31 shows how basic sections are defined.

```
section "test", rom0[$1000]
    ld b, $0F
    add a, b
    ldh a, [$FF80]

; The code below will be automatically placed into a suitable area of the ROM.
section "another test", rom0
    ld hl, $FF81
    ld [hli], $12
    ld [hli], $34
```

Listing 2-31: Storing code in a defined section

A section definition starts with the keyword `section`, followed by a section name between double quotes. Then, the type of the section is given. Here, the type is `rom0`, which means the section will be placed in bank 0 of the ROM. A *bank* is a chunk of the ROM. We'll explore this concept in Chapter 13; for now, let's assume the ROM consists of a single bank, which is automatically assigned the number 0.

Finally, there is an optional address specified between square brackets (here, `[$1000]`). If an address is present, then the code will be placed at that address. If no address is given, then the toolchain will automatically place the code at a suitable place for us.

Local Labels

The labels discussed earlier in this chapter were *global labels*, each of which must be unique. Under each global label, you can define *local labels*. The name of a local label must start with a dot (`.`), and the colon (`:`) at the end is optional. The big advantage of local labels is that they need to be unique only within their global label scope.

Listing 2-32 shows the two types of labels.

```
global_label:
    .local_label
        ld a, 10

another_global_label:
    .local_label
        ld a, 11
```

Listing 2-32: Global and local labels

Here, the name `.local_label` is used twice, under two different global labels.

To make this a bit more concrete, consider the example in Listing 2-33.

```
init_with_0:
    ld a, 0
    ld c, 16
    .loop
        ld [hli], a
        dec c
        jr .loop
    ret

init_with_1:
    ld a, 1
    ld c, 16
    .loop
        ld [hli], a
        dec c
        jr .loop
    ret
```

Listing 2-33: Using local labels within functions

There are two functions here that each contain a loop. Thanks to local labels, it is not necessary to come up with unique names for the loop labels; the generic name `.loop` can be reused.

Macros

Macros are sequences of instructions that you can reuse easily inside a program. Let's see how to leverage them through some examples.

The `ld` instruction lets you copy an 8-bit register into another 8-bit register. There is no 16-bit equivalent, so you'll need to use two 8-bit copies. For example, the code in Listing 2-34 copies `de` into `hl`.

```
ld h, d
ld l, e
```

Listing 2-34: Loading de into hl

Copying 16-bit registers is a common operation in assembly, and a macro is quite handy for this. Listing 2-35 defines a macro for the operation in Listing 2-34.

```
macro ld_hl_de
    ld h, d
    ld l, e
endm
```

Listing 2-35: Defining a macro to load de into hl

A macro definition starts with the keyword `macro`, followed by the macro name. After this are the instructions that form the body of the macro, and the definition ends with the keyword `endm`. In this example, the name is `ld_hl_de` and the instructions are exactly the same as in Listing 2-34.

The macro name can then take the place of any instruction. Listing 2-36 shows how to use the macro defined in Listing 2-35.

```
ld_hl_de
```

Listing 2-36: Using a macro

This new macro is great, but it works only on `de` and `hl`, and only in one direction. It cannot use `af` or `bc`, and it cannot copy `hl` into `de`. Fortunately, you can pass parameters to macros to make them more generic. The first parameter can be referenced inside the macro with `\1`, the second parameter with `\2`, and so on up to `\9`. Listing 2-37 shows how to use parameters.

```
; load 16-bit register \2 into 16-bit register \1
macro ld_r16_r16
    ld high(\1), high(\2)
    ld low(\1), low(\2)
endm
```

Listing 2-37: Defining a 16-bit load macro with parameters

In this code, `high` means the high byte of a 16-bit register, while `low` means the low byte of a 16-bit register. For example, `high(de)` means `d` and `low(de)` means `e`. It works for 16-bit numbers too: for example, `high($DB48)` means `$DB`. Both `high` and `low` are resolved at compilation time. They have no cycle cost, which means you can use them without any impact on performance.

Listing 2-38 shows how to use the generic macro, which is far more flexible than `ld_hl_de`.

```
    ; copy de into hl
    ld_r16_r16 hl, de

    ; copy hl into bc
    ld_r16_r16 bc, hl
```

Listing 2-38: Using the 16-bit load macro

A macro definition (like the one in Listing 2-37) does not consume any ROM space by itself. But each time the macro is used, the instructions in the body of the macro are copied into the ROM and consume bytes. Listing 2-38 would be equivalent to having the following instructions in the ROM:

```
ld h, d
ld l, e

ld b, h
ld c, l
```

This is different from a function, whose definition does consume bytes, but whose usage (`call`) costs only 3 bytes. For a short number of instructions, like in the case of `ld_r16_r16`, a macro is a good fit, because each `ld` instruction is only 1 byte.

However, for a lengthy bunch of instructions that need to be used over and over in the program, a function is most likely a more efficient choice. The `call` and `ret` instructions add to the overall cycle cost, but that cost will be increasingly less relevant as the number of instructions inside the function grows. The byte gain of using a function will be significant too.

Finally, you can have labels (global and local) inside macros, but you should add the `\@` suffix to every label, as shown in Listing 2-39.

```
macro dummy
    ld a, [$1000]

    or a
    jr nz, not_zero\@
        inc a
    not_zero\@:
endm
```

Listing 2-39: Macro labels

Without a `\@` appended to the `not_zero` label, the macro would not be reusable; the same label name would be copied each time the macro is used, which violates the rule that label names must be unique. To avoid label name clashes, use the `\@` suffix to generate a unique label ID each time the macro is used.

Constants

You will inevitably want to use constants in your programs. Thankfully, using the keywords `def` and `equ`, that's just as easy in assembly as in any other language.

Listing 2-40 shows a piece of code that will load the value 5 into the address `$FF80`.

```
def MY_VALUE    equ (3 + 2)
def MY_ADDRESS  equ $FF80

ld a, MY_VALUE
ld [MY_ADDRESS], a
```

Listing 2-40: Using constants

As this example shows, you can use mathematical expressions to define constants, as long they can be evaluated by the toolchain at compile time.

Constants do not consume any ROM space; they are replaced by their value wherever they are used. Constants in assembly are the equivalent of preprocessor constants in C.

Summary

In this chapter, you got your first taste of the DMG assembly language. If you're new to assembly, this might have been a lot to take in, but with the many code listings provided throughout the book you'll have plenty of opportunities to get familiar with the language.

If at any point you're unsure of what a particular instruction does or how to use it, you can refer back to the descriptions and examples in this chapter.

In the next chapter, you'll learn about the most important tools you'll use throughout the book: the RGBDS toolchain, which turns assembly code into DMG programs, and the BGB emulator, which runs DMG programs. You'll also write your very first program.

3

THE GAME BOY TOOLCHAIN

This chapter covers the basics of the RGBDS toolchain and the BGB emulator. You'll start by writing a simple program that demonstrates some of the fundamental material introduced in the previous chapter.

Then, you'll turn that program into a ROM with the RGBDS toolchain, an essential set of tools used for every program in this book. Finally, you'll run that ROM in the BGB emulator, which you can use to test, debug, and inspect your running programs if you're not using the actual hardware.

Writing a Game Boy Program

The first program you'll write includes several concepts introduced in the previous chapter: assembly instructions, sections, and labels. I'll explain each line here, and then we'll inspect the execution in the emulator.

Listing 3-1 shows the program. Follow along by using the *first_program* sample in the book's GitHub repository (*https://github.com/mdagois/gca*).

```
❶ def ROM_HEADER_ADDRESS      equ $0100
  def ROM_MAIN_ADDRESS        equ $0150

❷ section "header", rom0[ROM_HEADER_ADDRESS]
      di
```

```
    jr main

❸ section "main", rom0[ROM_MAIN_ADDRESS]
main:
    ld a, 0
    .loop
        ld [$C000], a
        inc a
        jr .loop
```

Listing 3-1: Our first program

First, we define two constants: the start address of the ROM header and the address of the main section of the code ❶.

Next, we define a section, instructing RGBDS to place all the following code at address `$0100` of ROM bank 0 ❷. The section name is `"header"`, its type is `rom0`, and its address is `$0100` (via the `ROM_HEADER_ADDRESS` constant).

Section names don't matter much code-wise (you can name your sections pretty much anything you like, as long as they're unique), but they help keep the code organized. The name `"header"` is a good fit because the DMG considers that the ROM space between addresses `$0100` and `$014F` is the *cartridge header*. This region of memory contains information about the ROM, like the title of the game, the cartridge type, and the ROM size. You don't need to set any header fields directly in the code, because RGBDS has a handy tool, rgbfix, to do this.

When you're writing code, the most important part of the header is located from address `$0100` to address `$0103`. This small 4-byte region at the very start of the header is the *entry point* where the CPU starts reading the ROM instructions when the DMG is powered on. There's not much we can do in 4 bytes, so the most common approach is to disable the interrupts so that they don't interfere with the program initialization and jump to an address outside the header.

We disable interrupts using the first instruction in this section, `di`. While interrupts should be disabled by default, it doesn't hurt to be explicit. The second instruction, `jr main`, performs the jump. The `main` label is defined later in this program. The `di` instruction is 1 byte (located at `$0100`) and the `jr` instruction is 2 bytes (located at `$0101` and `$0102`), so 3 out of 4 bytes of the entry point are filled already.

NOTE *The `main` label is located at `$0150`, which is 77 bytes after the jump instruction. That's within the range of the `jr` instruction, which, as mentioned in Chapter 2, is both smaller and faster than `jp`. We save a byte by using `jr` instead.*

Right after the `"header"` section, we define another section, `"main"`, starting at address `$0150` (via the `ROM_MAIN_ADDRESS` constant) ❹. Again, you can call this anything you want, but `"main"` is a good choice because this section has a similar role to the `main` function in C. The section address is right after the cartridge header, which ends at `$014F`. The ROM region between addresses `$0150` and `$7FFF` has no special purpose, so you can put whatever you like

there. In future programs, you'll write more instructions and even add assets (for example, graphics) to that region.

The `main` label at the very start of the `"main"` section is what the jump in the entry point uses. The next block of instructions gives us something to observe when running the program in BGB. First, we initialize `a` to zero using `ld a, 0`. Then, an infinite loop starts. Each loop iteration writes `a` to the first address of the WRAM (`$C000`) and then increments `a` with `inc a`. Finally, `jr .loop` jumps back to the start of the loop. Instructions in the loop repeat over and over until the DMG is powered off. This is a very simplistic example, but all games (and all the samples in this book) will include a loop similar to this one.

Building a ROM

Now that we've walked through the program, let's turn the source code into a ROM using the RGBDS toolchain. There are three steps to the conversion process: assembly, linkage, and fixing. This section covers what each step does, the necessary commands, and the parameters.

Assembly

The *assembly* step turns a source code file into an object file containing machine code for the DMG CPU. The tool responsible for this step is *rgbasm*.

Here's the assembly command. This assumes that the code from Listing 3-1 is in the *main.rgbasm* file:

```
rgbasm -Werror -Weverything -o main.o main.rgbasm
```

We run the assembler with the `rgbasm` command. The `-Werror` option turns all warnings into errors. I highly recommend using this, because ignoring warnings will eventually backfire.

Next, `-Weverything` enables all warnings. While this is completely optional, taking warnings into account dramatically improves code health and makes programs more robust. However, some warnings may incorrectly indicate errors in valid code, generating false positives. For personal projects, you can use `-Wall -Wextra` instead, which covers the most important warnings while dramatically lowering the odds of false positives.

Next on the command line, `-o main.o` specifies the output object file. It's common practice to keep the filename of the source file and switch the extension.

Finally, `main.rgbasm` is the input source file, which contains the code introduced in the previous section.

This command creates the *main.o* object file, which contains the machine code corresponding to the assembly in the *main.rgbasm* source file. The object file will be fed into the linkage command.

Linkage

The *linkage* step takes one or more object files produced during the assembly step and turns them into a single ROM. We have only one object file for now, *main.o*, so you can use the following command for the linkage step:

```
rgblink --dmg --tiny -o sample.gb main.o
```

We run the linker with the `rgblink` command. The `--dmg` option disables VRAM and WRAM *bank switching*, a technique to increase the amount of memory available. These types of bank switching are available only on the CGB, which I'll cover in Chapter 13.

Next, `--tiny` disables ROM bank switching. This type of bank switching is available on the DMG, but it's only necessary when the ROM size is over 32KiB. Most of the programs in this book won't require it.

The next part of the command line, `-o sample.gb`, specifies the output ROM file. The extension is *.gb* because that's what emulators usually expect. Finally, `main.o` is the input object file that the assembly step created.

When this command completes, it creates a ROM file called *sample.gb*, which will be fed into the fix command.

Fixing

The *fix* step is the final step of the toolchain. It puts proper values inside the header and makes sure the ROM is valid, which means it will pass the DMG checks.

The following is the command for the fix step:

```
rgbfix --title game --pad-value 0 --validate sample.gb
```

We run the fix tool with the `rgbfix` command. Then, `--title game` sets the title of the game, which should be kept under 11 characters because there are only 11 bytes reserved for the title in the cartridge header.

The `--pad-value 0` option tells the tool to pad the unused parts of the ROM with zeros. While not strictly necessary, this simplifies debugging the ROM by making the unused ROM regions easier to spot.

Then, `--validate` fixes the Nintendo logo and the checksums in the header. The DMG will check these before running the ROM and won't execute any instructions if any of the checks fail.

Finally, `sample.gb` is the input ROM file to fix, created during the linkage step. It is modified in place (that is, overwritten) during the fix step, becoming a valid ROM that we can load into any emulator or even run on real hardware.

From now on, this book will refer to the end-to-end process of assembling the source files, linking the object files, and fixing the ROM as *compilation*.

Running a ROM

To run your first ROM in the BGB emulator, launch BGB, which should display the screen in Figure 3-1.



Figure 3-1: Launching the BGB emulator

Right-click anywhere in the BGB window to open the context menu, shown in Figure 3-2.



Figure 3-2: The BGB context menu

Choose **Load ROM** and load the ROM produced during compilation. This will display the Nintendo logo and nothing else. The program does run in the background, but because it does not draw any graphics or produce any sound, its effects are not directly visible. You'll need to inspect the program execution in the debugger to see what's going on.

You can also run the ROM in BGB from the command line. Assuming that BGB is in your `PATH` and that *sample.gb* is in the current folder, enter the following command to run the ROM:

```
bgb --rom sample.gb
```

For the 64-bit version of BGB, the executable might be named *bgb64*.

Using the Debugger

At any time during the execution of a program in BGB, you can press the ESC key to launch the debugger. This pauses the execution of the program and displays a screen similar to Figure 3-3.

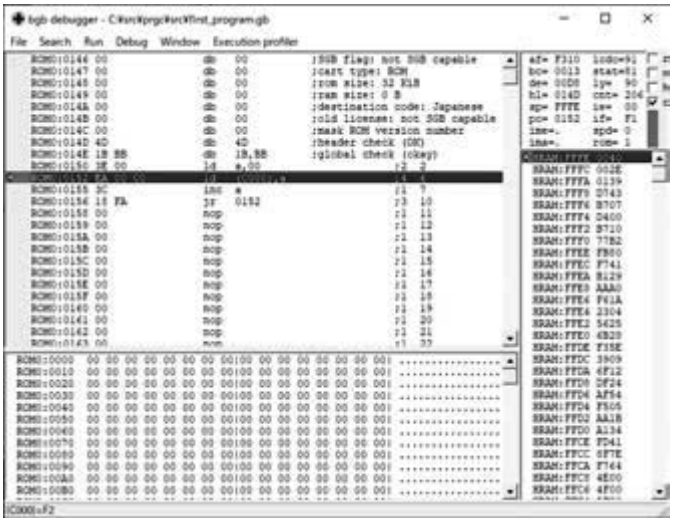


Figure 3-3: The BGB debugger

There are four views in the debugger: the code view, register view, data view, and stack view. The code, data, and stack views are different representations of the memory map. The register view is a quick and handy way to check the values of the CPU registers, status flags, and some important I/O registers. Let's take a closer look at what you'll find in each view; then I'll share a few shortcuts and tips that will be helpful when you're debugging your programs.

Code View

The *code view* presents the memory map as a bunch of instructions, as shown in Figure 3-4.

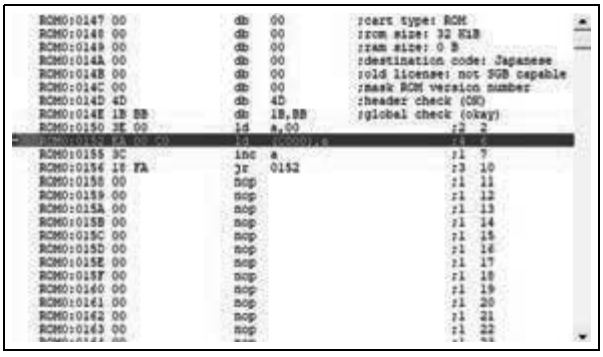


Figure 3-4: The code view

This view allows you to step into the code and place breakpoints. This is where programmers typically spend most of their time when debugging.

Each line in this view has the following format:

```
memory:address bytes disassembly [;comment] ;cycles cumulative cycles
```

The only exception is the header, which has a different layout to make its values easier to read. Otherwise, all the lines are formatted in the same way, with only the *[;comment]* being optional.

Here's an example from Listing 3-1:

```
ROM0:0152 EA 00 C0      ld (C000),a             ;4 6
```

The `ROM0` indicates the memory type. Examples of other memory types are WRA0, I/O, and HRAM. Memory types that end with a number can have several banks. As far as the DMG is concerned, banks are available only for ROM and ERAM, but BGB supports more than just the DMG, so WRAM and VRAM also end with numbers.

The address is `0152`, which is the address inside the memory map between `$0000` and `$FFFF`.

The bytes are `EA 00 C0`, which represent the line's instruction opcode and the parameters that go along with it. `EA` is the opcode for loading `a` into an address (see Appendix B), and `00 C0` is how the 16-bit address `$C000` is stored in the ROM. The low byte (`00`) is at the lower address, `$0153`, and the high byte (`C0`) is at the higher address, `$0154`. Systems using this common way to represent multibyte integers are called *little-endian* systems.

The disassembly is `ld (C000),a`. BGB reconstructs a piece of assembly from the binary opcode and parameters in the ROM. By default, the format is a bit different from what we use in assembly (`ld [$C000], a`), but I'll share a tip later for making it look more similar.

There is no comment in this example. Comments are usually present only for specific registers. For example, if an instruction uses an I/O register, BGB will add the register name as a comment. This is helpful when debugging.

The cycles count is `4`, which represents the cycle cost of the instruction on the line; this indicates that the `ld (C000),a` instruction takes 4 cycles to execute. Be aware that for instructions that have a variable cost, like conditional jumps whose cost depends on whether the jump occurs or not, BGB will display only the lower cost.

The cumulative cycles count is `6`, which represents the total cost of the instructions to that point from the top of the code view. That's pretty handy to evaluate the cost of a piece of code.

Because each line in the code view corresponds to an instruction, not all addresses are shown. For example, in Figure 3-3, the instruction at address `ROM0:0150` is `ld a, 00`, but because that instruction is 2 bytes long, the next line is `ROM0:0152`, not `ROM0:0151`.

Even data (graphics and audio assets) in the ROM will display as instructions in the code view, so to inspect data you'll want to use the data view instead.

Register View

The *register view* (Figure 3-5) shows the values of several registers. A handy feature of this view is that you can edit some of these (for example, the status flags) before the execution of the next instruction.

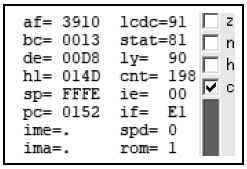


Figure 3-5: The register view

Some of these registers should be familiar, like the CPU registers. The others are I/O registers that I'll cover in later chapters.

Data View

The *data view* (Figure 3-6) allows you to check that memory contains the expected values at the expected addresses. It displays the memory map as a bunch of bytes.

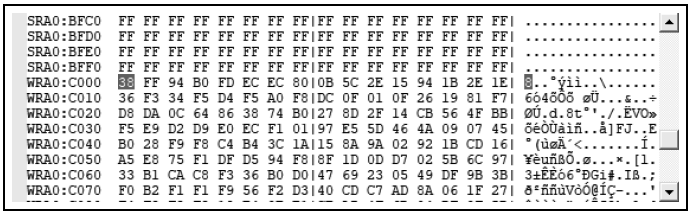


Figure 3-6: The data view

There are 16 bytes per line, so a single screen represents a decent chunk of data.

The data view also lets you modify any byte in the memory, which is handy for testing or reproducing rare scenarios.

Stack View

The *stack view* (Figure 3-7), like the data view, shows the full memory map, but it features several characteristics that make it well adapted to stack inspection.

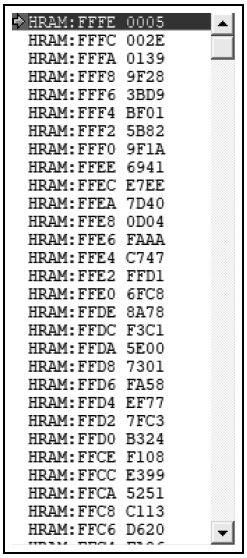


Figure 3-7: The stack view

First, each line shows only 2 bytes of data, which is the number of bytes in a single address. Addresses are the values that are the most commonly pushed onto the stack. Second, the top address is `$FFFF` and the addresses go down from there, which we expect because the DMG stack is a push-down stack. Third, the position of the view resets each time the programmer steps through the code in the debugger, keeping the bottom of the stack visible at all times. Finally, double-clicking a line in the stack view jumps to the address represented by the 2 bytes on that line. This makes it easier to navigate through the stack as `call` instructions fill it with return addresses.

Just like in the data view, you can modify bytes in the stack view. This comes in handy when you need to change values near the bottom of the stack.

Debugging Shortcuts

The most important functionalities, such as placing breakpoints and stepping through the code, are available via the Run and Debug menus. Take a look at these two menus to see what options they offer for debugging running programs.

Right-clicking each view triggers a context menu with handy features, such as ways to modify bytes or go to arbitrary addresses. There are also several shortcuts available in BGB. Table 3-1 shows the ones you'll probably find most useful when getting started.

Table 3-1: Useful Shortcuts

Shortcut	Action	Applicable view	Description
CTRL-G	Go to address	All except register	Makes the specified address visible in the window
CTRL-A	Go to PC	Code	Makes the current pc visible in the window
F2	Toggle breakpoint	Code	Places or removes a breakpoint on the line
F3	Step over	Code	Executes the next line of code but does not jump into functions
F7	Trace	Code	Executes the next line of code and jumps into functions
F8	Step out	Code	Executes code until the current function is over
F9	Run	Code	Executes code until a breakpoint is encountered

Shortcuts can save you a lot of time when debugging, so it's worth making the effort to learn them early. You can learn more about the debugger by reading the BGB manual at *https://bgb.bircd.org/manual.html*.

Debugging Tips

There are a few other options that it's handy to be aware of when debugging. First, in the BGB options (accessible from the Window menu), there is a checkbox named Start in Debugger on the Debug tab. When you enable this checkbox, BGB always starts in debugger mode, which pauses execution in the entry point each time a ROM is loaded instead of running the game right away. You can then press F9 to run the program.

Second, also on the Debug tab, there's a combo box called Disasm Syntax that is set to no$gmb by default. Changing this to rgbds will make the disassembly in the code view look more like the original assembly from the source file.

Finally, you have the option to enable watch mode when launching BGB from the command line interface:

```
bgb --rom sample.gb --watch
```

This mode has BGB continuously *watch* your ROM file (the *.gb* file). Whenever the ROM is recompiled, BGB detects the change and automatically reloads it. This removes the need to manually reload the ROM after compilation, while preserving breakpoints. I highly recommend using watch mode for debugging sessions.

Inspecting a ROM

Now that you know the basics of the debugger, let's use it to inspect a ROM. Load the *sample.gb* ROM you built earlier and open the debugger by pressing the ESC key. Select **Run►Reset** to make sure the execution is in the entry point. Your screen should look like Figure 3-8.

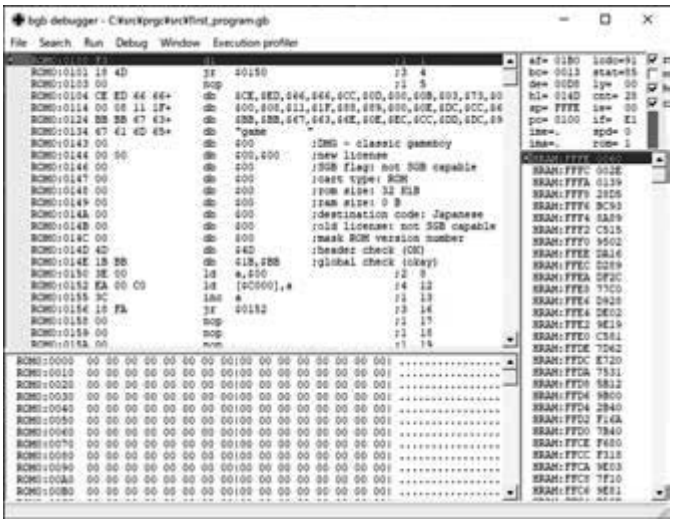


Figure 3-8: Inspecting the ROM in the debugger

Assuming you enabled the Start in Debugger option mentioned in the previous section, the execution will be paused at address `$0100`, right at the start of the entry point. As expected, the `pc` register in the register view is set to `$0100`. The small arrow on the left in the code view indicates the current line. The instruction is `di`, which is the first instruction you wrote in Listing 3-1.

All addresses from `$0000` to `$00FF` will be filled with `nop` in the code view. This is an unused region of the ROM, and we asked the toolchain to fill unused ROM regions with zeros with `--pad-value 0`. Zero happens to be the opcode for `nop`. The same applies for any addresses after our program, from `$0158` to `$7FFF`. The data view shows the actual zeros. Again, these views represent the memory map in different ways, but the underlying data is fundamentally the same. You can scroll (or use the shortcut CTRL-G) to `$0100` in the data view, and you'll see that the byte value is `F3`, which is the opcode for `di`.

Let's go back to the entry point in the code view, shown in Figure 3-9.

```
ROM0:0100 F3          di
ROM0:0101 18 4D       jr   $0150                ;3  4
ROM0:0103 00          nop                       ;1  5
```

Figure 3-9: The entry point

Below the `di` instruction, we have our jump, as expected, and below the jump there is a `nop`.

We filled 3 bytes in the entry point: `F3` for `di` and `18 4D` for `jr $0150`. The last byte was filled with a `nop` because we didn't use it. Notice also the cumulative cycle cost up to the jump. Our entry point will take 4 cycles to execute: 1 cycle for `di` and 3 cycles for `jr`.

Now, step into the code by pressing the F3 key, as shown in Figure 3-10.

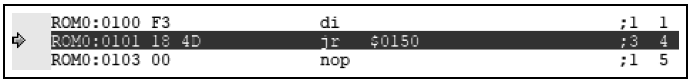


Figure 3-10: Stepping into the code

In this line the disassembly is `jr $0150`, echoing our original code from Listing 3-1, `jr main`, where `main` was a label located at `$0150`.

The two bytes of this line are `18 4D`. The first one, `18`, is the opcode for `jr`, so there's nothing peculiar here. The second one, `4D`, is an offset from the end of the current instruction to `$0150`. The `pc` is `$0101` and the `jr` instruction is 2 bytes, so we have `$0101 + $2 + $4D`, which indeed equals `$0150`. The point is that we never had to calculate this offset ourselves, and BGB won't even surface it in the disassembly. The toolchain and the emulator hide a lot of the complexity from us.

Pressing **F3** again should bring up Figure 3-11.

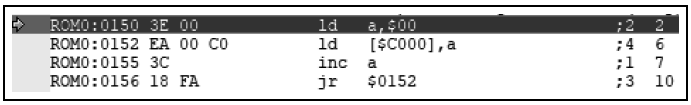


Figure 3-11: Stepping into "main"

We are now at the start of the `"main"` section, right where the `main` label is located. The `main` function initializes `a` to zero and then enters an infinite loop in which each iteration increments `a` and stores the value in WRAM (see Listing 3-1 if you need a refresher).

Now we'll use a breakpoint, to see how they work in BGB. Click and select the last line of the main loop, `jr $0152`, and press **F2**. The whole line should turn red to indicate the breakpoint. You can remove the breakpoint by pressing F2 again, but let's keep it for now. Your screen should look like Figure 3-12.

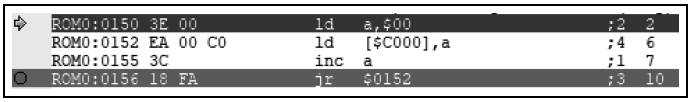


Figure 3-12: Setting a breakpoint

Next, in the data view, go to `$C000` (on line `WRA0:C000`). The first byte on the line is where our code stores the value of `a`. In Figure 3-13, `$C000` holds `B8`, but because WRAM is not initialized by default, it could be any value.

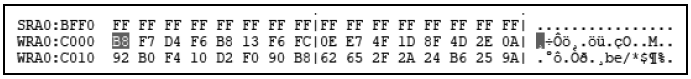


Figure 3-13: Inspecting the memory at the breakpoint

Finally, press **F9** to run the code up to the breakpoint. The arrow, visible at the top of Figure 3-12, will now be on the same line as the breakpoint, and the value at `$C000` will have changed to `$00`. Press **F9** a few more times, and you'll see the value at `$C000` get incremented each time.

This also updates `a` in the register view. Let's see how to modify the value of `a` directly in that view. Right-click the `af` register and choose **Edit Register**, as shown in Figure 3-14.

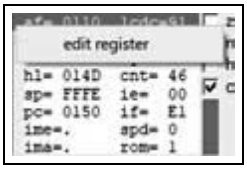


Figure 3-14: Editing a register

This should open a window where you can enter up to four hexadecimal digits to override the value of `af`. Input any two-digit hexadecimal number, such as `A5` or `DC`, to override `a` only, and press **F9** again. Now, the value you put in `a` is at `$C000`.

Editing is not limited to registers. You can also edit flags in the register view and modify bytes in the data view. You can even edit the code in the code view. For example, on line `ROM0:0156`, you could turn the jump bytes `18 FA` into `18 F8` to include the first instruction of the `main` function (`ld a, 0`) in the loop. That would set `a` back to zero for each iteration of the loop.

You now know enough debugging basics to go through the rest of this book. You can step into the code, place breakpoints, and edit values. Continue exploring the menus to discover more BGB functionality.

ROM Memory Map

In Table 1-1 you got an overview of the whole memory map, from `$0000` to `$FFFF`. The ROM spans half of that, from `$0000` to `$7FFF`. Table 3-2 shows a more granular map of the ROM.

Table 3-2: Focusing In on the ROM Section of the Memory Map

Start	End	Size (bytes)	Purpose
$0000	$0039	64	Fast function calls (through `rst`)
$0040	$0067	40	Interrupts
$0068	$00FF	152	Any purpose
$0100	$0103	4	Entry point
$0104	$0133	48	Nintendo logo
$0134	$013E	11	Title
$013F	$0142	4	Manufacturer code
$0143	$0143	1	CGB flag
$0144	$0145	2	New licensee code
$0146	$0146	1	SGB flag
$0147	$0147	1	Cartridge type
$0148	$0148	1	ROM size
$0149	$0149	1	RAM size
$014A	$014A	1	Destination code
$014B	$014B	1	Old licensee code
$014C	$014C	1	Mask ROM version number
$014D	$014D	1	Header checksum
$014E	$014F	2	Global checksum
$0150	$7FFF	32,432	Any purpose

The total size of the ROM is 32,768 bytes, and most of it is free to use for any purpose (for example, for instructions and assets). The only other important memory spaces to be aware of in the ROM are the address ranges reserved for the entry point, interrupts, and fast function calls. You've already learned how to use the entry point, and I'll cover interrupts when we start drawing animated graphics in Chapter 7. The space reserved for fast function calls has to do with using the `rst` instruction introduced in Chapter 2. If you never use `rst`, you are free to use the 64 bytes between `$0000` and `$0039` for any purpose, like storing data or instructions.

The remaining header fields are set by the rgbfix tool and are just for your reference.

You might be wondering why the entry point is located at address `$0100`, rather than `$0000`. The reason for this lies in the DMG boot-up sequence. When the DMG is powered on, the CPU starts executing instructions from address `$0000`. But at that moment, the addresses from `$0000` to `$00FF` are not yet mapped to the cartridge ROM. Instead, they are mapped to a 256-byte program called the *boot ROM* inside the CPU itself. The boot ROM's job is to check that the cartridge header is valid, scroll the Nintendo logo down to the middle of the LCD, and finally play that short memorable sound when the logo is done scrolling. The final action of the boot ROM is to disable itself by writing a nonzero value to an I/O register located at `$FF50`. Once the

boot ROM is disabled, the CPU sees the cartridge ROM at the address range `$0000–$00FF` instead of the boot ROM.

Because the last instruction of the boot ROM is at `$00FE` and is 2 bytes long, the CPU pulls its next instruction from address `$00FE` + 2 = `$0100`, after the boot ROM execution is over. That's why the entry point is located at `$0100`. The header is also read from the boot ROM, which is why it is located beyond address `$0100`.

Summary

This chapter introduced you to your very first program. While the program was simple, it helped you understand the necessary steps to turn source code into an executable ROM.

The next chapter presents the template that provides the basis for all the remaining code samples in this book. It also introduces some additional functionalities of the toolchain that come in handy when inspecting and debugging a program.

Game Boy Coding Adventure (Early Access) © 2025 by Maximilien Dagois

4

INTRODUCING THE SAMPLE TEMPLATE

This chapter introduces a simple template that you'll be using for all future code samples in this book. The template splits the assembly code over four files. This organization keeps reusable parts separate from sample-specific ones and makes it easier to find the code associated with the features each sample illustrates.

You'll learn how to compile the source code of the template, and you'll learn a few tricks involving the map and symbol files that you can instruct the linker to generate during compilation. These provide additional information that may come in handy as you inspect and debug the samples. Finally, you'll run the *base* sample, our first sample based on the template, in BGB to verify that everything works as intended.

The Four Files of the Sample Template

In this section, I'll cover the four files that make up the template: *hardware.rgbinc*, *main.rgbasm*, *sample.rgbasm*, and *utils.rgbinc*.

hardware.rgbinc

Many addresses, values, and flags are used to program the DMG, and memorizing all of them is just not practical. The good news is that people in the DMG community have already done the hard work of gathering all these constants into a single file called *hardware.inc*, which is available in the gbdev GitHub repository at *https://github.com/gbdev/hardware.inc*. To keep the extension consistent with the other include files in this book, I've renamed this file to *hardware.rgbinc*.

NOTE *In this book, I use the .rgbasm and .rgbinc extensions for source and include files, respectively. Which extension you use does not matter for compilation; it comes down to personal preference. You can even use the same extension for both source and include files.*

To understand the purpose of the *hardware.rgbinc* file, let's use a snippet of code from Listing 3-1 as an example. At some point in the program, the value of the `a` register is loaded into WRAM using the following code:

```
ld [$C000], a
```

The starting address of the WRAM, `$C000`, is used as is in this code. That's all well and good for a simple address like `$C000`, but imagine if we used another address, like `$D2F3`, instead. That address is still in WRAM, but that's not nearly as obvious as it is when you use `$C000`.

A better solution is to leverage the constant already defined in *hardware.rgbinc* for the WRAM base address, `_RAM`, and use relative positions for any location in WRAM, as shown next:

```
; _RAM equals $C000 in hardware.rgbinc

ld [_RAM], a          ; load (a) into $C000
ld [_RAM + $12F3], a  ; load (a) into $D2F3
```

These two `ld` instructions use the `_RAM` constant to make their intention to store values in WRAM clearer.

Here's a more complicated example using flags and an I/O register:

```
ld a, %10010001
ld [$FF40], a
```

This puts a binary value into `a` and then copies `a` to address `$FF40`. The intent is not obvious at all, but this actually configures the LCD.

Here's an improved version using constants from *hardware.rgbinc*:

```
ld a, LCDCF_ON | LCDCF_BG8000 | LCDCF_BG9800 | LCDCF_BGON
ld [rLCDC], a
```

Although it's longer, this is far more readable than the original version because it replaces the raw values `%10010001` and `$FF40` with flags and the

rLCDC I/O register constant. Even if you don't know the meaning of these constants, you can still intuit that the code is related to the LCD.

main.rgbasm

The second file of the template, *main.rgbasm*, is shown in Listing 4-1. This is a revised version of Listing 3-1.

```
include "hardware.rgbinc"

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

section "header", rom0[$0100]
entrypoint:
    di
    jp main
    ds ($0150 - @), 0

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

section "main", rom0
main:
    call InitSample
    .loop
        call UpdateSample
        jr .loop
```

Listing 4-1: main.rgbasm

The code begins by including the hardware definitions from the previous section. The `include` directive inserts the content of the specified file (here, *hardware.rgbinc*) into the current file.

I've made a few modifications in the header section. First, I removed the constant for the address `$0100`, because it was used only once and isn't really necessary (the header is, by definition, at `$0100`, after all).

Second, I added an `entrypoint` label at the start of the header to reinforce the fact that the following instructions are in the entry point. This is purely informational.

Next, I replaced the relative jump (`jr`) instruction with an absolute jump (`jp`) instruction, because from now on we'll let the linker place the main section in the ROM. The main section used to be at `$0150`, which is in reach of a relative jump from the entry point. But when we let the linker place a section, that section could end up anywhere in the ROM, including outside the reach of a relative jump, which would cause a compilation error. Switching to an absolute jump removes this risk. The `jp` instruction takes one more byte than the `jr` instruction, so now all the bytes in the entry point are used: 1 byte for `di` and 3 bytes for `jp`, for a total of 4 bytes.

Finally, the remainder of the header section gets filled with zeros. The header section must be exactly 80 bytes long, from `$0100` to `$014F`. If it isn't

filled properly, the linker could decide to place code right after the entry point. That code would then be overwritten by rgbfix when it sets the header values, which is clearly something we want to avoid.

To fill in the rest of the header, we use `ds ($0150 - @), 0`. This is not an instruction, but a functionality offered by the RGBDS toolchain's assembler. The `ds` stands for *data store*, and it takes two parameters: the number of times to store a given value and the value to store. For example, if we had `ds 12, $4B`, the assembler would store the value `$4B` in the ROM 12 times.

The formula `$0150 - @` computes the number of times that the value 0 must be stored to fill the header exactly. The address `$0150` is located right after the header, and `@` means the current address (`$0104`, here). So, 76 zeros will be stored. Adding the 4 bytes of instructions in the entry point, that will make the header section 80 bytes long, as desired.

I've also made some changes to the main section. To start, it's not forcibly placed at `$0150` anymore; I've removed the constant for that address, and no address is now specified. Unlike the header section, which must be located at `$0100`, the main section can be located anywhere in the ROM. As long as the jump instruction in the entry point uses the `main` label (and not a hardcoded address), it's safe to let the linker decide where to put it.

Most importantly, the main section now consists of two function calls. The `InitSample` function performs the necessary initialization of the sample, such as clearing memory or setting up graphics parameters. It is called only once, at the start of the program. The `UpdateSample` function updates the sample by, for example, getting the d-pad input, moving graphics around, or playing sound. It is called once for each main loop iteration until the DMG is powered off.

These two functions are defined in the next file, *sample.rgbasm*.

sample.rgbasm

The template aims at separating reusable code from sample-specific code. Because the `InitSample` and `UpdateSample` functions are sample-specific, they are defined in the third file of the template, *sample.rgbasm*, shown in Listing 4-2.

```
include "utils.rgbinc"

section "sample", romO

InitSample:
    ret

UpdateSample:
    ret

export InitSample, UpdateSample
```

Listing 4-2: sample.rgbasm

We start by including the *utils.rgbinc* file, which I'll discuss in the next section. Next, we define the section containing the initialization and update functions. Like for the main section in *main.rgbasm*, no address is specified for the sample section; we let the linker decide where to put it in the ROM.

Next, we have the two function definitions, `InitSample` and `UpdateSample`, which were referenced in *main.rgbasm*. They are empty for now, because this is just a sample template, not an actual sample yet.

Finally, we export the two function labels. This is an important step because exporting instructs the linker to make the labels available to other files. In this case, the labels become accessible from *main.rgbasm*. If you don't export the labels, using them in *main.rgbasm* will generate an error.

You can export any label; it doesn't have to be related to a function. Macros, however, are not exportable. To share macros, they must be part of an included file.

There are two ways to export a label: you can use the `export` keyword, as in the template, or you can use two colons after the label name instead of a single colon. For example, in the case of the `InitSample` and `UpdateSample` functions, it would look like this:

```
InitSample::
    ret

UpdateSample::
    ret
```

Spotting what's exported in a file is a bit harder with this approach (using `export` is more explicit), but you can use whichever technique you prefer.

utils.rgbinc

The last template file is *utils.rgbinc*, which contains macros and constants you can use in all samples. Listing 4-3 shows the contents of the file.

```
if !def(UTILS_RGBINC)
def UTILS_RGBINC equ 1

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

include "hardware.rgbinc"

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

endc
```

Listing 4-3: utils.rgbinc

It's pretty empty right now, but we'll add code that is generic enough to reuse in many samples (for instance, you might want to add the 16-bit load

macro in Listing 2-37). For now, there is only a single `include` directive for the hardware definitions, which we'll use in all the samples.

The first two lines and the last line are are *include guards*, used to avoid compilation errors that might come from including the file's contents more than once. For example, this could happen if two *.rgbinc* files include the same *.rgbinc* file, and a *.rgbasm* file includes both of those *.rgbinc* files. In *utils.rgbinc*, we first specify that if the constant `UTILS_RGBINC` is defined, the whole file should be ignored. We then define the constant, so that any future inclusions are ignored. It's good practice to use include guards in any file you include. The hardware definitions file, *hardware.rgbinc*, also uses them.

Compiling the Template

Let's move on to compiling the source code of the template to produce a ROM we could run in an emulator. We have an additional source file to handle this time, *sample.rgbasm*, compared to the situation in Chapter 3, where we had only *main.rgbasm*. Assuming all the files are in the same folder, you can use the following commands to create a ROM:

```
> rgbasm -Werror -Weverything -o main.o main.rgbasm
> rgbasm -Werror -Weverything -o sample.o sample.rgbasm
> rgblink --dmg --tiny --map sample.map --sym sample.sym -o base.gb main.o sample.o
> rgbfix --title game --pad-value 0 --validate base.gb
```

First, we turn the two source files, *main.rgbasm* and *sample.rgbasm*, into object files. You can have as many source files as needed; just include a command of the same format for each of them.

Then, the linker takes all the object files at once and joins them into a ROM. There are two new options here, `--map sample.map` and `--sym sample.sym`, which tell the linker to generate a map file and a symbol file, respectively. We'll look at these files momentarily.

Finally, the ROM is fixed; that step is exactly the same as in Chapter 3, because it is independent of the number of source files.

Include files are not part of any compilation command. Their content is temporarily added to the source code during compilation (via the `include` directive), so they are only implicitly compiled as part of the *.rgbasm* files.

The Map File

As you've just seen, the compilation process generates a map file when you include the linker option `--map`. A *map file* lists the addresses of all sections, labels, and local labels in the ROM. It also contains information about the sizes of the sections and the ROM, making it easy to locate and inspect specific sections and to spot which parts of the program consume the most bytes and might require some space optimization.

Listing 4-4 shows the map file generated for the sample template.

```
SUMMARY:
    ROM0: 90 bytes used / 32678 free

ROM0 bank #0:
    SECTION: $0000-$0007 ($0008 bytes) ["main"]
        $0000 = main
        $0003 = main.loop
    SECTION: $0008-$0009 ($0002 bytes) ["sample"]
        $0008 = InitSample
        $0009 = UpdateSample
    EMPTY: $000a-$00ff ($00f6 bytes)
    SECTION: $0100-$014f ($0050 bytes) ["header"]
        $0100 = entrypoint
    EMPTY: $0150-$7fff ($7eb0 bytes)
    TOTAL EMPTY: $7fa6 bytes
```

Listing 4-4: The template's map file

The `SUMMARY` segment at the top shows the used and free space in the ROM. The *used space* is where the instructions and resources are located in the ROM. The *free space* is the empty part of the ROM. As we add code and resources to the ROM, the used space gradually increases, and the free space decreases by the same amount. Here, you can see that the template uses 90 bytes of space and there are still 32,678 bytes of free space left.

The rest of the file shows the locations of the three sections defined in the source files, main, sample, and header, and which areas of the ROM are empty.

The linker has put the main section at address `$0000`. It contains one label, `main`, and one local label, `main.loop`. The whole section is 8 bytes long.

The sample section is very short (just 2 bytes) because there is barely any code in the `InitSample` and `UpdateSample` functions yet.

The header section is in the expected `$0100–$014F` range, with a size of 80 bytes (`$0050` in hexadecimal).

There are two regions of empty space, from `$000a` to `$00ff$` (before the header) and from `$0150` to `$7fff` (after the header). The final piece of information in the map file is the total amount of empty space (`$7fa6`). This is the same information as in the `SUMMARY` segment (32,678 bytes of free space), but expressed in hexadecimal.

The Symbol File

Along with a map file, you can instruct the linker to generate a symbol file by providing the option `--sym`. This is another handy inspection tool you can use to debug your programs. A *symbol file* contains all the label addresses in each bank of the ROM in an easily parsable format. A line in a symbol file has the following format:

```
bank:address label
```

Listing 4-5 shows the symbol file generated for the sample template.

```
; File generated by rgblink
00:0000 main
00:0003 main.loop
00:0008 InitSample
00:0009 UpdateSample
00:0100 entrypoint
```

Listing 4-5: The template's symbol file

Here, the `InitSample` label refers to address `$0008` in bank `00`.

While the symbol file may not appear to contain any information that isn't in the map file, its strength lies in how emulators leverage it. Figure 4-1 shows the tremendous benefits compiling a ROM with a symbol file brings to BGB.

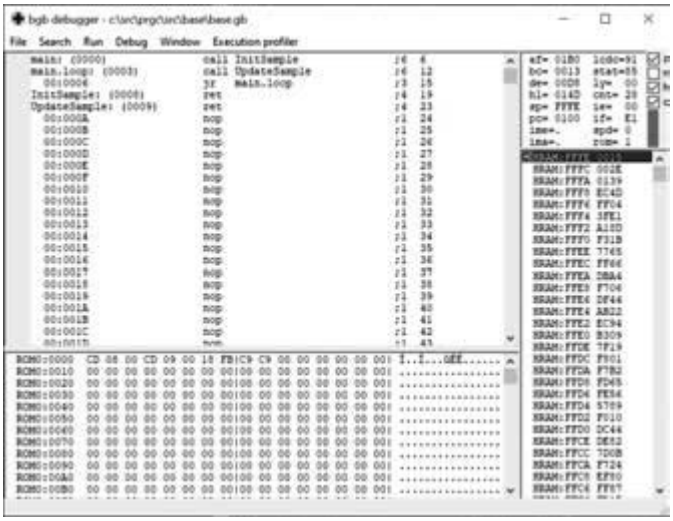


Figure 4-1: Debugging in BGB with symbols

Take a look at the code view. By using symbols instead of raw addresses, we get the associated label names for lines that have one. Label names also make function calls and jumps more descriptive. Overall, the code is far easier to comprehend.

For comparison, Figure 4-2 shows what we would have without symbols.

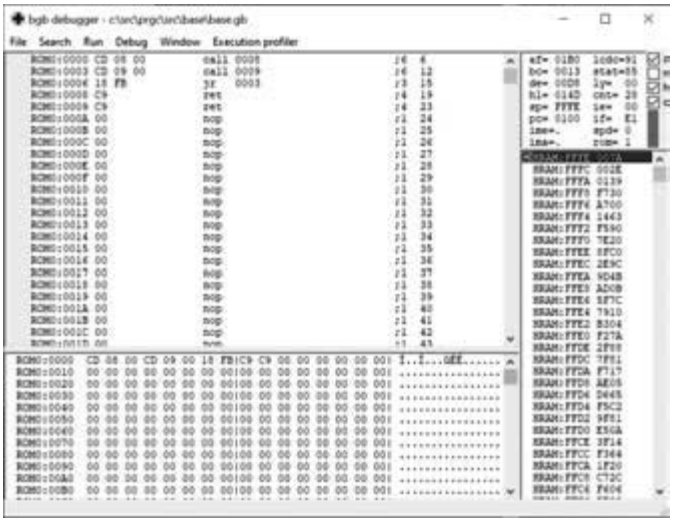


Figure 4-2: Debugging in BGB without symbols

The first line of the code view in Figure 4-2 contains mostly numbers that don't make much sense without inspecting the map file carefully. In contrast, on the first line in Figure 4-1, we can identify a label and a function name from the template code right away. The same applies to the following lines.

The only benefit of not using symbols is that we can see the machine code for each instruction in the code view. For example, in Figure 4-2, we can see that the machine code for the `jr` instruction is `18 FB`. In practice, however, this information is not very useful, and it's available in the data view anyway. At the end of the day, there is very little incentive for not using symbols.

BGB loads the symbol file automatically as long as this file has the same name as the ROM file and is placed in the same folder. Only the extension should differ. For example, if a ROM is named *test.gb*, then the symbol file should be named *test.sym*.

Running the Template

Now that we have a ROM, let's run and inspect it in BGB. We'll check the entry point of the ROM first, shown in Figure 4-3.

```
entrypoint: (0100)          di                          :1  12
  00:0101                   jp   main                   :4  16
```

Figure 4-3: Inspecting the entry point

No surprises here: The `jr` instruction has become a `jp`, reflecting our code change at the start of this chapter. Notice the presence of the labels, `entrypoint` and `main`, thanks to the symbol file.

Using the map file, we can find the address of any label. For example, Listing 4-4 tells us that the `main` label is located at `$0000`. (The address is likely the same for you because the template code is relatively short and simple, but be aware that the linker could place the main section anywhere, so make sure you refer to your own map file.) If we go to address `$0000` in the code view, we indeed find the `main` label, as shown in Figure 4-4.

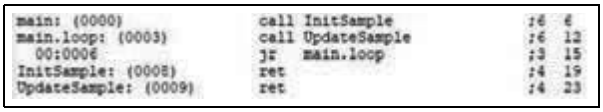


Figure 4-4: Inspecting the sections

The first three lines are the main section with the main loop. The last two lines are the sample section. We find the `InitSample` and `UpdateSample` functions here. Because they are both still empty, we only have single `ret` instructions.

If you're still not sure how all this works, reset the ROM (**Run►Reset**) and try stepping into the code with F7. This way, you can easily follow the flow of execution. Make sure you use F7 (Trace) and not F3 (Step Over), as F3 will not go inside the functions.

Summary

In this chapter, you learned about the basic template on which all the samples in this book are built. You also learned about map and symbol files, which are important inspection tools. Map files let you know where the linker put your code in the ROM, while symbol files enable the emulator to provide you with additional information during debugging sessions.

The next chapter introduces one of the most exciting topics we'll cover: the DMG's graphics capabilities. In most programs, graphics are the main way to communicate with the user. As a result, a lot of the DMG's hardware is dedicated to graphics. Graphics programming is a fascinating and rewarding field, combining both creativity and technical skills.

PART II

ESSENTIAL DMG PROGRAMMING

Game Boy Coding Adventure (Early Access) © 2025 by Maximilien Dagois

5

GRAPHICS OVERVIEW

This chapter introduces graphics on the DMG. We'll begin with the fundamental concepts, like vblank and the frame. You'll also get an overview of how the background, window, and sprites work, before diving into them in detail in Chapter 7. Next, you'll learn about the various structures that hold the graphics data, like tiles and tilemaps, which can display large graphics efficiently.

This chapter is heavy on theory and might be difficult to digest on first read. Don't worry, the content will make more sense once you start writing graphics samples!

The LCD, Vblank, and Frame

The DMG's LCD has a resolution of 160×144 pixels. The pixel processing unit (PPU) constantly redraws all the pixels of the LCD. A redraw is called an *LCD refresh*. It takes 114 cycles for the hardware to process each line of 160 pixels. Because the CPU frequency (the number of cycles per second) is 2^{20} Hz, a cycle is about 954 nanoseconds ($1 \ / \ 2^{20}$). That means it takes about 108.7 microseconds (μs) to process a line and about 15.67 milliseconds (ms) for the whole LCD (144 lines). The PPU processes the lines from the top to the bottom of the LCD and updates the pixels of each line from left to right.

Between two LCD refreshes, there is a period of time called the *vertical blanking interval (vblank)* during which nothing is drawn. The vblank is equivalent to the processing time for 10 lines, which is about 1.08 ms.

The combination of a refresh and a vblank is called a *frame*, which lasts about 16.74 ms. Thus, the number of frames per second, or *frame rate*, is 59.73. The number of cycles per frame is 17,556 ($2^{20} \ / \ 59.73$).

For reference, Listing 5-1 summarizes the frame-related calculations. To keep the numbers easy to read, units may change from one line to another.

```
frequency = 2^20 = 1.048576 MHz
cycle_duration = 1 / frequency = 954 nanoseconds

cycles_per_line = 114
time_per_line = cycles_per_line * cycle_duration = 108.72 microseconds

lines_per_lcd = 144
time_per_lcd = time_per_line * lines_per_lcd = 15.67 milliseconds

lines_per_vblank = 10
time_per_vblank = time_per_line * lines_per_vblank = 1.08 milliseconds

time_per_frame = time_per_lcd + time_per_vblank = 16.74 milliseconds
frames_per_second = 1 / time_per_frame = 59.73
cycles_per_frame = frequency / frames_per_second = 17556
```

Listing 5-1: Frame-related calculations

We'll refer to those numbers from time to time in the book, when the execution timing of our programs becomes important.

Figure 5-1 is a visual representation of various numbers in a single frame.

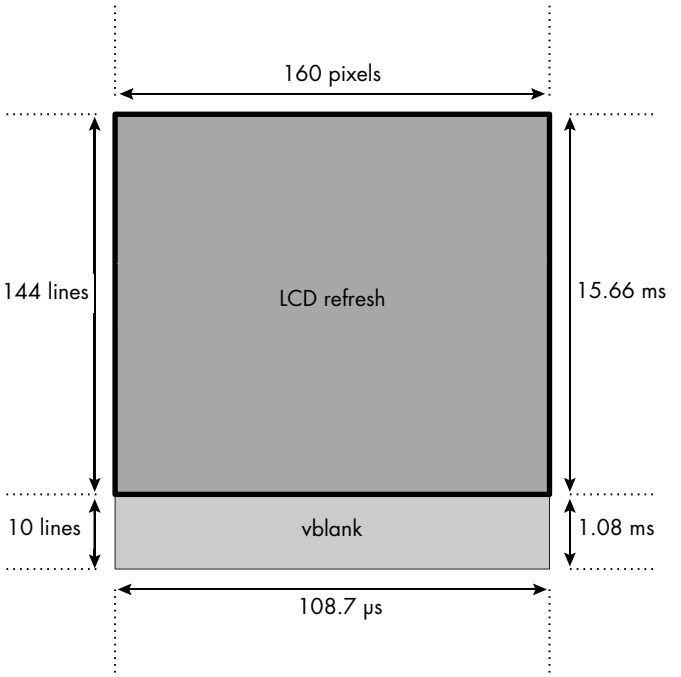


Figure 5-1: Decomposition of a single frame

The frame is a fundamental concept in graphics, because we can perform some tasks only at certain times in the frame. For example, the VRAM, which holds all the graphics data displayed to the LCD, is inaccessible during most of the LCD refresh, while it is freely accessible during the vblank.

High-Level Features

The DMG has three high-level graphics features: the background, window, and sprites. The *background* is a 256×256-pixel square area that can scroll horizontally and vertically. It wraps around in both directions, so it covers the whole LCD at all times. The LCD, as you'll recall, is 160×144 pixels, so it displays only a portion of the background at any time.

The *window* is a 160×144-pixel area drawn over the background. You can move the window around, like a window on a modern operating system, but it's less flexible in where it can be positioned. It does not wrap around horizontally or vertically. We often use the window to display some kind of user interface (UI), but a few games use it in more creative ways, like for creating huge bosses at the bottom of the screen.

The *sprites*, or *objects*, are small graphics elements used to display characters or projectiles that move around the screen independently. Their size is either 8×8 or 8×16. Several options influence how the DMG draws sprites to the LCD.

I'll cover the background, window, and sprites in detail in Chapter 7, but to give you an overview of each element, let's look at a real example. Figure 5-2 shows a simple scene with a background, a window, and two sprites.



Figure 5-2: A simple scene

Figure 5-3 shows what the background looks like in that scene.

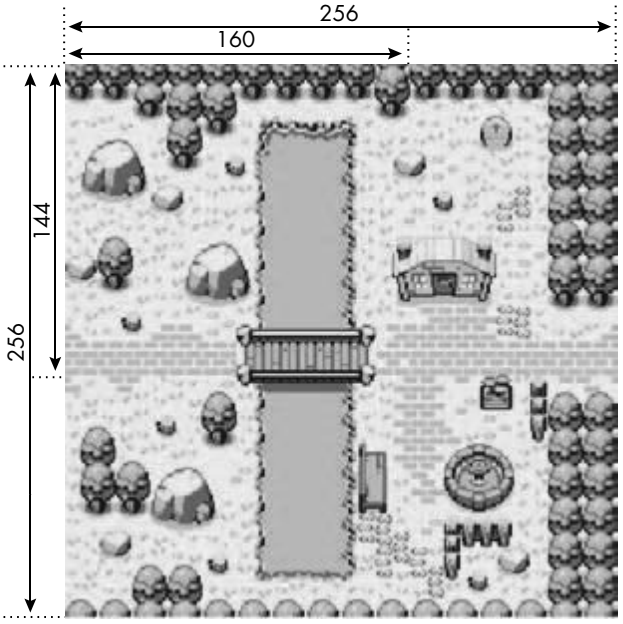


Figure 5-3: The background from the simple scene

This shows the full 256×256-pixel area, but the LCD displays only 160×144 pixels of this.

Figure 5-4 shows a 256×256-pixel area containing the window graphics.



Figure 5-4: The window from the simple scene

The window has a fixed size of 160×144 pixels. Here, it is partially covered by graphics, and only the top part is used in the scene.

Figure 5-5 shows the sprites used in the scene.

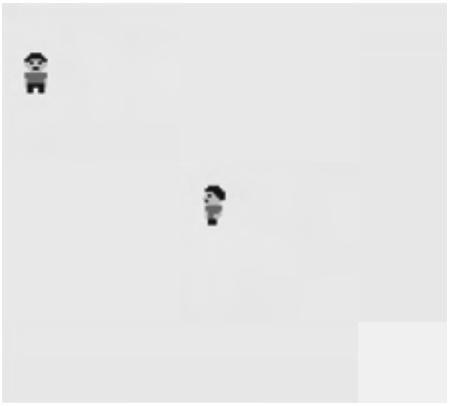


Figure 5-5: The sprites from the simple scene

This scene uses sprites to display characters. Each character is 8×16 pixels and uses one sprite.

Combining all three of these gives you the scene in Figure 5-2.

Elements of Graphics Data

In this section, we'll cover the four elements that make up graphics data: colors, palettes, tiles, and tilemaps.

Colors

In the DMG, the most basic graphics element is *color*. There are only four colors available, and each takes up 2 bits. Table 5-1 lists the colors you can use.

Table 5-1: DMG Colors

Name	Value	Bits
White	0	00
Light gray	1	01
Dark gray	2	10
Black	3	11

The color names are generic, and the actual colors depend on the DMG model. For example, on the original DMG, all colors are shades of green.

Palettes

A *palette* is a group of four colors. Because each color is encoded with 2 bits, a palette is 8 bits, or 1 byte. A palette can contain the same color several times. Figure 5-6 shows a few examples.

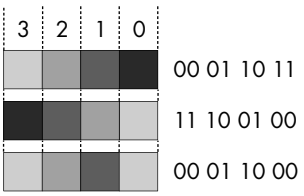


Figure 5-6: Palette examples

The numbers above the palettes are the indices of the colors inside each palette. The numbers on the right are the palette values in binary. The most significant 2 bits are for the color at index 3 in the palette. The least significant 2 bits are for the color at index 0 in the palette. For example, the color at index 3 of the top palette in Figure 5-6 is white (value `00`), while the color at index 1 of the bottom palette is dark gray (value `10`).

Tiles

A *tile* is an 8×8 array of palette indices. Each of the 64 indices is 2 bits (because a palette has four colors), so a tile takes up 128 bits, or 16 bytes.

Figure 5-7 shows the flow of data from the tile back to the colors.

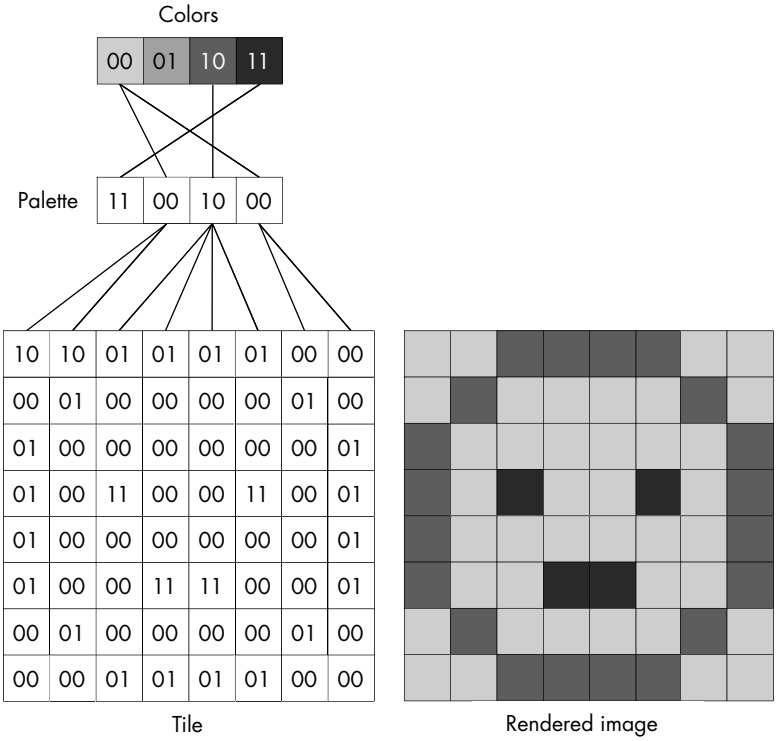


Figure 5-7: The relationship between colors, palette, and tile

The tile is just a collection of indices into the palette, and the palette holds a color at each index. Using the tiles, palettes, and colors, the PPU can determine the right color to render for each pixel of the LCD.

You might wonder why we use an indirection through the palette instead of just storing the colors directly into the tiles. Colors take 2 bits and palette indices, used in the tiles, take 2 bits as well, so the gain is not immediately obvious.

The reason is that you can use palettes for special effects. For example, you could fade out the screen by modifying the palette one color at a time. You could also briefly swap the palette of a character to show, for example, that it's taken damage from a monster. All you need to do for this is change the value of a single palette, which is only a byte.

This kind of effect would be harder to achieve if we stored the colors in the tiles; it would require editing the tile data directly, and a tile is much larger than a palette.

For reference, Figure 5-8 shows the tiles that were in VRAM during the simple scene in the previous section (see Figure 5-2). This screenshot comes from BGB's VRAM viewer, accessible through the Window menu of the debugger.

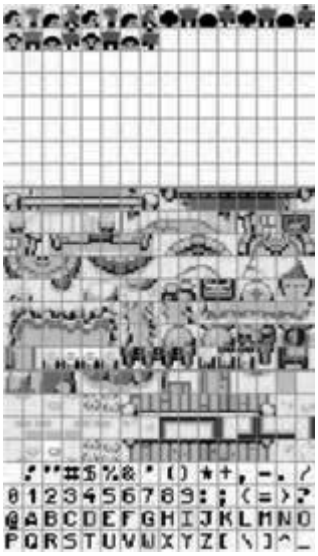


Figure 5-8: Tiles from the simple scene

The data inside a tile has to be organized in a specific way. As you just learned, a tile is an 8×8 array of 2-bit indices, for a total size of 16 bytes. A tile is encoded one row at a time, and each row takes 2 bytes. The first byte contains the least significant bit of each of the eight indices in that row, and the second byte contains the most significant bit of those same indices.

Figure 5-9 shows how this works for the fourth row of the image in Figure 5-7.

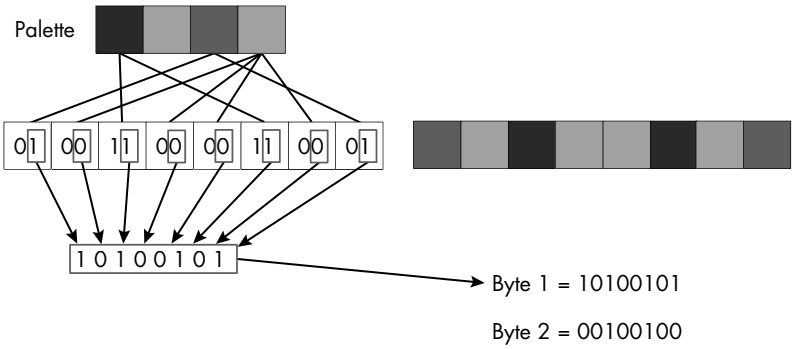


Figure 5-9: Encoding the first row of the tile

The first byte of the first tile row consists of the bits of the binary indices that are inside boxes. The second byte consists of the remaining bits.

Figure 5-10 shows how we would encode the whole tile from Figure 5-7.



	Byte 1	Byte 2
Row 1	00111100	11000000
Row 2	01000010	00000000
Row 3	10000001	00000000
Row 4	10100101	00100100
Row 5	10000001	00000000
Row 6	10011001	00011000
Row 7	01000010	00000000
Row 8	00111100	00000000

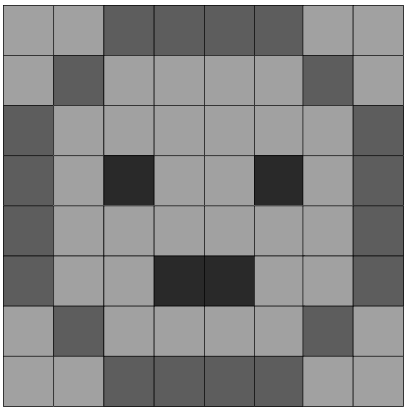


Figure 5-10: Encoding the entire tile

Understanding tile encoding is very important for writing image exporters because we have to convert the data from the original image format, such as BMP or PNG, to the DMG tile format.

Tilemaps

A *tilemap* is a 32×32 collection of tile indices. Each tile index is 1 byte, so a tilemap is 1,024 bytes. Because tiles are 8×8 in size, a tilemap is equivalent to an area of 256×256 pixels. The DMG has two tilemaps, both located in VRAM. As you'll see in the following chapters, the background and the window leverage tilemaps to display large graphics in a very efficient manner.

Tile Addressing Modes

We must place tiles and tilemaps in VRAM before the high-level features can use them. The VRAM is split into two areas: the tiles area and the tilemap area. These areas cover the whole 8KiB of the VRAM, and the PPU accesses the data inside them so that it can render graphics to the LCD. The tiles area extends from `$8000` to `$97FF`. That's 6,144 bytes (6KiB), and since a tile is 16 bytes, there can be up to 384 tiles in VRAM. The tilemap area contains up to 2 tilemaps. It extends from `$9800` to `$9FFF` for a total of 2KiB, with each tilemap occupying 1KiB.

Tilemaps use 1-byte-long indices (0 to 255) to reference tiles. However, as there can be up to 384 tiles in VRAM, a 1-byte index may not be enough to reference all the available tiles in the tile area. To solve this issue, the tiles area is further split into three logical blocks of 128 tiles each (blocks 0, 1, and 2), and we use two *tile addressing modes* to reference them: the $8000 mode and the $8800 mode.

The *$8000 mode* interprets the tile indices as *unsigned* integers. They go from 0 to 255, and the base address is `$8000`. The first tile is at `$8000` and the last tile is at `$8FFF`, so tile blocks 0 and 1 are accessible.

The *$8800 mode* interprets the tile indices as *signed* two's complement integers. They go from −128 to 127, and the base address is `$9000`. The first tile is at `$8800` and the last tile is at `$97FF`, so tile blocks 1 and 2 are accessible.

The background and window can use both addressing modes, but the sprites are limited to the $8000 mode. Both addressing modes share block 1, while blocks 0 and 2 are available in only one of the modes.

Table 5-2 sums up the addressing mode information.

Table 5-2: Tile Addressing Modes

Mode	Index	Base address	Blocks	Features
$8000	Unsigned	`$8000`	0 and 1	Background, window, sprites
$8800	Signed	`$9000`	1 and 2	Background, window

Table 5-3 shows the range of each area and the blocks in VRAM.

Table 5-3: VRAM Organization

Area	Size	Sub-area	Start	End
Tiles	6KiB	Block 0	`$8000`	`$87FF`
		Block 1	`$8800`	`$8FFF`
		Block 2	`$9000`	`$97FF`
Tilemaps	2KiB	Tilemap 0	`$9800`	`$9BFF`
		Tilemap 1	`$9C00`	`$9FFF`

You might wonder why the $8800 mode uses base address `$9000` coupled with signed integers and not base address `$8800` with unsigned integers. The key hint is that block 1 can be accessed in both addressing modes. In the $8000 mode, block 1 tile indices go from 128 to 255. In the $8800 mode, block 1 tile indices go from −128 to −1. For 8-bit integers, 128 equals −128 (both are `$80` in hexadecimal) and 255 equals −1 (both are `$FF` in hexadecimal). So, for any tile in block 1, we can use the same index in both addressing modes. That's a very handy property, because it allows high-level features using different addressing modes to share tile indices for tiles in block 1.

Putting It All Together

This kind of system leveraging colors, palettes, tiles, and tilemaps is quite common in 8-bit and 16-bit game consoles. For the DMG, it offers two main advantages. First, it helps control and reduce the memory consumption, as tiles are easily reusable. Second, it lowers the PPU processing time, as tiles can be processed faster than single pixels.

The drawback is that there are many instances of indirection between the high-level features and the actual colors displayed on the LCD.

Figure 5-11 visualizes the relationship between the various graphics elements.

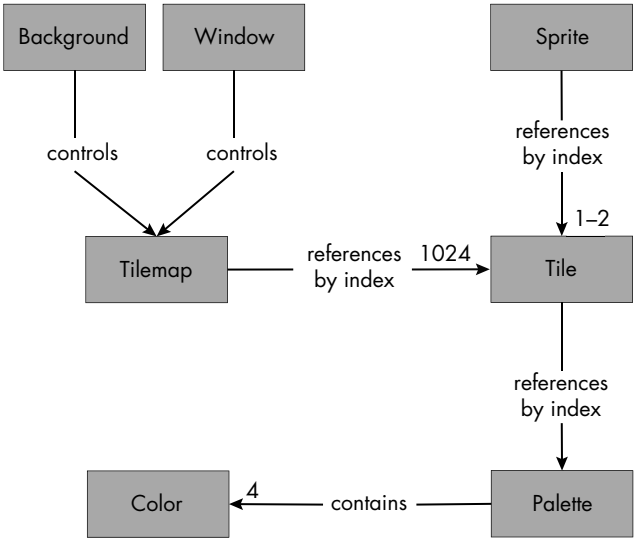


Figure 5-11: Visualizing how graphics elements are related

At the top of this diagram are the high-level features: the background, the window, and the sprites. The background and window control the display of the tilemaps, which consist of 1,024 tile indices. A sprite references up to two consecutive tiles directly, without going through a tilemap. A tile consists of 64 color indices from a palette. Finally, a palette contains four colors.

Summary

In this chapter you learned about the graphics capabilities of the DMG, from the most basic elements, like colors, to the high-level features, like the background.

You should now have a good grasp of how colors, palettes, tiles, and tilemaps are represented and connected to each other. You should also understand how the graphics data is organized in memory. The following chapters assume you've mastered those two aspects of DMG graphics.

In the next two chapters, you'll learn how to move the graphics data to the right place and how to set the DMG registers to leverage the high-level graphics features.

Game Boy Coding Adventure (Early Access) © 2025 by Maximilien Dagois

6

GRAPHICS ASSET MANAGEMENT

This chapter covers what you need to know to display custom graphics on the DMG, from converting image files to filling the VRAM with tiles and tilemap data.

First, you'll learn how to convert PNG files to DMG-compatible graphics, using either the RGBDS toolchain or a custom tool I created for this book. Then, we'll walk through a sample demonstrating how to add converted graphics data to a ROM. Finally, you'll see how to copy the graphics data from the ROM to VRAM so it can be displayed on the DMG LCD.

The graphics assets in this book are all available in the *assets* folder of the GitHub repository. They are four-color PNG files used as tilesets (groups of tiles) or tilemaps. The tileset PNG files for this book are 128×192 pixels and contain 384 tiles (16×24 tiles). The tilemap PNG files are 256×256 pixels and use the tile graphics from one of the tilesets. You don't need to stick to those conventions when you create your own tileset and tilemap PNG files; you can make them any size you want as long as the converted data fits in VRAM.

NOTE *Creating graphics assets for the DMG is a broad topic that is beyond the scope of this book. To learn more, search for online courses on creating low-resolution pixel art.*

Converting Assets

We cannot use PNG files in their native format on the DMG. We have to convert them so that we can put them into the ROM and then transfer them to VRAM.

In this book, we'll be converting tilesets to character (CHR) files, which have the *.chr* extension. (Tiles are often called *chars* or *characters* in the Japanese game industry.) CHR files are collections of tile data in the format explained in Chapter 5.

We'll convert tilemaps to tilemap layout (TLM) files with the *.tlm* extension. A TLM file contains a bunch of tile indices referring to tiles in a CHR file, which is produced as part of the conversion from PNG to TLM.

NOTE *The extension* .chr *stands for the word* character. *Tiles are often called* chars *or* characters *in the Japanese game industry. This book sticks to this extension, but any other extension works just as well.*

The samples in this book contain all the necessary CHR and TLM files (converted from PNG files), so you won't need to convert them yourself. That said, you'll need to convert any assets required for your own projects, so it's important to understand the conversion process.

There are two main options for asset conversion: use the RGBDS toolchain or write a custom tool. Which option is best will depend on the level of control you need in your program.

Using the RGBDS Toolchain

Let's start by going over how to convert assets using *rgbgfx*, the conversion tool that comes with RGBDS. There are two main use cases: tileset conversion and tilemap conversion.

The following command converts a tileset to generate a single CHR file with all the 8×8 tiles from the input PNG file.

```
rgbgfx --output tileset.chr tileset.png
```

You use the `rgbgfx` command to invoke the PNG converter of the RGBDS toolchain. The `--output` option specifies the output tileset file (here, *tileset.chr*). Finally, you provide the input source PNG image (here, *tileset.png*). The image should contain only up to four colors.

The following command converts a tilemap, generating a CHR file and a TLM file containing indices to tiles in the CHR file:

```
rgbgfx --unique-tiles --output tileset.chr --tilemap tilemap.tlm tilemap.png
```

The `--unique-tiles` option tells rgbgfx to generate a tileset with only unique tiles. This results in a CHR file that contains just the tiles necessary to represent the tilemap, with no doubles or unused tiles. If you don't include this option, the tool generates a tile for each 8×8-pixel block in the input tilemap PNG. The `--output` option specifies the output tileset (here,

tileset.chr), and the `--tilemap` option specifies the output tilemap file (here, *tilemap.tlm*). The final argument is the input source PNG image.

The main advantage of rgbgfx is that it's readily available in the toolchain, so it requires no additional development effort. However, there are some disadvantages. First, you can't convert more than one tilemap at a time, making it difficult to generate several TLM files referencing tiles from the same CHR file. Second, you can't control how the tileset is organized when converting tilemaps, because rgbgfx generates the tileset automatically. It will likely generate a different CHR file for each input tilemap PNG, with tiles at different indices.

The rgbgfx tools offers several additional options. You should evaluate whether it's good enough for your use case and write a custom tool only if necessary. For more details on rgbgfx, see the online documentation at *https://rgbds.gbdev.io/docs/master/rgbgfx.1*.

Using a Custom Tool

You can also use a custom tool to convert PNG files to CHR and TLM files. I created a tool called *gfxconv* for this book, which I used to convert all the sample assets. The tool's source code is available in the GitHub repository; see the *README.md* file at *https://github.com/mdagois/gca/blob/main/README.md* for more details.

The basic usage of gfxconv is as follows:

```
gfxconv tileset.png tilemap_0.png tilemap_1.png ...
```

You invoke the custom PNG converter with the `gfxconv` command and pass it the input tileset PNG (here, *tileset.png*). This file should contain 384 8×8 tiles that the tilemaps will reference (some tiles can be empty). The tool will generate a CHR file with the same name as the PNG.

Next, you list the input tilemap PNG files. You can pass any number of tilemap PNGs to gfxconv. If you don't specify any tilemap PNG files, it won't generate any TLM files. Otherwise, it will generate a TLM file for each input file.

The gfxconv tool is not meant to fit all use cases. In fact, it makes quite a few assumptions about the size and organization of the PNG files. Still, you can use the code as a reference to create other custom tools.

A custom tool gives you full control over the conversion process. For example, gfxconv converts the tileset as is (unlike rgbgfx, which does not allow you to control how the tileset is organized), so tilemaps can easily share the same tileset. The main disadvantage of writing a custom tool is the extra effort it takes to develop.

You can also use a combination of rgbgfx and custom tools. For example, you could use rgbgfx to export the tileset and a custom tool to handle the tilemaps.

Exporting Text Files

Although gfxconv and rgbgfx export binary files, when writing a custom tool it's possible to export text files instead. The assembly language has a functionality called `db` (define byte) to define data, as shown in Listing 6-1.

```
db %00111100
db %11000000
db %01000010
db %00000000
db %10000001
db %00000000
db %10100101
db %00100100
db %10000001
db %00000000
db %10011001
db %00011000
db %01000010
db %00000000
db %00111100
db %00000000
```

Listing 6-1: Defining data with `db`

This defines the tile data corresponding to Figure 5-7. To get a full tileset, you would just add more tile data.

You can also use `dw` (define word) combined with 16-bit numbers, as shown in Listing 6-2.

```
dw $C03C
dw $0042
dw $0081
dw $24A5
dw $0081
dw $1899
dw $0042
dw $003C
```

Listing 6-2: Defining data with `dw`

This is a more concise way to define each row in the tiles.

In some programming languages, generating text might be easier than generating binary data. Text also makes it easier to debug export issues, as the converted file is human-readable. Using text won't affect the ROM size or the runtime execution speed. The compilation time is longer, but nowadays it's unlikely to make a noticeable difference even on a low-end computer. Whether you choose binary or text is a matter of what's more convenient.

Adding Assets to the ROM

After converting your assets, you'll need to add them to the ROM. Let's see how this is done using the *init* sample, which is available in the GitHub repository.

First, we use a bit of arithmetic to put the data at the end of the ROM, as shown in Listing 6-3 (an excerpt from the *sample.rgbasm* file).

```
def TILES_COUNT                 equ (384)
def BYTES_PER_TILE              equ (16)
def TILES_BYTE_SIZE             equ (TILES_COUNT * BYTES_PER_TILE)
def TILEMAP_BYTE_SIZE           equ (1024)
def GRAPHICS_DATA_SIZE          equ (TILES_BYTE_SIZE + TILEMAP_BYTE_SIZE)
def GRAPHICS_DATA_ADDRESS_END   equ ($8000)
def GRAPHICS_DATA_ADDRESS_START \
    equ (GRAPHICS_DATA_ADDRESS_END - GRAPHICS_DATA_SIZE)
```

Listing 6-3: Defining constants to position the graphics assets at the end of the ROM

The goal of all these constants is to determine the starting address of the graphics data to ensure that it goes at the end of the ROM. The tileset size (384 tiles × 16 bytes) and tilemap size (1,024 bytes) are known constants, as is the end of the ROM (`$8000`), so we can subtract both sizes from the end of the ROM to compute a starting address.

NOTE *An assembly statement is usually written on a single line, but the backslash (\) at the end of the penultimate line lets us continue that statement on the following line. This lets you keep lines under a certain length, like the typical 80 or 100 characters.*

Listing 6-4 creates the section that holds all the graphics data in the ROM.

```
section "graphics_data", ROM0[GRAPHICS_DATA_ADDRESS_START]
incbin "tileset.chr"
incbin "background.tlm"
```

Listing 6-4: Creating the graphics_data section

We use the starting address we computed to place the section at the right address inside the ROM. Then we include the tileset and tilemap files using the `incbin` directive, which stands for *include binary*. This adds a file's content to the ROM as a raw collection of bytes, without giving it any kind of meaning (as opposed to code, which is compiled). Note that we use `incbin` because the CHR and TLM files exported from gfxconv are in binary format. If they were text files, we would use `include` instead.

The include order is important, because the data gets organized exactly like it would be in VRAM (see Chapter 5). The tile data comes first, then the tilemap. This order makes it easier to copy the data over to VRAM.

Most of the samples in this book use *tileset.chr*. Figure 6-1 shows the tileset image structure.

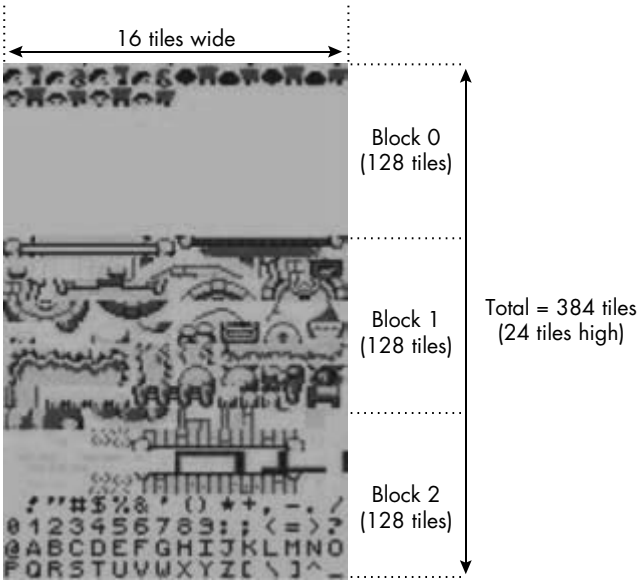


Figure 6-1: The sample tileset

The tileset PNG is an image containing 24 rows of 16 tiles converted to 384 tiles. It will completely fill the VRAM reserved for tile data, from `$8000` to `$97FF`.

The tilemaps vary visually from sample to sample. For example, this chapter's sample uses *background.tlm*, which is shown in Figure 6-2.



Figure 6-2: A sample tilemap

Tilemaps are converted to 1,024 tile indices, which will completely fill one of the two tilemap spaces in VRAM.

NOTE *In most of the samples in this book, we'll use the formulas in this section to place the graphics assets at the end of the ROM for convenience. However, placing the assets at the end of the ROM is not mandatory. You can place them anywhere in the ROM*

you like, in whatever format you choose, as long as those assets end up in the right format and location in VRAM.

Copying Assets to VRAM

Now that the tiles are in the ROM, we need to copy them over to VRAM. This is because the PPU, which renders graphics to the LCD, can only access the VRAM.

The VRAM is mostly inaccessible during the LCD refresh, so if the LCD is on, we'll have to copy the data over during the vblank. (There are ways to access the VRAM during a refresh, but they require more advanced techniques that we won't cover until later in the book.) As you learned in Chapter 5, the vblank is 1.08 milliseconds long (see Figure 5-1). Even with very tight code, copying 384 tiles and a tilemap to VRAM would take around 8 milliseconds, so we'd have to execute the copy over several vblanks. That's not very practical at initialization time, so it's better to just turn off the LCD. When the LCD is off, we can do whatever copying and initialization we need to without worrying about timing. When we're done, we can turn the LCD back on.

By default, the DMG LCD is on. We can turn it off with a single line of assembly, but there's a *huge* catch. While you can technically turn off the LCD at any point, and it will work as expected most of the time, there's a nonnegligible chance that doing so will damage the LCD. The LCD *must* be turned off during the vblank, as otherwise the LCD hardware might break irreversibly. The odds of this occurring are high enough that Nintendo effectively prohibits games from turning the LCD off outside of the vblank. Thankfully, it's easy enough to wait for the vblank to disable the LCD.

To keep things simple, the LCD is turned off in the *main.rgbasm* file, at the very start of the program. This will be true for all samples in this book. Each sample will be responsible for turning the LCD back on during initialization. This can be done at any point, without risks.

Listing 6-5, from the *init* sample's *main.rgbasm* file, shows the macro used to wait for the vblank and turn the LCD off.

```
macro DisableLCD
    ; wait for the vblank
    .wait_vblank\@
        ld a, [rLY]
        cp a, SCRN_Y
        jr nz, .wait_vblank\@

    ; turn the LCD off
    xor a
    ld [rLCDC], a
endm
```

Listing 6-5: The `DisableLCD` macro

The I/O register `rLY` contains the number of the current line. Its value goes from 0 to 153. From 0 to 143, the PPU is drawing to the LCD (as the LCD has 144 lines). That's the LCD refresh. From 144 to 153, the PPU is idle. That's the vblank. To make sure we are in the vblank before disabling the LCD, we need to wait for `rLY` to be equal to 144. But instead of using the raw value 144, we use a convenient constant defined in *hardware.rgbinc*, `SCRN_Y`, that corresponds to this value, which is the height of the screen in pixels.

Once we're in the vblank, we can safely disable the LCD by clearing the I/O register `rLCDC`. This is the LCD control register, which contains flags to customize the behaviors of the background, window, and sprites. Technically, we only need to clear bit 7 to turn the LCD off. That bit controls whether the LCD and PPU are enabled or not. But because we'll set `rLCDC` fully again in the `InitSample` function later, we can just clear it completely for now.

We use the `xor a` instruction to set the register `a` to zero. This is a shortcut for `xor a, a`. As xor'ing a number with itself is equivalent to setting it to zero, `xor a` is equivalent to `ld a, 0`. The difference is that `xor a` takes 1 byte in the ROM and 1 cycle to execute, while `ld a, 0` takes 2 bytes in the ROM and 2 cycles to execute. In short, this is a common pattern that saves 1 byte and 1 cycle.

Now that we've turned off the LCD, we can copy the graphics data to VRAM. Listing 6-6 shows the macro defined in the *sample.rgbasm* file to copy over all the tileset and tilemap data.

```
macro LoadGraphicsDataIntoVRAM
    ld de, GRAPHICS_DATA_ADDRESS_START
    ld hl, _VRAM8000
    .load_tile\@
        ld a, [de]
        inc de
        ld [hli], a
        ld a, d
        cp a, high(GRAPHICS_DATA_ADDRESS_END)
        jr nz, .load_tile\@
endm
```

Listing 6-6: Copying the graphics data to VRAM

The idea here is to copy the bytes one by one from the ROM to the VRAM. The `de` register holds the address of the source byte in the ROM, and `hl` holds the address of the destination in VRAM.

Because the data is at the end of the ROM, the copy is over once `de` is equal to `$8000`. We use a shortcut here, and the code only checks whether `d` is equal to `$80`. Checking that `e` is `$00` would be redundant, because no valid ROM address would start with `$80` anyway.

Finally, once the copy is done, we turn the LCD back on, as shown in Listing 6-7 (an excerpt from the `InitSample` function in *sample.rgbasm*). In

future samples, we'll also set the parameters for the background, window, and sprites here.

```
    ld a, LCDCF_ON
    ld [rLCDC], a
```

Listing 6-7: Turning the LCD back on

If you run the *init* sample in the BGB emulator, you won't see any graphics because this sample just loads the data in VRAM. However, you can use the VRAM viewer in BGB to inspect the contents of the VRAM. You can access the VRAM viewer from the debugger by pressing F5. The tool's Tiles menu is shown in Figure 6-3.

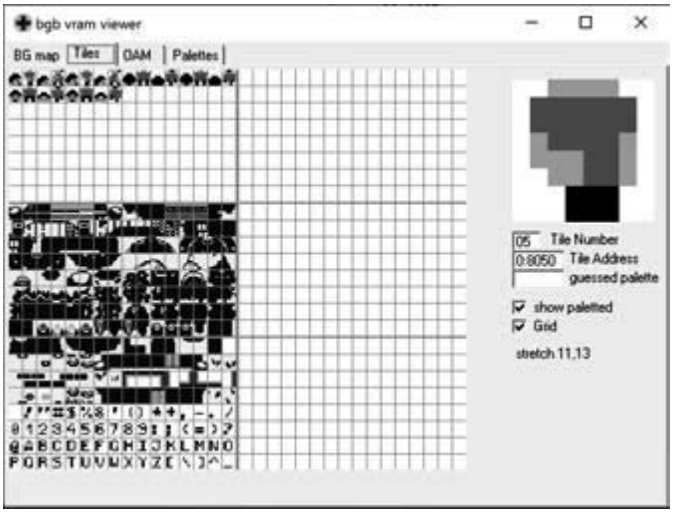


Figure 6-3: The BGB VRAM viewer

The tiles are visible in the viewer, so the data has been loaded as expected. You can check the tilemaps under the BG Map menu.

Sample Code

To close out this chapter, let's take a look at the complete code for the *init* sample. Listing 6-8 shows the contents of the *main.rgbasm* file.

```
include "hardware.rgbinc"

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

section "header", rom0[$0100]
entrypoint:
    di
    jp main
    ds ($0150 - @), 0
```

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

macro DisableLCD
    ; wait for the vblank
    .wait_vblank\@
        ld a, [rLY]
        cp a, SCRN_Y
        jr nz, .wait_vblank\@

    ; turn the LCD off
    xor a
    ld [rLCDC], a
endm

section "main", rom0
main:
    DisableLCD
    call InitSample
    .loop
        call UpdateSample
        jr .loop
```

Listing 6-8: The main.rgbasm *file*

As you can see, this file now includes the routine to disable the LCD. Listing 6-9 shows the contents of the *sample.rgbasm* file.

```
include "utils.rgbinc"

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

def TILES_COUNT                 equ (384)
def BYTES_PER_TILE              equ (16)
def TILES_BYTE_SIZE             equ (TILES_COUNT * BYTES_PER_TILE)
def TILEMAP_BYTE_SIZE           equ (1024)
def GRAPHICS_DATA_SIZE          equ (TILES_BYTE_SIZE + TILEMAP_BYTE_SIZE)
def GRAPHICS_DATA_ADDRESS_END   equ ($8000)
def GRAPHICS_DATA_ADDRESS_START \
    equ (GRAPHICS_DATA_ADDRESS_END - GRAPHICS_DATA_SIZE)

; load the graphics data from ROM to VRAM
macro LoadGraphicsDataIntoVRAM
    ld de, GRAPHICS_DATA_ADDRESS_START
    ld hl, _VRAM8000
    .load_tile\@
        ld a, [de]
        inc de
        ld [hli], a
```

```
        ld a, d
        cp a, high(GRAPHICS_DATA_ADDRESS_END)
        jr nz, .load_tile\@
endm

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

section "sample", rom0

InitSample:
    LoadGraphicsDataIntoVRAM

    ; turn the LCD on
    ld a, LCDCF_ON
    ld [rLCDC], a

    ret

UpdateSample:
    ret

export InitSample, UpdateSample

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

section "graphics_data", rom0[GRAPHICS_DATA_ADDRESS_START]
incbin "tileset.chr"
incbin "background.tlm"
```

Listing 6-9: The sample.rgbasm *file*

This file now contains the logic to load the tileset and the tilemap.

Summary

In this chapter, you learned how to convert graphics assets to a form the DMG can display and add them to a ROM. Note that there are many ways to organize and manage assets. The data in VRAM has strict rules regarding order and location, but in the ROM, there are no such rules. For example, you don't have to place the tileset and tilemap data at the end of the ROM, as I have chosen to do. You could place this data in distinct sections and copy them over separately; whatever matches your program's needs is fine.

You're now ready to learn how to leverage the high-level graphics features of the DMG. In the next chapter, we'll take an in-depth look at working with the background, window, and sprites.

Game Boy Coding Adventure (Early Access) © 2025 by Maximilien Dagois

7

HIGH-LEVEL FEATURES

This chapter covers the high-level graphics features of the DMG—the background, the window, and sprites—and explains how to leverage them. By the end of the chapter, you'll have all the knowledge you need to display custom graphics on the LCD.

You'll begin by learning about the parameters and functionalities of the background, including scrolling and wrapping. Building on that knowledge, you'll learn about frame pacing, a fundamental technique for producing smooth animations.

Then, we'll take a closer look at the window, comparing its parameters and functionalities to those of the background. Finally, you'll learn how to use sprites and gain an understanding of their behavior and limitations.

Background

The *background* is a large scrollable tilemap that forms the visual backdrop for a DMG game. You configure the background by specifying the palette, selecting the tilemap, and setting a few other parameters. There are also a couple of useful built-in functionalities to be aware of.

Palette

The first element you need to set up is the background palette, as demonstrated in the *bg* sample. The relevant code from *sample.rgbasm* is shown in Listing 7-1.

```
    ld a, %11100100
    ld [rBGP], a
```

Listing 7-1: Setting the background palette

The background has only one palette, which is set through the I/O register `rBGP`. In this code snippet, we place white at color index 0 (bits 0 and 1), light gray at index 1 (bits 2 and 3), dark gray at index 2 (bits 4 and 5), and black at index 3 (bits 6 and 7). This is just one possible palette setup; you can put the colors in any order you like (for example, `%00011011`), or even reuse the same color multiple times (for example, `%00001111`). For a refresher on the available colors, refer back to Table 5-1.

LCD Parameters

Now that we've set up the palette, we need to set the LCD control register, `rLCDC`, which will enable the background and set both the addressing mode ($8000 or $8800) and the tilemap address (`$9800` or `$9C00`).

In Chapter 6, you learned that the `rLCDC` register is a collection of flags that control the behavior of the DMG's graphics system. The *hardware.rgbinc* file defines the possible flag values, as shown in Listing 7-2.

```
DEF LCDCF_OFF     EQU %00000000 ; LCD Control Operation
DEF LCDCF_ON      EQU %10000000 ; LCD Control Operation
DEF LCDCF_WIN9800 EQU %00000000 ; Window Tile Map Display Select
DEF LCDCF_WIN9C00 EQU %01000000 ; Window Tile Map Display Select
DEF LCDCF_WINOFF  EQU %00000000 ; Window Display
DEF LCDCF_WINON   EQU %00100000 ; Window Display
DEF LCDCF_BG8800  EQU %00000000 ; BG & Window Tile Data Select
DEF LCDCF_BG8000  EQU %00010000 ; BG & Window Tile Data Select
DEF LCDCF_BG9800  EQU %00000000 ; BG Tile Map Display Select
DEF LCDCF_BG9C00  EQU %00001000 ; BG Tile Map Display Select
DEF LCDCF_OBJ8    EQU %00000000 ; OBJ Construction
DEF LCDCF_OBJ16   EQU %00000100 ; OBJ Construction
DEF LCDCF_OBJOFF  EQU %00000000 ; OBJ Display
DEF LCDCF_OBJON   EQU %00000010 ; OBJ Display
DEF LCDCF_BGOFF   EQU %00000000 ; BG Display
DEF LCDCF_BGON    EQU %00000001 ; BG Display
```

Listing 7-2: The `rLCDC` flags

Three flags, or bits, are relevant to the background. Table 7-1 summarizes the possible values of these flags and what they mean.

Table 7-1: Background Flag Values

Bit	Value	Meaning
0	`LCDCF_BGOFF`	The background is disabled.
	`LCDCF_BGON`	The background is enabled.
3	`LCDCF_BG9800`	The tilemap is located at `$9800`.
	`LCDCF_BG9C00`	The tilemap is located at `$9C00`.
4	`LCDCF_BG8000`	The $8000 addressing mode is used.
	`LCDCF_BG8800`	The $8800 addressing mode is used.

Listing 7-3 shows how the LCD control register is set in the *bg* sample.

```
    ld a, LCDCF_ON | LCDCF_BG8800 | LCDCF_BG9800 | LCDCF_BGON
    ld [rLCDC], a
```

Listing 7-3: Background setup

We need to enable the background before we can use it, so we raise bit 0 using `LCDCF_BGON`. The tilemap is at address `$9800` in VRAM and the tileset was organized for the $8800 addressing mode (see Figure 6-1), so the flags for bits 3 and 4 are set accordingly. The `LCDF_ON` flag ensures that the LCD is on.

Listing 7-4 shows the fully updated `InitSample` function from the *sample.rgbasm* file.

```
InitSample:
    ; init the palette
    ld a, %11100100
    ld [rBGP], a

    LoadGraphicsDataIntoVRAM

    ; set the graphics parameters and turn back LCD on
    ld a, LCDCF_ON | LCDCF_BG8800 | LCDCF_BG9800 | LCDCF_BGON
    ld [rLCDC], a

    ret
```

Listing 7-4: The `InitSample` function for the background (`bg`) sample

First, the background palette is set. Then, the graphics data, which consists of tiles and a tilemap, is loaded into VRAM. Finally, the remaining background parameters are set through the LCD control register so that the background displays properly on the LCD.

Figure 7-1 shows what you should see when you compile the code for this sample and run it in BGB.



Figure 7-1: The background sample

Notice that we see only a part of the tilemap, because the tilemap is 256×256 pixels but the LCD can only display 160×144 pixels. We'll use one of the background's key functionalities in the next section to reveal the rest of the tilemap.

Scrolling and Wrapping

The background has two key built-in functionalities: scrolling (controlled by the programmer) and wrapping (automatic).

You can control the scrolling of the background using the `rSCX` and `rSCY` registers. The `rSCX` register controls the horizontal scrolling and `rSCY` controls vertical scrolling.

The `rSCX` and `rSCY` registers store the coordinates of the screen relative to the top-left corner of the tilemap. In this book, I'll use the term *abscissa* and *ordinate* to refer to the x-coordinate and y-coordinate, respectively. Figure 7-2 shows how background scrolling works.

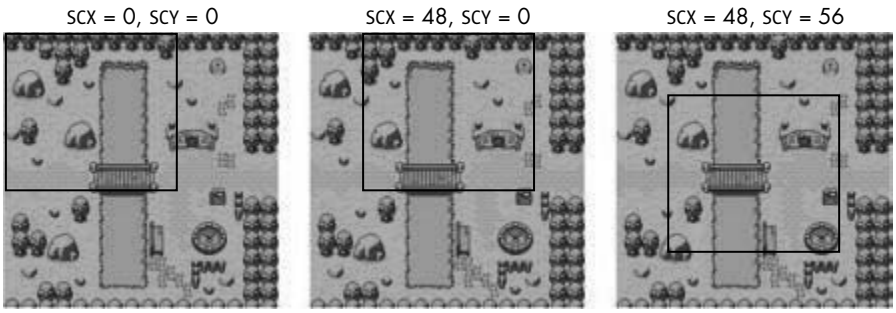


Figure 7-2: Three background scrolling positions and their corresponding coordinates

The square area represents the screen. The default position on the left is what we got previously, in Figure 7-1. The image in the center shows the

background scrolled horizontally, and the right-hand image shows the background scrolled in both directions.

When you scroll far enough in one direction, the boundary of the background's tilemap ends up displayed on the screen. That's when the wrapping kicks in. Figure 7-3 illustrates how wrapping works.

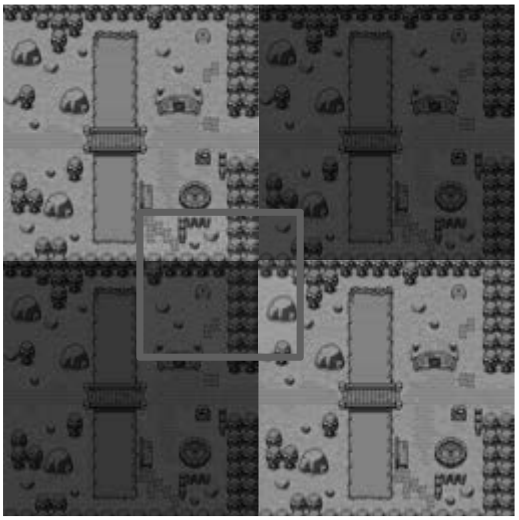


Figure 7-3: Background wrapping

The background's tilemap repeats in every direction. I've highlighted the repeat tilemaps to make the wrapping effect stand out, but in real life, the colors would not change.

To experiment with wrapping in BGB, you can update the `UpdateSample` function. In the sample, the scrolling values are zero by default, as shown in Listing 7-5.

```
UpdateSample:
    ld a, 0
    ld [rSCX], a
    ld a, 0
    ld [rSCY], a
    ret
```

Listing 7-5: The `UpdateSample` function

Try using values such as 176 and 184 for `rSCX` and `rSCY`, respectively. There's nothing special about those two values, but they allow you to see both scrolling and wrapping in effect, as in Figure 7-3.

While the background has only a few configurable parameters and functionalities, it's an essential part of the DMG graphics system, and most, if not all, released games make use of it.

Frame Pacing

In this section, you'll learn about an issue in the update function that you must address before configuring the other high-level graphics features. I'll illustrate the issue using the *scrolling_anim_broken* sample, then show you how to fix it using a technique called *frame pacing*, which synchronizes the update function's call with the DMG's frame mechanism to produce smooth animations.

Understanding the Synchronization Issue

For the purpose of this example, we'll add a simple animation to the background. Listing 7-6 shows the code for this.

```
UpdateSample:
    ld a, [rSCX]
    inc a
    ld [rSCX], a

    ret
```

Listing 7-6: A simple background animation

To create this basic animation, the code increments the value of `rSCX` each time the `UpdateSample` function is called. From this, you would expect that the background will scroll smoothly to the left, one pixel at a time, as each frame is drawn. But the actual result, shown in Figure 7-4, is quite different.

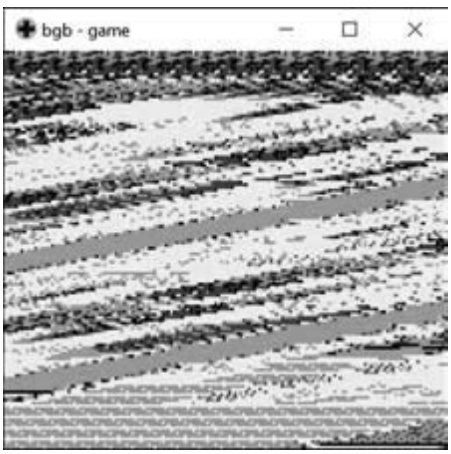


Figure 7-4: Broken background animation

The problem is that the animation is not synchronized with the DMG frame mechanism, described in Chapter 5. Currently, the `UpdateSample` function is called from the main loop as many times as the DMG CPU speed will allow. That's well over a thousand times per frame, but it should be

called only once per frame. The result is that each line of the screen scrolls independently of the other lines, causing the visual mess in Figure 7-4.

Fixing the Pacing

To fix the pacing issue, we'll wait for the next vblank at the start of the `UpdateSample` function. This ensures that `UpdateSample` executes only per frame because, by definition, there is only one vblank per frame.

In Listing 6-5, we waited for the vblank when we disabled the LCD in the main function. To accomplish this, we waited for the current line number register, `rLY`, to equal 144, which meant we had entered the vblank period. That technique is good enough for initialization, but a better way to wait for the vblank in the main loop is to use `halt` and the vblank interrupt. The `halt`/interrupt method is superior because it has the potential to prolong the device's battery life. When using `halt`, the DMG enters low power mode. On the other hand, the DMG will be fully working during the loop of the `rLY` method.

Before implementing the `halt`/interrupt method, we need to define what an interrupt is. For the purposes of this book, an *interrupt* is a function that gets called when a particular event occurs. When the event arises, the CPU will stop, or interrupt, any work it's currently doing, execute a function associated with the event (the interrupt function), and then go back to its previous work. In the sample, the event is the start of the vblank, and the associated interrupt function address for that event is `$0040`. There are four other event types that trigger interrupts, and each has its own fixed function address; I'll cover these later in the book.

Now that we know what an interrupt is, let's use the vblank interrupt in the sample.

Enabling the Vblank Interrupt

The first step is to enable the vblank interrupt. We do this during the initialization of the *scrolling_anim_fixed* sample.

Listing 7-7 shows the relevant code from `InitSample`.

```
ld a, IEF_VBLANK
ld [rIE], a
ei
```

Listing 7-7: Enabling the vblank interrupt

We perform two actions here. First, the code sets the `rIE` (interrupt enable) register. This register contains flags to tell the DMG which interrupts are enabled. For now, only the vblank interrupt is necessary, so we raise a single flag, `IEF_VBLANK`. Second, we enable the interrupt functionality. As you saw in Chapter 4, we disabled the interrupts in the template by using `di` (disable interrupts) in the entry point so they wouldn't get in the way during the sample initialization. Here, we use `ei` (enable interrupts) to re-enable the interrupts.

Defining the Interrupt Function

The second step of using the vblank interrupt is to define the interrupt function, as shown in Listing 7-8.

```
section "vblank_interrupt", rom0[$0040]
    reti
```

Listing 7-8: The vblank interrupt function

We define a section at `$0040`, because this is the address of the function called when the vblank event occurs. The function contains a single instruction, `reti`, which returns from the function (like a regular `ret`) with the additional effect of enabling interrupts.

We need to use `reti` and not `ret` here because the DMG disables the interrupts (like `di` would) before executing any interrupt function. This prevents interrupting the interrupt function, which could lead to a bunch of issues, like CPU hangs. Because we want to detect subsequent vblank events, we need to use `reti` to enable interrupts again after the function returns.

Waiting for the Vblank Interrupt

The last step is to wait for the interrupt to occur. This is done with the previously mentioned `halt` instruction, which puts the DMG into low power mode. In this mode, no instructions are executed, and the DMG wakes up only when an interrupt triggers. In the sample, the only enabled interrupt is the vblank interrupt. This means that each time the `halt` instruction completes, a new vblank period has just started.

We can add `halt` to the `UpdateSample` function, as shown in Listing 7-9.

```
UpdateSample:
    halt

    ld a, [rSCX]
    inc a
    ld [rSCX], a

    ret
```

Listing 7-9: Using `halt` to wait for the vblank

Now, each time the main loop calls `UpdateSample`, the DMG waits for the next vblank. The result is that `UpdateSample` is called only once per frame.

If you compile and run the sample again in BGB, you should now see that the scrolling animation is fluid because we have properly synchronized the update function with the DMG display mechanism.

Summing Up the Frame Pacing Mechanics

We now have the proper frame pacing in place, allowing for smooth animations. Let's quickly recap the three steps we implemented to fix the problem and the timing of each step.

The first step is to wait for the next vblank using the `halt` instruction. This step's duration is whatever time was left of the 16.74 ms of the previous frame.

The next step is to perform any operation that requires VRAM access or affects graphics directly before the end of the vblank. This step can take up to 1.08 ms, which is the duration of the vblank. As discussed in "Copying Assets to VRAM" on page 87, VRAM access is restricted during the LCD refresh, so it's critical to carry out any VRAM updates during the vblank. For example, the PPU will ignore a tile update, which is a write to VRAM, if it's executed outside the vblank. This will likely cause graphical artifacts, as the intended graphics data will not be in VRAM. Likewise, while operations such as scrolling are allowed at any time, they might cause visual glitches if not executed during the vblank (if the `rSCX` register is updated outside the vblank, the screen will end up split in two halves that use different scrolling values).

The final step of the update function is to perform any remaining program operations, such as checking input or running the sample logic. This step can take 15.66 ms (the duration of the LCD refresh) plus whatever time remains from the vblank's 1.08 ms.

Window

In this section we'll look at the *window*, which can display part of a tilemap in a rectangular area of the screen superimposed on the main background. You'll learn about the window's similarities and differences compared with the background as well as its main pitfalls.

Comparing the Window and the Background

The window and background share a few parameters. The first one is the palette. We set the palette for the background in the `InitSample` function, as shown in Listing 7-10.

```
ld a, %11100100
ld [rBGP], a
```

Listing 7-10: Setting the background and window palette

This sets the palette for the window too.

The second shared parameter is the tile addressing mode. The addressing mode is set in the LCD control register, `rLCDC`, using either the `LCDCF_BG8000` flag or the `LCDCF_BG8800` flag. These flag names are a bit misleading, because it looks like they apply only to the background, but the addressing mode setting actually applies to both the background and the window.

Earlier, we set the background addressing mode to `LCDCF_BG8800`, as shown in Listing 7-11.

```
    ld a, LCDCF_ON | LCDCF_BG8800 | LCDCF_BG9800 | LCDCF_BGON
    ld [rLCDC], a
```

Listing 7-11: Setting the background and window tile addressing mode

The window will also use the $8800 addressing mode.

There are three differences between the window and the background. The first difference is that they don't need to point to the same tilemap. Remember that there can be up to two tilemaps in VRAM. We can set up the window and background to point to the same tilemap or use a separate tilemap for each.

The second difference is that the window does not wrap. The window is just a square area that moves around over the background (although there are some limitations and particularities to be aware of, which I'll explain in more detail in the next section).

The final difference is that the window has its own enable flag (bit 5) in the LCD control register. For example, in Listing 7-11 we enabled the background with the `LCDCF_BGON` flag, but to enable the window, we would need to add the `LCDCF_WINON` flag too.

There's a catch, though: disabling the background (by clearing bit 0 in `rLCDC`) will also disable the window. That means we can enable the window if and only if we enable the background as well. The possible combinations are background on/window on, background on/window off, and background off/window off. There is no way to get a background off/window on combination.

Table 7-2 summarizes the LCD flags that affect the window.

Table 7-2: Window Flag Values

Bit	Value	Shared	Meaning
0	`LCDCF_BGOFF`	Yes	The background and window are disabled.
	`LCDCF_BGON`	Yes	The background and window are enabled.
4	`LCDCF_BG8000`	Yes	The $8000 addressing mode is used.
	`LCDCF_BG8800`	Yes	The $8800 addressing mode is used.
5	`LCDCF_WINOFF`	No	The window is disabled.
	`LCDCF_WINON`	No	The window is enabled.
6	`LCDCF_WIN9800`	No	The window tilemap is located at `$9800`.
	`LCDCF_WIN9C00`	No	The window tilemap is located at `$9C00`.

These flags are used in the `rLCDC` register to control the window parameters, alongside the flags for controlling the other high-level graphics features.

Moving the Window

Moving the window is similar to scrolling the background. We use the `rWX` and `rWY` registers to set the position of the top-left corner of the window.

In the `rWY` register, 0 represents the top line of the LCD. A value of 144 or more will put the window beyond the bottom of the screen, making it invisible.

The `rWX` register is a bit peculiar. First, the leftmost column of the LCD is at abscissa 7, not 0, as one would expect. This means that the coordinate for the top-left corner of the LCD is (7, 0). Second, there is a hardware bug that will very likely cause the window to display improperly for certain values of `rWX`: namely, anything in the 0–6 range and the value 166. Entering a value in the 0–6 range would technically bring the window slightly beyond the left edge of the LCD. A value of 166 would display only a single column of pixels from the window on the right side of the LCD. The bug will cause the window to either display at the wrong location, lack some columns of pixels, or flicker on the LCD. It's best to stay away from using the buggy values of `rWX` altogether.

In practice, you can safely move the window anywhere between coordinates (7, 0) and (165, 143). At (7, 0), the window completely covers the background. For any abscissa beyond 167 and any ordinate beyond 143, the window will be outside the bounds of the LCD screen and thus invisible. For any coordinates between (7, 0) and (165, 143), the window will partially cover the background, moving up and to the left from the bottom-right corner.

Figure 7-5 shows these three cases, using the background and window from the simple sample in Figures 5-3 and 5-4.

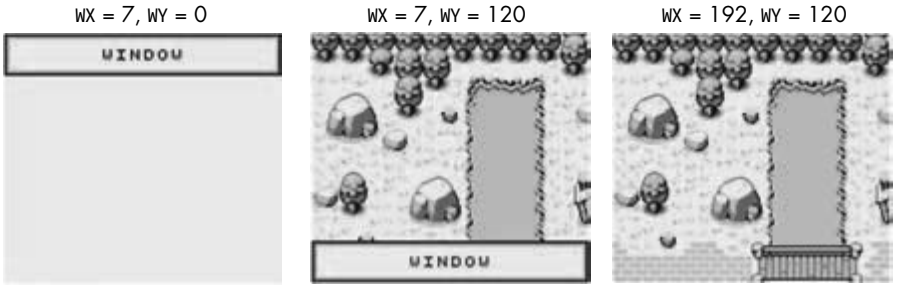


Figure 7-5: Window location examples

On the left, the window is at the origin and covers the whole background. In the center, the window is at its actual position in the sample. It covers only the portion of the background at the bottom of the LCD. Finally, on the right, the window is positioned beyond the bounds of the LCD display (to the right) and is not visible at all.

The window uses a tilemap for its graphics, which is 256×256 pixels, but the window itself has a fixed size of 160×144 pixels (the same dimensions as the LCD screen). If you consider the tilemap surface in Figure 7-6, the light area would potentially display on the LCD, while the dark area would never display.

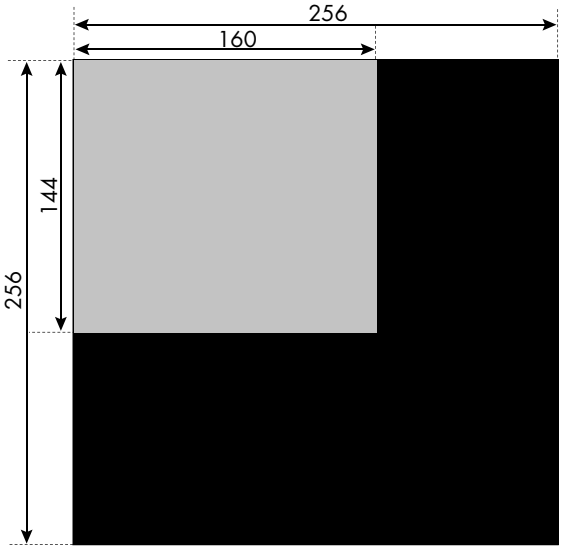


Figure 7-6: Unused tilemap parts

Keep the light area in mind when designing tilemaps for use with the window. The dark area won't necessarily be wasted, because the background can use it. This would most likely require using advanced graphics tricks, though, which I'll cover later in this book.

Using the Window

To wrap up this section, let's explore the *window* sample that displays the window over the background.

We'll start by adding the necessary graphics data to the ROM, as shown in Listing 7-12.

```
section "graphics_data", rom0[GRAPHICS_DATA_ADDRESS_START]
incbin "tileset.chr"
incbin "background.tlm"
incbin "window.tlm"
```

Listing 7-12: Adding the window tilemap

The *window.tlm* tilemap contains graphics for the window and is now at the end of the `"graphics_data"` section.

Next, we update the computation of this section's starting address, `GRAPHICS_DATA_ADDRESS_START`, to reflect the fact that two tilemaps are now used. Listing 7-13 shows the revised calculation.

```
def TILEMAPS_COUNT              equ (2)
def BYTES_PER_TILEMAP           equ (1024)
def TILEMAPS_BYTE_SIZE          equ (TILEMAPS_COUNT * BYTES_PER_TILEMAP)

def GRAPHICS_DATA_SIZE          equ (TILES_BYTE_SIZE + TILEMAPS_BYTE_SIZE)
```

```
def GRAPHICS_DATA_ADDRESS_END       equ ($8000)
def GRAPHICS_DATA_ADDRESS_START \
    equ (GRAPHICS_DATA_ADDRESS_END - GRAPHICS_DATA_SIZE)
```

Listing 7-13: Updating the constants to position the graphics assets

Finally, we initialize the window coordinates and set the tilemap address, as shown in Listing 7-14.

```
    ; place the window at the bottom of the LCD
    ld a, 7
    ld [rWX], a
    ld a, 120
    ld [rWY], a

    ; set the graphics parameters and turn the LCD back on
    ld a, LCDCF_ON | LCDCF_WINON | LCDCF_WIN9C00 | \
          LCDCF_BG8800 | LCDCF_BG9800 | LCDCF_BGON
    ld [rLCDC], a
```

Listing 7-14: Positioning the window and setting the `rLCDC` flags

We place the window at coordinates (7, 120) by setting `rWX` and `rWY`. Then, we add two flags, `LCDCF_WINON` and `LCDCF_WIN9C00`, to the LCD control register. The first flag enables the window, and the second flag makes the window point to the second tilemap (at `$9C00` in VRAM).

Figure 7-7 shows the results after compiling and running the sample in BGB.



Figure 7-7: The window *sample*

If you're feeling adventurous, try changing the `rWX` value to one of the unreliable values discussed earlier. You'll be able to see the hardware bug in action.

Sprites

Sprites are used to represent moving elements, such as characters or projectiles, which would be difficult, if not impossible, to depict with the background and window. In this section, we'll look at how they're used and what the limitations are.

Palettes

The first thing you need to set up for sprites are the palettes. Whereas the background and window have only one palette, sprites come with two palette options: `rOBP0` and `rOBP1`.

Each sprite can use only one palette at a time, but it's easy to swap palettes. Usually, we use the second palette for palette switching effects. For example, a character taking damage can briefly switch to the second palette to reflect its weakened status.

For sprites, the color at index 0 in a palette is the *transparent color*. If a sprite has pixels of transparent color, the background, window, and even other sprites will show through the transparent parts. This is a handy feature, because it lets you use sprites to represent non-square objects. For example, you could depict a round ball using the transparent color for pixels outside the ball graphics. The only downside to this feature is that it means sprites have only three visible colors. We'll see the transparent color in action in the samples in this section.

LCD Parameters

The LCD control register has two flags that affect sprites. The first one is the enable flag, which turns the sprites on and off. Unlike the window, which can be enabled only if the background is also enabled, sprites are totally independent of the background and window.

The second is the size flag, which controls whether sprites are 8×8 pixels (one tile) or 8×16 pixels (two tiles, aligned vertically). This parameter affects all sprites: they are either all 8×8 or all 8×16.

Table 7-3 summarizes the sprite-related flags.

Table 7-3: Sprite Flag Values

Bit	Value	Meaning
0	`LCDCF_OBJOFF`	The sprites are disabled.
	`LCDCF_OBJON`	The sprites are enabled.
3	`LCDCF_OBJ8`	The sprites are 8×8 (one tile).
	`LCDCF_OBJ16`	The sprites are 8×16 (two tiles).

Note that there is no tile addressing mode flag for sprites; sprites always use the $8000 tile addressing mode.

Object Attribute Memory

Each sprite has four attributes: an abscissa, an ordinate, a tile index, and control flags. Each attribute takes 1 byte in memory, so each sprite takes 4 bytes. The sprite attributes are set in a 160-byte dedicated memory area called the *object attribute memory (OAM)*. The OAM extends from `\$FE00` to `$FE9F`.

The PPU accesses the OAM when drawing the sprites to the LCD. Like the VRAM, the OAM is mostly inaccessible during the LCD refresh, so for now we'll set the sprites' attributes during the vblank. We'll look at an alternative technique for doing this in Chapter 12.

The first two attributes are the abscissa, X, and the ordinate, Y. Both attributes refer to the coordinates of the top-left corner of the sprite. X is equal to the horizontal position of the sprite plus 8. Y is equal to the vertical position of the sprite plus 16. That means when it comes to positioning sprites, the top-left corner of the LCD is actually at coordinates (8, 16). This allows the sprites to enter the left and upper sides of the LCD gradually. When a sprite's X or Y value is 0, it's outside the bounds of the LCD display and thus completely invisible. The same goes when X is above 168 (160 + 8) or Y is above 160 (144 + 16).

The third attribute to configure for sprites is the tile index. As mentioned previously, sprites always use the $8000 tile addressing mode. So, the indices go from 0 to 255 and can refer to any tiles in blocks 0 and 1. When using 8×16 sprites (two tiles), this attribute is the index of the first tile, which renders the top of the sprite. We cannot specify the second tile's index; it's always the value of the index attribute plus one. The second tile renders the bottom of the sprite.

The fourth and last attribute is a byte containing flags to control the sprite rendering. Table 7-4 summarizes these flags, which use only 4 bits (bits 0 to 3 are unused).

Table 7-4: OAM Flags

Bit	Description
4	Selects the palette number of the sprite
5	Flips the sprite horizontally when the bit is set
6	Flips the sprite vertically when the bit is set
7	Draws the background and window colors 1 to 3 over the sprite when the bit is set

Bit 4 selects one of the two sprite palettes. When the bit is zero, the first sprite palette is used; otherwise, the second palette is used.

Bit 5 and 6 flip (mirror) the sprite horizontally and vertically, respectively. This is a handy option to save on the number of tiles used for characters, because it avoids using separate tiles for the character facing left and right. Figure 7-8 shows an 8×16 sprite flipped in four different ways.

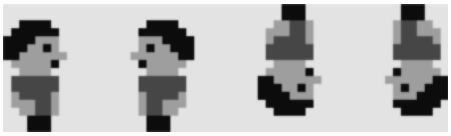


Figure 7-8: Flipping a sprite in different ways

From left to right, the four orientations are no flip, horizontal flip, vertical flip, and both flips. Combining both flips is equivalent to a 180-degree rotation.

By default, the sprites display over the background and window. Sprites are opaque except when pixels use color index 0, which lets the background and window show through. When bit 7 is set, the background and window display over the sprites. Figure 7-9 illustrates how this works.



Figure 7-9: The draw priority of a sprite

The house is part of the background and has two windows with color index 0, so what's *behind* shows through them. On the left, no sprites are visible, just the house. In the middle, a character sprite has bit 7 of its flags cleared and thus appears in front of the house. On the right, the sprite has bit 7 of its flags set and thus appears behind the house but is partially visible through the window.

Limitations

There are three limitations to sprites that you should be aware of. First is the number of sprites you can store. Because each sprite takes 4 bytes, there can be only 40 sprites in OAM at any given time. We'll see how to partially compensate for this limitation in Chapter 12.

Second, at most 10 sprites can display per LCD line. The PPU identifies the first 10 sprites in OAM that are on each line and ignores any additional sprites on the line. This limitation affects several commercial games, resulting in sprites flickering on the screen.

Finally, you can't make a single sprite invisible through flags. Instead, you have to move that sprite off the screen. The issue isn't too difficult to address, but it means that when you need to make a sprite invisible temporarily, you must save its position.

Also, keep in mind that it's better to move the sprite above or below the screen (using an ordinate equal to 0 or higher than 168) rather than to the sides (using an abscissa equal to 0 or higher than 168), because sprites that aren't visible on the LCD display will still count toward the maximum number of sprites allowed per line (10).

A Sprite Sample

Let's get some practice using sprites. You can follow along using the *sprites* sample.

First we'll set the parameters for all sprites, starting with the two sprite palettes, as shown in Listing 7-15.

```
    ld a, %11100100
    ld [rBGP], a
    ld [rOBP0], a
    ld a, %00011011
    ld [rOBP1], a
```

Listing 7-15: Configuring the palettes

We set the first palette to the same value as the background palette. The second palette is set to a different value, but it is not actually referenced by any sprite in the sample.

Next, we set the `rLCDC` flags to enable and configure the sprites, as shown in Listing 7-16.

```
    ld a, LCDCF_ON | LCDCF_WIN9C00 | LCDCF_WINON | LCDCF_BG8800 | \
          LCDCF_BG9800 | LCDCF_OBJ16 | LCDCF_OBJON | LCDCF_BGON
```

Listing 7-16: Setting the flags

The `LCDCF_OBJON` flag enables the sprites and the `LCDCF_OBJ16` flag makes them 8×16 pixels.

We'll set the sprite attributes inside the `UpdateSample` function. Although it's not really necessary in this sample (because the sprite attributes won't change over time), in a real-world situation you'd update the sprites frequently, and the update function is the right place to do this. Because the OAM is mostly inaccessible outside the vblank, we need to set the sprite attributes after `halt`, within the duration of the vblank (1.08 ms).

We use simple load instructions to set the attributes: one instruction to load the attribute value in `a`, and another to load `a` into an OAM attribute. These two instructions have been abstracted by a new macro, `copy`, shown in Listing 7-17. The macro is defined in *utils.rgbinc*.

```
; copy \2 into \1 through (a)
; example: copy [$FF84], 10
macro copy
    ld a, \2
    ld \1, a
endm
```

Listing 7-17: The copy macro

This macro covers the cases when two loads are necessary to move a value from a source to a destination, using `a` as an intermediary. We use it to set up two sprites over a few lines of code, as illustrated in Listing 7-18.

```
    ; set up the first sprite
def SPRITE_0_ADDRESS equ (_OAMRAM)
    copy [SPRITE_0_ADDRESS + OAMA_X], 16
    copy [SPRITE_0_ADDRESS + OAMA_Y], 32
    copy [SPRITE_0_ADDRESS + OAMA_TILEID], 16
    copy [SPRITE_0_ADDRESS + OAMA_FLAGS], OAMF_PAL0

    ; set up the second sprite
def SPRITE_1_ADDRESS equ (_OAMRAM + sizeof_OAM_ATTRS)
    copy [SPRITE_1_ADDRESS + OAMA_Y], 80
    copy [SPRITE_1_ADDRESS + OAMA_X], 80
    copy [SPRITE_1_ADDRESS + OAMA_TILEID], 0
    copy [SPRITE_1_ADDRESS + OAMA_FLAGS], OAMF_PAL0 | OAMF_XFLIP
```

Listing 7-18: Defining the sprites

The code defines two constants, `SPRITE_0_ADDRESS` and `SPRITE_1_ADDRESS`, that hold the addresses of the first two sprites in OAM. The attributes are filled in for each sprite. The first sprite is a front-facing character positioned in the upper-left corner of the screen. The second sprite is a right-facing character, but we flip it horizontally to make it face left instead. It is located close to the center of the screen.

If you compile and run the sample, the two sprites should display as expected. However, you will likely get some visual corruption, as shown in Figure 7-10. (If it isn't visible right away, try running the sample a few more times.)

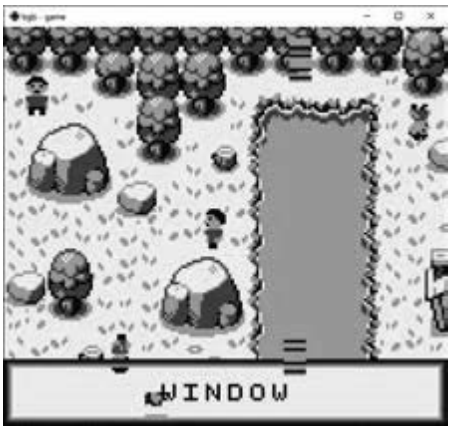


Figure 7-10: Visual corruption

Some graphics tiles appear randomly over the background and window. The issue is that the OAM, the memory that contains the sprite attributes, is not initialized by default. That means the 38 sprites that we're not using have attributes set to random values, causing them to display unexpectedly all over the screen.

To fix this visual corruption, we must initialize the OAM to position all sprites outside the screen by default. We do this with the `InitOAM` macro (Listing 7-20), which is included in the *sample.rgbasm* file in the *sprites_fixed* sample.

```
macro InitOAM
    ld c, OAM_COUNT
    ld hl, _OAMRAM + OAMA_Y
    ld de, sizeof_OAM_ATTRS
    .init_oam\@
        ld [hl], 0
        add hl, de
        dec c
        jr nz, .init_oam\@
endm
```

Listing 7-19: The `InitOAM` macro

This macro is a loop iterating over the OAM, setting the Y attribute of all 40 sprites to 0. It starts by initializing the registers used in the macro. First, `c` is set to `OAM_COUNT` (40). This acts as the loop index. Next, `hl` is set to point to the Y attribute of the first sprite in the OAM. Finally, `de` is set to `sizeof_OAM_ATTRS`, which is the number of bytes for the attributes of a single sprite (4).

The loop spans the remainder of the macro. For each iteration, the byte pointed to by `hl` (the Y attribute of one of the sprites in OAM) is set to zero. Then, `hl` is incremented by 4 so that it points to the next sprite's Y attribute. The register `de` holds the value 4, because the only 16-bit `add` instruction takes registers as parameters, not literals. Finally, the code jumps back to the start of the loop until all iterations are done.

Once the loop is completed, all the sprites' Y attributes are equal to zero, moving them outside the LCD display area and removing the graphical artifacts from Figure 7-10. You don't need to initialize any of the other attributes (X, tile index, and control flags), because you can initialize them before the sprites actually get used.

WARNING

As a rule of thumb, it's best to assume that memory is not initialized. BGB seems to do the right thing here by having random values in the OAM, just as the real hardware would, but some emulators might mistakenly zero-clear some of the memory areas, including the OAM. If you make the wrong assumptions about the state of the memory, you might find that your program works fine on an emulator but not on real hardware.

Finally, we use the `InitOAM` macro in the `InitSample` function, just before `LoadGraphicsDataIntoVRAM` (see Listing 7-20).

```
    ; init graphics data
    InitOAM
    LoadGraphicsDataIntoVRAM
```

Listing 7-20: Invoking the `InitOAM` macro

Now when you compile and run the sample, it should look like Figure 7-11.



Figure 7-11: A sprite sample

All the visual corruption is gone.

Summary

In this chapter, you learned how to leverage the high-level graphics features of the DMG. You should now be able to initialize and scroll the background as well as enable and move the window around the screen. Finally, you learned how to set up the OAM to display sprites.

Combined with what you learned in Chapter 6, you should now have a strong understanding of how to display custom graphics on the console. There is one more topic left with regard to sprites: the *DMA transfer*, which is a different method for copying data to the OAM. I'll cover this in Chapter 12 along with other advanced graphics topics.

Now we're ready to switch our focus to the input. The next chapter introduces the joypad, which allows us to interact with the DMG.

8

THE JOYPAD

The joypad is the DMG's sole input device, a crucial part of any program. In this chapter, you'll begin by learning how to *poll* the joypad to check the current status of its keys. Then you'll put this to use in a sample that scrolls the background and toggles the window based on input from the d-pad and buttons.

Next, you'll learn how to detect higher-level events, such as button presses and button releases. Finally, you'll learn about the joypad interrupt, a specific type of interrupt designed to respond to user input so you don't have to constantly poll the input state in your main loop. While you probably won't use this interrupt often in your programs due to its awkward behavior, I cover it here for the sake of completeness.

Input Polling

Input polling lets you check for user input at arbitrary intervals. The DMG has eight keys: UP, DOWN, LEFT, RIGHT, A, B, START, and SELECT. You can retrieve the status of these keys through an I/O register, `rP1`. Because there are eight keys and there are 8 bits in `rP1`, you might expect that each bit corresponds to a key, but that's not how it works. Instead, you need to poll the status of four keys at a time. Figure 8-1 illustrates the bits of `rP1`.

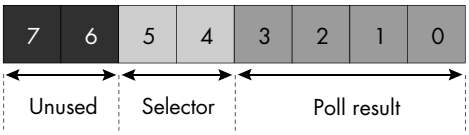


Figure 8-1: The bits of the `rP1` register

The top two bits, 6 and 7, are unused. The next two bits, 4 and 5, are used as a selector. Depending on the value written to the selector, the DMG will poll the status of either the d-pad (UP, DOWN, LEFT, and RIGHT) or the buttons (A, B, START, and SELECT). The low bits, from 0 to 3, hold the result of the input poll.

Listing 8-1 shows how to retrieve the d-pad status in code.

```
ld a, P1F_GET_DPAD
ldh [rP1], a
ldh a, [rP1]
ldh a, [rP1]
```

Listing 8-1: Polling the status of the d-pad

The first two lines write the d-pad selector value to `rP1` to have the DMG poll the d-pad status. Then, the `rP1` register is read twice with `ldh`. The first `ldh` is just a wait mechanism, which is necessary because the `rP1` register's value takes some time to stabilize. The second `ldh` is when the actual d-pad status is read into `a`.

You use similar code to retrieve the status of the buttons, as shown in Listing 8-2.

```
ld a, P1F_GET_BTN
ldh [rP1], a
ldh a, [rP1]
ldh a, [rP1]
ldh a, [rP1]
ldh a, [rP1]
ldh a, [rP1]
ldh a, [rP1]
```

Listing 8-2: Polling the status of the buttons

First, the button selector value is written to `rP1`. Then, the register is read six times. The first five `ldh` instructions are, again, just a waiting mechanism to let the polled value stabilize. It's unclear why this takes longer for the buttons than for the d-pad, but the extra `ldh` instructions are required. The last `ldh` reads the stabilized value into `a`.

After reading all the input, we must disable the input polling by writing to the `rP1` selector bits again, as shown in Listing 8-3.

```
ld a, P1F_GET_NONE
ldh [rP1], a
```

Listing 8-3: Disabling input polling

Having to poll for the status of the d-pad and buttons separately is not very convenient. It's best to poll only once per frame and gather the results into a single byte for later reference. Listing 8-4 shows a `GetPadInput` macro that does just that.

```
macro GetPadInput
    ; read the d-pad
    ld a, P1F_GET_DPAD
    ldh [rP1], a
    ldh a, [rP1]
    ldh a, [rP1]

    ; put the d-pad input into (b)'s high nibble
❶   swap a
    and a, $F0
    ld b, a

    ; read the buttons
    ld a, P1F_GET_BTN
    ldh [rP1], a
    ldh a, [rP1]
    ldh a, [rP1]
    ldh a, [rP1]
    ldh a, [rP1]
    ldh a, [rP1]
    ldh a, [rP1]

    ; merge the buttons' input into (b)'s low nibble
❷   and a, $0F
    or a, b
    ld b, a

    ; disable input polling
    ld a, P1F_GET_NONE
    ldh [rP1], a
endm
```

Listing 8-4: Polling the input into b

The macro polls the input of both the d-pad and the buttons and stores the results into register `b`. You've already seen the code for reading the d-pad and buttons, but there are two new pieces of code here that handle storing the results into `b`.

First, the d-pad's poll result is put into `b`'s high nibble ❶. The instruction `and a, $F0` applies a mask to make sure no bits are set in the low nibble, which is where we'll store the result for the buttons.

The buttons' poll result is then or'ed into `b`'s low nibble ❷. Again, we apply a mask to the polling result with `add a, $0F`, this time to avoid overwriting

the high nibble in `b`, which contains the d-pad's poll result. At the end of the macro, `b`'s bits represent the complete joypad status.

In `rP1`, input is pulled low, which means that when a key is held, the corresponding bit is equal to zero; otherwise, the bit is equal to one. This book will use that representation as well.

NOTE *Use of the term* held *here is intentional. The term* pressed *has a different meaning, introduced later in this chapter.*

Once the joypad status is in an 8-bit register, you can use the `and` and `bit` instructions to test whether a given key is held or not. Listing 8-5 shows flags and bits from *hardware.rgbinc* that come in handy when testing keys.

```
DEF PADF_DOWN   EQU $80
DEF PADF_UP     EQU $40
DEF PADF_LEFT   EQU $20
DEF PADF_RIGHT  EQU $10
DEF PADF_START  EQU $08
DEF PADF_SELECT EQU $04
DEF PADF_B      EQU $02
DEF PADF_A      EQU $01

DEF PADB_DOWN   EQU $7
DEF PADB_UP     EQU $6
DEF PADB_LEFT   EQU $5
DEF PADB_RIGHT  EQU $4
DEF PADB_START  EQU $3
DEF PADB_SELECT EQU $2
DEF PADB_B      EQU $1
DEF PADB_A      EQU $0
```

Listing 8-5: Input flags and bits for testing whether keys are held

The constants starting with `PADF_` are flags. The ones starting with `PADB_` are bits. You can use the flags with the `and` instruction and the bits with the `bit` instruction. Listing 8-6 shows a few examples of using `and` and `bit` to check whether some keys are held. A positive result would raise the zero flag.

```
    and a, PADF_LEFT       ; Is LEFT held in a?
    and a, PADF_A | PADF_B ; Are both A and B held in a?

    bit PADB_UP, a         ; Is UP held in a?
    bit PADB_B, b          ; Is B held in b?
```

Listing 8-6: Checking whether keys are held

The `and` instruction works only with `a`, but you can use it to test multiple keys at the same time. The `bit` instruction has the advantage of working with any 8-bit register, like `a` and `b` in these examples, but it can test only one flag at a time.

Declaring Constants

Before moving on to this chapter's samples, I'm going to introduce a command in the rgbasm assembly language dialect that's often used when declaring constants: `rsset`. The samples from this chapter on will regularly use variables, and we'll declare their offsets in HRAM and WRAM with `rsset`. Listing 8-7 shows a simple example of its use.

```
rsset $C000
def POSITION_X       rb 1
def POSITION_Y       rb 2
def LIFE_COUNT       rw 1
def SCORE            rl 1
def BACKPACK_WEIGHT  rb 1
```

Listing 8-7: Using the `rsset` command when declaring constants

The `rsset` command sets an internal counter in rgbasm. Here the counter is set to `$C000`, which happens to be the start of the WRAM. Each of the lines after `rsset` declares a constant equal to the counter's current value and then increments the counter according to the size provided at the end of the line.

There are three ways of specifying sizes, using the directives `rb` (1 byte), `rw` (2 bytes, or 1 word), and `rl` (4 bytes, or 1 long). For example, `rb 3` would mean 3 bytes, `rw 2` would mean 4 bytes (2 words), and `rl 4` would mean 16 bytes (4 longs).

The previous piece of code is equivalent to Listing 8-8, which uses `equ` to declare the constants.

```
def POSITION_X       equ ($C000)
def POSITION_Y       equ ($C001)
def LIFE_COUNT       equ ($C003)
def SCORE            equ ($C005)
def BACKPACK_WEIGHT  equ ($C009)
```

Listing 8-8: An alternative way of declaring the constants

When using constants to define contiguous locations in memory, it's far more convenient and robust to use `rsset` than `equ`. In Listing 8-7, any constant can be added, removed, or changed to a different size without impacting the other constants declared around it. Each constant's value will be properly recomputed during the next compilation. In Listing 8-8, changing a constant would require updating all the constants that are defined after it, which is quite error-prone.

Another good use of `rsset` is to declare data structures, like you would in C. You can also use `rsreset` for this purpose. For example, Listing 8-9 shows how the OAM attributes are defined in *hardware.rgbinc*.

```
rsreset
def OAMA_Y              rb  1   ; y pos
def OAMA_X              rb  1   ; x pos
def OAMA_TILEID         rb  1   ; tile id
```

```
def OAMA_FLAGS          rb  1   ; flags (see below)
def sizeof_OAM_ATTRS    rb  0
```

Listing 8-9: The OAM attributes structure

The `rsreset` command is equivalent to `rsset 0`. It basically sets the internal counter to zero.

Note that the name of the last constant in Listing 8-9 starts with `sizeof_`. That constant stores the number of bytes in the structure, which is convenient for writing formulas involving this structure. For example, we used `sizeof_OAM_ATTRS` in Chapter 7 to compute the address of the second sprite in the OAM. The samples in this book will follow the convention of adding this `sizeof_` constant at the end of each structure.

Listing 8-9 shows an alternative way of defining the constants in Listing 8-10.

```
def OAMA_Y              equ 0   ; y pos
def OAMA_X              equ 1   ; x pos
def OAMA_TILEID         equ 2   ; tile id
def OAMA_FLAGS          equ 3   ; flags (see below)
def sizeof_OAM_ATTRS    equ 4
```

Listing 8-10: Defining the OAM attributes structure with equ

We can use these offsets to get the address of the right attribute from the starting address of a sprite in OAM. That's exactly what we did in the *sprites* sample in the previous chapter, using code like the following:

```
def SPRITE_0_ADDRESS equ (_OAMRAM)
    copy [SPRITE_0_ADDRESS + OAMA_X], 16
    copy [SPRITE_0_ADDRESS + OAMA_Y], 32
    copy [SPRITE_0_ADDRESS + OAMA_TILEID], 16
    copy [SPRITE_0_ADDRESS + OAMA_FLAGS], OAMF_PAL0
```

Here, `SPRITE_0_ADDRESS` is the starting address of the sprite, to which we add `OAMA_X`, `OAMA_Y`, `OAMA_TILEID`, and `OAMA_FLAGS` to compute the addresses of the sprite's X position, Y position, tile index, and control flags, respectively.

Scrolling the Background

In this section, you'll put what you've just learned to practice by writing a sample that scrolls the background based on the d-pad input. The sample will also allow you to open and close the window with the A button.

To follow along, use the *input_bg* sample in the GitHub repository.

Handling Input

Input handling is the process of retrieving, storing, and using the status of the joypad in our programs. Let's start by adding a structure and some

macros to support this. Because we'll use input in almost all the remaining samples in the book, we'll add the code to *utils.rgbinc*.

First, we declare a structure to hold the joypad input, as shown in Listing 8-11.

```
rsreset
def PAD_INPUT_CURRENT   rb 1
def sizeof_PAD_INPUT    rb 0
```

Listing 8-11: Declaring a joypad input structure

The structure is only 1 byte for now, but it will grow soon. We'll use the byte to hold the latest input state. The high nibble will hold the d-pad's status, while the low nibble will be for the buttons.

Next, we add a macro that initializes an input structure passed as a parameter, as shown in Listing 8-12.

```
macro InitPadInput
    ld a, $FF
    ld [\1 + PAD_INPUT_CURRENT], a
endm
```

Listing 8-12: Initializing a joypad input structure

The `InitPadInput` macro initializes the byte to `$FF` to indicate that none of the keys are held.

We then add a second macro, shown in Listing 8-13, that updates a structure with the latest joypad status.

```
macro UpdatePadInput
    ; read the d-pad
    ld a, P1F_GET_DPAD
    ldh [rP1], a
    ldh a, [rP1]
    ldh a, [rP1]

    ; put the d-pad input into (b)'s high nibble
    swap a
    and a, $F0
    ld b, a

    ; read the buttons
    ld a, P1F_GET_BTN
    ldh [rP1], a
    ldh a, [rP1]
    ldh a, [rP1]
    ldh a, [rP1]
    ldh a, [rP1]
    ldh a, [rP1]
    ldh a, [rP1]
```

```
    ; merge the buttons' input with (b)'s high nibble
    and a, $0F
    or a, b

    ; store the current input
    ld [\1 + PAD_INPUT_CURRENT], a

    ; disable input polling
    ld a, P1F_GET_NONE
    ldh [rP1], a
endm
```

Listing 8-13: Updating a joypad input structure

This macro is very similar to the one in Listing 8-4, and the poll result gets stored into the structure's byte.

Finally, Listing 8-14 shows the last macro, which tests whether a set of keys is held or not.

```
macro TestPadInput_HeldAll
    ld a, [\1 + PAD_INPUT_CURRENT]
    and a, \2
endm
```

Listing 8-14: Checking whether joypad input keys are held

This macro takes a structure and an input mask and raises the zero flag if all keys in the mask are held. These macros might seem a little abstract for now, but they will become clearer over the next sections, when we actually use them.

Checking WRAM Usage

This section introduces the variables used in the *input_bg* sample, as well as a technique to compute the WRAM usage at compile time. The variables are declared using `rsset`, as shown in Listing 8-15.

```
rsset _RAM

; input struct
def WRAM_PAD_INPUT          rb sizeof_PAD_INPUT

; background scrolling
def WRAM_BG_SCX             rb 1
def WRAM_BG_SCY             rb 1

; window control
```

```
def WRAM_WIN_ENABLE_FLAG    rb 1

def WRAM_END                rb 0
```

Listing 8-15: Declaring the WRAM variables

All the variables are placed contiguously at the start of the WRAM (`_RAM` equals `$C000`). The `WRAM_PAD_INPUT` variable will hold the joypad input. The `sizeof_PAD_INPUT` constant, computed automatically from the input structure definition in *utils.rgbinc*, comes in handy here, because it tells us exactly the number of bytes we need to reserve in WRAM to hold all the variables necessary for input. The actual size of the structure, or even its content, does not matter. As long as enough space is reserved in WRAM, we can leverage the input macros to interact with the structure.

Next, there are two additional variables, `WRAM_BG_SCX` and `WRAM_BG_SCY`, to hold the background scrolling coordinates. We add one last variable, `WRAM_WIN_ENABLE_FLAG`, to control whether the window is enabled or not.

The last line is not strictly necessary for the program to run, but we can use it to perform some compile-time checks, as shown in Listing 8-16.

```
def WRAM_USAGE              equ (WRAM_END - _RAM)
println "WRAM usage: {d:WRAM_USAGE} bytes"
assert WRAM_USAGE <= $2000, "Too many bytes used in WRAM"
```

Listing 8-16: Checking the WRAM usage

The `WRAM_USAGE` constant computes the total WRAM usage, which we print out using `println`, a compile-time printing functionality of rgbasm. For example, compiling *sample.rgbasm* should output the following:

```
> rgbasm -Werror -Weverything -o sample.o sample.rgbasm
WRAM usage: 4 bytes
```

This allows us to track the WRAM consumption. The `println` statement is followed by an *assert*, which checks a condition at compile time (or link time if this is not possible at compile time) and generates an error if the condition is not met. Here, the condition is that the WRAM consumption must be below 8KiB. If the condition is not met, the tool displays the error message, generates an error, and aborts the assembly stage.

To trigger the failure artificially, you can add a dummy variable of 8KiB in the WRAM (for example, `def WRAM_DUMMY rb $2000`). Here's what you should see when you recompile and run the sample:

```
> rgbasm -Werror -Weverything -o sample.o sample.rgbasm
WRAM usage: 8196 bytes
ERROR: sample.rgbasm(56):
    Assertion failed: Too many bytes used in WRAM
error: Assembly aborted (1 error)!
```

If you try this, make sure to revert the change afterward.

You can use asserts any time you want to check an assumption at compile time, such as the value of a constant. When programming for the DMG, it's important to keep track of various statistics, including memory usage. This is because, unlike in modern operating systems, there are no safety nets on the DMG. If you use too much memory, it won't complain, and issues are likely to pop up. I highly recommend using asserts to catch potential problems.

NOTE *There are additional options available for asserts, such as static asserts, which get evaluated only at compile time (evaluation is never deferred to the linkage step). Read the rgbasm documentation for more details.*

Initializing the WRAM

When the DMG boots, the WRAM is filled with random values. If we don't initialize our WRAM variables, the sample might behave differently every time we run it. For that reason, the WRAM variables are all initialized in `InitSample`, as shown in Listing 8-17, to make sure they have known values before the main loop starts.

```
InitSample:
    ; init the WRAM state
    InitPadInput WRAM_PAD_INPUT
    xor a
    ld [WRAM_BG_SCX], a
    ld [WRAM_BG_SCY], a
    ld [WRAM_WIN_ENABLE_FLAG], a
```

Listing 8-17: WRAM initialization

The input initialization macro, `InitPadInput`, initializes the variables inside the joypad input structure. We then initialize all the other WRAM variables to zero.

Updating the Display and Input

We can now move on to the update function, which scrolls the background and toggles the window based on the joypad input. Unlike in the samples in the previous chapters, in this case the work done in the update function is explicitly split into two parts: vblank (for graphics handling) and logic (for the raw program logic).

Listing 8-18 shows the vblank part of the update function.

```
UpdateSample:
    halt

    ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
    ; vblank
    ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
```

```
    ; set the background position
    copy [rSCX], [WRAM_BG_SCX]
    copy [rSCY], [WRAM_BG_SCY]

    ; toggle the window on/off
    ldh a, [rLCDC]
    and a, ~LCDCF_WINON
    ld hl, WRAM_WIN_ENABLE_FLAG
    or a, [hl]
    ldh [rLCDC], a
```

Listing 8-18: The vblank section of the `UpdateSample` function

As usual, the function starts with `halt`, to synchronize it with the vblank. This is important because the following instructions must be executed during the vblank to avoid graphical glitches.

Next, it updates the background scrolling based on the `WRAM_BG_SCX` and `WRAM_BG_SCY` variables. Finally, the last few instructions toggle the window on or off. The `WRAM_WIN_ENABLE_FLAG` will be equal to either `LCDCF_WINON` or `LCDCF_WINOFF`, which are flags used to set bit 5 of the `rLCDC` register. The code merges the value of `WRAM_WIN_ENABLE_FLAG` into `rLCDC`, without influencing any of the other bits of the register.

Once the vblank-critical code executes, the regular logic of the program, shown in Listing 8-19, can run.

```
    ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
    ; logic
    ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

    UpdatePadInput WRAM_PAD_INPUT

    ; d-pad check
    TestPadInput_HeldAll WRAM_PAD_INPUT, PADF_LEFT
    jr nz, .left_checked
        ld hl, WRAM_BG_SCX
        dec [hl]
    .left_checked

    TestPadInput_HeldAll WRAM_PAD_INPUT, PADF_RIGHT
    jr nz, .right_checked
        ld hl, WRAM_BG_SCX
        inc [hl]
    .right_checked

    TestPadInput_HeldAll WRAM_PAD_INPUT, PADF_UP
    jr nz, .up_checked
        ld hl, WRAM_BG_SCY
        dec [hl]
    .up_checked
```

```
    TestPadInput_HeldAll WRAM_PAD_INPUT, PADF_DOWN
    jr nz, .down_checked
        ld hl, WRAM_BG_SCY
        inc [hl]
    .down_checked

    ; window toggle
    TestPadInput_HeldAll WRAM_PAD_INPUT, PADF_A
    jr nz, .window_toggle_done
        ld a, [WRAM_WIN_ENABLE_FLAG]
        cpl
        and a, LCDCF_WINON
        ld [WRAM_WIN_ENABLE_FLAG], a
    .window_toggle_done
```

Listing 8-19: The program logic section of the `UpdateSample` function

Here, the function updates the WRAM variables based on the joypad input. Note that there is no need to wait for the vblank to end before executing the program logic. Code that touches graphics must be executed within the vblank period, but the rest of the code can be executed anytime.

We start by updating the joypad input structure using the `UpdatePadInput` macro. Next, the `TestPadInput_HeldAll` macro checks the status of the d-pad's LEFT button. Holding the LEFT button raises the zero flag and updates the WRAM variable for horizontal scrolling accordingly. If the LEFT button is not held, the program jumps to the `.left_checked` label, bypassing the WRAM variable update. The following lines similarly check the other buttons (RIGHT, UP, and DOWN).

The last piece of code handles the window toggling by testing whether the A button is held. It first loads the current value of `WRAM_WIN_ENABLE_FLAG`, which will be either `LCDCF_WINON` (`%00100000`) or `LCDCF_WINOFF` (`%00000000`), into the `a` register. Next, the `cpl` instruction inverts all the bits in the `a` register. Then, `and a, LCDCF_WINON` ensures that the final value is either `LCDCF_WINON` or `LCDCF_WINOFF` by clearing all the bits but bit 5. Finally, the value of `a` is saved back to `WRAM_WIN_ENABLE_FLAG`.

Now the sample is ready to compile and run in BGB. The background scrolling should work as expected, but you may notice that something is wrong when holding the A button to toggle the window on and off. The problem is that the window toggling code continues to execute as long as we hold the A button, causing the window to flicker until we release the button. It's difficult for anyone to hold a button for just a single frame. To avoid this, we need code that detects button *presses* instead of checking whether the button is held.

Press and Release

In this section, we'll fix the window toggling issue by enhancing the joypad input mechanics to detect key presses and releases. You can follow along using the *input_window* sample on GitHub.

Expanding the Input Structure

First, we'll expand the input structure as shown in Listing 8-20, because we need more variables to track and store the state of the input.

```
rsreset
def PAD_INPUT_CURRENT   rb 1
def PAD_INPUT_PREVIOUS  rb 1
def PAD_INPUT_PRESSED   rb 1
def PAD_INPUT_RELEASED  rb 1
def sizeof_PAD_INPUT    rb 0
```

Listing 8-20: The expanded input structure

So far, we've stored only the current status of the input (`PAD_INPUT_CURRENT`). But now, the previous status (`PAD_INPUT_PREVIOUS`), the pressed status (`PAD_INPUT_PRESSED`), and the released status (`PAD_INPUT_RELEASED`) are part of the structure as well. The sole purpose of the previous status is to allow us to compute the pressed and released statuses.

We need to initialize these new variables in the initialization macro. Listing 8-21 shows the updated code in *utils.rgbinc*.

```
macro InitPadInput
    ld a, $FF
    ld [\1 + PAD_INPUT_CURRENT], a
    ld [\1 + PAD_INPUT_PREVIOUS], a
    ld [\1 + PAD_INPUT_PRESSED], a
    ld [\1 + PAD_INPUT_RELEASED], a
endm
```

Listing 8-21: Initializing the expanded input structure

The same convention applies to all the keys: if they are *activated* (held, pressed, or released) they have their corresponding flag set to zero, and otherwise it's set to one. By default, we assume none of the keys are activated, and thus we initialize every state-tracking byte to `$FF` (all bits are one).

Updating the Input Structure

Now that we have new variables in the input structure, we need to update them to reflect the latest joypad status. Let's see how this is done, starting with Listing 8-22.

```
macro UpdatePadInput
    ; store the previous input into the struct and (c)
    ld a, [\1 + PAD_INPUT_CURRENT]
    ld [\1 + PAD_INPUT_PREVIOUS], a
    ld c, a
```

Listing 8-22: Saving the previous input

At the very start of the update macro, before polling the latest current status, we copy the current status into the previous status variable. For convenience, we then store the previous status inside register `c`. We'll use this to compute the pressed status and released status.

Listing 8-23 shows a minor change to the current status polling for the buttons.

```
    ; merge the buttons' input into (b)'s low nibble
    and a, $0F
    or a, b
    ld b, a
```

Listing 8-23: Minor changes in the polling procedure

The load instruction added here stores the polled status into register `b` so that we can use it in later computations.

Finally, we compute the pressed status and released status based on the previous status and current status. If a key was not held before the poll and is held after, then we consider it *pressed*. Likewise, if the key was held before the poll and is not held after, then we consider it *released*. By definition, you cannot press and release a key at the same time. Listing 8-24 shows the code to compute both the pressed status and the released status.

```
    ; compute and store pressed buttons
    ld a, c
    cpl
    or a, b
    ld [\1 + PAD_INPUT_PRESSED], a

    ; compute and store released buttons
    ld a, b
    cpl
    or a, c
    ld [\1 + PAD_INPUT_RELEASED], a
```

Listing 8-24: Computing the pressed and released status of the buttons

At that point in the macro, the `c` register contains the previous polled status and the `b` register contains the current polled status. The macro uses those registers in combination with `cpl` and `or` instructions to compute the pressed and released status.

Adding Test Macros

The final update to the input system is to add new test macros for pressed and released keys. The samples will use those macros to detect whether given keys are pressed or released. Listing 8-25 shows the first macro, which tests for pressed keys.

```
macro TestPadInput_Pressed
    ld a, [\1 + PAD_INPUT_PRESSED]
    and a, \2
endm
```

Listing 8-25: Checking whether keys are pressed

Listing 8-26 shows the second macro, which tests for released keys.

```
macro TestPadInput_Released
    ld a, [\1 + PAD_INPUT_RELEASED]
    and a, \2
endm
```

Listing 8-26: Checking whether keys are released

The code is the same as for held keys, except that it uses different variables.

Fixing Window Toggling

Now that we can detect key presses and releases, we can fix the sample so that the window toggling works as expected. We can do this with either of the new test macros. The sample uses the released status test macro, as shown in Listing 8-27, but it could just as well use the macro in Listing 8-25.

```
TestPadInput_Released WRAM_PAD_INPUT, PADF_A
```

Listing 8-27: Using the released keys macro to fix window toggling

None of the code using the input structure and macros (for WRAM variable declaration, initialization, and d-pad checks) changed when we enhanced the structure. Everything is updated automatically during the next compilation, because the internals of the input system are hidden inside the structure and macros. This kind of abstraction of the inner workings of a system greatly facilitates code maintenance.

You can confirm that the window toggling issue described earlier is fixed by compiling the code for the *input_window* sample and running it in BGB yourself.

Going Further

You could expand the input structure and macros further to suit your program's needs. For example, you could add a macro to detect if *any* of a set of keys is pressed, which you could use to decide whether to play a walking

animation (any of the d-pad keys is held) or an idle animation (no d-pad key is held).

Key repeats are another nice feature. When a key is held, the game can automatically generate key press events every few frames. This is particularly helpful for puzzle games like *Tetris*. For example, instead of repeatedly pressing the RIGHT key to move a piece to the right side of the board, the player can simply hold down the key, and the game will generate RIGHT key presses at given intervals.

You can expect to use `InitPadInput` only once in a program, most likely during initialization. There's rarely a need to use it at several different places. Likewise, you can expect to use `UpdatePadInput` only once per frame, somewhere in the main loop. If you need to use it in more than one location, turn it into a function.

The Joypad Interrupt

To close out this chapter, we'll take a look at an interrupt associated with the joypad. The joypad interrupt is designed to notify the CPU when the state of the joypad changes. In theory it could be used as an alternative to polling the input state, but I highly recommend against using it for that purpose because it has some major drawbacks. First, the interrupt can trigger at any time during the frame, which does not work well with the main loop mechanics. Second, it might trigger multiple times for the same key press, due to the button bouncing issue mentioned earlier. Last but not least, the interrupt does not always allow us to differentiate exactly which key generated it.

This section presents the *input_interrupt* sample that handles joypad interrupts. It updates a counter to illustrate the issue of key presses being registered multiple times. It also shows the status of the four input bits from the last interrupt, so that we can track which key was held. It should look like Figure 8-2 once run.



Figure 8-2: The input_interrupt *sample*

This sample uses a single tilemap, *joypad.tlm*, to keep the graphics simple, because they're not the focus here.

Declaring the Variables

We need several WRAM variables to track the state of the sample, as shown in Listing 8-28.

```
rsset _RAM

def WRAM_IS_VBLANK      rb 1

def WRAM_INPUT_COUNTER  rb 1

def WRAM_HUNDREDS_TILE  rb 1
def WRAM_TENS_TILE      rb 1
def WRAM_UNITS_TILE     rb 1

def WRAM_P1_BITS        rb 4

def WRAM_END            rb 0
```

Listing 8-28: Sample WRAM variables

First we have the previously mentioned counter, `WRAM_INPUT_COUNTER`, which tracks the number of key presses. Then there are three counter tile indices (`WRAM_HUNDREDS_TILE`, `WRAM_TENS_TILE`, and `WRAM_UNITS_TILE`), because the counter is represented as a three-digit number in the middle of the screen. Finally, `WRAM_P1_BITS` is a 4-byte variable, with each byte holding the tile index for one of the bits displayed on the screen. We'll come back to `WRAM_IS_VBLANK` in a bit.

Initializing the Variables

Listing 8-29 shows the WRAM variable initialization.

```
InitSample:
    ; init the WRAM state
    xor a
    ld [WRAM_INPUT_COUNTER], a
    ld [WRAM_IS_VBLANK], a

    ld a, TILE_UI_NUMBER_0
    ld [WRAM_HUNDREDS_TILE], a
    ld [WRAM_TENS_TILE], a
    ld [WRAM_UNITS_TILE], a

    ld a, TILE_UI_NUMBER_0 + 1
    ld [WRAM_P1_BITS + 0], a
```

```
    ld [WRAM_P1_BITS + 1], a
    ld [WRAM_P1_BITS + 2], a
    ld [WRAM_P1_BITS + 3], a
```

Listing 8-29: Sample WRAM initialization

This initializes the counter to zero and the counter's tile indices to point to the graphics for the character "0". The tile indices that show the bits' statuses are initialized to point to the graphics for the character "1" (remember that released keys correspond to raised bits).

When using the joypad interrupt, we must decide whether the d-pad, the buttons, or both will generate interrupts. We can do this by writing the proper value into the selector of the `rP1` register, as we did when polling for the input state. Listing 8-30 shows this process.

```
    ; choose which keys will trigger an interrupt
    ld a, 0             ; all keys
    ;ld a, P1F_GET_BTN   ; buttons only
    ;ld a, P1F_GET_DPAD  ; d-pad only
    ld [rP1], a
```

Listing 8-30: Selecting the interrupt triggers

I've set the selector so that both the d-pad and the buttons generate interrupts, but you can comment out that line and uncomment one of the lines below it to generate interrupts for the buttons or d-pad only.

Finally, in Listing 8-31, the joypad interrupt flag, `IEF_HILO`, is added to the `rIE` register alongside the vblank interrupt we've been using so far.

```
    ; enable interrupts
    ld a, IEF_HILO | IEF_VBLANK
    ld [rIE], a
    ei
```

Listing 8-31: Adding the interrupt flag

This concludes the initialization of the sample.

Updating the Sample

Now that joypad interrupts are enabled, there is a chance that the `halt` instruction will complete before the vblank, because `halt` completes as soon as *any* interrupt occurs. That means we need to check whether the vblank period has really started and go back to `halt` if that's not the case. To do that, we use an additional variable, the *vblank flag*, to track whether the vblank interrupt function was executed or not. Listing 8-32 shows the new interrupt function.

```
section "vblank_interrupt", rom0[$0040]
    push af
    ld a, 1
```

```
    ld [WRAM_IS_VBLANK], a
    pop af
    reti
```

Listing 8-32: The updated vblank interrupt function

The code raises the vblank flag (`WRAM_IS_VBLANK`) to indicate that the vblank interrupt function has been called. Note that `af` is pushed and popped to keep the register in the state it was in before the interrupt occurred. That's particularly important here because the code relies on `a` being set to zero right before `halt` is executed. In general, any register used in an interrupt should be pushed and popped to avoid any nasty timing-dependent bugs.

Next, a more complex piece of code replaces `halt`, as shown in Listing 8-33.

```
UpdateSample:
    ld hl, WRAM_IS_VBLANK
    xor a
    .wait_vblank
        halt
        cp a, [hl]
        jr z, .wait_vblank
        ld [hl], a
```

Listing 8-33: Using `halt` and the vblank flag

This checks whether the vblank flag is raised. If the flag is raised, it means that the vblank period has started and the remainder of the update function can execute. Otherwise, a different interrupt trigger caused `halt` to complete, and the code should loop and call `halt` again to wait for the vblank.

Although the `WRAM_IS_VBLANK` variable is not shown here, it should be initialized to zero at the start of the program. We'll look at some caveats of using this approach in Chapter 14.

Next, during the vblank, the three digits that represent the counter are updated based on the tile indices in WRAM. Listing 8-34 shows the updated code.

```
    ; update counter's tiles
    ld hl, _SCRN0 + (7 * SCRN_VX_B) + 13
    ld a, [WRAM_HUNDREDS_TILE]
    ld [hli], a
    ld a, [WRAM_TENS_TILE]
    ld [hli], a
    ld a, [WRAM_UNITS_TILE]
    ld [hli], a
```

Listing 8-34: Updating the counter

We use a formula to have `hl` point to the first digit in the tilemap, which is located at row 7 and column 13. The formula consists of adding the tilemap

starting address (`_SCRN0`) to 7 rows of 32 (`SCRN_VX_B`) tiles each and the column number 13. Then, the three digits are updated in order using `hl`.

The same idea applies to update the four digits representing the bits, as shown in Listing 8-35.

```
; update bits' tiles
ld hl, _SCRN0 + (9 * SCRN_VX_B) + 12
ld a, [WRAM_P1_BITS + 0]
ld [hli], a
ld a, [WRAM_P1_BITS + 1]
ld [hli], a
ld a, [WRAM_P1_BITS + 2]
ld [hli], a
ld a, [WRAM_P1_BITS + 3]
ld [hli], a
```

Listing 8-35: Updating the bits

We use the same formula we used for the counter digits here, with the first bit digit at row 9 and column 12.

After updating the graphics, we compute the counter tile indices in WRAM so that we can update the counter digits graphics during the next vblank. Listing 8-36 shows the instructions.

```
ld a, [WRAM_INPUT_COUNTER]

; compute the hundreds into (b)
; (a) is also updated to remove the hundreds
ld b, $FF
.hundreds
    inc b
    sub a, 100
jr nc, .hundreds
add a, 100

; set up the tile id for the hundreds
ld c, a
ld a, TILE_UI_NUMBER_0
add a, b
ld [WRAM_HUNDREDS_TILE], a
ld a, c

; compute the tens into (b)
; (a) is also updated to remove the tens
ld b, $FF
.tens
    inc b
    sub a, 10
jr nc, .tens
add a, 10
```

```
    ; set up the tile id for the tens
    ld c, a
    ld a, TILE_UI_NUMBER_0
    add a, b
    ld [WRAM_TENS_TILE], a
    ld a, c

    ; at this point, (a) contains the units
    ; set up the tile id for the units
    ld b, TILE_UI_NUMBER_0
    add a, b
    ld [WRAM_UNITS_TILE], a
```

Listing 8-36: Computing the tile indices

This loops over the counter to find the values of the hundreds, the tens, and the ones. Then it computes and saves each tile index in the corresponding WRAM variable.

Defining the Interrupt Function

The final piece of the sample is the interrupt function, located at address `$0060`. It updates the counter and the bits' tile indices, as shown in Listing 8-37.

```
section "joypad_interrupt", rom0[$0060]
joypad_interrupt:
    push af
    push hl

    ; counter update
    ld hl, WRAM_INPUT_COUNTER
    inc [hl]

    ; bits' tile indices update
    ld a, [rP1]

    ld hl, WRAM_P1_BITS
    ld [hl], TILE_UI_NUMBER_0 + 1
    bit 0, a
    jr nz, .skip_bit_0
        ld [hl], TILE_UI_NUMBER_0
    .skip_bit_0

    inc hl
    ld [hl], TILE_UI_NUMBER_0 + 1
    bit 1, a
    jr nz, .skip_bit_1
```

```
    ld [hl], TILE_UI_NUMBER_0
.skip_bit_1

inc hl
ld [hl], TILE_UI_NUMBER_0 + 1
bit 2, a
jr nz, .skip_bit_2
    ld [hl], TILE_UI_NUMBER_0
.skip_bit_2

inc hl
ld [hl], TILE_UI_NUMBER_0 + 1
bit 3, a
jr nz, .skip_bit_3
    ld [hl], TILE_UI_NUMBER_0
.skip_bit_3

pop hl
pop af
reti
```

Listing 8-37: The joypad interrupt function

The `af` and `hl` registers are pushed onto the stack at the start and popped back off at the end. The reason for this is the same as that discussed in the previous section: Because the interrupt could occur at any time during the main code execution, it's important to restore any registers to their original values before the end of the interrupt to avoid bugs creeping in.

When a joypad interrupt occurs, the bit corresponding to the input that triggered that interrupt will be pulled low (set to zero) in `rP1`. But because we have only 4 bits for eight keys, it's impossible to differentiate between certain keys when both the d-pad and the buttons can generate interrupts. Button A uses the same bit as RIGHT, B the same as LEFT, SELECT the same as UP, and START the same as DOWN. This limitation of the joypad interrupt makes it unsuitable for programs that need to leverage more than four keys.

Try compiling and running the sample in BGB. When you press any joypad keys, the counter should increase faster than the number of times you press the keys. That's due to the button bouncing issue described earlier causing the interrupt to occur more often than is necessary.

Press A then RIGHT, and notice that the same bit is pulled low in both cases. That's because in Listing 8-30 we specified that all eight keys will generate interrupts, but there are only 4 bits to cover them.

Summary

You can now leverage the joypad to add interaction to your programs. Many of the samples in subsequent chapters use the joypad extensively, so feel free to return to this chapter at any point if you need a refresher.

Polling is the best and only practical method to retrieve the joypad input. It's a reliable way to get the status of all the buttons at specific times in the frame. Using the joypad interrupt is not recommended. Its timing and frequency are messy, and it's impossible to definitively determine which key press caused the interrupt when using more than four keys. I covered it here to be thorough, but we won't use it again in any of this book's samples.

In the next chapter, we'll change gears and look at how to generate sound on the DMG.

Game Boy Coding Adventure (Early Access) © 2025 by Maximilien Dagois

9

AUDIO

In this chapter, you'll learn about the audio capabilities of the DMG. It's impossible to represent sounds in a book, so it's particularly important to run the samples in this chapter and experiment on your own.

We'll start with the basics of the DMG audio system, such as enabling the audio hardware and controlling the sound volume. Then we'll move on to the four channels of the DMG, each of which has its own audio capabilities. Channels 1 and 2 are suitable for melodies and sound effects. Channel 3 is highly customizable, and channel 4 handles noise.

By the end of the chapter, you'll be able to output and control sounds in your programs.

Getting Started

The DMG has four distinct channels for audio. Each channel can produce a sound, so it's possible to play up to four sounds simultaneously. The channels are each controlled using up to five I/O registers, whose usage varies between channels.

The register names follow a particular format. They start with `rNR`, followed by two numbers: the channel number (1 to 4) and the register number for that channel (0 to 4). For example, `rNR13` is the fourth register of channel 1, while `rNR30` is the first register of channel 3.

The sounds the channels produce are mixed together and output to two terminals, which are similar to the left and right audio channels in a stereo system. I'll refer to them as the *left terminal* and *right terminal* in this book. The DMG has two I/O registers to control the sound mixing on the terminals. It has only one speaker, so it produces mono sound, but you can experience stereo sound using earphones. BGB also supports stereo sound.

NOTE *In addition to the four channels we'll discuss, the DMG supports a fifth channel, commonly called Vin (for voltage input). It requires some extra hardware in the cartridge, and no commercial game I know of has ever used it. For those reasons, I won't cover the Vin channel in this book.*

Control Registers

There are three registers dedicated to controlling the sound hardware. They have names like `rNR50`, with the last digit ranging from 0 to 2, but they're not related to a fifth channel.

The first register is `rNR50`, which controls the volume of the sound on each terminal. Table 9-1 summarizes how its bits are used.

Table 9-1: The `rNR50` Bits

Bits	Description
0–2	Controls the volume of the right terminal
3	Enables Vin on the right terminal
4–6	Controls the volume of the left terminal
7	Enables Vin on the left terminal

There are 3 bits allocated per terminal, so the volume for a terminal ranges between 0 (softest) and 7 (loudest). Even at volume 0, a terminal still makes a soft audible sound. The remaining 2 bits in the registers are dedicated to Vin.

The second register is `rNR51`, which controls the distribution of the channels' sound over the terminals. Table 9-2 summarizes the usage of its bits.

Table 9-2: The `rNR51` Bits

Bit	Description
0	Outputs channel 1 sound to the right terminal
1	Outputs channel 2 sound to the right terminal
2	Outputs channel 3 sound to the right terminal
3	Outputs channel 4 sound to the right terminal
4	Outputs channel 1 sound to the left terminal
5	Outputs channel 2 sound to the left terminal
6	Outputs channel 3 sound to the left terminal
7	Outputs channel 4 sound to the left terminal

There are three main combinations of interest: hard pan left (channel sound sent to the left terminal only), hard pan right (channel sound sent to the right terminal only), and center pan (channel sound sent to both terminals). For example, clearing bit 0 and setting bit 4 would hard pan the sound of channel 1 to the left. Setting bit 0 and clearing bit 4 would hard pan the sound of channel 1 to the right. Setting both bit 0 and bit 4 would pan the sound of channel 1 to the center. The same logic applies to the other channels and their respective bits.

The final register is `rNR52`. Its bits are used as described in Table 9-3.

Table 9-3: The `rNR52` Bits

Bits	Description
0	Indicates that channel 1 is playing sound (read-only)
1	Indicates that channel 2 is playing sound (read-only)
2	Indicates that channel 3 is playing sound (read-only)
3	Indicates that channel 4 is playing sound (read-only)
4–6	Unused
7	Enables the sound hardware

The `rNR52` register has two functionalities. First, it has one bit to enable the sound hardware on the DMG. When bit 7 is set to zero, the DMG doesn't make any sound. This can extend the battery life by about 15 percent, so it's worth clearing that bit if a program doesn't leverage any of the channels. Second, `rNR52` has four read-only bits to track whether a channel is playing sound or not. However, those bits are reliable only under certain circumstances, which I'll explain in the relevant sections.

You can set the `rNR50` and `rNR51` registers at any time to create a stereo effect for sounds. For example, you could make the sound move gradually from left to right by adjusting the volume of the two terminals.

Sound Sample

The *ch1_simple* sample is a sound sample with simple graphics, with just a single tilemap to fill the screen. We'll use this sample to test the DMG sound. The familiar Game Boy jingle should play when you press the A button. The jingle consists of two sounds, each of which is a single musical note. Sound 1 plays on channel 1 as soon as you press button A. After a few frames, sound 2 plays on the same channel. This causes sound 1 to stop, because only one sound can play at the same time on a given channel.

WRAM Initialization

For this sample, we need only two variables in WRAM: the joypad input and a counter to introduce a delay between sound 1 and sound 2. The variable declarations are shown in Listing 9-1.

```
def WRAM_PAD_INPUT                  rb sizeof_PAD_INPUT
def WRAM_FRAME_COUNTER              rb 1
```

Listing 9-1: Declaring the variables

We initialize them as shown in Listing 9-2.

```
    InitPadInput WRAM_PAD_INPUT
    copy [WRAM_FRAME_COUNTER], $FF
```

Listing 9-2: Initializing the variables

The counter is set to `$FF`, which disables the counter mechanics in the sample's update function, as we'll see shortly.

Finally, Listing 9-3 shows the DMG sound setup, which is the most relevant part of the initialization.

```
    copy [rNR52], AUDENA_ON
    copy [rNR50], $77
    copy [rNR51], $FF
```

Listing 9-3: Enabling and initializing the DMG audio

First, this code enables the sound hardware bit in `rNR52` to make sure the DMG outputs sound. Next, it sets the volume to the maximum on both terminals by setting `rNR50` to `$77`, with each nibble representing the volume of one terminal. Finally, it raises all bits in `rNR51` to output all sound channels to both terminals.

Make sure to enable the sound hardware through `rNR52` before writing to any other sound register, because writes to those registers are ignored when the hardware is disabled.

The Update Function

Now that we've enabled and initialized the audio hardware, we'll play some sounds in the update function. This function is split into two parts.

The first part, shown in Listing 9-4, triggers sound 1 when you press the A button.

```
    TestPadInput_Pressed WRAM_PAD_INPUT, PADF_A
    jr nz, .no_start_sound_1
        ; play sound 1
        copy [rNR10], $00
        copy [rNR11], $80
        copy [rNR12], $F3
        copy [rNR13], $83
        copy [rNR14], $87
```

```
        ; activate the counter to play sound 2 after a delay
        copy [WRAM_FRAME_COUNTER], 4
    .no_start_sound_1
```

Listing 9-4: Starting sound 1 and activating the counter

First, the code uses the `TestPadInput_Pressed` macro from Listing 8-25 to check whether the A button is pressed.

Next, it starts sound 1 on channel 1 by setting five channel-specific registers. I'll explain what these registers mean in more detail in the next section.

Finally, the code activates the `WRAM_FRAME_COUNTER` counter by setting it to a value different from `$FF`. The value is the number of frames that will elapse before sound 2 starts and overrides sound 1 on channel 1 (in this case, 4).

The second part of the update function, shown in Listing 9-5, checks the counter and starts sound 2 if it has run out (that is, if the counter value has dropped to zero).

```
; skip counter check if the counter is disabled (equals $FF)
ld a, $FF
ld hl, WRAM_FRAME_COUNTER
cp a, [hl]
jr z, .no_trigger_sound_2

    ; play sound 2 when the counter reaches zero
    dec [hl]
    jr nz, .no_trigger_sound_2
        ; play sound 2
        copy [rNR10], $00
        copy [rNR11], $80
        copy [rNR12], $F3
        copy [rNR13], $C1
        copy [rNR14], $87

        ; deactivate the counter
        dec [hl]

.no_trigger_sound_2
```

Listing 9-5: Checking the counter and playing sound 2

The first few lines check whether the counter is active by comparing it to `$FF`. In this sample, the counter is disabled when its value is `$FF`. Any other value will enable the counter, which typically happens after you press the A button.

If the counter is active, the code decrements it and checks whether it is zero (if it's disabled, the code skips these steps). When the counter equals zero, sound 2 starts on channel 1. To start sound 2 on channel 1, the code sets the same five channel-specific registers used for starting sound 1. Only the values change, in order to produce a different sound.

Once sound 2 starts, the counter is disabled by setting it back to `$FF`. As the counter is already equal to zero at that point, a simple decrement instruction, `dec [hl]`, is enough to set it back to `$FF` (that is, we take advantage of the counter underflow).

Once sound 2 starts, sound 1 stops, because a channel can only ever play one sound at the same time. To play several sounds at the same time on the DMG, we would need to use more than one channel.

That's it for the first sample. We enabled the sound system, initialized the volume of each terminal, and redirected the sound of all channels to both terminals. We then played two sounds in succession on channel 1 to produce a jingle. To help you better understand how to use channel registers, the following sections cover each channel in more detail.

Channel 1

In this section, you'll learn about channel 1 and everything you need to know to play sound on this channel.

Channel 1 produces a *quadrangular wave* that alternates between maximum and minimum amplitudes, where the transition between amplitudes is instantaneous. Figure 9-1 shows an example of a quadrangular wave.

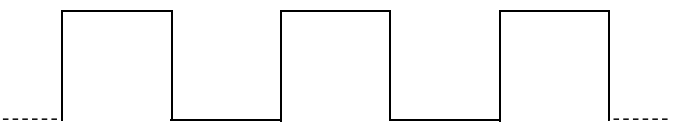


Figure 9-1: A quadrangular wave

You might be more familiar with sine waves, which are smooth curves. If you're wondering why channel 1 produces a quadrangular wave, it's because this type of wave is easier to generate on digital systems such as the DMG. Quadrangular waves are often used for melody and sound effects, and they're one of the elements that give old consoles their distinctive "retro" sound.

Channel 1 has two control flags (restart sound and length counter), four parameters (frequency, duty cycle, length, and volume), and two optional functions (volume envelope and frequency sweep). You can set these up through five registers, as described next.

Control Flags

The first control flag, *restart sound*, instructs the DMG to start playing a sound on channel 1 based on the parameters in registers `rNR10` to `rNR14`. Unless that flag is raised, the DMG will not play any sound at all on channel 1. The flag is set through bit 7 of `rNR14`.

The second flag, *length counter*, enables the length counter mechanics. When the flag is cleared, any sound on channel 1 will play indefinitely, until we deliberately stop it. When the flag is raised, we can specify the duration of the sound on the channel through another register, which I'll cover in

detail in “Possible Sound Lengths” on page 148. We set the length counter flag through bit 6 of `rNR14`.

Parameters

Channel 1 has four parameters: frequency, duty cycle, length, and volume. Let’s explore what each of them means and how to set them.

Frequency

The first parameter of channel 1 is *frequency*, which is related to the pitch of a sound. A low frequency corresponds to a low-pitched sound, while a high frequency corresponds to a high-pitched sound.

Frequency is encoded in 11 bits, which is a value from 0 to 2,047. Here’s the formula to get a frequency in Hertz (Hz) from an 11-bit value:

```
frequency = 131,072 / (2,048 - value)
```

For the minimum 11-bit value of 0, the frequency is 64 Hz. For the maximum 11-bit value of 2,047, the frequency is 131,072 Hz. That means the frequency of channel 1 ranges between 64 and 131,072 Hz, which is good enough for the average human hearing range (usually between 20 and 20,000 Hz). Figure 9-2 maps the 11-bit values to the corresponding frequencies.

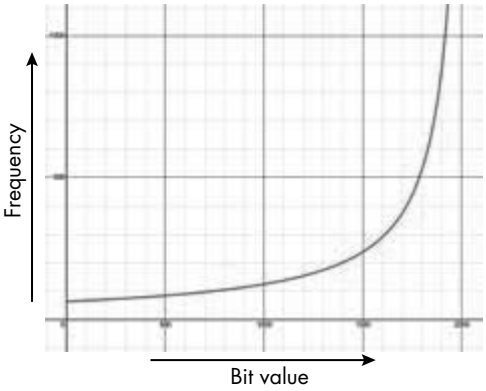


Figure 9-2: The relationship between bit value and frequency

Most of the bits represent low frequencies, which means the precision drops off quickly at high frequencies. This is fine, because most of the sounds used in music and sound effects are rather low frequency. For example, the highest note on an 88-key piano is 4,186 Hz, and on a flute it is 2,093 Hz.

The formula is also related to how octaves work. Each time the sound goes up an octave, the frequency doubles. There are still only 12 notes (C, C#, D, D#, E, F, F#, G, G#, A, A#, and B) per octave, so the frequency interval between notes gets larger and larger for higher octaves. For example, the interval between a C4, which is the note C at octave 4 (261 Hz), and a D4 (293 Hz) is only 32 Hz, but the interval between a C8 (4,186 Hz) and

a D8 (4,698 Hz) is 512 Hz. Consequently, less precision is required as the frequency gets higher.

We can also convert values in the opposite direction. Here's the formula to get an 11-bit value (for the sound registers) from a frequency:

```
value = 2,048 - (131,072 / frequency)
```

This formula is more useful than the previous one, because we'll often want to convert notes of known frequencies to bit values that the DMG understands. For example, suppose we want to find the 11-bit value of the note A4, commonly used for tuning instruments. A4's frequency is 440 Hz. Using the formula shown here, we can convert that frequency into an 11-bit value of about 1,750.11. Because we cannot use floating-point numbers, we round down to 1,750. In hexadecimal, that value would be `$6D6`.

Now, if we were to convert `$6D6` back to a frequency (using the first formula), we'd get about 439.84 Hz. That's not exactly an A4 (440 Hz), but it's close enough.

We need two registers to set the frequency: `rNR13` and `rNR14`. For the note A4, whose 11-bit value is `$6D6`, `rNR13` would hold `$D6`, and the lower 3 bits of `rNR14` would hold `$6`. The note C4 (261.63 Hz) would have an 11-bit value of `$60B`, so `rNR13` would hold `$0B`, and the lower 3 bits of `rNR14` would hold `$6`.

Figure 9-3 shows a screenshot of the *ch1_frequency* sample related to this section.

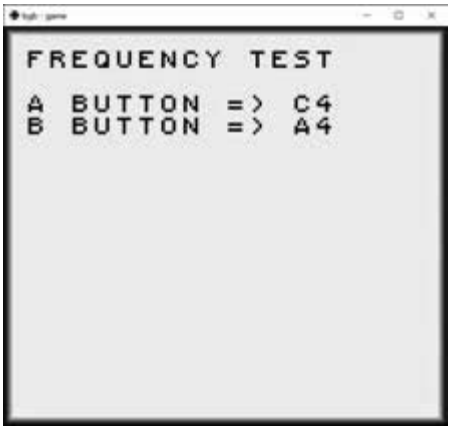


Figure 9-3: The frequency sample

It plays a C4 (`$60B`) on button A and an A4 (`$6B6`) on button B. Listing 9-6 shows the corresponding code.

```
    TestPadInput_Pressed WRAM_PAD_INPUT, PADF_A
    jr nz, .button_a
        ld de, $C60B
        jr .play_sound
    .button_a

    TestPadInput_Pressed WRAM_PAD_INPUT, PADF_B
    jr nz, .button_b
        ld de, $C6D6
        jr .play_sound
    .button_b

    ret

    .play_sound
    copy [rNR10], $00
    copy [rNR11], $80
    copy [rNR12], $F0
    copy [rNR13], e
    copy [rNR14], d

    ret
```

Listing 9-6: Playing notes on channel 1

This code focuses on `rNR13` and `rNR14`. The lower 8 bits of each frequency end up in `rNR13`. The higher 3 bits end up in the low nibble of `rNR14`. The high nibble of `rNR14` is set to `$C` to raise both the restart sound and length counter flags.

The code uses `de` as a temporary place to hold the values of `rNR13` and `rNR14`, because the sound will start playing as soon as the restart sound flag is set in `rNR14`. If `rNR14` were used directly instead of passing through `de`, the sound would start before `rNR10`, `rNR11`, and `rNR12` were properly set, and the DMG would not play the correct sound. The `rNR14` register must always be set last.

Duty Cycle

The second parameter of channel 1 is the duty cycle of the quadrangular wave. The *duty cycle* is the percentage of time during which the wave amplitude is high. There are four different duty cycles for channel 1: 12.5 percent, 25 percent, 50 percent, and 75 percent, as illustrated in Figure 9-4.

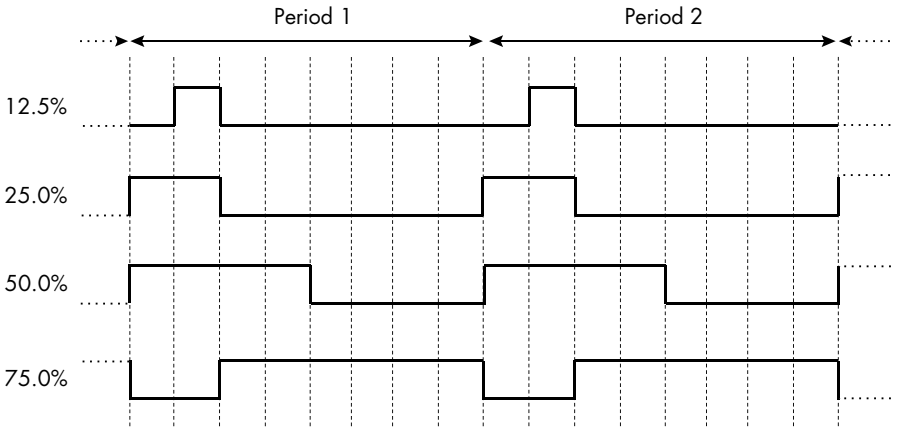


Figure 9-4: Duty cycles

The duty cycle is related to *timbre*, a quality of sound that helps us distinguish the source of the sound. For example, a flute and a piano could play the same note, yet most people could easily distinguish between the two instrument sounds because they have different timbres.

The duty cycle is set in the higher 2 bits (bits 7–6) of `rNR11`. Figure 9-5 shows a screenshot of the *ch1_duty* sample.



Figure 9-5: The duty cycle sample

This sample plays a C4 note with a different duty cycle based on the key pressed. Listing 9-7 shows the corresponding code.

```
    TestPadInput_Pressed WRAM_PAD_INPUT, PADF_LEFT
    jr nz, .left
        copy [rNR11], $00
        jr .play_sound
    .left
```

```
    TestPadInput_Pressed WRAM_PAD_INPUT, PADF_UP
    jr nz, .up
        copy [rNR11], $40
        jr .play_sound
    .up

    TestPadInput_Pressed WRAM_PAD_INPUT, PADF_RIGHT
    jr nz, .right
        copy [rNR11], $80
        jr .play_sound
    .right

    TestPadInput_Pressed WRAM_PAD_INPUT, PADF_DOWN
    jr nz, .down
        copy [rNR11], $C0
        jr .play_sound
    .down

    ret

    .play_sound
    copy [rNR10], $00
    copy [rNR12], $F0
    copy [rNR13], $0B
    copy [rNR14], $C6

    ret
```

Listing 9-7: Switching duty cycles

Only the upper 2 bits of `rNR12` are modified. A duty cycle of 50 percent (the RIGHT button) is a square wave, often considered the default duty cycle. Duty cycles of 25 percent (the UP button) and 75 percent (the DOWN button) sound exactly the same, so there are only three distinct duty cycles available.

Sound Length

The third parameter of channel 1 is the *sound length*, which controls how long the sound plays after it starts. An internal counter tracks the length. Once that counter runs out, the sound stops automatically.

The length is a 5-bit value (0 to 63). You can use the following formula to get the sound length in seconds from the 5-bit value:

```
length = (64 - value) / 256
```

The inverse formula that gets a 5-bit value from a target length in seconds is as follows:

```
value = 64 - (256 * length)
```

This is a bit unintuitive, but the minimum 5-bit value of 0 corresponds to the longest sound length (250 ms), while the maximum 5-bit value of 63 corresponds to the shortest sound length (3.9 ms). Table 9-4 lists all the possible sound lengths.

Table 9-4: Possible Sound Lengths

5-bit value	Length (ms)	5-bit value	Length (ms)
0	250.0	32	125.0
1	246.09375	33	121.09375
2	242.1875	34	117.1875
3	238.28125	35	113.28125
4	234.375	36	109.375
5	230.46875	37	105.46875
6	226.5625	38	101.5625
7	222.65625	39	97.65625
8	218.75	40	93.75
9	214.84375	41	89.84375
10	210.9375	42	85.9375
11	207.03125	43	82.03125
12	203.125	44	78.125
13	199.21875	45	74.21875
14	195.3125	46	70.3125
15	191.40625	47	66.40625
16	187.5	48	62.5
17	183.59375	49	58.59375
18	179.6875	50	54.6875
19	175.78125	51	50.78125
20	171.875	52	46.875
21	167.96875	53	42.96875
22	164.0625	54	39.0625
23	160.15625	55	35.15625
24	156.25	56	31.25
25	152.34375	57	27.34375
26	148.4375	58	23.4375
27	144.53125	59	19.53125
28	140.625	60	15.625
29	136.71875	61	11.71875
30	132.8125	62	7.8125
31	128.90625	63	3.90625

The length is set in the lower 5 bits of register `rNR11`, which is the same register as the duty cycle.

There are two important things to note. First, the length is an optional parameter that is used only when bit 6 of `rNR14` is set. When bit 6 is cleared, the sound never ends (that is, the length is totally ignored). Second, when the length is enabled (that is, when bit 6 of `rNR14` is set), you can use bit 0 in `rNR52` to check whether the sound is playing or not. Bit 0 is raised automatically when the sound starts playing and gets cleared when the length counter expires and the sound stops. You can use this technique, for example, to trigger an event like a menu transition at the end of a sound.

The *ch1_length* sample plays a C4 note at different lengths based on which key you press. This sample demonstrates the use of the 5-bit length in `rNR11`, the length counter flag in `rNR14`, and the play flag in `rNR52`. A screenshot is shown in Figure 9-6.



Figure 9-6: The length sample

The sample plays a note with a length that varies depending on the key pressed. It also features a visual indicator for the play flag from `rNR52`. Listing 9-8 shows the portion of the update function that updates the graphics based on the play flag.

```
def PLAY_FLAG_ROW           equ (8)
def PLAY_FLAG_COLUMN        equ (12)
def TILE_INDEX_ZERO         equ (80)

UpdateSample:
    halt

    ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
    ; vblank
    ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

    ld hl, _SCRN0 + PLAY_FLAG_ROW * SCRN_VX_B + PLAY_FLAG_COLUMN
    ld [hl], TILE_INDEX_ZERO

    ld a, [rNR52]
```

```
    bit 0, a
    jr z, .play_flag
        ld [hl], TILE_INDEX_ZERO + 1
    .play_flag
```

Listing 9-8: The vblank section of the update function for the length sample

This code displays a `1` on the LCD when the channel is playing and a `0` otherwise.

NOTE *The value of the channel 1 play flag might be `1` at the very start of a program, even if there is no sound playing. The flag is uninitialized by default, and you should not rely on it being set to any given value until a sound properly starts on the channel.*

Listing 9-9 is the program logic part that actually plays a sound based on the d-pad input. There are four length options, one of which (the LEFT key) is to not use the length at all and play the sound indefinitely.

```
    UpdatePadInput WRAM_PAD_INPUT

    ld c, $40

    TestPadInput_Pressed WRAM_PAD_INPUT, PADF_LEFT
    jr nz, .left
        ld b, $00
        ld c, $00
        jr .play_sound
    .left

    TestPadInput_Pressed WRAM_PAD_INPUT, PADF_UP
    jr nz, .up
        ld b, $3C
        jr .play_sound
    .up

    TestPadInput_Pressed WRAM_PAD_INPUT, PADF_RIGHT
    jr nz, .right
        ld b, $31
        jr .play_sound
    .right

    TestPadInput_Pressed WRAM_PAD_INPUT, PADF_DOWN
    jr nz, .down
        ld b, $00
        jr .play_sound
    .down

    ret
```

```
.play_sound
copy [rNR10], $00

ld a, $80
or a, b
ld [rNR11], a

copy [rNR12], $F0
copy [rNR13], $0B

ld a, $86
or a, c
ld [rNR14], a

ret
```

Listing 9-9: The program logic section of the update function for the length sample

Register `b` holds the 5-bit length of the sound. It is or'ed into `rNR11` along with the duty cycle (`$80`) when the sound plays.

Register `c` holds the value of the length counter flag. It is `$40` by default (flag raised) but gets cleared when you press the LEFT button, which corresponds to "length off."

Volume

The fourth and last parameter of channel 1 is the volume of the sound. You can set the volume through the high nibble (bits 7–4) of `rNR12`. Its value ranges from 0 to 15. At 0, the sound is mostly inaudible; the higher the value, the louder the volume.

The *ch1_volume* sample lets you control channel 1's volume and play a sound by pressing the A button. The sample also includes some control over the terminals' volume from `rNR50`. Figure 9-7 is a screenshot of the sample.



Figure 9-7: The volume sample

Listing 9-10 shows the first part of the sample's code, which includes the vblank section of the update function.

```
def TILE_INDEX_ZERO         equ (80)
def TILE_INDEX_A            equ (97)

def TERMINAL_LEFT_TILE_ADDRESS  equ (_SCRN0 + 4 * SCRN_VX_B + 10)
def TERMINAL_RIGHT_TILE_ADDRESS equ (_SCRN0 + 5 * SCRN_VX_B + 10)
def CHANNEL_TILE_ADDRESS        equ (_SCRN0 + 8 * SCRN_VX_B + 11)

UpdateSample:
    halt

    ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
    ; vblank
    ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

    ; left terminal volume
    ld a, [rNR50]
    swap a
    and a, $07
    add a, TILE_INDEX_ZERO
    ld hl, TERMINAL_LEFT_TILE_ADDRESS
    ld [hl], a

    ; right terminal volume
    ld a, [rNR50]
    and a, $07
    add a, TILE_INDEX_ZERO
    ld hl, TERMINAL_RIGHT_TILE_ADDRESS
    ld [hl], a

    ; channel volume
    ld a, [WRAM_CHANNEL_VOLUME]
    swap a
    and a, $0F
    cp a, 10
    jr c, .compare_to_ten
        sub a, 10
        add a, TILE_INDEX_A
        ld hl, CHANNEL_TILE_ADDRESS
        ld [hl], a
        jr .compare_to_ten_end
    .compare_to_ten
        add a, TILE_INDEX_ZERO
        ld hl, CHANNEL_TILE_ADDRESS
        ld [hl], a
    .compare_to_ten_end
```

Listing 9-10: The vblank section of the update function for the volume sample

The code uses a combination of `swap` and `and` masks to get the current volume values for the terminals and the channel, and it updates the background tiles accordingly with the proper tile indices. This is handled a bit differently for the channel because the value can go up to 15. Since we express the volume in hexadecimal (`$0` to `$F`), we need more code to display the proper letters when the volume is over 9.

Listing 9-11 shows the second part of the update function.

```
    UpdatePadInput WRAM_PAD_INPUT

    ; cycle through left terminal's volumes
    TestPadInput_Pressed WRAM_PAD_INPUT, PADF_LEFT
    jr nz, .left
        ld a, [rNR50]
        swap a
        inc a
        and a, $07
        swap a
        ld b, a

        ld a, [rNR50]
        and a, $8F
        or a, b
        ld [rNR50], a
    .left

    ; cycle through right terminal's volumes
    TestPadInput_Pressed WRAM_PAD_INPUT, PADF_RIGHT
    jr nz, .right
        ld a, [rNR50]
        inc a
        and a, $07
        ld b, a

        ld a, [rNR50]
        and a, $F8
        or a, b
        ld [rNR50], a
    .right

    ; increase the channel volume and loop to $00 if overflow
    TestPadInput_Pressed WRAM_PAD_INPUT, PADF_UP
    jr nz, .up
        ld a, [WRAM_CHANNEL_VOLUME]
        swap a
        inc a
        and a, $0F
        swap a
```

```
    ld [WRAM_CHANNEL_VOLUME], a
.up

; decrease the channel volume and loop to $0F if underflow
TestPadInput_Pressed WRAM_PAD_INPUT, PADF_DOWN
jr nz, .down
    ld a, [WRAM_CHANNEL_VOLUME]
    swap a
    dec a
    and a, $0F
    swap a
    ld [WRAM_CHANNEL_VOLUME], a
.down

; play a sound with the proper channel volume
TestPadInput_Pressed WRAM_PAD_INPUT, PADF_A
jr nz, .play_sound
    copy [rNR10], $00
    copy [rNR11], $80
    copy [rNR12], [WRAM_CHANNEL_VOLUME]
    copy [rNR13], $0B
    copy [rNR14], $C6
.play_sound

ret
```

Listing 9-11: The program logic section of the update function for the volume sample

The code cycles through all possible volumes of the terminals with the d-pad's LEFT/RIGHT keys, with some extra `and` masking to avoid any overflow or underflow. Likewise, it is possible to control the volume of channel 1 with the d-pad's UP/DOWN keys.

Note that this updates `rNR50` directly, while it uses a WRAM variable as a buffer for `rNR12`. When we modify the value of any of the `rNR5X` registers, we can expect the change to apply immediately to the sound system, without any issue. This is true for many channel-specific registers, but `rNR12` is one exception that requires special care. Changes to `rNR12` do not take effect until the restart sound flag is raised in `rNR14`. This is not a problem in itself, but unpleasant and unwanted accidental sounds, known as *sonic artifacts*, may occur if a sound is already playing on channel 1. To avoid any artifacts in our sample, we delay the modification of the `rNR12` register (through a WRAM variable) to as close as possible to where the restart sound flag is raised.

Functions

Channel 1 has two functions that can modify a sound over time while it is playing on the channel. Let's see how to configure each of them.

Volume Envelope

The first function of channel 1 is the *volume envelope*, which increases or decreases the volume of the channel over time. Its main use is to fade sounds in and out. The initial volume is whatever is set as the volume in `rN12` when the sound starts.

The volume envelope has two parameters: the direction of the envelope and the length of each envelope step.

The envelope direction is either up or down. When the direction is up, the volume of the sound increases until it reaches the maximum value (15). When it's down, the volume decreases until it reaches 0. The direction is set through bit 3 of `rNR12`. When the bit is set, the volume goes up; otherwise, it goes down.

The step length is the amount of time the CPU waits before increasing or decreasing the volume by one unit. This is set in the lower 3 bits of `rNR12` and ranges from 0 to 7. When the length is 0, the volume envelope function is completely disabled (that is, the volume won't change at all). For any other 3-bit value, you can calculate the step length (in seconds) with the following formula:

```
length = value / 64
```

The shorter the step length, the faster the volume will change. Table 9-5 summarizes all the possible step lengths.

Table 9-5: Envelope Step Lengths

3-bit value	Step length (ms)
0	Function is disabled
1	15.6
2	31.2
3	46.9
4	62.5
5	78.1
6	93.8
7	109.4

Figure 9-8 shows how the volume envelope decreases the sound volume over time, for a value of `$65` in `rNR12`.

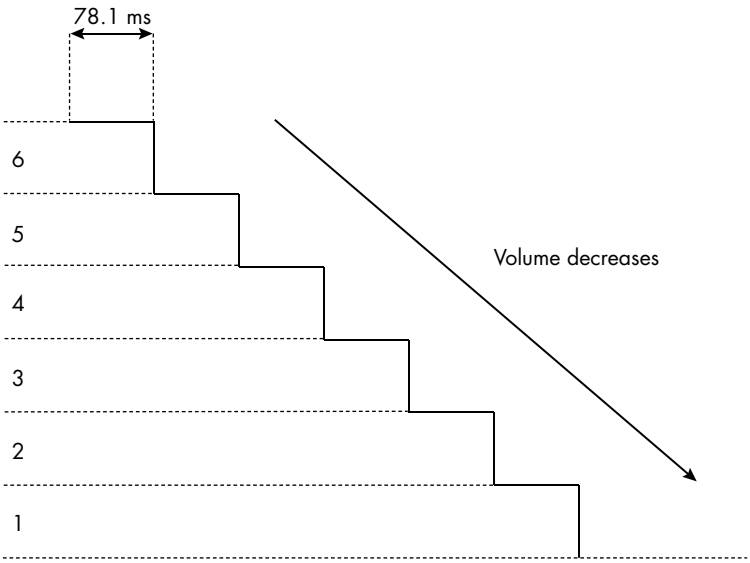


Figure 9-8: The effect of the volume envelope

The initial volume is 6, and the step length between two volume changes is 78.1 ms. The volume drops until it is at zero, at which point it does not change anymore. Be aware, though, that even when the volume drops to zero due to the envelope function, the sound does not technically stop, and bit 0 in `rNR52` (which tracks whether channel 1 is playing) does not clear.

Figure 9-9 is a screenshot of the *ch1_envelope* sample. This sample allows you to change the step length before playing the sound.



Figure 9-9: The envelope sample

Listing 9-12 shows the update function.

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
; vblank
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

; update the step length tile
ld a, [WRAM_STEP_LENGTH]
add a, TILE_INDEX_ZERO
ld [STEP_LENGTH_TILE_ADDRESS], a

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
; logic
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

UpdatePadInput WRAM_PAD_INPUT

; increase the step length
TestPadInput_Pressed WRAM_PAD_INPUT, PADF_UP
jr nz, .up
    ld a, [WRAM_STEP_LENGTH]
    inc a
    and a, $07
    ld [WRAM_STEP_LENGTH], a
.up

; decrease the step length
TestPadInput_Pressed WRAM_PAD_INPUT, PADF_DOWN
jr nz, .down
    ld a, [WRAM_STEP_LENGTH]
    dec a
    and a, $07
    ld [WRAM_STEP_LENGTH], a
.down

; play the sound
TestPadInput_Pressed WRAM_PAD_INPUT, PADF_A
jr nz, .play_sound
    copy [rNR10], $00
    copy [rNR11], $80

    ld a, [WRAM_STEP_LENGTH]
    or a, $F0
    ld [rNR12], a

    copy [rNR13], $0B
    copy [rNR14], $86
.play_sound
```

Listing 9-12: The update function for the envelope sample

This should look pretty straightforward, because its structure is very similar to the code we used for previous channel 1 samples. The initial volume is hardcoded to the maximum of 15 and the direction is set to down. You can change the step length with UP and DOWN.

Frequency Sweep

The second function of channel 1 is the *frequency sweep*, which increases or decreases sound frequency over time. This means the pitch changes, making the sound higher or lower while it's playing.

The frequency sweep is mainly used for special sound effects accompanying events like a character jumping or getting a power-up. It has three parameters: the function, the step length, and the shift number.

The function is either addition or subtraction. Addition increases the frequency over time, so the sound becomes higher. Inversely, subtraction decreases the frequency, so the sound becomes lower. The function is set through bit 3 of `rNR10`. When the bit is set, subtraction is used; otherwise, addition is used.

The step length is the amount of time the CPU waits between two frequency changes. This is set through bits 6–4 of `rNR10`, so it's a 3-bit value from 0 to 7. When the value of this parameter is 0, the DMG disables the frequency sweep (no changes in frequency occur). Otherwise, it computes the step length in seconds as follows:

```
length = value / 128
```

Table 9-6 shows all the possible step lengths.

Table 9-6: Sweep Step Lengths

3-bit value	Step length (ms)
0	Function is disabled
1	7.6
2	15.6
3	23.4
4	31.3
5	39.1
6	46.9
7	54.7

The last parameter is the shift number, which controls how much the frequency changes at each sweep step. This is easier to understand using the formula for the sweep. Here's the formula for the addition function:

```
frequency(step) = frequency(step - 1) + (frequency(step - 1) / (1 << shift_number))
frequency(0) = frequency set in rNR13 and rNR14 when the sound is started
```

At each step of the sweep, the audio system computes a new frequency, `frequency(step)`. It calculates the change in frequency based on the frequency

at the previous step, `frequency(step - 1)`, and the shift number, `shift_number`. This change corresponds to `(frequency(step - 1) / (1 << shift_number))`. The higher the shift number is, the smaller the frequency change is. We get the new frequency by adding the frequency change to the frequency at the previous step.

To better understand how this works, let's use an example. We'll assume a starting frequency of `$400` and a shift number of 2. Then, the first three steps of the sweep will yield the following frequencies:

```
frequency(0) = $400

frequency(1) = frequency(0) + (frequency(0) / 4)
frequency(1) = $400 + ($400 / 4)
frequency(1) = $400 + $100
frequency(1) = $500

frequency(2) = frequency(1) + (frequency(1) / 4)
frequency(2) = $500 + ($500 / 4)
frequency(2) = $500 + $140
frequency(2) = $640
```

The frequency goes up from 128 Hz (`$400`) to 170 Hz (`$500`) to 292 Hz (`$640`). The sound's pitch gets higher and higher at each step.

For subtraction, the formula is very similar, except a subtraction sign replaces the addition sign, as shown next:

```
frequency(step) = frequency(step - 1) - (frequency(step - 1) / (1 << shift_number))
frequency(0) = frequency set in rNR13 and rNR14 when the sound is started
```

With subtraction, using a shift number of zero has no effect; in this case, the sweep function simply gets disabled. This behavior occurs because with a shift number of zero, the frequency falls to zero after the first step, preventing much of a sweep effect.

If we reuse the same parameters as for the addition function example (a frequency of `$400` and a shift number of 2), the first three steps of the sweep will yield the following frequencies:

```
frequency(0) = $400

frequency(1) = frequency(0) - (frequency(0) / 4)
frequency(1) = $400 - ($400 / 4)
frequency(1) = $400 - $100
frequency(1) = $300

frequency(2) = frequency(1) + (frequency(1) / 4)
frequency(2) = $300 - ($300 / 4)
frequency(2) = $300 - $180
frequency(2) = $180
```

This time, the frequency goes down from 128 Hz ($400) to 102 Hz ($300) to 78 Hz ($180). The sound gets lower and lower at each step.

NOTE *The frequency update formulas for addition and subtraction act on the 11-bit binary value of the frequency. Because the relationship between the binary value of a frequency and its value in Hertz is not linear, the formulas cannot be applied to Hertz values directly.*

In addition mode, the frequency might become so high that 11 bits are not enough to store the value anymore. In that case, the sound stops and bit 0 in `rNR52` is cleared.

In subtraction mode, the shifted frequency (the rightmost part of the equation) will eventually become zero. When that happens, the frequency stops changing, but the sound itself does not stop playing. Thus, bit 0 in `rNR52` is never cleared.

Figure 9-10 is a screenshot of the *ch1_sweep* sample.



Figure 9-10: The sweep sample

This simple sample plays two sounds: one using addition (you might recognize this as a power-up sound) and one using subtraction (the sound of an object dropping from high above).

Listing 9-13 shows the relevant code from the update function.

```
; play addition sample
TestPadInput_Pressed WRAM_PAD_INPUT, PADF_A
jr nz, .play_sound_add
    copy [rNR10], $45
    copy [rNR11], $80
    copy [rNR12], $F0
    copy [rNR13], $0B
    copy [rNR14], $86

    copy [WRAM_STOP_COUNTER], STOP_SOUND_COUNTER_INIT
.play_sound_add
```

```
; play subtraction sample
TestPadInput_Pressed WRAM_PAD_INPUT, PADF_B
jr nz, .play_sound_sub
    copy [rNR10], $4E
    copy [rNR11], $80
    copy [rNR12], $F0
    copy [rNR13], $E1
    copy [rNR14], $87

    copy [WRAM_STOP_COUNTER], STOP_SOUND_COUNTER_INIT
.play_sound_sub
```

Listing 9-13: The code to play the two sounds

The sounds have different starting frequencies and, of course, different values for the sweep in `rNR10`. A stop counter is set each time a sound plays, stopping the sound after about 2 seconds. In the case of the addition function, the sound will automatically stop when the frequency value goes over 11 bits, so the counter isn't really necessary. But in the case of subtraction, the sound will never stop, eventually converging to a very low frequency value and staying there. To stop the sound automatically, you need a different mechanism, based on the stop counter.

Listing 9-14 shows the code to stop the sound on channel 1.

```
; stop the sound when the counter reaches zero to avoid any infinite sound
ld a, [WRAM_STOP_COUNTER]
inc a
ld [WRAM_STOP_COUNTER], a
jr nz, .stop_sound
    copy [rNR10], $00
    copy [rNR11], $3F
    copy [rNR12], $00
    copy [rNR13], $00
    copy [rNR14], $C0
.stop_sound
```

Listing 9-14: Stopping the sound with a counter

Most registers are set to zero. The length is set to `$3F` (63), which is the shortest possible length, as you saw in Table 9-4. This clears the play flag of the channel after 3.9 ms. The two control flags are also set.

These values work well to forcibly cut the sound and clear the play flag shortly after. In a real program, though, it's better to set the length parameter to the actual desired length of the sound instead of using a stop counter like in this sample. That way, the sound will stop automatically and you won't need any extra code to handle the counter.

Continuous Sound

The samples we've seen up to now have set all the registers and raised the restart sound flag every time a sound plays. That's because these samples are using single sounds, just to demonstrate the effect of each parameter and function. You can modify some parameters while a sound is playing without raising the restart sound flag again.

Let's explore a sample that plays a series of notes. It starts a continuous sound (without any length specified) and then switches to different notes by modifying the frequency, but without restarting the sound. You can follow along using the *ch1_continuous* sample on GitHub.

The first step is to define some notes to play, as shown in Listing 9-15.

```
Notes:
dw $060b, $0642, $0672, $0689, $06b2, $06d6, $06f7, $0706
dw $06f7, $06d6, $06b2, $0689, $0672, $0642, $060b, $0000
```

Listing 9-15: Defining the notes

The frequency sequence goes from C4 to C5 and back to C4, playing only the notes of the C major scale. The last frequency is `$0000`, used as a sentinel to mark the end of the note sequence.

Listing 9-16 shows the code that starts the sound.

```
    TestPadInput_Pressed WRAM_PAD_INPUT, PADF_A
    jr nz, .start_notes
        xor a
        ld [WRAM_NOTE_INDEX], a

        copy [rNR10], $00
        copy [rNR11], $80
        copy [rNR12], $F0

        ld hl, Notes
        ld a, [hli]
        ld [rNR13], a
        ld a, [hl]
        or a, $80
        ld [rNR14], a

        copy [WRAM_FRAME_COUNTER], TIME_BETWEEN_NOTES
    .start_notes
```

Listing 9-16: Starting the sound

The note index is initialized to zero (the first note) and the channel 1 registers are set accordingly, getting the frequency from the first note. The restart sound flag is raised here, so the sound starts playing. A frame counter, used to track the duration of the notes, is also initialized here.

Finally, Listing 9-17 shows the code to switch to the next note.

```
    ld a, $FF
    ld hl, WRAM_FRAME_COUNTER
    xor a, [hl]
    jr z, .play_notes

        ; decrease the timer and play the next sound when zero is reached
        dec [hl]
        jr nz, .sound_switch
            ld a, [WRAM_NOTE_INDEX]
            inc a
            ld [WRAM_NOTE_INDEX], a

            sla a
            ld d, 0
            ld e, a

            ld hl, Notes
            add hl, de

            ld a, [hli]
            or a, a
            jr nz, .stop_sound
                copy [rNR12], $00
                copy [rNR14], $C0
                ld a, $FF
                ld [WRAM_FRAME_COUNTER], a
                jr .play_notes
            .stop_sound

            ld [rNR13], a
            ld a, [hli]
            ld [rNR14], a

            copy [WRAM_FRAME_COUNTER], TIME_BETWEEN_NOTES
        .sound_switch

    .play_notes
```

Listing 9-17: Updating the frequency

Once the frame counter runs out, the code increments the note index. Then it extracts the frequency of the next note and puts it in the `rNR13` and `rNR14` registers. The restart sound flag is not set here, so the sound keeps playing continuously while reflecting the change in frequency. There's a bit of management in the code too, updating the frame counter and stopping the sound when the sentinel (`$0000`) is encountered.

Besides the frequency, you can change the duty cycle and the sweep function on the fly for interesting effects. However, you should not modify

volume-related registers like `rNR12` without raising the restart sound flag again; the changes will not be reflected and, worse, may produce sonic artifacts.

Tips for Using Channel 1

To end this section on channel 1, here are a few recommendations:

- When starting a sound, set `rNR14` last. The sound will start as soon as the restart sound flag is raised, so setting other registers after `rNR14` might create sonic artifacts.
- Be aware that not all registers are fully read/write. Write all 8 bits at once, as `set` (writing 1 bit at a time) might not work as expected.
- The values of registers may change during playback. This is true when using the envelope or sweep functions. Do not assume the values of the frequency-, length-, and volume-related registers.
- During playback, writes to `rNR12` are not reflected until the restart sound flag is set. To prevent sonic artifacts, this flag should be set as soon as possible after any change to `rNR12`.
- When disabling the envelope function, clear all the bits of the low nibble of `rNR12`.
- When disabling the sweep function, write `$00` in `rNR10`. Values from `$08` to `$0F` should work too.

Table 9-7 summarizes the usage of the channel 1 registers.

Table 9-7: Channel 1 Registers

Register	Bits	Access	Needs restart	Usage
`rNR10`	6–4	Read/write	—	Sweep time
`rNR10`	3	Read/write	—	Sweep function (0 = addition, 1 = subtraction)
`rNR10`	2–0	Read/write	—	Number of sweeps
`rNR11`	7–6	Read/write	—	Duty cycle
`rNR11`	5–0	Write-only	—	Sound length
`rNR12`	7–4	Read/write	Yes	Sound volume
`rNR12`	3	Read/write	Yes	Volume envelope direction (0 = down, 1 = up)
`rNR12`	2–0	Read/write	Yes	Envelope step length
`rNR13`	7–0	Write-only	—	Frequency lower bits (8 out of 11 bits)
`rNR14`	7	Write-only	—	Restart sound flag (1 = restart)
`rNR14`	6	Read/write	—	Length counter flag (0 = disabled, 1 = enabled)
`rNR14`	2–0	Write-only	—	Frequency higher bits (3 out of 11 bits)

The rNR10 register is dedicated to the sweep function. The rNR11 register holds the duty cycle and the length. The rNR12 register controls the volume and the envelope function. The rNR13 register holds the low bits of the frequency, while the rNR14 register holds the upper bits of the frequency and the two control flags.

The *ch1_full_test* sample will let you test all the parameters of channel 1. Figure 9-11 shows a screenshot of this sample.

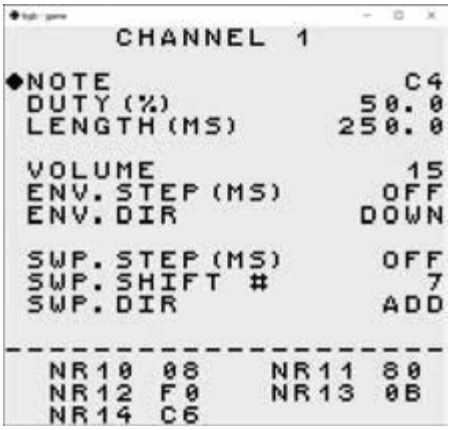


Figure 9-11: The combined test sample for channel 1

The *ch1_full_test* sample supports notes from C2 to B8 and uses more convenient units, like milliseconds for times and percentages for the duty cycle. You select a parameter to configure by moving up and down in the list (with the UP/DOWN keys) and configure it by pressing the LEFT/RIGHT keys. By holding the B button, you can cycle faster through the notes (one octave at a time) and length (10 values at a time).

Channel 2

Channel 2 works almost exactly like channel 1, except that it doesn't have a sweep function. This means it's less handy for sound effects, but it's good enough for creating melodies.

Table 9-8 lists the channel 2 registers, for the sake of completeness.

Table 9-8: Channel 2 Registers

Register	Bits	Access	Needs restart	Usage
rNR21	7–6	Read/write	—	Duty cycle
rNR21	5–0	Write-only	—	Sound length
rNR22	7–4	Read/write	Yes	Sound volume
rNR22	3	Read/write	Yes	Volume envelope direction (0 = down, 1 = up)
rNR22	2–0	Read/write	Yes	Envelope step length
rNR23	7–0	Write-only	—	Frequency lower bits (8 out of 11 bits)
rNR24	7	Write-only	—	Restart sound flag (1 = restart)
rNR24	6	Read/write	—	Length counter flag (0 = disabled, 1 = enabled)
rNR24	2–0	Write-only	—	Frequency higher bits (3 out of 11 bits)

The recommendations for channel 1 apply for channel 2 as well.

The *ch2_full_test* sample will let you test the various parameters of channel 2. Figure 9-12 shows a screenshot of that sample.

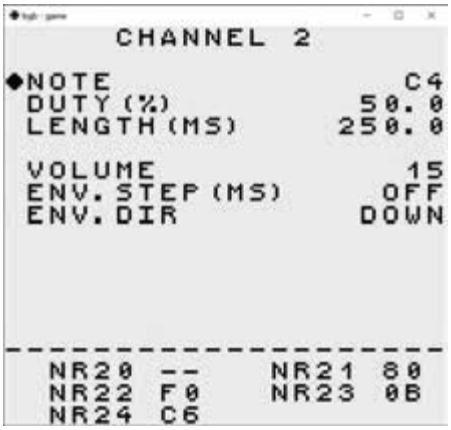


Figure 9-12: The combined test sample for channel 2

The sample is the same as the one for channel 1, with the sweep function removed.

Channel 3

Channel 3 produces a *free-form wave*, which means that, unlike channels 1 and 2, it's not limited to a quadrangular form. The waveform is customizable.

This channel has three control flags (sound enable, restart sound, and length counter) and three parameters (frequency, length, and volume). The parameters are set up through five registers. There is also a dedicated

16-byte space in the memory map (the wave RAM) to store 32 4-bit values (the samples) that define the waveform. The duty cycle parameter is absent and would not make much sense anyway, because the concept really only applies to quadrangular waves. The envelope and sweep functions are also missing, most likely to keep the hardware complexity and price down.

While the details differ, many of the concepts you learned while working with the first two channels apply to channel 3. This section covers the main differences with channel 3 and explains how to leverage its sound capabilities.

Control Flags

There are three control flags in channel 3. Two of these, the restart sound flag and the length counter flag, work exactly like the corresponding flags in channels 1 and 2. They are in `rNR34` at bits 7 and 6, respectively.

The *sound enable* flag is new. Located at bit 7 in `rNR30`, it controls whether channel 3 sound is enabled: When the bit is set, the sound is on, and otherwise it's off. The sound enable flag can forcibly stop any sound on the channel, which is important for two reasons. First, the wave RAM cannot be safely written to when the sound is enabled. Second, channel 3 has issues when some parameters are set while sound is already playing on the channel; sonic artifacts might occur, and the wave RAM might get corrupted.

Parameters

While the three parameters of channel 3 are very similar to those of the other channels, there are some differences it's important to be aware of.

The first parameter, frequency, is set the same way as in channels 1 and 2, using an 11-bit value. The lower 8 bits are set in `rNR33`, while the higher 3 bits are set in the lower bits of `rNR34`. The difference is in the interpretation of the 11-bit value. The following formulas calculate a frequency in Hertz from an 11-bit value and vice versa:

```
frequency = 65,536 / (2,048 - value)
value = 2,048 - (65,536 / frequency)
```

For a given frequency, the corresponding 11-bit values for channel 1/2 and channel 3 would be quite different. For example, for the note C4 (261.63 Hz), the 11-bit value for channel 1/2 would be 1,547 (`$60B`), while it would be 1,798 (`$706`) for channel 3.

The second parameter is the length. While other channels use 5 bits for this value, in channel 3 it's stored in an 8-bit register of its own, `rNR31`. This means you can make the sound last longer: up to 1 second, instead of the 250 ms allowed by the other channels. The following formulas calculate the actual length in seconds from the 11-bit value and vice versa:

```
length = (256 - value) / 256
value = 256 - (256 * length)
```

The last parameter is the volume. Only the higher 2 bits of `rNR32` are reserved for the volume, while in channels 1 and 2 it gets 4 bits. Table 9-9 shows the settings the possible 2-bit values correspond to.

Table 9-9: Channel 3 Volume

2-bit value	Volume
%00	0% (no sound)
%01	100%
%10	50%
%11	25%

As this indicates, the volume settings for channel 3 are less flexible than for channels 1 and 2.

Custom Waveforms

Its support for custom waveforms is the main attribute that sets channel 3 apart from the other channels. The waveform is the last piece necessary to output sound on this channel. A dedicated place in the memory map called the *wave RAM* defines the waveform, which extends from `$FF30` to `$FF3F`, for a total of 16 bytes. The wave RAM should be written to only when the sound enable flag is cleared. The behavior of a write operation when the sound enable flag is raised depends on the DMG hardware revision and is basically considered undefined.

The waveform is defined as a set of 32 4-bit samples. The first sample is stored in bits 7 to 4 of `$FF30`, the second in bits 3 to 0 of `$FF30`, the third in bits 7 to 4 of `$FF31`, and so on until the last sample, which is stored in bits 3 to 0 of `$FF3F`.

Figure 9-13 shows a few typical waveforms in DMG audio.

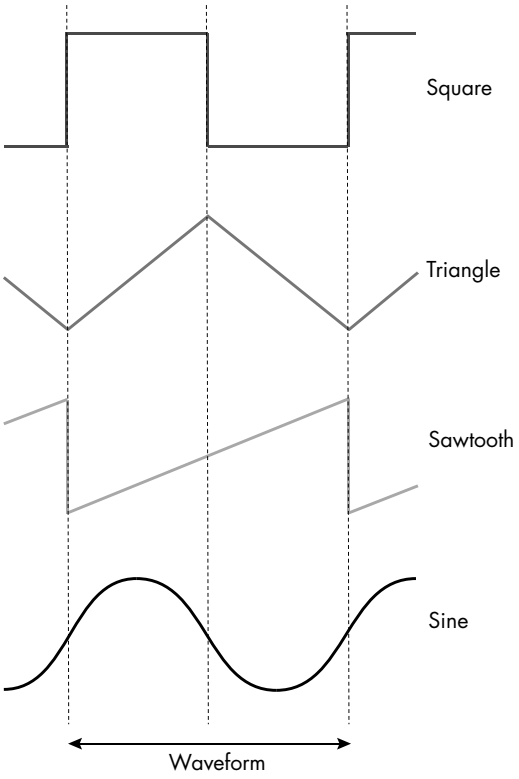


Figure 9-13: Waveform examples

You're already familiar with the square wave, as it's what's used in the first two channels. The other waveforms allow you to create sounds with different characteristics. For example, sine waves sound smooth, and sawtooth waves sound harsh. Other kinds of waveforms are possible as well, as long as they can be represented accurately enough with 32 samples.

Figure 9-14 shows how the waveforms would be converted to 32 samples in the wave RAM, with the actual 4-bit binary values of the samples on the right.

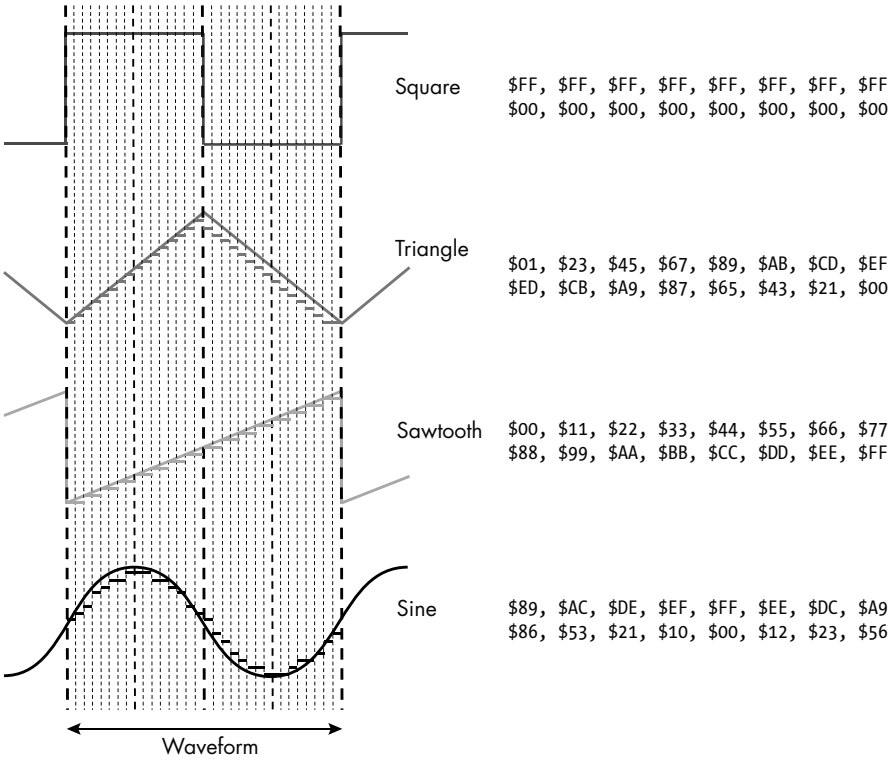


Figure 9-14: Waveforms converted to 32 samples

Apart from with square waves, getting a very accurate representation of the original wave is impossible. As a result, the sound will lose a bit of its quality and accuracy when playing on channel 3. Still, the wave approximations are good enough for most purposes.

Producing a Square Wave

Let's explore a simple sample that produces a square wave on channel 3. You can follow along using the *ch3_simple* sample on GitHub.

Listing 9-18 shows the waveform data.

```
ch3_waveform:
db $FF, $FF, $FF, $FF, $FF, $FF, $FF, $FF
db $00, $00, $00, $00, $00, $00, $00, $00
ch3_waveform_end:
```

Listing 9-18: Sample waveform data

First, we put the waveform data in the ROM using `db`. This is a square wave (quadrangular, with a 50 percent duty cycle). Then, during the sample initialization, the waveform data is copied over to the wave RAM, as shown in Listing 9-19.

```
copy [rNR30], $00
ld de, ch3_waveform
ld hl, _AUD3WAVERAM
ld c, ch3_waveform_end - ch3_waveform
.ch3_data_copy
    ld a, [de]
    inc de
    ld [hli], a
    dec c
jr nz, .ch3_data_copy
```

Listing 9-19: Copying the waveform

The sample disables channel 3 by clearing the sound enable flag in `rNR30` before copying the data over. This is a necessary step to make sure the behavior is consistent over the various hardware revisions of the DMG.

Finally, we play the same sound on channels 1 and 3. This is a C4 note at 50 percent duty cycle. If we properly set up channel 3 to produce a 50 percent duty cycle square wave, then the sounds playing on both channels should sound the same. The code is in Listing 9-20.

```
; play channel 1 sound
TestPadInput_Pressed WRAM_PAD_INPUT, PADF_A
jr nz, .play_ch1
    copy [rNR10], $00
    copy [rNR11], $80
    copy [rNR12], $F0
    copy [rNR13], $0B
    copy [rNR14], $C6
.play_ch1

; play channel 3 sound
TestPadInput_Pressed WRAM_PAD_INPUT, PADF_B
jr nz, .play_ch3
    copy [rNR30], $00 ; stop the sound to avoid any corruption
    copy [rNR30], $80
    copy [rNR31], $C0
    copy [rNR32], $20
    copy [rNR33], $06
    copy [rNR34], $C7
.play_ch3
```

Listing 9-20: Channel 1 and 3 sound

We set the volume to the maximum on both channels, and the length is 250 ms for both sounds (the maximum length for channel 1). The frequency is set on both channels to produce a C4 note. The frequency formulas are different for the two channels, so the 11-bit values are not the same.

Notice how channel 3 is disabled before setting the registers. When no sound is playing, this step is not strictly necessary, but setting the restart sound flag during sound playback might corrupt the wave RAM. To be safe, we disable the channel first, forcibly stopping any sound playback. We then enable the channel again on the next line.

When you compile and run the sample both channels should sound the same, which is what we expect. This means we set up channel 3 correctly and are ready to try out other waveforms.

Producing Different Waveforms

Now let's examine a sample that plays a C4 note using four different types of waveforms: square, triangle, sawtooth, and sine. Each waveform will sound different, even though the sound frequency remains the same. This unique ability of channel 3 to leverage custom waveforms opens up new possibilities for sound designers.

Figure 9-15 shows a screenshot of the *ch3_waveforms* sample on GitHub, which you can use to follow along.



Figure 9-15: The waveforms sample

Each key on the d-pad is associated with a waveform. Listing 9-21 shows the data declarations in the waveform section.

```
def WAVEFORM_BYTE_LENGTH            equ (16)

section "waveform", rom0

ch3_square_waveform:
db $FF, $FF, $FF, $FF, $FF, $FF, $FF, $FF
db $00, $00, $00, $00, $00, $00, $00, $00

ch3_triangle_waveform:
db $01, $23, $45, $67, $89, $AB, $CD, $EF
db $ED, $CB, $A9, $87, $65, $43, $21, $00
```

```
ch3_sawtooth_waveform:
db $00, $11, $22, $33, $44, $55, $66, $77
db $88, $99, $AA, $BB, $CC, $DD, $EE, $FF

ch3_sine_waveform:
db $89, $AC, $DE, $EF, $FF, $EE, $DC, $A9
db $86, $53, $21, $10, $00, $12, $23, $56
```

Listing 9-21: The waveform-related data

The `WAVEFORM_BYTE_LENGTH` constant is the size of the wave RAM, and all waveforms are of that size. Each waveform has a label so we can refer to it easily. Listing 9-22 shows the first function, whose goal is to copy any waveform into wave RAM.

```
copy_waveform_func:
    copy [rNR30], $00
    ld hl, _AUD3WAVERAM
    ld c, WAVEFORM_BYTE_LENGTH
    .ch3_data_copy
        ld a, [de]
        inc de
        ld [hli], a
        dec c
    jr nz, .ch3_data_copy
    ret

macro copy_waveform
    ld de, \1
    call copy_waveform_func
endm
```

Listing 9-22: The copy function

The channel sound is first turned off with `copy [rNR30], $00`, which is necessary to avoid wave RAM corruption. To make the function easier to use, we add a helper macro that puts the parameter (a waveform label/address) into the register `de`, where the function expects it to be.

Listing 9-23 shows the next function of the waveform section, which plays a C4 note for 1 second on channel 3.

```
play_sound:
    copy [rNR30], $00 ; stop the sound to avoid any corruption
    copy [rNR30], $80
    copy [rNR31], $00
    copy [rNR32], $20
    copy [rNR33], $06
```

```
    copy [rNR34], $C7
    ret
```

Listing 9-23: The play function

This function first disables the channel to avoid any waveform corruption in case a sound is already playing on the channel, then sets all the registers to play the desired sound.

Listing 9-24 shows how these functions are used in the `UpdateSample` function.

```
    TestPadInput_Pressed WRAM_PAD_INPUT, PADF_LEFT
    jr nz, .play_square
        copy_waveform ch3_square_waveform
        call play_sound
    .play_square

    TestPadInput_Pressed WRAM_PAD_INPUT, PADF_UP
    jr nz, .play_triangle
        copy_waveform ch3_triangle_waveform
        call play_sound
    .play_triangle

    TestPadInput_Pressed WRAM_PAD_INPUT, PADF_RIGHT
    jr nz, .play_sawtooth
        copy_waveform ch3_sawtooth_waveform
        call play_sound
    .play_sawtooth

    TestPadInput_Pressed WRAM_PAD_INPUT, PADF_DOWN
    jr nz, .play_sine
        copy_waveform ch3_sine_waveform
        call play_sound
    .play_sine
```

Listing 9-24: Playing the waveform sounds

For each d-pad key press, the helper macro copies the correct waveform into wave RAM. Then, the sound is played.

Compile and run the sample and compare the sounds of the different waveforms. The sine wave sounds the cleanest, because the transitions between samples are smooth. You could use this to emulate the sound of a tuning fork or a bass. The triangle wave sounds clean as well, and a bit brighter (good for melodic sounds like a flute, for example). The transitions are almost as smooth as for the sine wave, but not quite. The square wave sounds the brightest of all, but it's not nearly as clean as the first two waveforms. Clarinets produce similar waveforms to a square wave. The sawtooth wave sounds the harshest; this could be used to emulate bowed string instruments, like the violin.

These are the most common waveform examples, but there are many others that you could define here. Feel free to experiment.

Duty Cycle

Earlier, we defined a 50 percent duty cycle quadrangular (square) wave for channel 3. The code is repeated in Listing 9-25 for reference.

```
db $FF, $FF, $FF, $FF, $FF, $FF, $FF, $FF
db $00, $00, $00, $00, $00, $00, $00, $00
```

Listing 9-25: Defining a quadrangular wave with a 50 percent duty cycle

By varying the ratio of high (`$F`) and low (`$0`) samples, you can fine-tune the duty cycle by increments of 3.125 percent. That's a lot more precision than channels 1 and 2 offer, giving you more control over the timbre of the sound. For example, you could define a square wave with a 3.125 percent duty cycle, like in Listing 9-26.

```
db $F0, $00, $00, $00, $00, $00, $00, $00
db $00, $00, $00, $00, $00, $00, $00, $00
```

Listing 9-26: Defining a quadrangular wave with a 3.125 percent duty cycle

Or you could define a square wave with a 31.25 percent duty cycle, as shown in Listing 9-27.

```
db $FF, $FF, $FF, $FF, $FF, $00, $00, $00
db $00, $00, $00, $00, $00, $00, $00, $00
```

Listing 9-27: Defining a quadrangular wave with a 31.25 percent duty cycle

You cannot produce these two duty cycles on channels 1 and 2. Being able to define the duty cycle to this level of specificity is probably meaningful only to finicky artists, and it's less straightforward than using a parameter, but it's good to keep in mind that the option is available.

Volume Control

The volume control on channel 3 is less granular than on the first two channels: Only 4 levels are available, while the other channels have 16. However, you can also control the volume by adjusting the waveform itself.

To get an idea of how this works, we'll use the triangle waveform defined in Listing 9-28.

```
db $01, $23, $45, $67, $89, $AB, $CD, $EF
db $ED, $CB, $A9, $87, $65, $43, $21, $00
```

Listing 9-28: Triangle waveform (base)

When the volume is set to 50 percent using the bits in register `rNR32`, what the hardware really does is divide all the sample values in the waveform by 2, a right shift of each sample value. The result in this case would be equivalent to the waveform in Listing 9-29.

```
db $00, $11, $22, $33, $44, $55, $66, $77
db $76, $65, $54, $43, $32, $21, $10, $00
```

Listing 9-29: Triangle waveform (50 percent)

Likewise, for volume at 25 percent, the sample values in the waveform are divided by 4. This is like two right shifts, and the equivalent waveform is shown in Listing 9-30.

```
db $00, $00, $11, $11, $22, $22, $33, $33
db $33, $32, $22, $21, $11, $10, $00, $00
```

Listing 9-30: Triangle waveform (25 percent)

By changing the waveform yourself, you can further control the volume, going beyond the four volume presets. The flexibility is limited, but you can access volumes that are not equivalent to right shifts with this technique. For example, Listing 9-31 shows a triangle waveform at 75 percent volume.

```
db $00, $12, $33, $45, $66, $78, $99, $AB
db $A9, $98, $76, $65, $43, $32, $10, $00
```

Listing 9-31: Triangle waveform (75 percent)

Like the technique for adjusting the duty cycle described in the previous section, it's good to know this option exists should you ever need additional control over the volume.

Tips for Using Channel 3

To end our discussion of channel 3, here are some recommendations:

- Always clear the sound enable flag to make sure the sound is stopped before editing the waveform in wave RAM.
- When starting a sound, set `rNR34` last. The sound will start as soon the restart sound flag is raised, so setting other registers after `rNR34` might create sonic artifacts.
- Clear the sound enable flag before setting the restart sound flag. Setting this flag while a sound is playing might create sonic artifacts or corrupt the waveform in wave RAM.
- Not all registers are fully read/write. Write all 8 bits at once, as `set` (writing 1 bit at a time) might not work as expected.

Table 9-10 summarizes the usage of the channel 3 registers.

Table 9-10: Channel 3 Registers

Register	Bits	Access	Usage
rNR30	7	Read/write	Sound enable flag (0 = off, 1 = on)
rNR31	7–0	Read/write	Sound length
rNR32	6–5	Read/write	Sound volume
rNR33	7–0	Write-only	Frequency lower bits (8 out of 11 bits)
rNR34	7	Write-only	Restart sound flag (1 = restart)
rNR34	6	Read/write	Length counter flag (0 = disabled, 1 = enabled)
rNR34	2–0	Write-only	Frequency higher bits (3 out of 11 bits)

The rNR30 register contains only the sound enable flag. The rNR31 and rNR32 registers are dedicated to the length and volume, respectively. The rNR33 register holds the low bits of the frequency, while the rNR34 register holds the upper bits of the frequency and the two remaining control flags.

The *ch3_full_test* sample will let you test all the parameters of channel 3. Figure 9-16 shows a screenshot of this sample.

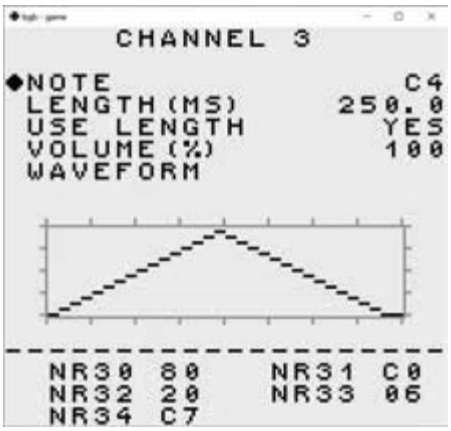


Figure 9-16: The combined test sample for channel 3

Besides adjusting the settings for the parameters and flags, it's possible to set and experiment with the waveform. After navigating to that option, press the B button to enter or exit the waveform edit mode. In edit mode, a sample selector should appear above the samples. You can move that selector between samples using the LEFT and RIGHT buttons and change the sample values using the UP and DOWN buttons.

Channel 4

Channel 4 produces pseudorandom noise by alternating between high and low levels at a given frequency. This channel has two control flags (restart sound and length counter), two parameters (length and volume), an optional function (volume envelope), and a polynomial counter. Because most

of these features work exactly as they do for channel 1, in this section we'll focus on the new feature: the polynomial counter.

The Polynomial Counter

Channel 4 makes static-like noise. Unlike other channels, it does not have any purposely defined waveform or frequency; instead, it uses the outcome of a pseudorandom number generator to produce the waveform. Because there is no clear repeating pattern in that waveform, the frequency is not properly defined.

Figure 9-17 shows an overview of the mechanics of the polynomial counter.

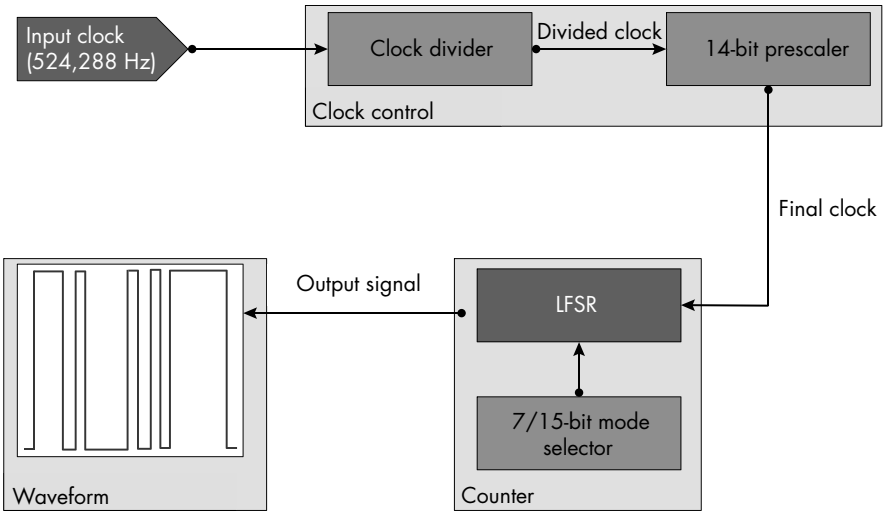


Figure 9-17: How the polynomial counter operates

Let's go through the various components in turn. At the top left is the input clock of the DMG. This runs at 524,288 Hz, the frequency of the DMG base clock divided by 8.

The input clock is fed into a clock divider, which further reduces the frequency. For example, when the divisor is set to 4, the divided clock is equal to 131,072 Hz, the result of dividing 524,288 by 4. The divided clock then becomes the input of a prescaler, which again reduces the frequency of the clock by dividing it by a power of 2 (we'll look at the possible values for both of these shortly). The clock divider and the prescaler are part of a bigger conceptual block whose purpose is to control the clock. Its output is the clock that drives the counter. Each tick of that clock updates the counter.

The counter is a *linear-feedback shift register (LFSR)*, which counts in a pseudorandom order. That is, whereas a regular 8-bit counter would generate 256 distinct values from 0 to 255, in order, an 8-bit LFSR would generate $2^8 - 1 = 255$ distinct values between 1 and 255 (0 is never generated by an LFSR) in a seemingly random but deterministic order.

There are two modes of operation for the LFSR: 15-bit and 7-bit. In 15-bit mode, the LFSR will generate $2^{15} - 1 = 32{,}767$ values, producing sound that is close to white noise. In 7-bit mode, the LFSR will generate $2^7 - 1 = 127$ values. This mode produces a sound with a more distinct pattern.

The output of the LFSR is a single bit used to create a pseudorandom quadrangular waveform. If the bit is zero, the waveform level is low; otherwise, it's high.

In Figure 9-17, there are three red boxes: the clock divider, the prescaler, and the 7/15-bit mode selector. We control these parts through the `rNR43` register, which is dedicated to the management of the polynomial counter. The high 4 bits of `rNR43` are dedicated to the 14-bit prescaler.

Table 9-11 sums up all the possible values of the prescaler.

Table 9-11: Clock Prescaler Values

4-bit value	Divisor	4-bit value	Divisor
0	2	1	4
2	8	3	16
4	32	5	64
6	128	7	256
8	512	9	1,024
10	2,048	11	4,096
12	8,192	13	16,384
14	Invalid	15	Invalid

The prescaler works by shifting the clock value. All possible divisors are powers of two. The prescaler is 14 bits wide, so you should not use the last two values (14 and 15); they're invalid.

Bit 3 of `rNR43` is for the 7/15-bit mode selection. When the bit is zero, the 15-bit mode is selected; otherwise, the 7-bit mode is active.

The low 3 bits of `rNR43` are for the clock divider. Table 9-12 shows the possible values.

Table 9-12: Clock Divider Values

3-bit value	Divisor
0	0.5
1	1
2	2
3	3
4	4
5	5
6	6
7	7

The only nonintuitive value in this table is 0. Because zero cannot be a divisor, it has a different meaning and is replaced by 0.5 (which is equivalent to multiplying by 2).

The following formula computes the frequency of the counter update, based on the clock and prescaler divisors:

```
counter_update_frequency = 524,288 / clock_divisor / prescaler_divisor
```

Note that this is distinct from the sound frequency; it's the frequency of change of the waveform. Keep in mind, though, that the waveform could be high or low for several consecutive cycles. The switch is pseudorandom and depends on the value of the LFSR output bit; it won't necessarily switch between high and low at each clock cycle.

The output noise is smoother for higher LFSR update frequencies and harsher at low frequencies. Using the 7-bit mode produces more recognizable patterns and overall less noisy sound because the counter goes through only 127 values, instead of 32,767 values. It is even possible to output sounds resembling musical notes in that mode.

Sound Effect Examples

Table 9-13 shows several possible configurations of the registers and how they might be used as sounds in a game.

Table 9-13: Channel 4 Examples

rNR41	rNR42	rNR43	rNR44	Sound
00	F3	27	80	An object breaking
00	F4	46	80	An explosion
00	0F	57	80	Spaceship propulsion
00	F3	1F	80	A power-up (7-bit mode)

You can try playing these sounds using the *ch4_example* sample on GitHub. Figure 9-18 shows a screenshot of the sample run in BGB.



Figure 9-18: The ch4_examplesample

You can press the d-pad keys to play the various sound effects listed in Table 9-13.

Table 9-14 sums up the usage of the channel 4 registers.

Table 9-14: Channel 4 Registers

Register	Bits	Access	Needs restart	Usage
rNR41	5–0	Write-only	—	Sound length
rNR42	7–4	Read/write	Yes	Sound volume
rNR42	3	Read/write	Yes	Volume envelope direction (0 = down, 1 = up)
rNR42	2–0	Read/write	Yes	Envelope step length
rNR43	7–4	Read/write	—	Prescaler shift count
rNR43	3	Read/write	—	LFSR bits count (0 = 15 bits, 1 = 7 bits)
rNR43	2–0	Read/write	—	Clock divider
rNR44	7	Write-only	—	Restart sound flag (1 = restart)
rNR44	6	Read/write	—	Length counter flag (0 = disabled, 1 = enabled)

You can use the *ch4_full_test* sample to test the parameters of channel 4. When you run this sample, it should look like Figure 9-19.

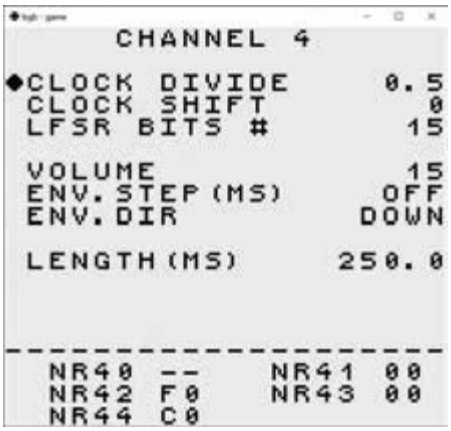


Figure 9-19: The test sample for channel 4

The effects of the polynomial counter's parameters can be difficult to predict. Hands-on experimentation is key when you're trying to generate sounds using this mechanism.

Summary

This chapter covered a lot of ground, providing an overview of the basics of DMG audio and the features of each of the four channels. You should now be ready to use any of them, but audio is a deep subject, and this chapter barely scratches the surface. Finding good references on creating audio for the DMG can be challenging, so a trial and error approach might be necessary.

As a final note, the *sound_test* sample is a combination of the four individual channel sound test samples where you can use the SELECT and START buttons to switch between channels.

In the next part of the book we'll explore some more advanced topics, building on what we've covered so far. I'll start by introducing two simple features of the DMG: the counter and the timer. The timer, in particular, will come in handy in several of the following samples.

PART III

ADVANCED GAME BOY PROGRAMMING

Game Boy Coding Adventure (Early Access) © 2025 by Maximilien Dagois

10

THE COUNTER AND TIMER

The DMG has a built-in counter, which tracks the clock ticks of the CPU, and a timer feature that lets programs schedule interrupts at specific intervals in time. This chapter covers both in detail.

While the counter is of limited use in general, it's still good to know how it affects the timer and how we can leverage it as a basic random number generator.

On the other hand, the timer is very handy, because it gives us precise control over the timing of execution of our code. We'll use the timer in various places in this book, including in the next chapter when we study serial communication.

Using the Counter

The *counter* is a 16-bit register incremented at the CPU frequency, which is 4.194304 MHz. The upper 8 bits are accessible through the `rDIV` register, but the lower 8 bits of the counter are hidden from the programmer, as shown in Figure 10-1.

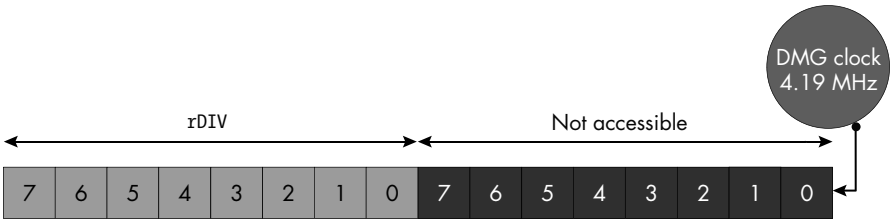


Figure 10-1: The contents of the built-in counter

Because `rDIV` reflects only the upper 8 bits of the counter, the register is equivalent to a counter updated at 16,384 Hz (4.194304 MHz / 256). The counter is always active, and there is no way to stop it. However, it's possible to reset the counter to zero by writing *any* value to `rDIV`, which can also be read at any time.

The `rDIV` register is updated 16,384 times per second, or around 274 times per frame (16,384 / 59.73). That's enough to overflow it at least once in every single frame.

The update frequency of `rDIV` is too high to make it a good candidate to track time. However, `rDIV` can be useful in two cases. The first one is related to the timer feature, as the counter influences the way the timer is triggered (we'll come back to this discussion in the next section). The second use case is for generating semirandom numbers. Because of the somewhat unpredictable nature of the counter value at the frame level, it can be used as a basis for a random number generator (RNG).

Figure 10-2 is a screenshot of the *counter_rng* sample, which illustrates this idea.



Figure 10-2: The counter RNG sample

The sample takes the lower 2 bits of `rDIV` (a value from 0 to 3) in each frame and tracks their occurrence over a period of 60 frames. It then displays how many times 0, 1, 2, and 3 came out. The purpose is to demonstrate that the distribution is good enough for a simple RNG.

Let's take a look at the code, starting with Listing 10-1.

```
xor a
ld hl, WRAM_RESULTS
ld [hli], a
ld [hli], a
ld [hli], a
ld [hli], a
copy [WRAM_UPDATE_TRACKER], UPDATE_INTERVAL
```

Listing 10-1: Initialization

During initialization, 4 bytes (located at `WRAM_RESULTS`) are set to zero. These bytes are used to count the occurrences of the values 0, 1, 2, and 3. Additionally, a frame count tracker (`WRAM_UPDATE_TRACKER`) is set to 60 (`UPDATE_INTERVAL`) frames.

Listing 10-2 shows how the tracker is decremented in each frame.

```
ld hl, WRAM_UPDATE_TRACKER
dec [hl]
jp nz, .display_results
    ; display the results on the LCD
    ld a, [WRAM_RESULTS + 0]
    DisplayRegisterA 3, 7
    ld a, [WRAM_RESULTS + 1]
    DisplayRegisterA 4, 7
    ld a, [WRAM_RESULTS + 2]
    DisplayRegisterA 5, 7
    ld a, [WRAM_RESULTS + 3]
    DisplayRegisterA 6, 7

    ; reset the results and update the counter
    xor a
    ld hl, WRAM_RESULTS
    ld [hli], a
    ld [hli], a
    ld [hli], a
    ld [hli], a
    copy [WRAM_UPDATE_TRACKER], UPDATE_INTERVAL
.display_results
```

Listing 10-2: Updating the tracker and displaying the results

Once the tracker reaches zero, the occurrence counts display on the screen; then they and the tracker reset.

Finally, Listing 10-3 shows the code that checks the lower 2 bits of `rDIV` and updates the results.

```
ld a, [rDIV]
and a, $03
ld h, 0
ld l, a
ld de, WRAM_RESULTS
add hl, de
inc [hl]
```

Listing 10-3: Gathering the results

If the 2 bits are equal to 0, the first byte of the results is incremented. If the 2 bits are equal to 1, the second byte is incremented, and so on.

You could use any number of bits from `rDIV`, in any order. Those bits should be random enough for many RNG use cases, but you'll have to check if the results meet your goals.

Using the Timer

The *timer* is a way to trigger an interrupt at regular time intervals. We'll only cover the basics of setting the time interval and starting the timer in this section, but we'll use the timer to implement communication protocols in Chapters 11 and 17.

Three registers control the timer's behavior. The first is the timer counter, `rTIMA`. The DMG increments this 8-bit register automatically. When the register overflows, it triggers a timer interrupt. You can control the frequency of the increment.

The second register is the timer modulo, `rTMA`. When `rTIMA` overflows, it resets to the value of `rTMA`, providing an additional level of control over the interval of time between the timer interrupts.

The last register is the timer control, `rTAC`. It contains the *timer enable* flag, which let us enable and disable the timer, and the clock frequency setting of the timer counter. Table 10-1 sums up the bits of the `rTAC` register.

Table 10-1: The `rTAC` Register Flags

Bits	Description
2	Timer enable flag
0–1	Clock frequency (0 = 4,096 Hz, 1 = 262,144 Hz, 2 = 65,536 Hz, 3 = 16,384 Hz)

Only four clock frequencies are available for the timer counter, but the timer modulo register, `rTMA`, gives us a finer degree of control. For example, let's say the clock frequency is set to 16,384 Hz. This means the timer counter is updated 16,384 times per second. If we set `rTMA` to 0, the timer counter will overflow 64 times per second (16,384 / 256). If we set it to 192, the timer counter will overflow 256 times per second (16,384 / (256 − 192)).

We could also set the timer modulo to 255 to get the timer counter to overflow 16,384 times per second. Each overflow triggers an interrupt.

To calculate the number of times the timer interrupt will be triggered per second, use the following formula:

```
interrupts_per_second = clock_frequency / (256 - timer_modulo)
```

The lowest possible interrupt frequency is 16 Hz, which you can achieve with a clock of 4,096 Hz and a modulo of 0. The highest possible interrupt frequency is 262,144 Hz, with a clock of 262,144 Hz and a modulo of 255.

The Built-in Counter Connection

In the previous section, you learned that the 16-bit counter (the one partially visible through `rDIV`) has an effect on the timer feature. Here, I'll explain the influence of the counter on the timer so you understand why we need to care about the value of the counter when using the timer. Figure 10-3 illustrates the connection between the two.

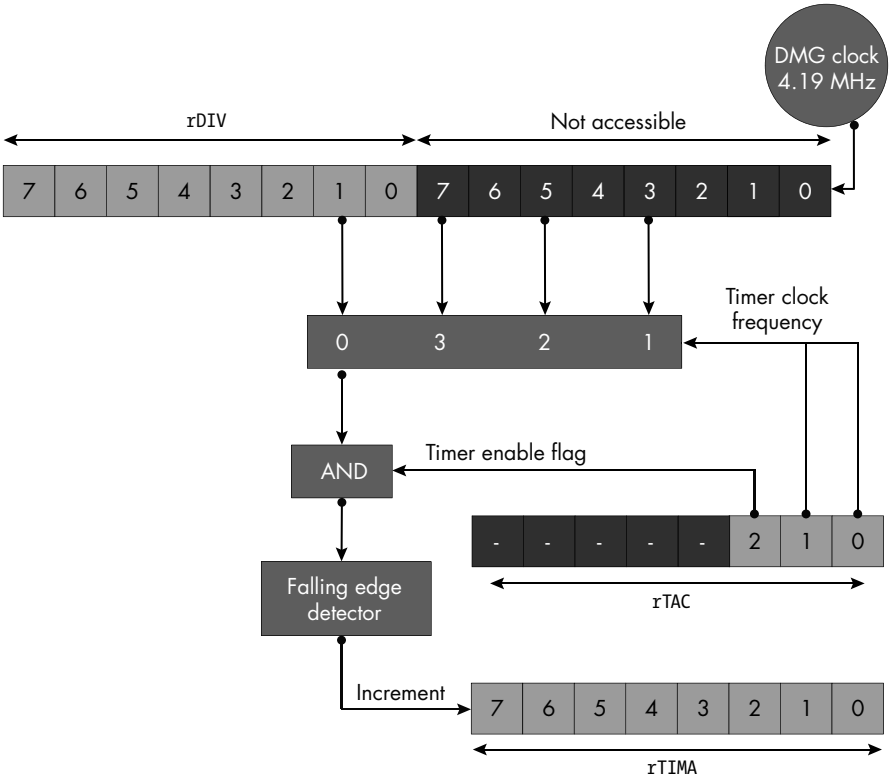


Figure 10-3: The architecture of the counter and timer

Setting a clock frequency via bits 0 and 1 in `rTAC` actually selects a bit in the counter. That bit is and'ed with the timer enable flag and fed into a falling edge detector. A *falling edge* occurs when the bit value changes from one to zero. When the timer enable flag is cleared, that condition never occurs because the edge detector is fed with only zeros. But when the timer enable flag is raised, the falling edge condition occurs at the desired frequency, and each time the condition is met, the timer counter (`rTIMA`) gets incremented.

Let's say the clock frequency is set to 65,536 Hz (2 in `rTAC`). In that case, bit 5 of the counter is forwarded to the falling edge detector. That bit transitions from one to zero (a falling edge) every 64 cycles, so the timer counter will be incremented 65,536 times per second (4.194304 MHz / 64), which is exactly the target frequency set in `rTAC`. Other clock frequencies follow the same principle by forwarding a different bit of the counter to the falling edge detector.

Because the timer uses the bits of the counter, you need to take into account a couple of subtle behaviors. First, when starting the timer you should reset the counter (by writing to `rDIV`), as otherwise the first `rTIMA` increment might be triggered earlier than desired. That's because the counter never stops and could contain any value, potentially causing a falling edge to occur as soon as the timer starts. Subsequent timer increments are unaffected, but it's important to reset the counter if an accurate timing is necessary for the first interrupt.

Second, a falling edge might be triggered when the counter is reset, causing an `rTIMA` increment. That increment occurs only if the *forwarded* bit of the counter is equal to one when the counter is reset. Still, that could in turn trigger an unwanted timer interrupt if `rTIMA` overflows. You can prevent this issue by enabling the interrupt only after resetting the counter.

A Timer Sample

The *timer_sec* sample illustrates how to set and start the timer. It leverages the timer to increment a byte every second.

The sample sets the timer clock frequency to 4,096 Hz and keeps the modulo to 0. In this configuration, the timer interrupt triggers 16 times per second (4,096 / 256), the lowest trigger frequency possible. But since the goal is to count the number of seconds, it uses a WRAM variable called the *timer scaler* to count down from 16. When the timer scaler value reaches 0, the total number of seconds is incremented.

Listing 10-4 shows how the WRAM variables are set at the start of the initialization function.

```
; initialize the WRAM state
xor a
ld [WRAM_IS_VBLANK], a
ld [WRAM_SECONDS], a
copy [WRAM_TIMER_SCALER], TIMER_SCALING
```

```
    ; enable interrupts
    copy [rIE], IEF_VBLANK | IEF_TIMER
    ei
```

Listing 10-4: Timer initialization

This resets the number of seconds and sets the scaler to 16 (`TIMER_SCALING`). Notice that the timer interrupt is also enabled in `rIE`.

At the end of the initialization function, the timer is set and started, as shown in Listing 10-5.

```
    ; set and start the timer
    xor a
    ld [rTMA], a
    ld [rTIMA], a
    ld [rDIV], a
    ld a, TACF_START | TACF_4KHZ
    ld [rTAC], a
```

Listing 10-5: Starting the timer

The `rDIV` register is reset to ensure an accurate interval before the first interrupt. Normally, the value of `rTMA` should also be copied into `rTIMA` to make sure the first interrupt triggers at the correct time. Because we used a value of zero for `rTMA` here, resetting both registers has the same effect. The `rTAC` register (in particular, the timer enable flag) is set last, to make sure all the other registers that influence the timer behavior have the desired values before the timer is started.

Listing 10-6 shows the interrupt function.

```
section "timer_interrupt", rom0[$0050]
timer_interrupt:
    push af
    push hl

    ld hl, WRAM_TIMER_SCALER
    dec [hl]
    jr nz, .update_seconds
        ld hl, WRAM_SECONDS
        inc [hl]
        copy [WRAM_TIMER_SCALER], TIMER_SCALING
    .update_seconds

    pop hl
    pop af
    reti
```

Listing 10-6: The timer interrupt function

The address for the timer interrupt is `$0050`, and a section makes sure the code is placed there. During the interrupt, the scaler is decremented. When it reaches zero, it means a second has passed and the number of seconds must be incremented.

Finally, the graphics on the LCD are updated to display the number of seconds, as shown in Listing 10-7.

```
ld a, [WRAM_SECONDS]
DisplayRegisterA 3, 12
```

Listing 10-7: Updating the timer display

Here, we use the `DisplayRegisterA` macro to display the number of seconds in hexadecimal on tile row 3, column 7 of the LCD.

This method of counting seconds should be pretty accurate. It would be harder to implement it on a frame timing basis, because the number of frames per second is 59.73, which is not an integer.

Tips for Using the Timer

To wrap up this discussion, here are a few points to keep in mind:

- Before starting the timer (by raising the timer enable flag), you must copy the value of `rTMA` into `rTIMA`. Otherwise, the first interrupt may not trigger at the correct time. You should also reset the counter by writing to the `rDIV` register, for the same reason.
- The `rTAC` register, or more precisely the timer enable flag, should be set last, after `rTMA` and `rTIMA`. This will avoid the timer running with incorrect parameters for several cycles.
- You can set the timer to trigger interrupts at a very high rate, up to thousands of times per frame. However, the interrupt function code should be kept short and return as soon as possible. If the CPU gets blocked for too long in the interrupt, critical code (such as a VRAM transfer during the vblank) might not execute in time.

Summary

This chapter introduced the counter and, more importantly, the timer. The timer is particularly useful when communicating with external devices, such as another DMG or the Game Boy Printer. You'll have the opportunity to use it in several subsequent chapters.

The next chapter focuses on the serial communication capabilities of the DMG, often used in games to exchange data between players or implement multiplayer modes.

11

SERIAL COMMUNICATION

This chapter introduces the DMG's serial communication capabilities. *Serial communication* is the process of sending and receiving data over an accessory called the Game Link Cable.

We'll start by looking at the hardware required for serial communication and the alternatives available in the emulator. Then, you'll learn how to send and receive single bytes of data through serial communication. Finally, we'll cover more advanced techniques, such as defining communication protocols and sending multiple bytes of data in succession. By the end of the chapter, you'll be able to make two DMGs communicate with each other, just like they do in multiplayer games.

Communication Devices

The DMG can communicate with many kinds of devices serially. The most obvious device is another DMG. Figure 11-1 illustrates serial communication between two DMGs connected via a Game Link Cable.



Figure 11-1: Serial communication using the Game Link Cable

This setup allows for multiplayer features, such as the two-player mode in *Tetris*.

Another device that leverages serial communication is the Game Boy Printer (covered in Chapter 17). Several games, such as *Pokémon Yellow*, *Pokémon Crystal*, and *Donkey Kong Country*, support this accessory, as does the Game Boy Camera.

This chapter uses two connected DMGs to demonstrate serial communication. This is the most common setup, and you can easily emulate it in BGB. Most importantly, the fundamental principles behind communicating between two DMGs apply to communication with any other kind of device.

Using Real Hardware

If you plan on using real hardware to run serial communication programs, there are several versions of the Game Link Cable and a few different adapters you can use. The reason for this diversity of options is that the connection port size differs between Game Boy models. The original Game Boy from 1989 has a big port, while other models (Game Boy Pocket, Game Boy Light, and Game Boy Color) feature a smaller port.

Table 11-1 describes the official cables and adapters that are available. You can connect any two Game Boys as long as you have the right equipment.

Table 11-1: Serial Cables and Adapters

Serial #	Name	Color	Description
DMG-04	Game Link Cable	Black	Connects to big ports on both ends
MGB-004	Game Link Cable Adapter	Black	Converts one end of a DMG-04 cable to a small port connection
MGB-008	Game Boy Pocket Game Link Cable	White	Connects to small ports on both ends
MGB-010	Universal Game Link Cable	Black	Connects to a small port on one end and either a small or a big port on the other end (fork)
CGB-003	Game Boy Color Game Link Cable	Black	Connects to small ports on both ends
DMG-14	Universal Game Link Adapter	Black	Converts one end of an MGB-008 or CGB-003 cable to a big port connection

Because not all of the cables and adapters in Table 11-1 are manufactured anymore, you might have a hard time finding them. Unofficial alternative cables are available; just make sure they support your particular Game Boy model.

NOTE *The Super Game Boy does not have a connection port, so it doesn't allow for serial communication. The Super Game Boy 2 does feature a small port.*

Using BGB

You can also use BGB to emulate serial communication between two DMGs. To do this, you start two instances of the emulator and connect them to each other. One instance becomes a server, and the other instance becomes a client of that server. This client/server setup is solely for the purpose of emulating the serial communication; it has nothing to do with the DMG itself. From the ROM's perspective, it's as if two DMGs were connected via a Game Link Cable.

There are two ways to set up the BGB instances: through the UI and through the command line. Let's take a look at each, starting with the UI.

From the UI

To set up the client and server instances from the UI, right-click anywhere in the BGB window and open the **Link** submenu in the context window, as shown in Figure 11-2.

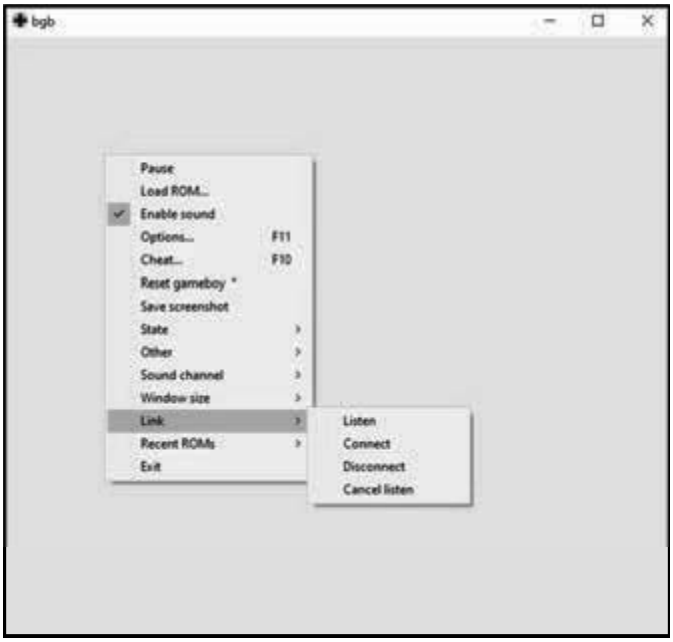


Figure 11-2: The BGB Link menu

In this menu, the Listen option launches a server, while the Connect option starts a client.

Choose **Listen** to open a pop-up window that asks for a port number, as shown in Figure 11-3.

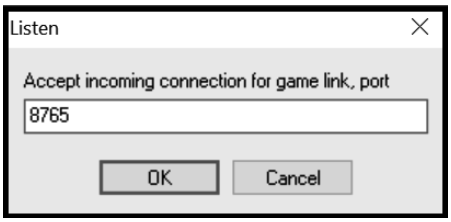


Figure 11-3: Launching a link server in BGB

The port number you enter doesn't matter, as long as the operating system accepts it. In this example, we're using port 8765.

Click **OK**, then repeat the process, this time choosing the **Connect** option. This will open a pop-up window that asks for an IP address to connect to. This section assumes that both BGB instances are run on the same PC, so we'll use the localhost IP address, 127.0.0.1, telling BGB to connect to the machine it's running on. You must also specify the port by adding it after the address. For example, assuming the server was launched with port 8765, the connection address for the client would be 127.0.0.1:8765, as shown in Figure 11-4.

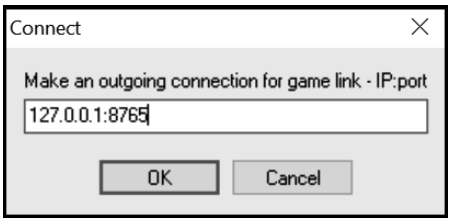


Figure 11-4: Connecting a link client in BGB

Once you've completed these three steps, you'll have two emulator instances that are connected to each other and ready to run serial communication samples. These instances will each display a new status in their title bar. The display depends on whether a ROM is loaded or not, but it should look like one of the bars in Figure 11-5.

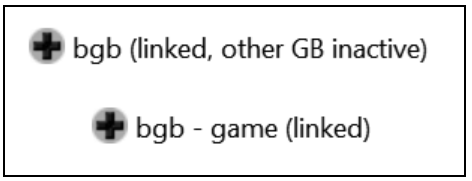


Figure 11-5: BGB linked

Next, we'll look at the alternative option, using the command line.

From the Command Line

Using the UI to set up the client and server instances is straightforward, but if you're doing this repeatedly it quickly becomes cumbersome to start and connect the instances manually. Thankfully, you can do all the setup via the command line as well.

The following command starts a server (here, using a ROM named *sample.gb* and 8765 as the port):

```
bgb --watch --rom sample.gb --listen 8765
```

To start a client, use a command like this one:

```
bgb --watch --rom sample.gb --connect 127.0.0.1:8765
```

These commands start BGB in watch mode. As you learned in Chapter 3, in watch mode BGB detects when any changes are made to the ROM and reloads it automatically. This is a handy feature for development, because it means we can start the two BGB instances, set them up for serial communication once, and have BGB handle reloading the ROM for us whenever we recompile the code.

Transfer

With the setup done, we're now ready to perform a *transfer*, an exchange of data between two DMGs. This is the basic action of serial communication. During a transfer, each DMG sends a byte and receives one at the same time.

The DMG uses an 8,192 Hz (2^{13} Hz) clock during transfers. At each tick of the clock, a single bit is exchanged. More precisely, one bit is sent out on the falling edge of the clock, and one bit is fed in on the rising edge of the clock. Consequently, it takes 8 clock ticks, or about 0.977 ms ($8 / 8{,}192$) or 1,024 cycles ($2^{20} \times 8/8,192$), to complete a transfer (8 bits out, 8 bits in).

The theoretical maximum transfer rate is 1,024 bytes per second ($2^{20}/1,024$). That's up to 17 bytes per frame ($1{,}024 / 59.73$).

As DMGs exchange data synchronously bit by bit, we need a reliable synchronization mechanism throughout a transfer. To achieve this, the DMGs share a transfer clock during serial communication: One DMG uses an internal clock of 8,192 Hz, while the other borrows the clock from its counterpart. In this book, we'll call the DMG using its internal clock the *parent* and the other DMG the *child*.

Registers and Interrupt

There are only two registers related to serial communication: the *serial buffer* register, `rSB`, and the *serial control* register, `rSC`.

The `rSB` register is a regular 8-bit register. Before the transfer, it must be filled with the data to send to the other device. Once the transfer is complete, it holds the data received from the other device. Because `rSB` is the only data register dedicated to serial transfers, only one byte can be exchanged at a time. You cannot modify the value of `rSB` during a transfer, and you must wait for the transfer to complete before exchanging another byte.

The `rSC` register contains two flags: the clock selection flag (bit 0) and the transfer start flag (bit 7). The *clock selection* flag is used to select the origin of the clock used to drive the transfer. When the flag is set, it selects the internal clock of the DMG, and the DMG becomes the parent. Otherwise, it selects an external clock, coming from the other device, and the DMG becomes the child.

The *transfer start* flag has a different effect based on whether the device is the parent or the child. On the parent side, raising the flag starts the transfer immediately. The flag is automatically cleared when the transfer completes. On the child side, raising the flag does not start the transfer. Instead, it readies the DMG for the next transfer. It's important to raise the flag even on the child's side, as otherwise the transfer won't occur when the parent initiates it. The flag is cleared when the next transfer completes.

Once a transfer is started, you cannot stop it. The transfer start flag is not writable until the transfer is over.

Table 11-2 sums up the registers and flags used in serial communication.

Table 11-2: Serial Communication Registers

Register	Bits	Usage
`rSB`	7–0	Holds the data to send before the transfer and received after the transfer
`rSC`	0	Clock selection flag (1 = internal, 0 = external)
`rSC`	7	Transfer start flag (1 on parent = start the transfer, 1 on child = get ready for the next transfer)

We can poll the transfer start flag to track whether a transfer is complete. While this is a viable solution, it's better to use the serial interrupt because it frees the CPU for doing other work while the transfer is in process. To enable the serial interrupt, raise the `IEF_SERIAL` flag in the `rIE` register. The interrupt function is at `$0058` and is called at the end of each transfer.

A Basic Transfer

In this section we'll explore the *serial_transfer* sample, which performs a single transfer when a player presses the A button on one of the DMGs. Two instances of BGB must be connected to each other to run this sample, as shown in Figure 11-6. You can also use two real DMGs linked with a cable.

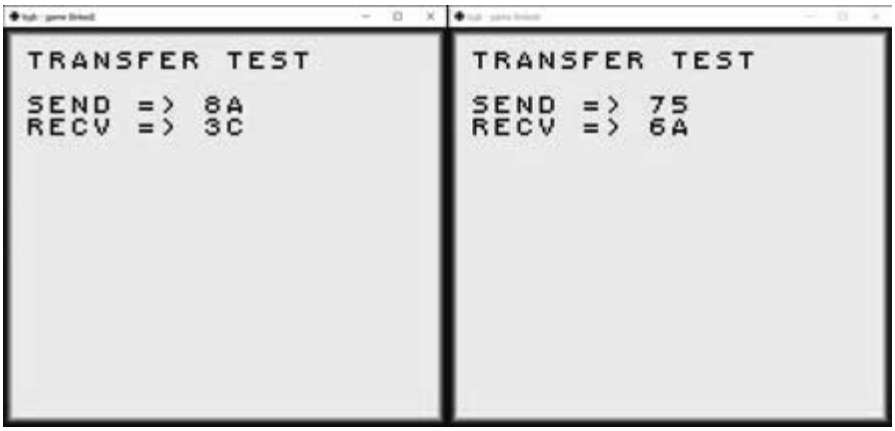


Figure 11-6: Connecting BGB instances for a serial transfer

The `SEND` field shows the value that will be sent to the other DMG during the next transfer. The `RECV` field shows the value received during the last transfer. Each time you press the A button on one of the DMGs, a transfer occurs. The `SEND` value of each DMG *moves* to the `RECV` value of the other DMG, showing a basic exchange of values. Then, another `SEND` value is semi-randomly selected.

Let's see how this works, starting with the initialization code in Listing 11-1.

```
    ld a, [rDIV]
    ld [WRAM_DATA_SEND], a
    ld [rSB], a
    ld a, $80
    ld [rSC], a
    xor a
    ld [WRAM_DATA_RECV], a
```

Listing 11-1: Serial WRAM initialization

This code initializes two WRAM variables, `WRAM_DATA_SEND` and `WRAM_DATA_RECV`, to hold the `SEND` and `RECV` values, respectively. The `WRAM_DATA_SEND` variable and the `rSB` register are initialized to the same random value. That's always the case throughout the sample, because `rSB` should always contain the value to send during the next transfer. The `rSB` register is the value the DMG actually references during the serial transfer, but its value changes during transfers; `WRAM_DATA_SEND` serves as a convenient buffer variable to display on the screen, with a stable value. The `rSC` register is set to `$80`, which means that, by default, both DMGs are set up as children ready for transfer. The `WRAM_DATA_RECV` value is set to zero as it has received no data yet.

Next, the serial interrupt is enabled as shown in Listing 11-2, because the sample relies on the serial interrupt function.

```
    copy [rIE], IEF_VBLANK | IEF_SERIAL
```

Listing 11-2: Serial interrupt initialization

The update function contains the code to start a transfer via a press of the A button, shown in Listing 11-3.

```
    TestPadInput_Pressed WRAM_PAD_INPUT, PADF_A
    jr nz, .transfer_data
        copy [rSC], $81
    .transfer_data
```

Listing 11-3: Polling input to initiate a transfer

The code switches the DMG on which the button was pressed from child to parent. This starts the transfer, as the other DMG is already in child mode.

The serial interrupt function is shown in Listing 11-4.

```
section "serial_interrupt", rom0[$0058]
    push af

    ld a, [rSB]
    ld [WRAM_DATA_RECV], a

    ld a, [rDIV]
    ld [WRAM_DATA_SEND], a
    ld [rSB], a
```

```
    ld a, $80
    ld [rSC], a

    pop af
    reti
```

Listing 11-4: The serial interrupt function

By the time the interrupt function is called, the `rSB` register contains the value received from the other DMG. First, we copy `rSB` into `WRAM_DATA_RECV`, to keep the value received and display it on the screen in the next frame. Second, we select a random value (using `rDIV` for convenience) and put it into both `WRAM_DATA_SEND` and `rSB`. That random value will be sent during the next transfer. Finally, the parent DMG is put back into child mode, ready for the next transfer.

This sample demonstrates how to perform a simple transfer between two DMGs. When the parent starts a transfer, and the child is ready, data flows between the two DMGs.

Tips on Transfers

Before we move on to the next section, here are some important notes on transfers:

- Even if a DMG is not connected to any other device, it is still possible to start a transfer. At the end of such a transfer, the interrupt is triggered and `rSB` is always equal to `$FF`. You can observe this easily by running the *serial_transfer* sample using a single DMG or BGB instance.
- If the parent starts a transfer and the child is not ready (that is, its transfer start flag is not set), the parent's behavior is the same as if there were no child at all. The interrupt is triggered, and `rSB` equals `$FF`. On the child side, there is no obvious way to determine that a transfer started on the parent, because the interrupt is not triggered and `rSB` remains unchanged. You can observe this behavior in the sample by not setting `rSC` back to `$80` in the interrupt function. The first transfer will succeed, but any subsequent transfers won't work as the child won't be ready. In general, it's the responsibility of the parent to make sure the delay between two consecutive transfers is long enough for the child to get ready. Likewise, it's the responsibility of the child to get ready before the next transfer begins. As a programmer, you'll have to write code to make sure the parent and child do their duty.
- Because the parent drives the transfer, it's easy to control the timing of the transfer relative to the vblank on the parent side. However, on the child side, the transfer could start and end at any time during the frame. Your code has to take this into account. Usually this

means using buffer variables, like `WRAM_DATA_SEND` and `WRAM_DATA_RECV` in the sample. The interrupt function loads values in those buffer variables, which we can reference at any point in the frame.

- In the sample, we were able to freely switch which DMG acts as parent because nothing prevents a user on either DMG from pressing the A button. This is not a typical setup. Generally, it's better to decide which DMG is the parent once and stick to that setup; otherwise, both DMGs might try to assume the role at the same time, and the data transfer won't go through. You'll see how to do this in the next section.
- Manually triggering transfers with a button press, as in the sample, is also atypical. Beyond the setup phase, communication should happen automatically during the frame. We'll explore that next.

A Simple Protocol

Serial communication on the DMG is quite rudimentary. As the programmer, you are responsible for making the data exchange reliable. In this section, we'll explore a sample (*serial_parent_child*) that uses a simple protocol to achieve stable transfers between the parent and child.

It does this by splitting the communication into two phases: the synchronization phase and the transfer phase.

Synchronization Phase

The synchronization phase is manual, and it requires some form of user input. At the start, both DMGs are placed in a ready state and wait. When a player presses an arbitrary key (like START) on one of the DMGs, that DMG becomes the parent, while the other DMG becomes the child. Their respective roles remain fixed until the serial communication is not necessary anymore.

Figure 11-7 shows a high-level overview of the main function during the synchronization phase.

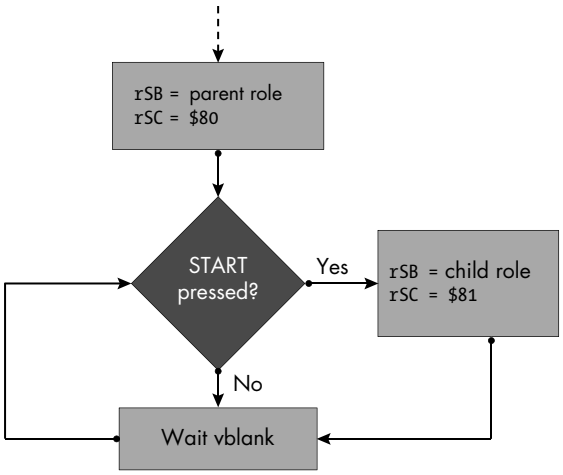


Figure 11-7: The main function during the synchronization phase

By default, each DMG expects to become a child and gets ready to send a value indicating the other device should take on the *parent role*. When the START button is pressed on a DMG, it switches to parent mode and starts a transfer to send a value indicating the other device should take on the *child role*.

Figure 11-8 is an overview of the interrupt function of the synchronization phase.

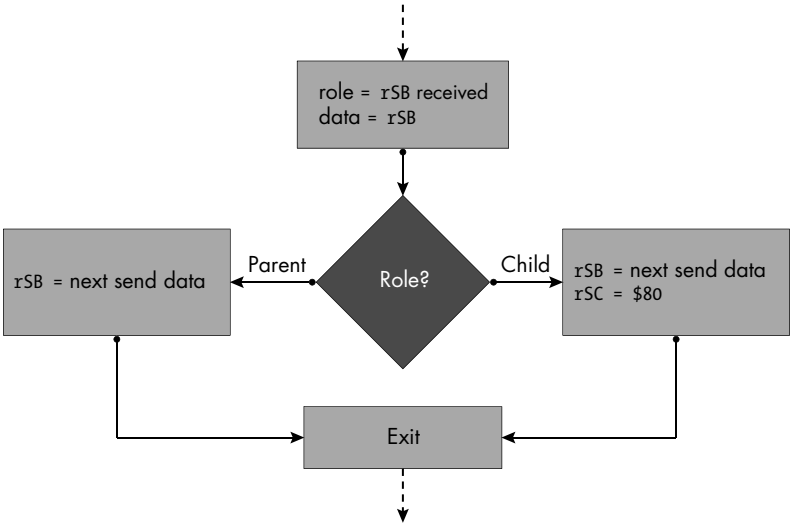


Figure 11-8: The interrupt function during the synchronization phase

Each DMG receives a role from the other DMG. That role specifies which task the DMG is assigned to during the serial communication. The DMG that initiated the first transfer (through the START button press) receives the parent role and thus becomes the parent. The other DMG receives the child role and becomes the child.

The assembly code of the *serial_parent_child* sample is slightly more complex than the overview suggests because some error checking is involved, but the idea is the same. Note that jump tables are used in this sample, but you don't need to understand them to grasp the concept of the section. I'll cover jump tables in Chapter 15.

Listing 11-5 shows how the roles are defined.

```
def SERIAL_ROLE_SYNC                equ ($00)
def SERIAL_ROLE_PARENT              equ ($01)
def SERIAL_ROLE_CHILD               equ ($02)
```

Listing 11-5: Defining the roles

Here we have the default role, `SERIAL_ROLE_SYNC`, and the parent and child roles. The values of these roles should be different from `$FF`, which is the value received when the transfer does not complete successfully. That value is used to detect errors.

The update function, shown in Listing 11-6, is a direct mapping of the overview from Figure 11-7.

```
Update_Sync:
    ; check whether the DMG becomes the parent
    TestPadInput_Pressed WRAM_PAD_INPUT, PADF_START
    ret nz
    copy [rSB], SERIAL_ROLE_CHILD
    copy [rSC], $81
    ret
```

Listing 11-6: The update function

As soon as we press the START button on one DMG, it sends a byte to the other DMG to let it know that it should become the child.

The interrupt function, seen in Listing 11-7, is also very similar to what's in the Figure 11-8 overview.

```
SerialInterrupt_Sync:
    ; check whether an error occurred
    ld hl, WRAM_DATA_IS_ERROR
    set 0, [hl]
    ld a, [rSB]
    cp a, SERIAL_NO_DATA
    ret z
    res 0, [hl]
```

```
; initialize the transfer based on the role
ld [WRAM_ROLE], a
cp a, SERIAL_ROLE_CHILD
jr nz, .parent
.child
    copy [WRAM_TRANSFER_COUNTER], SERIAL_TIMEOUT
    copy [rSC], $80
    jr .init_end
.parent
    copy [WRAM_TRANSFER_COUNTER], SERIAL_TRANSFER_INTERVAL
.init_end

; initialize the send value
xor a
ld [WRAM_DATA_SEND], a
ld [rSB], a
ret
```

Listing 11-7: The interrupt function

The beginning of the function is a simple error check, which raises the error flag `WRAM_DATA_IS_ERROR` (that is, an error is assumed by default). When the value received equals `$FF` (`SERIAL_NO_DATA`), it means something has gone wrong, such as the DMGs not being properly connected. In that case, the function ignores the interrupt and returns early, with the error flag still raised. After the error check, the error flag `WRAM_DATA_IS_ERROR` is cleared, with `SERIAL_ROLE_PARENT` and `SERIAL_ROLE_CHILD` as the only expected values in `a`. The error checking here is a bit weak, because we assume the `a` register to have a valid value (that is, one of the roles). You could improve it to catch other unexpected values, but that will do for the sample.

Assuming no error occurs, each DMG initializes a few variables based on its assigned role. The parent sets a counter variable, `WRAM_TRANSFER_COUNTER`, to start a transfer every 60 frames. In a real game, data would probably be sent every frame, but for this sample I kept the transfer frequency low to make it easier to visualize the effect of a transfer. The child uses the same counter variable for a different purpose: a timeout. When it does not receive any data for more than 120 frames, it considers the connection lost. The child also immediately sets `rSC` to get ready for the next transfer. Finally, the `WRAM_DATA_SEND` buffer variable and `rSB` are reset to zero.

Once the interrupt function completes on both parent and child, the synchronization phase is over, and the two DMGs transition to the transfer phase.

Transfer Phase

The transfer phase is automatic. The parent is responsible for programmatically driving transfers at arbitrary intervals. User input is not necessary to trigger the transfers.

During this phase the DMGs adopt different behavior based on their role, which we'll explore in this section. The update function for the parent, shown in Listing 11-8, keeps track of the transfer counter.

```
Update_Parent:
    ; check whether the transfer timer has expired
    ld hl, WRAM_TRANSFER_COUNTER
    dec [hl]
    ret nz

    ; start the transfer
    copy [rSC], $81
    ret
```

Listing 11-8: The parent update function

When the counter runs out, it triggers a transfer.
Listing 11-9 shows the interrupt function for the parent.

```
SerialInterrupt_Parent:
    ; reset the counter
    copy [WRAM_TRANSFER_COUNTER], SERIAL_TRANSFER_INTERVAL

    ; retrieve received data and check for error
    ld a, [rSB]
    cp a, SERIAL_NO_DATA
    jr nz, .error
        ; go back to the "sync" role on error
        copy [WRAM_ROLE], SERIAL_ROLE_SYNC
        copy [rSB], SERIAL_ROLE_PARENT
        copy [rSC], $80
        ret
    .error
    copy [WRAM_DATA_RECV], [rSB]

    ; update the send data
    ld a, [WRAM_DATA_SEND]
    inc a
    and a, SERIAL_VALUE_MASK
    ld [WRAM_DATA_SEND], a
    ld [rSB], a

    ret
```

Listing 11-9: The parent interrupt function

This code resets the transfer counter at the start and performs some error checking, with the DMG returning back to the synchronization phase if it receives the unexpected value `$FF` (`SERIAL_NO_DATA`). Then, the received

value is buffered into `WRAM_DATA_RECV` and the next value to send is put into `WRAM_DATA_SEND` and `rSB`.

The update function for the child uses the counter as a timeout, as shown in Listing 11-10.

```
Update_Child:
    ; timeout check
    ld hl, WRAM_TRANSFER_COUNTER
    dec [hl]
    ret nz

    ; go back to the "sync" role in case of timeout
    copy [WRAM_ROLE], SERIAL_ROLE_SYNC
    copy [rSB], SERIAL_ROLE_PARENT
    copy [rSC], $80
    ret
```

Listing 11-10: The child update function

If no transfer has occurred for more than 120 frames, it goes back to the synchronization phase.

The interrupt function for the child resets the timeout counter, because being in the interrupt means a transfer occurred as expected. Listing 11-11 shows how this works.

```
SerialInterrupt_Child:
    ; reset the timeout
    copy [WRAM_TRANSFER_COUNTER], SERIAL_TIMEOUT

    ; retrieve received data
    copy [WRAM_DATA_RECV], [rSB]

    ; update the send data
    ld a, [WRAM_DATA_SEND]
    inc a
    and a, SERIAL_VALUE_MASK
    ld [WRAM_DATA_SEND], a
    ld [rSB], a

    ; get ready for the next transfer
    copy [rSC], $80

    ret
```

Listing 11-11: The child interrupt function

Like on the parent side, the received value is buffered and the next value to send is updated. At the end, the child switches back to the ready state by writing `$80` to `rSC`.

Two main differences stand out between parent and child. First, the timing for raising the transfer start flag is quite different. On the parent, the flag is not raised until starting the transfer is required, because raising the flag on an internal clock starts the transfer right away. On the child, the flag is raised in the interrupt function, because it must be raised as early as possible so as not to miss the next transfer initiated by the parent.

Second, the error management is different. On the parent, getting `$FF` in `rSB` after a transfer is the main way of detecting errors. We can interpret the error in two ways: either the child is absent or the child is not ready. If there is a chance that the child could not get ready before the last transfer, it would be best to implement a retry mechanism and eventually consider the behavior an error only after multiple unsuccessful tries. On the child, the value of `rSB` has no particular meaning. When a transfer completes, it is because the parent started it. That means the main way to detect errors is by checking whether the transfers occur as expected. In the sample, we expect roughly one transfer per second, so a timeout after 2 seconds is an acceptable way to check for errors. If we expect some data to flow in every frame, then the lack of transfer for a couple of frames or more could be a good indicator of a problem.

Once run, the sample should look like Figure 11-9, where both DMGs send numbers in order, starting from 0 and going up to 127. We can expect both DMGs to always display the same `SEND` and `RECV` values, which will indicate that each transfer executed properly.

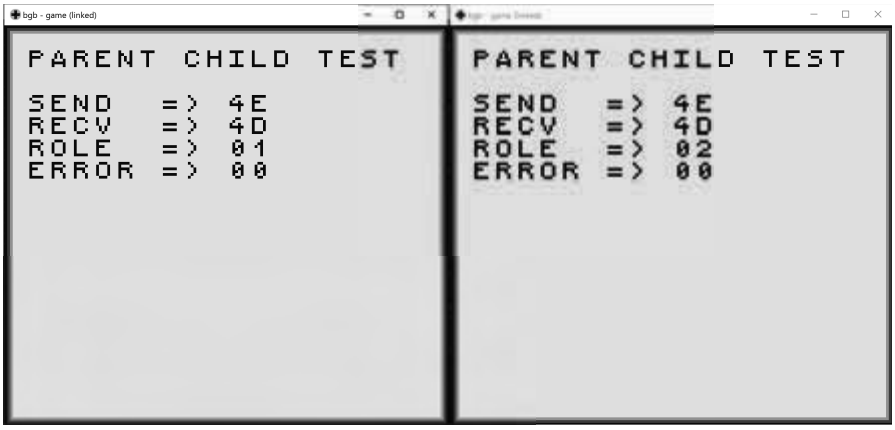


Figure 11-9: The parent/child sample

The sample is similar to the previous one, pictured in Figure 11-6, with two added fields: `ROLE` and `ERROR`. The `ROLE` field shows the value of `WRAM_ROLE`. The initial value is zero, but the screenshot was taken after the synchronization phase, so the parent and child roles are already assigned. The `ERROR` field equals 0 most of the time. The only situation where the `ERROR` field shows 1 is when the synchronization phase fails after the START button is pressed.

This section explored a simple protocol that enables reliable serial communication, but you may need to use a different approach that fits the needs

of your program. For example, it is possible to switch the parent and child after each transfer in the interrupt functions, as in the *serial_switch_roles* sample. Listing 11-12 shows the relevant code.

```
SerialInterrupt_Parent:
    ; reset the timeout
    copy [WRAM_TRANSFER_COUNTER], SERIAL_TIMEOUT

    ; retrieve received data and check for error
    ld a, [rSB]
    cp a, SERIAL_NO_DATA
    jr nz, .error
        ; go back to the "sync" role on error
        copy [WRAM_ROLE], SERIAL_ROLE_SYNC
        copy [rSB], SERIAL_ROLE_PARENT
        copy [rSC], $80
        ret
    .error
    copy [WRAM_DATA_RECV], [rSB]

    ; update the send data
    ld a, [WRAM_DATA_SEND]
    inc a
    and a, SERIAL_VALUE_MASK
    ld [WRAM_DATA_SEND], a
    ld [rSB], a

    ; get ready for the next transfer
    copy [rSC], $80

    ; switch role
    copy [WRAM_ROLE], SERIAL_ROLE_CHILD
    ret

SerialInterrupt_Child:
    ; reset the counter
    copy [WRAM_TRANSFER_COUNTER], SERIAL_TRANSFER_INTERVAL

    ; retrieve received data
    copy [WRAM_DATA_RECV], [rSB]

    ; update the send data
    ld a, [WRAM_DATA_SEND]
    inc a
    and a, SERIAL_VALUE_MASK
    ld [WRAM_DATA_SEND], a
    ld [rSB], a
```

```
; switch role
copy [WRAM_ROLE], SERIAL_ROLE_PARENT
ret
```

Listing 11-12: Switching roles in the interrupt functions

Take some time to compare this snippet of code to the ones from Listings 11-9 and 11-11. The core of the interrupt functions (the part that sends and receives a byte of data) doesn't change at all, but at the end of the interrupt, both sides change their role. The parent becomes passive, setting its timeout and getting ready as a child. The child sets a counter to trigger the next transfer as it becomes the parent.

The role switching does not fundamentally change the behavior of this particular sample, but in a situation where the serial communication takes the form of a request/response exchange, switching roles after a request has been sent might be convenient. The main takeaway here is that it is up to each programmer to create a protocol that fits the needs of their program.

Chaining Transfers

The samples we've used so far in this chapter performed at most one transfer per frame. In this section, you'll explore a sample that performs several transfers in a row. You'll learn how to make such transfers reliable and how to avoid the common pitfall of starting the transfers too fast.

In theory, it's possible to perform up to 17 transfers per frame, assuming the transfers are performed back to back. In practice, we need to add some delay between transfers, so reaching 17 transfers per frame is unlikely. These delays are necessary because we can't fully control when the child becomes ready for a transfer. When the parent triggers the transfer, the child could be at any stage of its frame. The parent has no direct way to verify that the child is ready. Adding a delay between transfers gives the child enough headroom to get ready.

When computing the necessary delay between transfers, you need to take three factors into account. First, you need to account for the duration of a transfer itself, which is constant and equal to 1,024 cycles. Second, you need to check the difference in duration between the parent serial interrupt and the child serial interrupt. That difference is only relevant when the child's serial interrupt is longer than the parent's, because that will mean the child takes more time to process the transfer. If the child's interrupt is shorter, then you can ignore the duration difference. Finally, you must consider the duration of any other potential interrupt on the child. For example, a vblank interrupt could occur at any time on the child, including between two transfers that happen during a single frame on the parent.

Let's see how all this works using the *serial_multibytes* sample on GitHub. This sample is very similar to the one in the previous section, but this time, instead of triggering a single transfer at a time the parent performs three transfers back to back. The sample includes a bunch of code to manage the fact that there are three received and sent values, which I won't show

here as it's not relevant to serial communication. Figure 11-10 shows what it looks like.



Figure 11-10: The multibytes sample

Listing 11-13 is the portion of the parent's update function that performs the transfers.

```
    copy [rSC], $81
    call WaitTransfer
    copy [rSC], $81
    call WaitTransfer
    copy [rSC], $81
```

Listing 11-13: Performing multiple transfers

The function starts three transfers, as opposed to the single transfer from Listing 11-8. It introduces a delay between each transfer by calling the `WaitTransfer` function.

Let's compute the delay that `WaitTransfer` should introduce. First, we have the transfer duration of 1,024 cycles. Analyzing the interrupt code reveals that the child's serial interrupt is shorter than the parent's (by a few cycles), so we can ignore the duration difference. Finally, we need to consider that a vblank interrupt might occur on the child during transfers. That interrupt is 17 cycles long, but when the CPU handles the interrupt it takes about 5 extra cycles, for a total of 22 cycles. So, we can conclude that the delay should be at least 1,046 cycles (1,024 + 17 + 5). A `call` takes 6 cycles, so the `WaitTransfer` function should last at least 1,040 cycles.

Listing 11-14 shows how the function is implemented.

```
WaitTransfer:
    ld c, 104
    .wait_loop
        nop
        nop
        nop
```

```
        nop
        nop
        nop
        dec c
        jr nz, .wait_loop
    ret
```

Listing 11-14: Implementing the delay function

This function lasts 1,045 cycles. With the 6 cycles of the `call` instruction, it adds up to 1,051 cycles, which is long enough to cover the required delay of 1,046 cycles.

This means we'll be waiting almost a millisecond between each transfer. In a real program, this would be a huge waste of CPU time. There are several ways to deal with that.

First, the CPU could do actual work instead of wasting cycles within an active wait such as `WaitTransfer`. For example, it could update the game logic, move data around in the memory map, or set audio parameters. Any task that reaches the required number of cycles between two transfers is a good fit.

Second, assuming there is no other work to be done, you could use `halt` to pause the CPU. The CPU would wake up when the serial interrupt triggers, and execution of the code after `halt` would resume after the serial interrupt function returns. Instead of 1,046 cycles, waiting 22 cycles (to take into account the potential vblank interrupt on the child) would be enough and would save a significant amount of battery life.

Finally, you could set the DMG timer so that it triggers an interrupt after at least 1,046 cycles. The next transfer would then start in this interrupt. When all three of the frame's transfers were done, you could stop the timer to prevent any other interrupts. Using the timer interrupt to manage the state of the data transfers removes the need for unwieldy tracking from the main loop.

The *serial_multibytes_timer* sample leverages the timer in this way. Listings 11-15 through 11-18 show the most relevant parts of this sample. The first transfer of the frame and the timer are both started in the parent's update function, as shown in Listing 11-15.

```
    copy [rSC], $81
    call StartTimer
```

Listing 11-15: Transfer and timer start

Listing 11-16 shows the `StartTimer` function.

```
StartTimer:
    ld a, 256 - 66
    ld [rTMA], a
    ld [rTIMA], a
    ld [rDIV], a
    copy [rTAC], TACF_START | TACF_65KHZ
    ret
```

Listing 11-16: The timer start function

This sets the timer frequency to 65,536 Hz in `rTAC`. The timer counter, `rTIMA`, gets incremented every 16 cycles ($2^{20} \div 65,536$), offering enough granularity to get close to 1,046 cycles. The timer modulo, `rTMA`, is set so that `rTIMA` overflows every 66 ticks. This means the interrupt occurs every 1,056 cycles ($16 \times 66 = 1056$), which is safely longer than the necessary minimum delay.

Listing 11-17 shows the `StopTimer` function.

```
StopTimer:
    copy [rTAC], TACF_STOP
    ret
```

Listing 11-17: The timer stop function

This function stops the timer once all of the current frame's transfers complete.

Listing 11-18 shows the interrupt function.

```
TimerInterrupt:
    call StopTimer

    ld a, [WRAM_DATA_INDEX]
    or a
    ret z

    copy [rSC], $81
    call StartTimer

    ret
```

Listing 11-18: The timer interrupt function

By default, the interrupt function stops the timer and checks the value of `WRAM_DATA_INDEX`. That variable represents the current transfer index and is updated inside the serial interrupt function each time a transfer ends. Before any transfer starts, it equals 0. After the first transfer, it equals 1. After the second transfer, it equals 2. Finally, after the third and last transfer, it resets to 0 to prepare for the next frame. An index with a value of 0 means that all the transfers for that frame are done. In that case, the interrupt just returns. If there are still transfers to be done, the next transfer starts (via `rSC`) and the timer restarts.

Using a timer to handle the delay allows us to avoid cumbersome tracking inside the frame. As long as the first transfer starts early enough in the frame that the timer interrupt and subsequent transfers don't bleed into the next frame, you can get a pretty reliable multibyte transfer.

Summary

In this chapter, you learned how to transfer bytes of data between a DMG and another device (typically, another DMG). You should now have a good grasp of the parent/child protocol, which you can build on to design new protocols that suit the needs of your programs.

You also learned different techniques for chaining transfers, including the timer interrupt that keeps the transfer management out of the main loop. We'll use serial communication again in Chapter 17, on the Game Boy Printer.

The next chapter introduces new graphics concepts and advanced graphics techniques.

12

ADVANCED GRAPHICS

This chapter covers advanced DMG graphics functionalities, such as horizontal blanking, that weren't covered in the previous chapters. You'll start by learning about the LCD modes and how you can access the VRAM even outside the vblank.

Then you'll see implementations of a few advanced techniques that allow you to enhance the graphics of your programs, such as parallax scrolling, which creates the illusion of depth with 2D graphics, and a palette trick to create nice screen transitions between menus or scenes in your programs.

Next, you'll learn about direct memory access, a hardware feature that shortens the duration of OAM updates dramatically, giving you more time in the vblank to implement additional graphics effects. You'll also learn a trick to go beyond the 40-sprite limit discussed in Chapter 7.

LCD Details

This section introduces two advanced LCD topics, modes and event tracking. We'll put these to use in the following section in a few samples that implement parallax scrolling.

Modes

Until now, this book has considered the frame as being split into two phases: the LCD refresh (15.67 ms for 144 lines) and the vblank (1.08 ms for the equivalent of 10 lines). The frame is actually split into more granular phases called *modes*. Understanding modes allows you to pull off some interesting graphics tricks like parallax scrolling, which we'll look at shortly. In this section, we'll explore a sample that you can only implement using modes.

There are four modes: OAM scan, rendering, horizontal blanking, and vertical blanking. During the LCD refresh, the DMG cycles between the first three modes. On each line, it first enters the *OAM scan*, also known as mode 2, during which the PPU determines which sprites are visible on the line. The DMG then switches to *rendering*, also known as mode 3, where the PPU draws to the LCD. Near the end of the line, the DMG goes into the *horizontal blanking* interval, or *hblank*, also known as mode 0. Graphics-wise, nothing happens during the hblank. Finally, after the LCD refresh, the DMG switches to mode 1, which is the vertical blanking interval (vblank) you learned about in previous chapters.

Each mode has its own duration and OAM/VRAM access rules, as summarized in Table 12-1.

Table 12-1: Modes

Mode	Name	Duration (cycles)	OAM/VRAM access	Notes
0	Hblank	22–51 per line	Both	Full OAM/VRAM access once per line
1	Vblank	1,140 per frame	Both	Full OAM/VRAM access once per frame
2	OAM scan	20 per line	VRAM only	PPU compiles a list of sprites on the line
3	Rendering	43–72 per line	—	PPU draws 160 pixels to the LCD

During modes 2 and 3, the OAM and VRAM are susceptible to access restrictions. This makes sense because the PPU accesses them to draw an image to the LCD. Other parts of the memory map (ROM, WRAM, HRAM, I/O registers, and so on) are fully accessible in any mode.

Figure 12-1 represents how the modes relate to the lines of the frame.

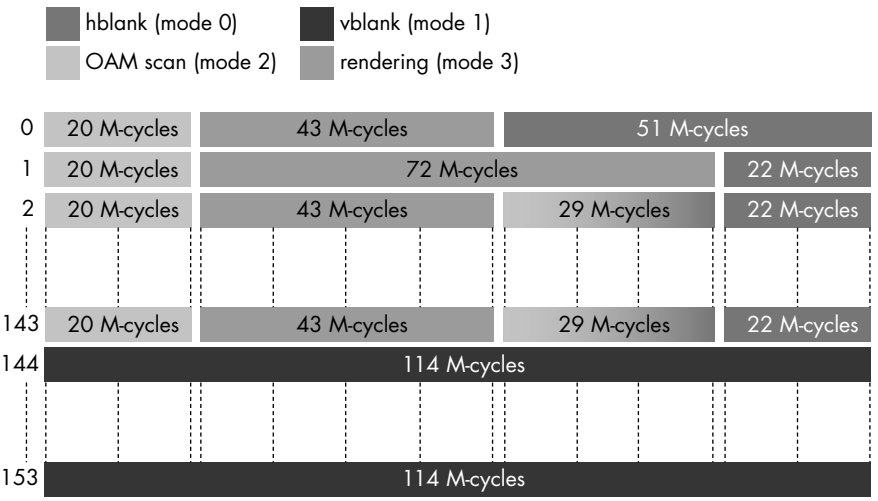


Figure 12-1: A visualization of the four modes

As described previously, each line starts with an OAM scan (mode 2), followed by rendering (mode 3) and finally the hblank (mode 0). The hblank and rendering have variable durations, but their combined duration is always 94 cycles. The factors that increase the rendering duration are window usage, background scrolling, and the presence of sprites. Sprites have the biggest impact on the rendering duration.

During the hblank, both OAM and VRAM are fully accessible, just like during the vblank. The hblank lasts at least 22 cycles, but you can expect more cycles if the line has barely any sprites (in which case the rendering duration will be shorter). The maximum duration is 51 cycles. At the end of the hblank, the OAM scan of the next line starts, prohibiting any OAM access. The VRAM is still accessible, though, because the OAM scan does not block VRAM access. This means that, even in the worst case, there are at least 42 cycles in each line (at least 22 from the hblank plus 20 from the OAM scan) during which VRAM is fully accessible.

Event Tracking

The DMG has three I/O registers dedicated to tracking modes and line status. The first is the *line Y* register, `rLY`, which you learned about in Chapter 6 (see Listing 6-5). This register is read-only and holds the current line number, from 0 to 153.

The second is the *LY compare* register, `rLYC`. The DMG constantly compares the value in this register to that of `rLY`, which is handy for tracking a particular line. We'll see how to leverage that in the samples in the next section.

The third and last register is the *LCD status* register, `rSTAT`. Table 12-2 summarizes how this register's bits are used.

Table 12-2: The LCD Status Register

Bits	Access	Usage
6	Read/write	Triggers the LCD interrupt when `rLY=rLYC`
5	Read/write	Triggers the LCD interrupt when entering mode 2 (OAM scan)
4	Read/write	Triggers the LCD interrupt when entering mode 1 (vblank)
3	Read/write	Triggers the LCD interrupt when entering mode 0 (hblank)
2	Read-only	`rLY=rLYC` flag (line comparison flag)
1–0	Read-only	Current mode

The lowest 3 bits are read-only. They provide information about the current mode and whether `rLY` and `rLYC` are equal.

The remaining 4 bits are the LCD interrupt trigger flags. The interrupt itself is enabled via a flag in the `rIE` register, like any other interrupt. When this occurs, an interrupt function (located at `$0048`) is called, again like any other interrupt. However, the LCD interrupt is a *four-in-one* interrupt, which means it can be triggered by four different events: entering mode 0, entering mode 1, entering mode 2, and when `rLY` is equal to `rLYC`. Each of these events is enabled independently by setting a corresponding trigger flag in `rSTAT`.

Bits 3, 4, and 5 in the status register are for tracking modes 0, 1, and 2, respectively. Mode 3 doesn't offer any particular benefits in terms of OAM/VRAM access, so tracking it would not be particularly useful. Bit 6 is used for tracking whether `rLY` equals `rLYC`.

When an LCD interrupt is triggered, bits 0 to 2 are leveraged to determine which condition(s) caused the interrupt. This is necessary because two different sources can request the interrupt at the same time: Modes are mutually exclusive (the DMG can be in only one mode at any given time), but the line comparison flag could be raised during any mode.

In addition, there's a bug called the *`rSTAT` bug* in the DMG that it's important to be aware of. When a value is written to the `rSTAT` register, it may trigger an LCD interrupt. This can occur even if you're setting `rSTAT` to zero to clear the LCD interrupt trigger flags. To counter this problem, you need to read the information bits of `rSTAT` from inside the interrupt function, to make sure the interrupt occurred under the expected conditions. Reading from `rSTAT` does not cause any issue. This bug has been fixed in more recent hardware models (CGB and later), but we'll look at a sample that triggers it on the DMG in the next section.

Graphics Tricks

In this section, we'll explore a few programming tricks that enhance the visual quality of programs: parallax scrolling and palette fades.

Parallax Scrolling

A common technique in retro video games, *parallax scrolling* creates the illusion of depth by scrolling parts of the background at different speeds. Parts that are supposed to be farther away from the point of view scroll slowly, while closer parts scroll faster.

Figure 12-2 is a screenshot of this section's sample, which implements parallax scrolling.

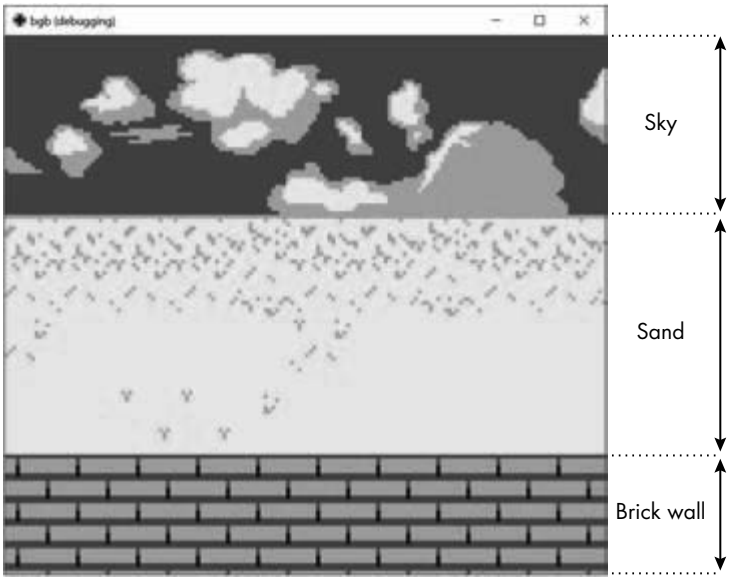


Figure 12-2: The parallax scrolling sample

The background has three parts: the sky, the sand, and a brick wall. The sky extends from line 0 to 47, the sand from line 48 to 111, and the wall from line 112 to 143. The sample causes the sky, sand, and wall to scroll horizontally at low, medium, and fast speed, respectively.

To accomplish this, we change the value of `rSCX`, the register that controls horizontal scrolling of the background, at carefully chosen times during the LCD refresh. At the very start of the LCD refresh, `rSCX` is set to the sky's scrolling value. When we enter the hblank at line 47, `rSCX` switches to the scrolling value for the sand. Then, at the end of the hblank (line 111), `rSCX` switches to the wall's scrolling value. It's important to change `rSCX` during the hblank, when the DMG does not draw anything to the LCD. Otherwise, it would cause graphical artifacts in the middle of the lines.

The *parallax* sample uses a 3-byte variable, declared in Listing 12-1, to track the parallax scroll state:

```
def WRAM_BG_SCROLL_X                rb 3
```

Listing 12-1: Declaring the parallax scroll variable

These three bytes will store the scrolling values for the sky, the sand, and the wall.

Listing 12-2 shows the interrupt setup.

```
copy [rLYC], 255
copy [rSTAT], STATF_MODE00
ld a, IEF_VBLANK | IEF_LCDC
ld [rIE], a
ei
```

Listing 12-2: Setting up the interrupts

First, the LY compare register (`rLYC`) is set to 255. This is equivalent to disabling line comparison because the line number (`rLY`) is between 0 and 153, so it's never equal to 255. Next, the mode 0 (hblank) trigger flag is raised in `rSTAT`, and the `IEF_LCDC` flag is added to `rIE` to enable the LCD interrupt. With `rSTAT` and `rIE` set this way, the LCD interrupt function will be called each time the DMG enters the hblank. Note that all this setup is performed during the initialization of the sample (`InitSample`). Because the LCD is disabled during initialization, the `rSTAT` bug I mentioned in the previous section won't occur.

Listing 12-3 shows the update function of the sample.

```
; update the 3 scrolling values
ld a, [WRAM_BG_SCROLL_X + 0]

inc a
ld [rSCX], a ; set the scrolling to the sky value
ld [WRAM_BG_SCROLL_X + 0], a

ld b, a
sla a
ld [WRAM_BG_SCROLL_X + 1], a

add a, b
ld [WRAM_BG_SCROLL_X + 2], a

; set the next target line
copy [rLYC], 47
```

Listing 12-3: The update of the parallax effect

Right after the vblank interrupt, the function updates the three scrolling values to their new values for the next frame. The sky scrolls 1 pixel at a time, the sand 2 pixels, and the wall 3 pixels. The `rSCX` register is set to the sky scrolling value, getting the parallax ready for the next LCD refresh. At the end of the vblank, `rLYC` is set to the first target line, the last line of the sky (47). This will cause the line comparison flag in `rSTAT` to be raised once the DMG is processing line 47.

The last piece of the parallax mechanics is the LCD interrupt, shown in Listing 12-4.

```
LcdInterrupt:
    ; check that we are in mode 0 on the target line
    ld a, [rSTAT]
    and a, STATF_LYCF | %00000011
    cp a, STATF_LYCF
    ret nz

    ; set up the sand scrolling
    ld a, [rLY]
    cp a, 47
    jr nz, .middle_section
        copy [rSCX], [WRAM_BG_SCROLL_X + 1]
        copy [rLYC], 111
        ret
    .middle_section

    ; set up the wall scrolling
    copy [rSCX], [WRAM_BG_SCROLL_X + 2]
    copy [rLYC], 255
    ret
```

Listing 12-4: The LCD interrupt function

The first few lines, up to `ret nz`, check that all the conditions are met to update the scrolling value in `rSCX`. In `rSTAT`, the mode must be equal to 0 (hblank) and the line comparison flag (bit 2) must be raised. These two conditions translate to the lowest 3 bits of `rSTAT` being equal to `STATF_LYCF` (`%00000100`).

Next, the code path splits depending on whether the current line is 47 or 111. For line 47, the scrolling value sets to that of the sand section and the target line updates to 111 to handle the wall section later. For line 111, the scrolling value sets to that of the wall section and line comparison is disabled by setting the target line back to 255.

Once compiled, the sample works as expected, but there is one catch: An interrupt is triggered on every single line between 0 and 143, but meaningful work is really only done on lines 47 and 111. That means there are 142 useless interrupts that just return early. An early return costs 33 cycles (handling of the interrupt by the DMG plus code at `$0048` for the instructions up to `ret nz`). Over 142 lines, that's 4,686 cycles for a single frame. Because a frame is 17,556 cycles long, this means over 26 percent of the frame CPU time is wasted. This is not a big issue for this particular sample, but in a real program, it would result in a massive and unacceptable loss of performance.

Thankfully, we can improve the situation by enabling only the line comparison interrupt at first. Once the interrupt triggers, it enables the hblank interrupt for the duration of the line.

Because we'll write to the `rSTAT` register every frame, we'll need to deal with the `rSTAT` bug. Let's see how this works in code using the *parallax_optimized* sample. Listing 12-5 shows the new initialization phase.

```
    copy [rLYC], 255
    xor a
    ld [rSTAT], a
    ld a, IEF_VBLANK | IEF_LCDC
    ld [rIE], a
    ei
```

Listing 12-5: The optimized parallax sample initialization

The only change is that this resets `rSTAT` to zero instead of enabling the hblank interrupt from the start, because we want hblank interrupts to trigger only when they are strictly required for the parallax effect.

We add a single line to the update function to set the line comparison interrupt to trigger on line 47, as shown in Listing 12-6.

```
    copy [rLYC], 47
    copy [rSTAT], STATF_LYC
```

Listing 12-6: The optimized parallax sample update function

After setting the first target line to 47, this function enables the line comparison interrupt. It will very likely trigger the `rSTAT` bug once, because a value is written to the register during the frame when the LCD is enabled. We'll deal with this issue from inside the LCD interrupt function. Listing 12-7 shows the changes to that function.

```
LcdInterrupt:
    ; check whether we are on the target line
    ld a, [rSTAT]
    bit 2, a
    ret z

    ; enable the hblank interrupt (as we are on the target line)
    ; and check if we are in hblank
    ld b, a
    copy [rSTAT], STATF_MODE00
    ld a, b
    and a, %00000011
    ret nz

    ; set up the sand scrolling
    ld a, [rLY]
    cp a, 47
    jr nz, .middle_section
        copy [rSCX], [WRAM_BG_SCROLL_X + 1]
        copy [rLYC], 111
        copy [rSTAT], STATF_LYC
        ret
    .middle_section
```

```
    ; set up the wall scrolling
    copy [rSCX], [WRAM_BG_SCROLL_X + 2]
    copy [rLYC], 255
    xor a
    ld [rSTAT], a
    ret
```

Listing 12-7: The optimized parallax sample interrupt function

The first few lines, up to `ret z`, check whether the target line was reached. This check is only necessary because of the `rSTAT` bug, which causes the interrupt to trigger on writes to the register.

The next lines, up to `ret nz`, enable the hblank interrupt (and disable the line comparison interrupt). They also check whether the current mode is hblank. When the line comparison interrupt occurs, the hblank interrupt is still disabled; it must be enabled (with `copy [rSTAT], STATF_MODE00`) so that the hblank of the target line is caught by an interrupt. But at that point, the DMG is not yet in the hblank—it's more likely to be in the OAM scan, which is the first mode on each line—so the function just returns early. When the actual hblank interrupt occurs, the code to enable that interrupt executes again. This is redundant but harmless. However, because the DMG is in the hblank, the function does not return early, and the code meant for the hblank executes.

The rest of the function is very similar to that of Listing 12-4. This version adds a bit of interrupt trigger management, which consists of updating the `rSTAT` register. After line 47, it disables the hblank interrupt to avoid a series of unnecessary interrupts between lines 47 and 111. The line interrupt is re-enabled to catch the start of line 111. After line 111, all LCD interrupt triggers are disabled, to avoid any needless interrupts until the next frame.

Writing to `rSTAT` from inside the interrupt function won't trigger any unwanted interrupts. Interrupts are automatically disabled inside interrupt functions (which is why we need a call to `reti` in the first place), so the `rSTAT` bug won't occur.

After all these changes, the LCD interrupt will trigger only five times per frame: once due to the `rSTAT` bug, twice for line comparisons, and twice for hblanks. This is a good improvement over the 144 interrupts (142 of which were useless) triggered in the previous sample. The only waste is due to the `rSTAT` bug and costs around 28 cycles per frame. Overall, the parallax mechanics amount to less than 0.2 percent of the total cycle count in the frame, compared to over 26 percent in the previous sample.

NOTE *Parallax scrolling is a nice technique, but on early DMG hardware the LCD suffers from heavy ghosting, where graphics from the previous frames are still partially visible on the LCD, causing an undesirable blurring effect. You'll need to test your program on real hardware to make sure it has the effect you're looking for. This section's sample is good enough to demonstrate the parallax technique, but the scrolling is actually way too fast to look good on the original DMG.*

Palette Fades

Palette fades are eye-pleasing ways to transition between menus or scenes in a program. To achieve a smooth transition, we switch gradually between at least four different palettes. There are two main types of fades: fade to black and fade to white.

The *fade to black* option gradually transitions from the normal palette to a palette containing only the color black. The *fade to white* option transitions to a palette containing only the color white. Figure 12-3 shows a series of screenshots of a fade to white animation from the *palette_fade* sample.

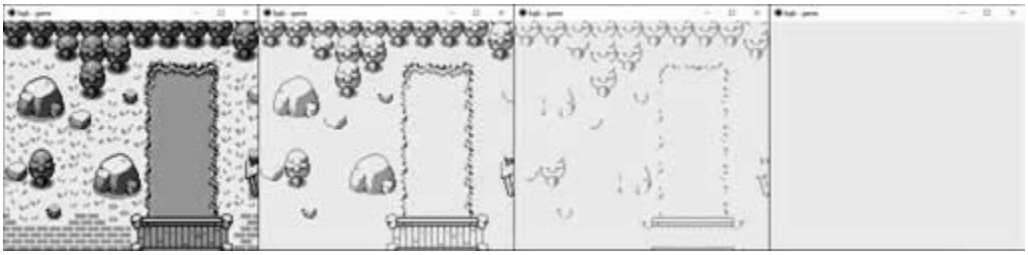


Figure 12-3: A sample showing palette fades

The visible effect is that colors gradually shift toward white until every pixel on the screen is white.

The sample demonstrates both fade types. The LEFT key starts a fade to black, the RIGHT key starts a fade to white, and the UP key restores the original state.

Listing 12-8 shows how to add the palettes to the ROM.

```
rsreset
def FADE_LEVEL_DARKEST      rb 1
def FADE_LEVEL_DARKER       rb 1
def FADE_LEVEL_DARK         rb 1
def FADE_LEVEL_NORMAL       rb 1
def FADE_LEVEL_LIGHT        rb 1
def FADE_LEVEL_LIGHTER      rb 1
def FADE_LEVEL_LIGHTEST     rb 1
def FADE_LEVELS_COUNT       rb 0

palettes:
db %11111111
db %11111110
db %11111001
db %11100100
db %10010000
db %01000000
db %00000000
palettes_end:
```

```
def PALETTES_COUNT          equ (palettes_end - palettes)

assert (palettes_end - palettes) == FADE_LEVELS_COUNT, \
```

Listing 12-8: Declaring palettes for fading to black and white

First, the code declares seven constants as a convenient way to refer to each palette in the code. For example, `FADE_LEVEL_DARKEST` refers to the fully black palette. Then, it adds seven palettes to the ROM, between the `palettes` and `palettes_end` labels. The first palette is full black (`%11111111`), the fourth one is the *common palette* (`%11100100`) we used in many of this book's samples, and the seventh and last one is full white (`%00000000`). These three palettes are *end states*. The other palettes are *in-betweens* used to transition smoothly from one end state to another.

The palette fading system uses only three variables: the current fade level (`WRAM_FADE_CURRENT_LEVEL`), the target fade level (`WRAM_FADE_TARGET_LEVEL`), and an animation frame counter (`WRAM_FADE_FRAME_COUNTER`). The *current fade level* corresponds to the palette currently used for rendering. The fade animation system updates this level automatically. The *target fade level* is the end state that the animation is transitioning to. Once the current fade level equals the target fade level, the animation is over. The *animation frame counter* controls the speed of the fade animation, because updating the fade level at every single frame would cause the animation to be too fast.

Listing 12-9 shows how we initiate a fade to black animation.

```
    copy [WRAM_FADE_TARGET_LEVEL], FADE_LEVEL_DARKEST
    copy [WRAM_FADE_FRAME_COUNTER], FADE_ANIM_FRAME_COUNT
```

Listing 12-9: Initiating a fade

This sets the target fade level and resets the animation counter.

Every frame, the code updates the palette based on the current fade level, as shown in Listing 12-10.

```
    ld hl, palettes
    ld a, [WRAM_FADE_CURRENT_LEVEL]
    ld d, 0
    ld e, a
    add hl, de
    ld a, [hl]
    ld [rBGP], a
```

Listing 12-10: Updating the palette

This code computes the address of the palette for the current fade level and saves it in the `hl` register. The code then updates the background palette, `rBGP`, with the palette stored at the address in `hl`. This update must occur during the vblank to avoid any change of color in the middle of the LCD refresh.

Finally, Listing 12-11 shows the code that animates the current fade level.

```
ld hl, WRAM_FADE_CURRENT_LEVEL
ld a, [WRAM_FADE_TARGET_LEVEL]
cp a, [hl]
jr z, .check_animation
    ld hl, WRAM_FADE_FRAME_COUNTER
    dec [hl]
    jr nz, .animate_fade
        ld [hl], FADE_ANIM_FRAME_COUNT

        ld hl, WRAM_FADE_CURRENT_LEVEL
        ld a, [WRAM_FADE_TARGET_LEVEL]
        cp a, [hl]
        jr c, .fade_up
            ; the target level is superior to the current level
            inc [hl]
            jr .fade_done
        .fade_up
            ; the current level is superior to the target level
            dec [hl]
        .fade_done
    .animate_fade
.check_animation
```

Listing 12-11: Animating the fade

This code first checks whether the animation is over by comparing the current fade level to the target fade level. Then, it handles the frame counter and determines whether the current fade level must be updated. Finally, it brings the current fade level one step closer to the target fade level.

NOTE *This sample only manages the background palette. To apply the fade to sprites, we would need to improve the sample so that the sprite palettes gets updated according to the fade level, just like the background palettes are.*

Direct Memory Access

In this section, you'll learn about another advanced technique: *direct memory access (DMA)*, a fast way to copy sprite attributes from ROM or WRAM to OAM. This allows you to free up time to process other VRAM-related operations during the vblank.

The OAM is 160 bytes, and using DMA, it's possible to copy all these bytes in 160 cycles. That's a single cycle per byte, about six times faster than what we can achieve using load operations. While the speed gain is tremendous, there is a catch. During a DMA copy, the CPU can only access the HRAM. That means it can't run instructions from anywhere else, including

from the ROM itself, so we'll have to run instructions from HRAM while the DMG is processing a DMA copy.

This section explains how to take full advantage of DMA. First, we'll go through a sample that does not use DMA to copy its sprite data to OAM. Then, we'll explore a sample that uses DMA to highlight the necessary changes to support the fast copy of sprite data. Finally, you'll learn about a DMA technique that lets you display more than 40 sprites per frame.

Copying to OAM Without DMA

The *dma_none* sample illustrates how a standard copy of 160 bytes to OAM would work. This sample will serve as a basis for adding DMA support in the next sample.

Listing 12-12 shows the code executed during the vblank, which performs the copy from WRAM (`WRAM_SPRITES`) to OAM.

```
    ; transfer the sprites to OAM
    ld hl, _OAMRAM
def REPT_BYTE_NUM = 0
    REPT OAM_BYTES_COUNT
        ld a, [WRAM_SPRITES + REPT_BYTE_NUM]
        ld [hli], a
def REPT_BYTE_NUM = REPT_BYTE_NUM + 1
    ENDR
```

Listing 12-12: Copying the data to OAM with load operations

The whole copy takes 963 cycles, which is quite good considering it uses load operations. This isn't the most byte-efficient way to copy 160 bytes of data (it takes 643 bytes of ROM), but this sample is a good illustration of the use of two new constructs that we'll examine now: repeat blocks and mutable constants.

Repeat Blocks

You can use *repeat blocks* to repeat a block of code a certain number of times. For example, the following code would repeat the `nop` operation five times:

```
REPT 5
    nop
ENDR
```

This code would generate the following output:

```
    nop
    nop
    nop
    nop
    nop
```

Repeat blocks can be really handy when used in conjunction with `inc`, `dec`, or `hli`, because they make the code shorter and easier to modify. They also work very well with mutable constants.

Mutable Constants

You can use mutable constants to generate even more complex code with repeat blocks. You might find the name confusing, because no one would expect a constant to be anything but immutable.

We've already used immutable constants, which are defined with `equ`, in several places in this book. Their value cannot change across the whole file in which they are defined. *Mutable constants* are defined or modified with `=`, and they work like immutable constants for the most part, except that their value can be changed at any time. While that might seem to defeat the purpose of using a constant in the first place, mutable constants are actually very powerful when used in conjunction with repeat blocks because they allow us to use compile-time formulas that get evaluated for each iteration of the repeat block.

The following code uses a repeat block and a mutable constant together:

```
    xor a
def COUNT = 1
    REPT 4
        add a, COUNT
def COUNT = COUNT + 1
    ENDR
```

The mutable constant's value is modified inside the repeat block. This generates the following code:

```
    xor a
    add a, 1
    add a, 2
    add a, 3
    add a, 4
```

As you can see, each iteration of the repeat block generates a slightly different line of code because the value of the mutable constant updates each time.

Using repeat blocks and mutable constants can dramatically lower the number of lines in a program, especially when some kind of boilerplate code is necessary. Don't overuse this technique, though, as the byte cost of the generated code can quickly add up. Sometimes, a good old loop is more efficient.

Returning to Listing 12-12, the code inside the repeat block copies a single byte from WRAM to OAM. Thanks to the mutable constant, a value is grabbed from a different address each time the code is repeated. Thanks to `hli`, that value is copied to a different location in OAM each time as well.

For the sake of completeness, Listing 12-13 shows an alternative implementation that consumes significantly fewer bytes than the version in Listing 12-12.

```
ld de, WRAM_SPRITES
ld hl, _OAMRAM
REPT OAM_BYTES_COUNT
    ld a, [de]
    ld [hli], a
    inc de
ENDR
```

Listing 12-13: A more efficient alternative

This code takes an extra three cycles to execute, but it takes only 486 bytes of ROM instead of the original 643 bytes because it uses load instructions working on registers instead of load instructions working on hard-coded addresses (for example, `ld a, [de]` instead of `ld a, [WRAM_SPRITES + REPT_BYTE_NUM]`).

The sample looks like Figure 12-4.



Figure 12-4: The DMA sample

There are 40 sprites moving left and right over 5 lines. In the next section, we'll modify the code to leverage DMA, which dramatically speeds up the sprite data copy to OMA, but the sample will keep the same look.

Leveraging DMA to Copy to OAM

Now let's explore how to use DMA by replacing the load operations of the previous sample with a DMA copy.

To start a DMA copy, we must specify the source address of the data to copy to OAM. The address format is `$XX00`, with the first two values ranging from `$00` to `$DF`. We always assume the low byte to be `$00`, so we don't specify it. We specify the high byte via the `rDMA` register. The copy starts as soon as a value is written to `rDMA`. We expect the source data to be 160 bytes long,

and that much will be copied to OAM for every DMA copy. We need to ensure that the data contains 160 properly initialized bytes, as otherwise random sprites may show up on the screen. The destination address is always the OAM (`$FE00` to `$FE9F`), so there's no need to specify it. An example of a valid source address would be `$6400`. The corresponding value written to `rDMA` would be `$64`, and the range of data copied to OAM would be `$6400` to `$649F`.

NOTE *Technically, the source address of a DMA copy could be in the ROM, VRAM, ERAM, or WRAM. However, VRAM and ERAM are unlikely places to store sprite data, because the quantity of VRAM is quite limited and ERAM is mostly used for game saves. I'll cover only the ROM and WRAM options in this book.*

We'll use the *dma* sample on GitHub in this section. In this sample, the data is held in `WRAM_SPRITES`, a 160-byte memory area in WRAM. Because `WRAM_SPRITES` is at the start of the WRAM, its starting address is `$C000`. This is a valid address, but it's good practice to check programmatically that the address stays valid. An assert, such as the one shown in Listing 12-14, is the perfect tool for this.

```
assert low(WRAM_SPRITES) == 0, "WRAM_SPRITES must be 256-byte-aligned"
```

Listing 12-14: A source address alignment check

Now, if the address ever becomes invalid (for example, due to it being moved around in WRAM), an error will occur during compilation, saving you a lot of debugging time.

Next comes the tricky part. Once a DMA copy starts, it takes 160 cycles to complete. The CPU can access only the HRAM during that time, so the copy must start from the HRAM. Then you must implement some kind of loop to wait for the copy to end, and that loop's code must reside in HRAM. This means you need to reserve some space in HRAM for a function that will both start the DMA copy and run the wait loop until the copy is done.

In Listing 12-15, we reserve a space of `DMA_FUNC_SIZE` bytes in HRAM to hold the DMA copy function. I'll show you the formula for computing `DMA_FUNC_SIZE` in Listing 12-17.

```
rsset _HRAM

def HRAM_DMA_FUNC                    rb DMA_FUNC_SIZE

def HRAM_END                         rb 0

; sanity checks
def HRAM_USAGE                       equ (HRAM_END - _HRAM)
println "HRAM usage: {d:HRAM_USAGE} bytes"
assert HRAM_USAGE <= $40, "Too many bytes used in HRAM"
```

Listing 12-15: Reserving the necessary space in HRAM

The assert avoids us consuming more than half (64 bytes) of the HRAM's 127 bytes. It's best to limit consumption in this way to avoid any unintended overlap with the stack, which uses the top of the HRAM. The function should easily fit in 64 bytes, because it's only 10 bytes long.

In the update function of the sample, we replace the load operations from Listing 12-12 with a single function call: `call DmaCopy`. The `DmaCopy` function, defined in Listing 12-16, jumps to the reserved address in HRAM.

```
DmaCopy:
    jp HRAM_DMA_FUNC
```

Listing 12-16: Jumping to the reserved location

This ensures that the CPU goes on executing code from HRAM, which is necessary because ROM access will be blocked as soon as the DMA copy starts. Listing 12-17 shows the function that starts the DMA copy and implements the wait loop.

```
DmaCopyFunc:
    ld a, high(WRAM_SPRITES)
    ldh [rDMA], a
    ld c, 40
    .wait_copy
        dec c
        jr nz, .wait_copy
    ret
DmaCopyFunc_End:

def DMA_FUNC_SIZE equ (DmaCopyFunc_End - DmaCopyFunc)
```

Listing 12-17: Initiating the DMA copy

This function first puts the high byte of `WRAM_SPRITES` into `rDMA`, starting the DMA copy. Then, the wait loop goes through 40 iterations, each of which is 4 cycles long, for a total of 160 cycles (The last iteration is only 3 cycles, but the `ret` instruction safely brings the total wait time over 160 cycles). Finally, we compute the function size into a constant (`DMA_FUNC_SIZE`), which was used previously to reserve space in the HRAM (see Listing 12-15).

Note that the `DmaCopyFunc` function in Listing 12-17 resides in the ROM. It cannot be called as is; its code must first be copied to HRAM, because that's where `DmaCopy` in Listing 12-16 expects it to be.

Listing 12-18 shows the `InitDmaCopy` function that performs the code copy. This function is called during the initialization of the sample.

```
InitDmaCopy:
    ld hl, HRAM_DMA_FUNC
    ld de, DmaCopyFunc
    ld c, DMA_FUNC_SIZE
    .func_copy
        ld a, [de]
```

```
        ld [hli], a
        inc de
        dec c
        jr nz, .func_copy
    ret
```

Listing 12-18: DMA initialization

Once this function has copied all the code from `DmaCopyFunc` into HRAM, the program is ready to leverage the `DmaCopy` function. The cost of the `DmaCopy` function is 182 cycles: 160 cycles from the wait loop plus 22 cycles of overhead from `call`, `jp`, `ret`, and the various loads required to make the DMA work.

To sum up, to leverage DMA, you'll need to take the following steps:

1. Reserve some space in HRAM for the function handling the DMA copy (here, `DmaCopyFunc`).
2. Copy that function into HRAM (we did this using `InitDmaCopy`).
3. Replace the load operations with a call to a function in HRAM (here, `DmaCopy`).

When you run this sample, it should look exactly the same as the previous one that used load operations. The only difference is that it leverages the DMA functionality.

Whether DMA is more efficient than load operations depends on the number of sprite attributes you set in each frame. Considering an optimal speed of 6 cycles per attribute using load operations, using DMA will become more efficient when more than 182 / 6 = 30 attributes are set per frame. In practice, though, it's probably more efficient for a lower number of attributes due to some overhead when using load operations. For example, if you use a loop instead of a repeat block to save bytes in the ROM, that will increase the number of cycles required for copying the attributes, making DMA potentially more attractive.

Displaying More Than 40 Sprites

In the previous sample, the DMA copy triggered during the vblank. That's the typical use case, allowing you to display up to 40 sprites on the screen. However, contrary to VRAM load operations, DMA is still usable during the OAM scan (mode 2) and rendering (mode 3). This enables an interesting technique that can be used to display more than 40 sprites on the screen.

This technique is illustrated in the *dma_double_sprite* sample on GitHub. Here's how it works. First, we display a batch of 40 sprites near the top of the screen. Then, we set an LCD interrupt to trigger once the LCD refresh passes the last sprite of the batch. Inside the LCD interrupt, a DMA copy brings a second batch of sprites to the OAM. Those sprites are drawn in the remaining part of the frame, near the bottom of the screen.

Figure 12-5 shows what it looks like.



Figure 12-5: Displaying 80 sprites

Here, the first batch of sprites displays from lines 32 to 72. Then, an LCD interrupt triggers on line 80. During that interrupt, a DMA copy of a second batch of sprites executes. Finally, the second batch of sprites displays from lines 96 to 136.

There are a few limitations to this technique. First, you need to organize the sprites in vertical batches. Second, sprites won't display during DMA. Because a DMA copy takes 160 cycles to complete, and a line lasts 114 cycles, this means there are roughly two lines during which sprites won't display. In other words, the batches will be separated by two or more lines. Finally, it cannot overcome the limit of 10 sprites per line, which is built into the LCD hardware itself.

Still, this technique is quite powerful, and you could theoretically use it to display up to four batches of 40 sprites per frame, for a total of 160 sprites.

Summary

This chapter introduced a few advanced graphics techniques that are available to DMG programmers. You began by learning about the different LCD modes and how you can track certain LCD events, like the hblank. You then put this knowledge to use to implement parallax scrolling in an efficient way and experimented with another nice visual effect, palette fades.

Next, you learned about the benefits of direct memory access and how to leverage it in your programs to speed up data transfer to OAM or to work around the 40-sprite limit.

This chapter concludes our coverage of DMG graphics. The next chapter introduces the memory bank controllers, which let us access more ROM memory and even save persistent data in the cartridge.

Game Boy Coding Adventure (Early Access) © 2025 by Maximilien Dagois

13

MEMORY BANK CONTROLLERS

This chapter covers the *memory bank controller (MBC)*, a piece of hardware inside the cartridge that increases the amount of memory accessible to programs.

You'll start by learning the basics of MBCs: why they are important, how they work, and how to enable them in a program. Then, we'll take a closer look at one particular type of MBC, the MBC5, which was very common in commercial games.

Finally, you'll put your new knowledge into practice using two samples: one that leverages ROM banks to extend the ROM memory, and one that leverages ERAM banks to implement game saves.

By the end of this chapter, you'll have all the theoretical and practical knowledge you need to leverage an MBC in your programs.

Fundamentals

This section covers the fundamentals of MBCs and their main use case, bank switching. Through bank switching, MBCs let us use more ROM so that we can create more complex programs. MBCs also give us access to ERAM, allowing us to save data into the cartridge.

The Evolution of Cartridges

The very first DMG cartridges were just pieces of ROM on a circuit board, like the *Tetris* cartridge in Figure 1-5. The ROM size was limited to 32KiB.

As games grew in complexity, cartridges started featuring additional hardware, like the *Zelda* cartridge in Figure 1-5. One of the chipsets commonly found in cartridges is an MBC, which extends the amount of ROM available via the bank switching technique introduced back in Chapter 3. They also give access to ERAM, which is often used for game saves.

While it is technically possible to use ERAM without an MBC, I'm not aware of any released games that did. In this book, we'll assume that an MBC is necessary for ERAM.

There are around 30 types of cartridges, featuring a variety of hardware. Most of these are either obsolete (because subsequent types improved upon them) or tailored to specific use cases linked to specialized hardware in the cartridge (like an accelerometer). This chapter covers the MBC5 cartridge, which is quite common and versatile and supports large amounts of both ROM and ERAM. You can read about the other cartridge types in the Pan Docs at *https://gbdev.io/pandocs/*.

Bank Switching

In the memory map, addresses from `$0000` to `$7FFF` are dedicated to ROM access. That's a total of 32KiB of addressable memory. For ROMs over 32KiB, you'll need to use the bank switching technique to increase the amount of usable ROM.

With this technique, the ROM is split into 16KiB areas called *banks*, with each bank assigned a number from 0 to the number of banks minus one. For example, a ROM of 512KiB splits into 32 banks, with bank numbers going from 0 to 31. The memory map addresses dedicated to the ROM are also split into two logical spaces of 16KiB each. The first space (space #0 extends from `$0000` to `$3FFF`, and the second space (space #1) extends from `$4000` to `$7FFF`. By default, space #0 maps to bank 0 and space #1 to bank 1, but it's possible to *remap* a space to a different bank during the program execution. This means that while the DMG can still only ever access 32KiB of ROM at any time, you can use bank switching to access many ROM banks over the execution of the program.

Figure 13-1 shows how bank switching works for a 64KiB ROM (4 banks). The first and second spaces map to banks 0 and 2, respectively.

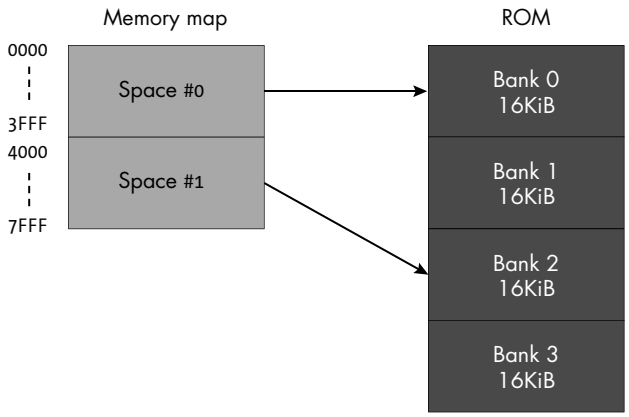


Figure 13-1: ROM bank switching

Space #0 has to contain the interrupt functions, so remapping it would require duplicating those functions in several banks. Because this is impractical to do, space #0 is often considered locked to bank 0; the MBC5 won't even allow it to map to any other bank. Thus, bank 0 should contain as much of the core code as possible, like interrupt functions and the main loop. Other banks should contain temporarily useful code and data, like graphics to be copied to VRAM, and map to space #1 when necessary.

Bank switching for ERAM works in a similar fashion to ROM, but there are two differences. First, ERAM banks are 8KiB. Second, there is only one space in the memory map dedicated to ERAM, extending from `$A000` to `$BFFF`, so only one bank is ever visible at a time.

ROM with MBC Enabled

To use bank switching in a program, you need to set three fields related to the MBC type, the ROM size, and the ERAM size in the cartridge header. We'll go through each of these fields in this section to see how they're set up.

First, the *cartridge type* field at address `$0147` specifies the type of MBC present in the cartridge (if any). Table 13-1 shows examples of possible values for the cartridge type.

Table 13-1: Examples of Cartridge Types

Code	MBC	ROM	ERAM	Battery
00	—	32KiB	—	No
19	MBC5	Up to 8MiB	—	No
1A	MBC5	Up to 8MiB	Up to 128KiB	No
1B	MBC5	Up to 8MiB	Up to 128KiB	Yes

Until now, our samples have all used cartridge type `$00`, which has a ROM of 32KiB and no ERAM. This table focuses on the MBC5, the subject of this chapter, but keep in mind that there are many other types. There are also several configurations for the MBC5, depending on whether the cartridge has some ERAM or a battery. Most MBCs come in several variations.

The second important field of the header is the *ROM size*, at address `$0148`. Table 13-2 lists the possible values for this field.

Table 13-2: ROM Sizes

Code	ROM	Banks	Comments
00	32KiB	2	No bank switching
01	64KiB	4	—
02	128KiB	8	—
03	256KiB	16	—
04	512KiB	32	—
05	1MiB	64	—
06	2MiB	128	—
07	4MiB	256	—
08	8MiB	512	—

The formula to compute the ROM size from the code in the cartridge header is `rom_size` = $2^{code+1} \times$ 16KiB, though it's probably easier to just refer to the table.

The third and last significant field of the header is the *ERAM size*, at address `$0149`. Table 13-3 shows the possible values for this field.

Table 13-3: ERAM Sizes

Code	ERAM	Banks	Comments
00	—	—	No ERAM
01	—	—	Unused
02	8KiB	1	—
03	32KiB	4	—
04	128KiB	16	—
05	64KiB	8	—

You can set the cartridge type, ROM size, and ERAM size fields using the rgblink and rgbfix tools, but you need to make a few changes for MBC and bank switching. The following shows what the typical build commands looked like in the samples up to now:

```
> rgblink --dmg --tiny --map rom.map --sym rom.sym -o rom.gb main.o sample.o
> rgbfix --title game --pad-value 0 --validate sample.gb
```

To start using MBC and bank switching, make the following updates. First, remove the `--tiny` option from the `rgblink` command's parameters, because in tiny mode the toolchain disables bank switching and considers that there is only one space of 32KiB instead of two spaces of 16KiB in the memory map.

Second, add the `--mbc-type` and `--ram-size` options to the `rgbfix` command's parameters. The first option, `--mbc-type`, sets the MBC type. It takes a value from the Code column of Table 13-1. The second option, `--ram-size`, is necessary only when using ERAM. It sets the ERAM size and takes a value from the Code column of Table 13-3.

Here's what the build commands would look like for a cartridge of type MBC5 with 32KiB of battery-backed ERAM:

```
> rgblink --dmg --map rom.map --sym rom.sym -o rom.gb main.o sample.o
> rgbfix --title game --mbc-type 0x1B --ram-size 0x03 --pad-value 0 --validate rom.gb
```

As mentioned, the `--tiny` option is gone from the `rgblink` command. The `rgbfix` command now has the `--mbc-type 0x1B` option and the `--ram-size 0x03` option to set the MBC type and the ERAM size, respectively. We don't specify the ROM size because the toolchain computes it automatically based on the size of the ROM passed to `rgbfix`.

MBC5

Now that you know how to create ROMs with bank switching capabilities, let's see how the MBC5 lets us access more ROM and save data inside the cartridge.

Characteristics

The MBC5 features up to 8MiB of ROM (512 banks) and up to 128KiB of ERAM (16 banks). That's the maximum amount of ROM and ERAM available among all the official cartridge types. It also has a rumble feature that makes the DMG vibrate, but I won't cover that in this book because it's not available in emulators or flashcarts.

The MBC5 features registers that select the ROM and ERAM banks. These registers do not have dedicated addresses in the memory map. Instead, they are accessible by writing to ROM addresses. That may sound odd since ROM stands for read-only memory and, generally speaking, writing to ROM has no effect, but an MBC5 interprets writes to the ROM as writes to its internal registers. Table 13-4 lists those registers.

Table 13-4: MBC5 Registers

Register	Address	Purpose	Values	Comments
`rRAMG`	`$0000`	ERAM enable	`$00` = disabled, `$0A` = enabled	ERAM must be enabled before reading or writing to it
`rROMB0`	`$2000`	ROM bank number (bits 0–7)	0–255	—
`rROMB1`	`$3000`	ROM bank number (bit 8)	0 or 1	Register can be ignored if number of ROM banks is 256 or fewer
`rRAMB`	`$4000`	ERAM bank number	0–15	—

The *ERAM enable* register, `rRAMG`, has only two valid values: `$00`, which disables the ERAM, and `$0A`, which enables it. Stick to these two values, because any other value might exhibit a different behavior on various hardware revisions. The default value is `$00`, so ERAM is disabled when the DMG is powered on. You'll need to enable it before reading or writing any values from or to ERAM. If the ERAM is battery powered (typically for game saves), disable it after each use to lower the risk of game saves getting corrupted when the DMG is powered down.

The *ROM bank number* is a 9-bit number, so two registers are dedicated to it. The `rROMB0` register holds the lower 8 bits of the bank number. This is the only register needed if there are 256 banks or fewer. If there are more than 256 banks, the `rROMB1` register holds the highest and last bit of the bank number. For example, to select bank 478 (`$1DE`), `$DE` and `$01` must be written to `rROMB0` and `rROMB1`, respectively.

The *ERAM bank number* is a 4-bit number, so only one register, `rRAMB`, is required to hold this.

All MBC5 registers are write-only. Reading from them yields whatever is in the ROM at their addresses, which is what we expect.

NOTE *There are several other addresses assigned to each MBC5 register. For example, addresses from `$0000` to `$1FFF` all map to the ERAM enable register, so writing `$0A` to address `$1234` would have the same effect as writing `$0A` to `$0000`. We'll use only the base addresses of the registers in this book.*

ROM Bank Switching

Let's put what you've learned about the MBC5 into practice using the *mbc_rom* sample, which illustrates the ROM bank switching mechanism. Based on the optimized parallax sample from Chapter 12, the *mbc_rom* sample uses an MBC5 with no ERAM. Here are the necessary build commands:

```
> rgblink --dmg --map rom.map --sym rom.sym -o rom.gb main.o sample.o
> rgbfix --title game --mbc-type 0x19 --pad-value 0 --validate rom.gb
```

Up to now, we've been putting all the sections into `rom0`, using the following syntax:

```
section "section_name", rom0
```

This is still the way to put sections in bank 0 when using bank switching. Bank 0 always maps to space #0, so code and data that are useful throughout the execution of the program should be placed there. To place a section in a different bank, you'll need to use a different syntax. Here's one example:

```
section "section_name", romx, bank[2]
```

This puts the section in bank 2 via two directives: `romx` and `bank`. The `romx` directive tells the linker to put the section in a bank other than bank 0, for which we would use `rom0` instead. The `x` in `romx` is not a placeholder, and you must use `romx` for any bank other than 0.

The `bank[2]` directive tells the linker to place the section in bank 2. To place the section in a different bank, you could instead specify any number from 0 to the total number of banks minus one.

The sample uses a total of three banks. Bank 0 contains the main loop code and interrupt functions. Bank 1 holds the sample initialization code, as shown in Listing 13-1.

```
section "sample_init", romx, bank[1]

InitSample:
    ...
```

Listing 13-1: Putting the sample initialization code in bank 1

Bank 1 also holds the data used during the sample initialization, shown in Listing 13-2.

```
section "graphics_data", romx[GRAPHICS_DATA_ADDRESS_START], bank[1]
incbin "parallax.chr"
incbin "parallax.tlm"
```

Listing 13-2: Putting the graphics data in bank 1

Finally, bank 2 contains the code for the sample update, as shown in Listing 13-3.

```
section "sample_update", romx, bank[2]

UpdateSample:
    ...
```

Listing 13-3: Putting the update code in bank 2

When the sample starts, bank 0 and bank 1 map to space #0 and space #1, respectively. After the initialization, access to bank 1 isn't necessary anymore, because the `InitSample` function is called only once and the graphics

data from the ROM has already been copied to VRAM. That means space #1 can safely map to bank 2, and `UpdateSample` can be called from there.

The bank switches from *main.rgbasm* (bank 0) right after the initialization function, as shown in Listing 13-4.

```
    call InitSample
    ld a, 2
    ld [rROMB0], a
    .loop
        call UpdateSample
        jr .loop
```

Listing 13-4: Switching the ROM bank

Bank switching is instantaneous, so unless there is a good reason for it, any bank switching instructions should be in bank 0, as in Listing 13-4. The code calls the `InitSample` function from bank 1, but the bank switching occurs only after that function returns and the program counter, `pc`, is back pointing to an address in bank 0 (the address of the `ld a, 2` instruction, in this case).

If we were to switch banks while executing code in space #1, `pc` would still point to space #1 after the switch, but the executed code would be taken from the freshly mapped bank. Listing 13-5 shows an example of this.

```
section "test", romx, bank[1]
test_function:
    ; some work is done here
    ...

    ; bank switch
    ld a, 2
    ld [rROMB0], a

    ; that code is never reached, as space #1 is mapped to bank 2
    ; at that point
    ret
```

Listing 13-5: Incorrect bank switching

The code for `test_function` is in bank 1, but after the bank switch, that code is gone; it's immediately replaced by the code in bank 2 at the same location. As a result, the `ret` instruction is never reached.

Before we end our discussion of ROM bank switching, I want to point out that the map file the linker generates contains some information about the banks in the ROM. Listing 13-6 shows the map file generated for the *mbc_rom* sample.

```
ROM0 bank #0:
  SECTION: $0000-$002e ($002f bytes) ["interrupts"]
           $0000 = LcdInterrupt
           $0022 = LcdInterrupt.middle_section
  SECTION: $0040-$0047 ($0008 bytes) ["vblank_interrupt"]
  SECTION: $0048-$004d ($0006 bytes) ["lcd_interrupt"]
  SECTION: $004e-$0063 ($0016 bytes) ["main"]
           $004e = main
           $004e = main.wait_vblank_u1
           $005f = main.loop
  SECTION: $0100-$014f ($0050 bytes) ["header"]
           $0100 = entrypoint
    SLACK: $3f5d bytes

ROMX bank #1:
  SECTION: $4000-$402f ($0030 bytes) ["sample_init"]
           $4000 = InitSample
           $4017 = InitSample.load_tile_u7
  SECTION: $6400-$7fff ($1c00 bytes) ["graphics_data"]
    SLACK: $23d0 bytes

ROMX bank #2:
  SECTION: $4000-$4025 ($0026 bytes) ["sample_update"]
           $4000 = UpdateSample
           $4004 = UpdateSample.wait_vblank
    SLACK: $3fda bytes

USED:
    ROM0: $00a3 bytes in 1 bank
    ROMX: $1c56 bytes in 2 banks
```

Listing 13-6: The map file for the sample

Anything in bank 1 is unnecessary beyond the initialization. Also notice that banks 1 and 2 share the same address space (like the `"sample_init"` and `"sample_update"` sections).

While banks do not increase the maximum size of ROM available at any one time (32KiB), they are a tremendous tool for optimizing the quantity of useful code and assets accessible across the various phases of the execution of a program. Next, we'll look at another feature of MBCs: the ability to save data.

Saves with ERAM

Let's check out how the ERAM works using an MBC5 with 32KiB of battery-backed ERAM (4 banks). We're using an arbitrary ERAM size for this sample, but for your programs, use whatever meets your needs. Make sure to use a battery-backed MBC (that is, type `1B`) if your goal is to implement game

saves or any equivalent persistent data mechanism. The build commands will look as follows:

```
> rgblink --dmg --map rom.map --sym rom.sym -o rom.gb main.o sample.o
> rgbfix --title game --mbc-type 0x1B --ram-size 0x03 --pad-value 0 --validate rom.gb
```

When ERAM is battery-backed, its content is kept alive even when the DMG is powered down. That was the cheapest way to enable game saves back in the day. The *Zelda* cartridge shown in Figure 1-5 features such an ERAM. On emulators and flashcarts, instead of an actual ERAM, a *.sav* file with the same name as the ROM is generated, which is loaded automatically when a program is run.

We'll use the *mbc_eram* sample in this section. Derived from the ROM bank switching sample of the previous section, this updated sample does not scroll the background automatically. Instead, you use the LEFT and RIGHT keys to scroll the background. Pressing the A button saves the three parallax scrolling values to ERAM. Listing 13-7 shows the `WriteSave` function that writes a game save into ERAM.

```
WriteSave:
    ; enable the ERAM
    ld a, $0A
    ld [rRAMG], a

    ; write the header
    ld hl, _SRAM
    ld a, $44
    ld [hli], a
    ld a, $4D
    ld [hli], a
    ld a, $47
    ld [hli], a

    ; write the scrolling values
    ld de, WRAM_BG_SCROLL_X + 0
    REPT 3
        ld a, [de]
        inc de
        ld [hli], a
    ENDR

    ; disable the ERAM
    ld a, $00
    ld [rRAMG], a
    ret
```

Listing 13-7: Writing a game save into ERAM

The first step is to enable the ERAM by writing `$0A` into `rRAMG`. Otherwise, we won't be able to write to the ERAM, since it's disabled by default. We then write a simple data header to `_SRAM`, which is the starting address of the ERAM. The header consists of the three ASCII letters D, M, and G, whose values are `$44`, `$4D`, and `$47`, respectively. This is a crude way of validating the data during loading, which I'll explain soon. Next, the scrolling values stored in the 3-byte variable `WRAM_BG_SCROLL_X` are written to the ERAM. Finally, we disable the ERAM by writing `$00` into `rRAMG`, as recommended for battery-backed ERAM. Because the `WriteSave` function is only called from the `UpdateSample` function, it is in bank 2.

Listing 13-8 shows the `LoadSave` function, the companion of the `WriteSave` function that loads a game save from ERAM. The `LoadSave` function is in bank 1 because it is only called from the `InitSample` function.

```
LoadSave:
    ; enable the ERAM
    ld a, $0A
    ld [rRAMG], a

    ; check the header
    ld hl, _SRAM
    ld a, [hli]
    cp a, $44
    jr nz, .done
    ld a, [hli]
    cp a, $4D
    jr nz, .done
    ld a, [hli]
    cp a, $47
    jr nz, .done

    ; read the scrolling values in
    ld de, WRAM_BG_SCROLL_X + 0
    REPT 3
        ld a, [hli]
        ld [de], a
        inc de
    ENDR

    .done
    ; disable the ERAM
    ld a, $00
    ld [rRAMG], a
    ret
```

Listing 13-8: Loading a game save from ERAM

The function enables the ERAM, then performs a simple header check. If any of the expected letters (D, M, and G) is not there, it considers the game save invalid and keeps the scrolling values to their default values. Assuming the header is valid, the scrolling values are read in. Finally, it disables the ERAM.

When the sample is run in BGB, a *.sav* file is generated in the same folder as the ROM file. Scroll the background with LEFT and RIGHT, press the A button, close BGB, and then relaunch the program. The background scrolling should be back to the same position as when you last pressed the A button. Figure 13-2 shows the top of the *.sav* file BGB generated, viewed in the hexadecimal editor HxD (available at *https://mh-nexus.de/en/hxd/*).

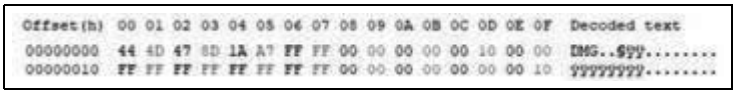


Figure 13-2: The game save file generated by BGB

The header is visible (`DMG`), as well as the three scrolling values. The rest of the file is garbage, because the sample does not initialize any of these values. Never expect these values to be zero by default, because the hardware does not initialize ERAM.

The sample in this section is using a very crude header. As long as ERAM is properly enabled before access and disabled after use, its content should not get corrupted. Still, in a real program, you might want to perform the equivalent of a cyclic redundancy check to ensure the validity of the data stored in ERAM. See *https://en.wikipedia.org/wiki/Cyclic_redundancy_check* for a detailed explanation.

Summary

In this chapter, you learned about MBCs and bank switching. You should now be able to enable bank switching in your ROMs with the RGBDS toolchain. You should also understand how to switch ROM banks to access larger amounts of ROM memory. Finally, you should be able to implement game saves by writing and reading data in ERAM.

This chapter focused mostly on the MBC5, but there are many types and variations of MBCs that offer access to interesting additional hardware. For example, the MBC3 features a real-time clock, and the MBC7 features an accelerometer. The only issue is that, in general, emulators and flashcarts might not support these specific use cases. The MBC5 is probably the most useful MBC nowadays, especially when it comes to extending ROM or using ERAM.

In the next chapter, we'll take a closer look at interrupts to give you a more complete understanding of this important feature of the DMG.

14

INTERRUPTS

The previous chapters introduced the five types of hardware interrupts on the DMG: vblank, joypad, LCD, timer, and serial. This chapter provides additional information related to interrupts. While this information won't be directly applicable to most programs, it will come in handy in those rare situations where you need full control over your interrupts.

You'll begin by learning a bit more about the execution of interrupts and the level of control you can exert over their relative priority. This knowledge will help you prioritize the interrupts that are the most time-sensitive in your program. Then, you'll get a few tips on writing interrupts. We've already put these to use in earlier chapters, and we'll formalize that knowledge here.

Finally, you'll learn about some pitfalls related to using `halt` with interrupts. In particular, we'll look at a hardware bug that occurs rarely but will have you scratching your head if you're not aware of it.

Executing Interrupts

In this section, we'll study the various methods at our disposal to prioritize the execution of interrupts. We'll begin by looking at the default priority

of interrupts. Then, we'll see how to nest and manually trigger interrupts. Finally, you'll learn an alternative way to handle the vblank interrupt.

Understanding Interrupt Priorities

Let's start by discussing the default priority of the DMG interrupts. Each interrupt is assigned a priority. When several interrupts trigger at the same time, they execute in the order shown in Table 14-1.

Table 14-1: Interrupt Addresses and Priorities

Interrupt	Address	Priority
Vblank	$0040	0 (highest)
LCD	$0048	1
Timer	$0050	2
Serial	$0058	3
Joypad	$0060	4 (lowest)

For example, when a vblank and a timer interrupt occur at the same time, the vblank interrupt executes before the timer interrupt.

Nesting Interrupts

Now that you have a good grasp of the default priority of the interrupts, let's see the first way we can influence the execution priority. By default, the DMG disables all interrupts before executing any interrupt function, to prevent interrupts from interrupting each other. However, you can use *nested interrupts* to force execution even if another interrupt is currently executing. This allows you to prioritize time-sensitive, high-priority interrupts over interrupts that won't suffer from a slight delay before being handled.

To do this, you need to re-enable the interrupts (using `ei`) from inside any interrupt function that should have low priority. This allows high-priority interrupts to take precedence over low-priority interrupts. For example, consider the code in Listing 14-1.

```
section "vblank_interrupt", rom0[$0040]
    reti

section "serial_interrupt", rom0[$0058]
    call serial_function
    reti

section "timer_interrupt", rom0[$0050]
    ei
    call timer_function
    reti
```

Listing 14-1: Three interrupt definitions

Three interrupts are defined in this example. The vblank and serial interrupts are high priority, and we do not allow nested interrupts in their corresponding functions. On the other hand, the timer interrupt function re-enables interrupts at the start of its execution (with the `ei` instruction) to allow for nesting. If a vblank or serial interrupt occurs during the timer interrupt (after the `ei` instruction), the CPU suspends the execution of the timer interrupt function. It then fully executes the nested interrupt function (vblank or serial). Finally, the CPU resumes the execution of the timer interrupt function from where it was suspended.

An interrupt function with a lengthy execution time could allow nesting to let other interrupts with tighter timing requirements to execute in priority.

The main caveat is that once an interrupt allows nesting, all other interrupts will systematically interrupt it. We cannot pick and choose which interrupts can be nested. For example, in Listing 14-1, both serial interrupts and vblank interrupts will interrupt the timer interrupt, which allows nesting.

Triggering Interrupts Manually

Another way we can control the execution of interrupts is by triggering them manually. This typically comes in handy when debugging, because we can force the timing of an interrupt and see how it interacts with other pieces of a program.

Typically, the DMG triggers interrupts automatically. When an interrupt occurs, the corresponding flag is raised in the interrupt flag register, `rIF`. If the interrupt master enable (IME) flag and the interrupt flag in the interrupt enable register (`rIE`) are both raised, the interrupt function is called and the flag in `rIF` gets cleared. The DMG handles the process of raising and clearing the flags in `rIF`; the program's responsibility is only to enable the interrupts in IME (via `ei`) and set the interrupt flags in `rIE`. However, it is also possible to trigger an interrupt manually, because the `rIF` register is writable.

The *interrupt_trigger* sample in the GitHub repository demonstrates this. At initialization, this sample enables interrupts with `ei`, and the serial interrupt flag is raised in `rIE`. You can trigger a serial interrupt on a key press, like in Listing 14-2.

```
    TestPadInput_Pressed WRAM_PAD_INPUT, PADF_A
    jr nz, .trigger_interrupt
        ld hl, rIF
        set 3, [hl]
    .trigger_interrupt
```

Listing 14-2: Triggering an interrupt

Bit 3 of the `rIF` register corresponds to the serial interrupt. When the program sets this bit, the serial interrupt function is called, just as if an interrupt had occurred naturally.

Handling the Vblank Interrupt Differently

To end this section, let's look at an alternative way to handle the vblank interrupt. It has both advantages and drawbacks, so whether you opt for this new implementation will depend on your program's needs.

Most samples in this book manage the vblank as in Listing 14-3.

```
section "vblank_interrupt", rom0[$0040]
    ; do nothing here
    reti

section "sample", rom0
UpdateSample:
    halt
    ; perform VRAM transfers here
```

Listing 14-3: Typical vblank handling

The vblank interrupt is very short, and the bulk of the work happens in the update function, after handling the interrupt.

Listing 14-4 shows an alternative method of handling vblank interrupts.

```
section "vblank_interrupt", rom0[$0040]
    ; perform VRAM transfers here
    reti

section "sample", rom0
UpdateSample:
    halt
    ; do nothing here (VRAM-wise)
```

Listing 14-4: An alternative vblank handling method

This time, the heavy lifting of updating the VRAM is done inside the interrupt. The advantage of this method is that it makes the VRAM transfers less likely to extend beyond the vblank period (which would cause them to fail), because other interrupts cannot block the execution of the interrupt. However, that's only really a concern when you're using multiple interrupts and have a high number of VRAM transfers.

The main issue with the alternative method stems from the advantage just mentioned: Because we can't interrupt the execution of the interrupt, it plays poorly with the timer and serial interrupts. For example, timer ticks could end up being skipped, causing the timer interrupt to not trigger as many times as expected. Likewise, in a multiplayer game, serial transfers could be completely missed on the child, causing the game to not function properly.

In general, it's safer to keep the interrupt functions as short as possible, which is the approach I've used for vblank handling throughout this book. If the VRAM transfers are likely to go beyond the vblank, it's best to optimize or split the work over two frames.

Writing Interrupts

Now that you know how to exert some control over your interrupts, let's review a couple of general notes to keep in mind when writing interrupts to optimize your ROM space and avoid bugs.

Repurposing Reserved Addresses

Each interrupt has an associated function address and 8 bytes of reserved ROM space. If the interrupt function code fits into 8 bytes, it can be written right into its reserved space. Otherwise, a jump or a function call can get to the actual code located anywhere in the ROM.

However, when an interrupt is unused, it is perfectly acceptable to repurpose its space. For example, if a program does not leverage the serial interrupt at all, we can use the memory space from `$0058` to `$0060` for any other purpose. We could use it as extra space for the timer interrupt (starting at `$0050`), or for code not related to interrupts in any way.

Keeping Registers Clean

Interrupts may occur at any time while code is executing. To avoid interfering with the interrupted code, which causes bugs, you need to use the `push` and `pop` instructions to save and restore any registers used in the interrupt function. The code snippet in Listing 14-5 illustrates this.

```
section "serial_interrupt", rom0[$0058]
    call SerialFunction
    reti

SerialFunction:
    push af
    push bc
    push de
    push hl

    ; do some work using registers

    pop hl
    pop de
    pop bc
    pop af
    ret
```

Listing 14-5: Using push and pop to save and restore registers in interrupts

We use the serial interrupt here, but the same principle applies to any other interrupt. This code pushes and pops all eight registers, but you can save a bit of ROM space and some cycles by only pushing or popping the

registers actually used in the interrupt function. For example, if you use neither `b` nor `c`, you can safely remove `push bc` and `pop bc`.

The `af` register requires some extra caution. For example, even if you don't use the `a` register at all, some instructions could still affect the flags in `f`. As a rule of thumb, it's best to start by pushing and popping all registers. Then, once the interrupt function code is stable, you can remove any unnecessary `push` and `pop` instructions.

More About halt

We used the `halt` instruction repeatedly in previous chapters, because it's an efficient way to implement frame pacing. Here are some additional things to keep in mind when using `halt`.

IME Influence

When the IME flag is set, `halt` causes the DMG to enter low power mode until an interrupt occurs. The DMG then handles any enabled interrupt by calling the corresponding interrupt function. No surprises here.

When the IME flag is not set (that is, the interrupts are disabled), the behavior of `halt` depends on whether an interrupt is pending or not. An interrupt enters the pending state when its interrupt flag is raised in the `rIE` register and the interrupt event occurs (the flag is set in register `rIF` by the DMG). Because the interrupts are disabled, the interrupt function is not called, and thus, the interrupt is pending.

When there are no pending interrupts, `halt` causes the DMG to enter low power mode until an interrupt occurs. But the interrupt function is not called upon exiting low power mode, because the IME flag is not set.

When there is a pending interrupt, `halt` does not cause the DMG to enter low power mode, which is fine. But due to a hardware bug, the instruction right after the `halt` executes twice. To counter this bug, add a `nop` right after `halt`. The `nop` executes twice when the bug conditions are met, but that has no bad side effects.

In the end, when there is a chance that `halt` may get called when interrupts are disabled, it is safer to add a `nop` right after `halt`. Note that this is not the case in the samples of this book, because `halt` is only ever called when interrupts are enabled.

Multiple Interrupts and Vblank

In this book's samples, we typically implement frame pacing by synchronizing the main loop with the vblank using the `halt` instruction. However, as discussed in Chapter 8, other interrupts might interfere with the frame pacing mechanics because they also cause `halt` to complete. The code in Listing 14-6, initially presented in Listing 8-33, counters that issue.

```
    ld hl, WRAM_IS_VBLANK
    xor a
    .wait_vblank
        halt
        cp a, [hl]
        jr z, .wait_vblank
        ld [hl], a
```

Listing 14-6: Using `halt` with multiple interrupts

There are a few things to be aware of when using this code. First, the vblank flag is stored in WRAM. HRAM is also a perfect valid choice for storing that flag. Using a single bit in a byte is possible too. That would require using bit operations, but it would save the 7 bits wasted with the current code.

Second, the code might fail to catch some vblank interrupts. If a vblank interrupt occurs between the `cp` and `jr` instructions, then the value of the zero flag will be stale when it's used in `jr`, and the interrupt will be missed. Unfortunately, there is no way to completely prevent this issue. Thankfully, the odds of a vblank interrupt occurring at that particular timing are low, and the impact of a missed vblank isn't too serious; a frame is simply skipped.

Finally, you should evaluate whether it's even necessary to use the code in Listing 14-6. For example, most LCD interrupts won't ever overlap with the vblank interrupt. Likewise, a timer interrupt resolved within the LCD refresh should not cause any issue. The serial interrupt, covered in Chapter 11, is usually the problematic one. On the child side, there's no way to control the timing of that interrupt, so you must assume that it could occur at
any time.

Summary

In this chapter, you learned new ways to control the behavior of your interrupts, such as nesting them or triggering them manually.

We also reviewed some tips for writing interrupts, such as keeping registers clean to avoid any interference with the main loop. Finally, you learned about some pitfalls of using the `halt` instruction, such as the hardware bug and the potential for missed vblank interrupts.

In the next chapter, I'll cover programming and debugging tricks that will make your life easier when developing programs.

Game Boy Coding Adventure (Early Access) © 2025 by Maximilien Dagois

15

PROGRAMMING AND DEBUGGING TRICKS

This chapter covers several topics related to debugging and developing programs. This knowledge is not strictly necessary for programming the DMG, but you may find it handy in many situations.

We'll start by looking at jump tables, an efficient way to handle states in your programs. Then I'll introduce two BGB features, code break points and debug messages, that will help you debug programs.

Next, you'll learn a technique to strengthen your programs by detecting jumps to unintended addresses. You'll also learn about bringing the stack into WRAM to extend its size beyond the 127 bytes available in HRAM.

Finally, we'll take a closer look at linear-feedback shift registers, which generate pseudorandom numbers with relatively few instructions and little complexity. Pseudorandom numbers are often used in games to simulate unpredictable events, such as dice rolls.

Jump Tables

Jump tables are an indispensable assembly programming tool. They are very common constructs typically used to handle states in a program. There are several variations, but the main idea is the same: *jump tables* use an index to choose a jump address from a table of addresses.

To understand the purpose of jump tables, let's first explore what happens when we don't use them using the *jump_table_equivalent* sample on GitHub. Then, we'll rewrite the sample using jump tables to illustrate their advantages.

The Classic Method

There are four update functions in the *jump_table_equivalent* sample. The sample code calls the right one based on the value of an index variable, `WRAM_FUNC_INDEX`. When the index equals 0, the code calls `Update_0`. When the index equals 1, it calls `Update_1`, and so on up to index 3. Listing 15-1 shows the first update function.

```
def DISPLAY_ROW        equ (1)
def DISPLAY_COLUMN     equ (1)
def VRAM_DISPLAY_START equ (_SCRN0 + SCRN_VX_B * DISPLAY_ROW + DISPLAY_COLUMN)
def TILE_INDEX_SPACE   equ (64)
def TILE_INDEX_A       equ (97)

Update_0:
    ld hl, VRAM_DISPLAY_START
    ld a, TILE_INDEX_A + 25
    ld [hli], a
    ld a, TILE_INDEX_A + 4
    ld [hli], a
    ld a, TILE_INDEX_A + 17
    ld [hli], a
    ld a, TILE_INDEX_A + 14
    ld [hli], a
    ld a, TILE_INDEX_SPACE
    ld [hli], a
    ret
```

Listing 15-1: Function 0

The function displays `ZERO` to the screen by updating the background tilemap. The other update functions are similar, only displaying different text, so I won't reproduce them here.

Listing 15-2 shows a classic way to call the right function based on the value of `WRAM_FUNC_INDEX`.

```
    ld a, [WRAM_FUNC_INDEX]
    cp a, 0
    jr nz, .zero
        call Update_0
        jr .done
    .zero
    cp a, 1
    jr nz, .one
```

```
        call Update_1
        jr .done
    .one
    cp a, 2
    jr nz, .two
        call Update_2
        jr .done
    .two
    cp a, 3
    jr nz, .three
        call Update_3
        jr .done
    .three

    .done
```

Listing 15-2: Using cp to call the update functions

The code loads the value of the index into `a`, then performs a series of comparisons (`cp`). As soon as it finds a match, it calls the corresponding function and skips any remaining comparisons.

In the sample, the UP and DOWN keys change the value of the index. The characters displayed on the screen reflect the identity of the update function called.

The Jump Table Method

The main issue with the classic method that doesn't use jump tables is that it requires more and more code as the number of update functions grows. Moreover, the cost of calling an update function isn't homogeneous and depends on the index value, making the cycle cost difficult to evaluate. For example, calling `Update_0` requires a single `cp` instruction, while calling `Update_3` requires four of those instructions.

Let's examine the *jump_table* sample, which uses a jump table to address the drawbacks of the classic method. The update functions in this sample do not change. The only difference is in the `UpdateSample` function, shown in Listing 15-3.

```
UpdateFuncTable:
dw Update_0
dw Update_1
dw Update_2
dw Update_3

UpdateSample:
    halt

    ; compute a pointer to the address of the function
    ; pointer = table address + (index * 2)
```

```
    ld a, [WRAM_FUNC_INDEX]
    ld d, 0
    ld e, a
    ld hl, UpdateFuncTable
    add hl, de
    add hl, de

    ; unpack the function address and call it
    ld a, [hli]
    ld h, [hl]
    ld l, a
    CallHL
```

Listing 15-3: Using a jump table to call the update functions

First, we add the `UpdateFuncTable` table containing the four function addresses to the code. Next, in the first block of code after `halt`, we use the index to get the *address of the address* of a function in the table. The formula in pseudocode is as follows:

```
address_of_address = table_address + (function_index * 2)
```

Because addresses are 2 bytes long, the index is multiplied by 2. In assembly, you can do this by simply adding the index twice to the value of the starting address of `UpdateFuncTable`. The two consecutive additions yield the address of the function address in `hl`.

The value in `hl` at that point is not the function address itself. It's just the address at which the function address is stored. For example, let's assume the address of the table is `$1000` (that is, `UpdateFuncTable` equals `$1000`). When the index equals 2, `hl` will contain `$1004` after the second `add hl, de` instruction executes. That's different from the actual address of `Update_2`, which we need to unpack with additional instructions.

With `hl` pointing to where the function address is located, it's easy to retrieve the actual function address. In the three lines of code right before `CallHL`, the 2 bytes stored at the address in `hl` are read into `hl` itself. Assuming the index equals 2, `hl` will contain the address of the `Update_2` function after these three lines execute.

The last action is to call the update function using the `CallHL` macro. There's no instruction to call a function from an address stored in `hl`, but there is a `jp` instruction that takes `hl` as a parameter. The `CallHL` macro, shown in Listing 15-4, leverages that `jp` instruction to implement a custom function call from an address in `hl`.

```
macro CallHL
    ld de, .call_return_address\@
    push de
    jp hl
    .call_return_address\@
endm
```

Listing 15-4: Calling a function from an address stored in `hl`

This defines an address via the `.call_return_address@` label right *after* the `jp hl` instruction, which will act as the return address of the function. It is pushed onto the stack *before* the `jp` instruction and will get popped by the `ret` instruction at the end of the function at address `hl`. This kind of address push is what other `call` instructions do, and the `CallHL` macro just emulates that behavior.

This is only one example of using a jump table. Many variations are possible, but the principle stays the same: you use an index to select an address in a table and jump to it.

Once run, the sample looks just like the previous one, but jump tables make the code easier to maintain. For example, supporting a new function is just a matter of adding its address to the table. There's no need to change any of the calling code. It's also possible to keep track of the number of entries in the table, as shown in Listing 15-5.

```
UpdateFuncTable:
dw Update_0
dw Update_1
dw Update_2
dw Update_3
UpdateFuncTable_End:

def FUNC_COUNT equ ((UpdateFuncTable_End - UpdateFuncTable) / 2)
assert(FUNC_COUNT == 4)
```

Listing 15-5: Tracking the number of function entries

Performing a range check on the index (to make sure it doesn't go over `FUNC_COUNT`) becomes trivial and strengthens the code.

Another advantage is that the performance of jump tables is more predictable. With the classic method, the cost of the function call depends on the value of the index. When the index equals 0, the function call will cost only a few cycles, because it corresponds to the very first comparison. On the other hand, when the index equals 3, all the comparisons execute. With a jump table, the cost is always the same no matter the value of the index. This means the performance of jump tables is independent of the number of functions; the efficiency of function lookup doesn't change with the size of the function table. The performance when using a jump table is not necessarily better for a small number of functions, but it stays fixed as the number of functions grows, unlike with the classic approach.

Finally, jump tables are more byte-efficient. Adding a new function has an overhead of 2 bytes, because we only need to add the function address to the function table. With the classic method, 9 bytes would be required to handle a new index value. Even in the simple contrived samples used in this section, the jump table version is already 11 bytes smaller.

More Debugging Features

We've used BGB's debugger multiple times in previous chapters. This section covers two additional BGB debugging functionalities that are useful when tracking bugs: code breakpoints and debug messages.

Source Code Breakpoints

We usually place breakpoints manually in BGB's code view (see Figure 3). However, it's sometimes convenient to trigger breakpoints systematically without having to set them up every time through BGB's GUI. For example, you could place a breakpoint in some rare error condition, or right inside code that is currently being written and changes often. That's where *source code breakpoints* come in handy.

To add a source code breakpoint, you must write the instruction `ld b, b` directly in the code (`b` stands for *breakpoint*). The instruction has no side effects aside from taking a byte and a cycle, but BGB will interpret the instruction as a breakpoint. Figure 15-1 shows such a breakpoint in the code view.

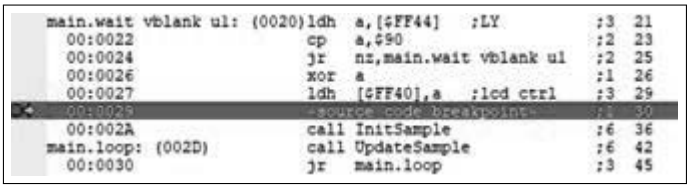


Figure 15-1: A source code breakpoint

Before you can use source code breakpoints, you need to enable them in BGB. The option is available in the Exceptions tab of the Options window (which you can open by pressing the F11 key from the debugger), as shown in Figure 15-2.

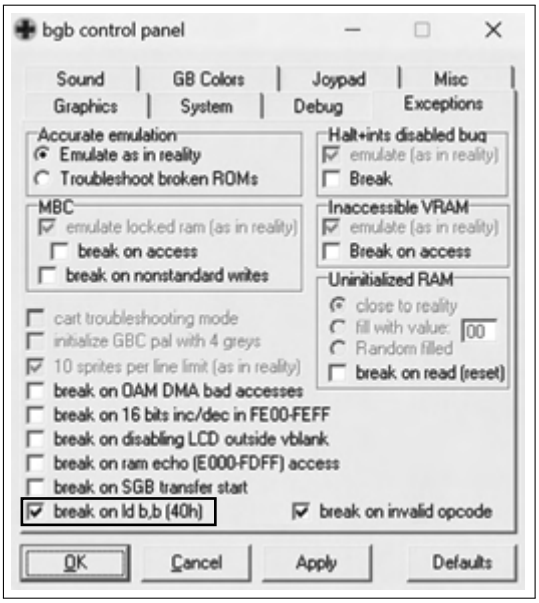


Figure 15-2: Enabling source code breakpoints in BGB

When you select the Break on ld b,b (40h) option, BGB interprets all `ld b,b` instructions as code breakpoints.

Logging

Logging is a common technique used to track events during program execution, such as the end of the initialization phase or an error. Most modern programming languages offer built-in support for it, and it's particularly useful when debugging programs.

Unfortunately, the DMG and its assembly language don't have any native logging abilities. While it's possible to display messages on the LCD directly, it's not very convenient to do so due to the limited resolution of the screen. Thankfully, BGB supports *debug messages*, which are akin to logs. Debug messages require a specific set of assembly instructions. These instructions have no side effects when run on real hardware (other than their byte and cycle costs). However, BGB interprets them as logs and outputs messages to a dedicated logging window or a file.

Listing 15-6 shows the set of instructions necessary to display the message "Hello, DMG."

```
ld d, d
jr .message
db $64, $64, $00, $00, "Hello, DMG"
.message
```

Listing 15-6: Debug message instructions

On real hardware, the `ld d, d` instruction would execute followed by the unconditional jump, `jr`. That would leave no observable side effects, which is what we want when running on real hardware. On the other hand, BGB detects the `ld d, d` instruction, ignores the jump, and interprets the data after `db $64, $64, $00, $00` as a message to output. The `$64, $64, $00, $00` sequence is an arbitrary chain of bytes that BGB recognizes as the start of a message. The `d` in `ld d, d` is a mnemonic for the term *debug message*.

Typing this piece of code every time we want a log would be cumbersome, so it's best to wrap the instructions in a macro, as shown in Listing 15-7.

```
; output debug message \1
macro debug
    ld d, d
    jr .message\@
    db $64, $64, $00, $00, \1
    .message\@
endm
```

Listing 15-7: A debug message macro

Using this macro, you need only a single line to use debug messages, as shown in Listing 15-8.

```
debug "Hello, DMG"
```

Listing 15-8: Using the debug message macro

You can only use debug messages with static messages. For example, you can't use them to display the value of a register. However, there is a syntax that supports Boolean tests. Listing 15-9 shows one example.

```
debug "%a=$01%one;not one;"
```

Listing 15-9: A debug message that includes a Boolean test

This would print `one` when `a` equals 1 and `not one` otherwise. You can use this syntax to test registers, addresses, and flags. You must use hexadecimal numbers in the test condition, or the test will always evaluate to false. Read the BGB manual at *https://bgb.bircd.org/manual.html* for more information on the available options.

You must also enable debug messages before you use them in BGB. Several options are available in the Debug tab of the Options window, which

you can access by pressing F11 from the debugger. Figure 15-3 shows the recommended options for using debug messages.

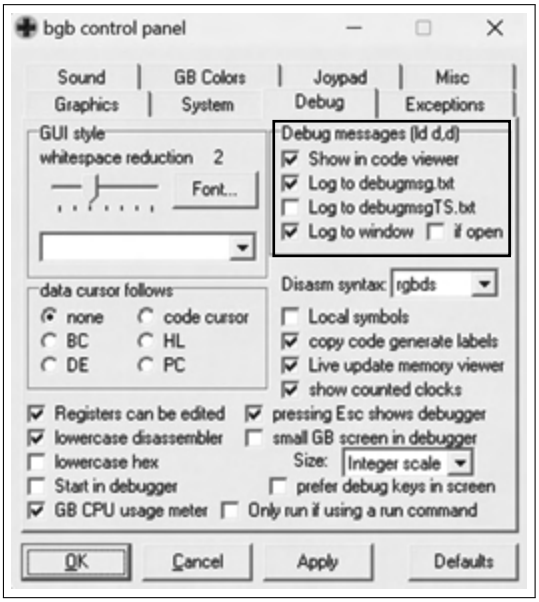


Figure 15-3: Debug message options

The Show in Code Viewer option displays debug messages in the code view instead of the `ld d, d` instruction. The Log to debugmsg.txt option outputs the logs into the *debugmsg.txt* file in BGB's folder. The Log to Window option displays the logs in the debug messages window, which is accessible through the debugger's Window menu.

Faulty Jumps

A common error in assembly is to jump to the wrong address, which typically causes bugs that are difficult to reproduce and understand. In this section, we'll cover a simple technique to detect those unintended jumps early on.

The samples in this book use the `--pad-value` parameter of the `rgbfix` command to pad the ROM with zeros (see Chapter 3). By default, rgblink also adds zero padding when allocating space for the sections in the ROM. While this makes unused ROM areas easy to identify, it does not help with catching bugs caused by faulty jumps that set the program counter to an address in an unused area. Because `0` is the opcode for `nop`, jumping to an unused area leads to the CPU doing nothing until the program counter (`pc`) points back to a non-padded area. Once a bug finally triggers, the programmer is usually left with very little information about the cause of the issue.

We can address this by using a different padding value, like `$C7`, for both rgblink and rgbfix, coupled with a restart vector function (see `rst` in Chapter 2). When we use `$C7` as padding value, the instruction linked to the unused areas changes from `nop` to `rst $00`. This means that, as soon as the `pc` register points to an invalid area, the CPU executes a call to the function at address `$0000`. What that function does is up to the programmer, but typically it should catch the error and notify the programmer of the issue.

The *catch* sample illustrates how this works. First, we set the padding parameters for both the `rgblink` and `rgbfix` commands, as shown here:

```
> rgblink --pad 0xC7 ...
> rgbfix --pad-value 0xC7 ...
```

Next, we define a minimalist `catch` function, as shown in Listing 15-10.

```
section "catch", rom0[$0000]
    ld b, b
catch:
    jr catch
```

Listing 15-10: Defining the catch function

This uses a source code breakpoint, described in the previous section, so that the execution stops right after a faulty jump occurs. It then enters an infinite loop, because there is little point in trying to execute any other code when such a bug occurs, and you should investigate the problem right away.

In the sample, pressing the A button triggers a faulty jump with `call $4000`, as shown in Listing 15-11. The other sections (main and sample) are at hard-coded addresses to make sure `$4000` is an invalid address, as the linker could technically choose to put any address-less section at address `$4000`.

```
    TestPadInput_Pressed WRAM_PAD_INPUT, PADF_A
    jr nz, .faulty_jump
        call $4000
    .faulty_jump
```

Listing 15-11: Triggering a faulty jump

Once the faulty jump triggers, the `catch` function is called and the execution stops on the source code breakpoint. Now you can inspect the stack to find the origin of the issue. That task is easier for faulty `call` instructions than for faulty `jp` and `jr` instructions, because the latter two do not leave any trace on the stack. However, it is always possible to know the value the faulty jump sets the `pc` register to, because the address right after that value is put on the stack by `rst $00` (`$4001` in the sample).

You could use any restart vector in Chapter 2 for the technique explained in this section. I used `$C7` because it leaves the ROM area between the restart vector code and the interrupts as a single contiguous piece. This maximizes the odds that the linker will be able to put a section there.

Moving the Stack to WRAM

As you learned in Chapter 2, at the start of a program the stack pointer (`sp`) is typically initialized to the top address of the HRAM. All the samples in this book keep it there. However, like any other 16-bit register, we can modify the value of `sp` at any time. That means you can move the stack to another location, and WRAM is a particularly interesting alternative to HRAM.

Moving the stack to WRAM has two significant advantages. First, there is way more room in WRAM, which has 8,192 bytes compared to HRAM's meager 127. As a result, we can push more values and call more nested functions at any given time. Second, it frees the HRAM for more meaningful tasks. Because the cost of `push` and `pop` is the same no matter the memory location, the stack does not benefit from being in HRAM. It's better to reserve the HRAM for variables that we access more frequently during the frame, and thus that will profit from the reduced byte and cycle costs of `ldh`.

To move the stack to WRAM, simply assign the `sp` register to a location in WRAM at the very start of the program. Listing 15-12 shows a snippet of code that sets the stack to the top of the WRAM.

```
section "main", rom0
main:
    ld sp, _RAM + $2000
```

Listing 15-12: Configuring a WRAM stack

You must set the `sp` register *before* any `push` or `call` instructions, or at least the corresponding `pop` and `ret` should also happen before the `sp` change. Otherwise, values and return addresses would be pushed into HRAM but popped from WRAM, which would inevitably cause bugs.

Implementing a Linear-Feedback Shift Register

I briefly introduced the concept of the linear-feedback shift register (LFSR) in Chapter 9 when dealing with sound channel 4, because an LFSR is the source of the random noisy sound of that channel. As a reminder, an LFSR is a counter that counts in a pseudorandom order. It's a convenient way to generate apparently random numbers, often used in games to simulate unpredictable behaviors like the result of a dice roll or the amount of gold in a treasure chest. In this section, you'll learn how to implement your own LFSR in assembly and use it to generate random numbers.

An LFSR has two properties: the *number of bits*, which is just the width of the LFSR counter in bits, and the *period*, which is the number of different values it generates. The maximum period is $2^{num_of_bits} - 1$.

Given a number X of N bits, an LFSR generates a new value from X using the following algorithm:

1. Exclusive-or K bits from X together (those bits are called *taps*) to get a new bit.
2. Shift X one bit to the left.
3. Insert the new bit to the right.

Figure 15-4 illustrates the algorithm with a 4-bit LFSR.

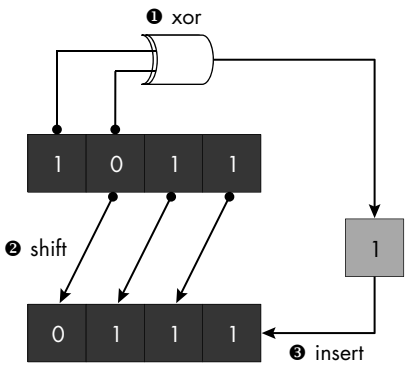


Figure 15-4: Visualizing a 4-bit LFSR

Bits 2 and 3 (the taps) are xor'ed together to get a new bit. Then, the value is shifted to the left, dropping the leftmost bit. Finally, the new bit is inserted on the right end.

This 4-bit LFSR is a *maximal LFSR*, which means it has the longest possible period of 15 ($2^4 - 1$) for its number of bits. Assuming the original 4-bit number equals `$E`, the sequence of generated numbers would be `$E`, `$C`, `$8`, `$1`, `$2`, `$4`, `$9`, `$3`, `$6`, `$D`, `$A`, `$5`, `$B`, `$7`, `$F`, finally cycling back to `$E` again. An LFSR will always generate zero from a zero, but it will never generate zero from a nonzero value. This is good, because otherwise the LFSR would get locked into generating infinite zeros.

Let's look at the *lfsr* sample, where pressing A cycles through a 4-bit LFSR's values. Listing 15-13 shows the 4-bit LFSR's code, but this is only one of many possible implementations.

```
; produce the next 4-bit LFSR value from the value in (a)
; (a) should not be zero, otherwise zero will be returned
; this will never produce zero if (a) is not zero
macro LFSR4_GetNextValue
    ld d, a ; Save the value of (a).
    ld e, a ; (e) is used to perform the taps.

    ; bits 2 and 3 are used as taps; the new bit will be in bit 3 of (a)
        ; bit 3 is already at the right place in (a)

        ; bit 2
        sla e
        xor a, e

    ; bring the new bit into the carry
    swap a
    rla
    ; transfer the new bit into the original value
```

```
    rl d
    ; save the result in (a) and mask to leave only 4 bits
    ld a, d
    and a, $0F
endm
```

Listing 15-13: The code for a 4-bit LFSR

The B button cycles through an 8-bit LFSR's values. This LFSR uses four taps to produce a maximal sequence of 255 values. Its implementation is very similar to the 4-bit LFSR. Check out the sample's code for more details.

Table 15-1 lists a few maximal LFSRs. You can find many more online, and it's a good exercise to try and implement one.

Table 15-1: Examples of Maximal LFSRs

Number of bits	Taps	Period
4	2, 3	15
8	3, 4, 5, 7	255
12	3, 9, 10, 11	4,095
16	3, 12, 14, 15	65,535

LFSRs are very useful for generating pseudorandom numbers. To see a more realistic example, check out the minesweep game in the GitHub repository at *https://github.com/mdagois/gca/tree/main/games/minesweep*; it uses an 8-bit LFSR to randomly place 40 mines at the start of play.

Summary

In this chapter, you learned a bunch of tricks that are likely to come in handy when programming the DMG. The debugging tricks are particularly important when developing complex programs over weeks or months, because they'll help you detect bugs early and save you time when fixing them.

In the next part of the book, we'll turn our attention to DMG variations and peripherals. The next chapter introduces the Game Boy Color, a massive upgrade of the DMG with updated features such as a faster GPU and colored graphics.

Game Boy Coding Adventure (Early Access) © 2025 by Maximilien Dagois

PART IV

BEYOND THE GAME BOY

Game Boy Coding Adventure (Early Access) © 2025 by Maximilien Dagois

16

GAME BOY COLOR

The Game Boy Color (CGB), released in 1998, is a significant upgrade to the original DMG, offering colored graphics, a faster CPU, more memory, and various new functionalities. This chapter covers the full range of CGB features.

We'll begin by diving into the CGB's enhanced graphics capabilities, including extended color palettes, a new VRAM bank, high-level feature improvements, and DMA transfers to VRAM. Next, you'll learn how to activate the CPU's double speed mode, allowing it to run at twice the frequency of the DMG CPU. Then we'll look at additional improvements, such as the CGB's support for WRAM banks, faster serial communication transfer rates, and infrared capabilities.

Finally, you'll see how to write programs that run on both the DMG and the CGB, while still unlocking all the advanced features of the CGB. Because CGB programs are very similar to those that run on the DMG, much of what you've already learned will still apply here.

CGB Graphics

In this section, you'll learn about the colored palette system, the additional VRAM bank that makes this system possible, the updated parameters of the background and sprites, and the VRAM DMA capabilities offered by the CGB. You'll need to use the new palette system and set the background and sprites parameters so that graphics display properly, but using VRAM DMA is optional.

NOTE *Because the CGB has such a strong focus on color graphics, the images in this chapter won't be very useful when viewed in black and white. Full-color versions of the images are available in the GitHub repository at* https://github.com/mdagois/gca/tree/main/images.

All the features of the background apply to the window as well. The two terms are interchangeable in this chapter, so I'll use only the term *background* from this point forward.

Setting Palettes

As you learned in Chapter 5, the DMG uses one background palette and two sprite palettes. Each palette has four colors, with each color represented by 2 bits. You can set the palettes using three registers: `rBGP`, `rOBP0`, and `rOBP1`.

The CGB expands on the palette options: There are eight palettes for the background and eight for the sprites. Each palette still has four colors, but these colors are now 15-bit BGR555 values. *BGR555* is a color format that represents colors as blue, green, and red components, as illustrated in Figure 16-1.

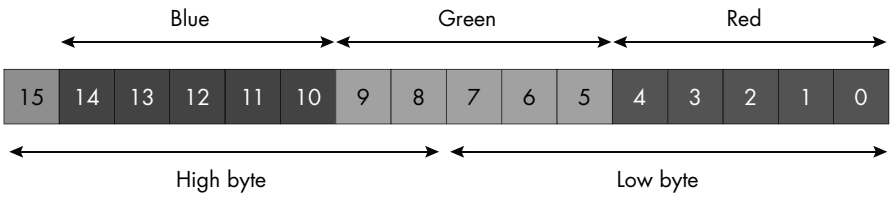


Figure 16-1: The BGR555 color format

Each component takes 5 bits, and they are organized over 2 bytes. The highest bit is unused.

A dedicated piece of RAM in the CGB PPU stores all the palettes. It's split into two areas, one for the background palettes and one for the sprite palettes. Figure 16-2 shows how these areas are organized.

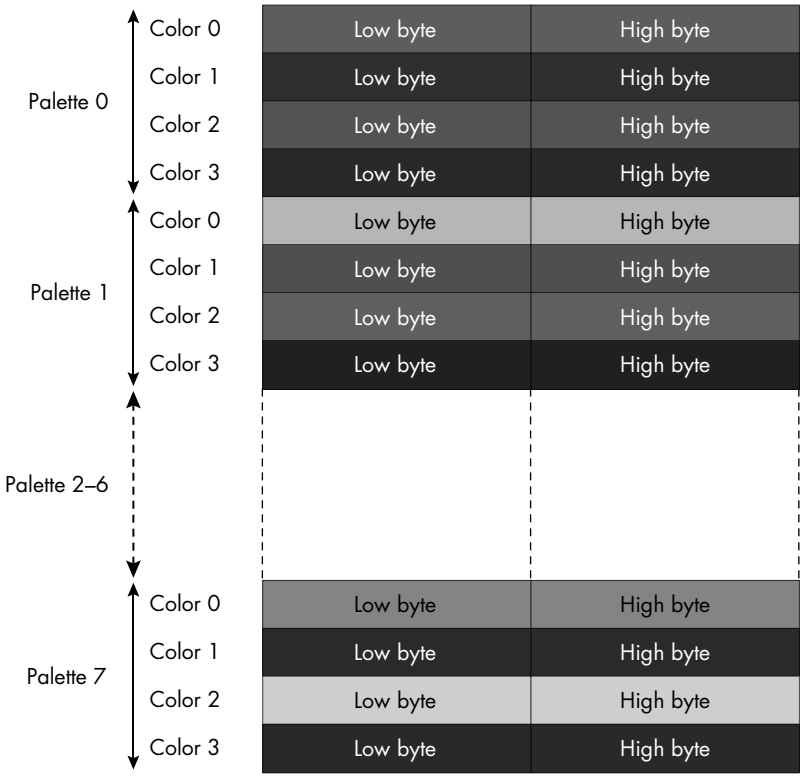


Figure 16-2: The organization of a palette area

Each area contains eight palettes of four BGR555 colors, for a total of 64 bytes (8 palettes × 4 colors × 2 bytes). Each set of 2 bytes represents a single BGR555 color. The low byte of the color is stored first.

On the CGB, palettes are no longer set using the DMG register `rBGP`, as described in Chapter 7. For the background palettes, the CGB uses the *background color palette specification* (`rBCPS`) and *background color palette data* (`rBCPD`) registers. The lower 6 bits of `rBCPS` are the *palette data index*, which specifies which of the 64 palette bytes to set next. The value of that byte is then set through `rBCPD`. Listing 16-1 shows how this works in code.

```
; set the low byte of the first color of the third palette to $1F
ld a, 16
ld [rBCPS], a
ld a, $1F
ld [rBCPD], a

; set the high byte of the third color of the fifth palette to $72
ld a, 37
ld [rBCPS], a
ld a, $72
ld [rBCPD], a
```

Listing 16-1: Setting palette bytes

When computing a palette byte index, each palette is 8 bytes, each color is 2 bytes, and the low byte of each color comes before its high byte. For example, in Listing 16-1, the high byte of the third color of the fifth palette is at index 37 because there are four palettes (4 × 8 bytes = 32 bytes), two colors (2 × 2 bytes = 4 bytes), and the low byte (1 byte) before it. Here's the formula to determine a palette byte index from a palette and color:

```
byte_index = (palette_number * 8) + (color_number * 2) + (0 if low_byte, otherwise 1)
```

This formula assumes palette numbers from 0 to 7 and color numbers from 0 to 3. The final part of the formula selects either the low byte or the high byte of the target color.

The `rBCPS` register has one additional flag on bit 7: the *auto increment* flag. When this flag is raised, the CGB automatically increments the palette data index each time a value is written to `rBCPD`. This is a handy feature because it's common to set all the bytes of one or more palettes sequentially when loading graphics from the ROM. Listing 16-2 shows how to set the third palette using the auto increment functionality.

```
; set the index to point to the first byte of the third palette
; with auto increment enabled
ld a, 16 | BCPSF_AUTOINC
ld [rBCPS], a

; set the 8 bytes of the 4 colors of the palette
ld hl, rBCPD

    ; color 0
    ld a, $FC
    ld [hl], a
    ld a, $6B
    ld [hl], a

    ; color 1
    ld a, $11
    ld [hl], a
    ld a, $3B
    ld [hl], a

    ; color 2
    ld a, $A6
    ld [hl], a
    ld a, $29
    ld [hl], a
```

```
; color 3
ld a, $61
ld [hl], a
ld a, $10
ld [hl], a
```

Listing 16-2: Setting the third palette

The `rBCPS` register is set only once, thanks to the auto increment flag. Without that flag, you would need to set the byte index in the `rBCPS` register before writing each byte to the `rBCPD` register, which would roughly double the number of instructions.

Setting the sprite palettes works exactly the same way as for the background. The registers used are the *object color palette specification* (`rOCPS`) and *object color palette data* (`rOCPD`) registers. The DMG registers `rOBP0` and `rOBP1`, covered in Chapter 7, are not used anymore.

The PPU accesses the palette RAM when rendering to the LCD (mode 3, discussed in Chapter 12). During that time, we cannot access the palettes from the CPU; access to that memory area is restricted, just like the VRAM. You'll need to set the palettes at initialization time or during the vblank to avoid issues.

Table 16-1 lists the registers related to palette management, including the DMG registers for reference.

Table 16-1: Palette Registers

Register	Bits	Description
`rBCPS`	0–5	Background palette data index
	7	Background palette data index auto increment flag
`rBCPD`	All	Background palette byte data
`rBGP`	—	DMG background palette (unused in the CGB)
`rOCPS`	0–5	Sprite palette data index
	7	Sprite palette data index auto increment flag
`rOCPD`	All	Sprite palette byte data
`rOBP0`	—	DMG sprite palette 0 (unused in the CGB)
`rOBP1`	—	DMG sprite palette 1 (unused in the CGB)

Now that you've seen how the palettes are organized and how to set them up, let's go over the *cgb_palettes* sample on GitHub. You can use the *color_picker.html* file in the *tools* folder of the repository to translate web colors (RGB888) into CGB colors (BGR555).

This sample sets the first three background palettes using 12 colors from the color wheel in Figure 16-3.

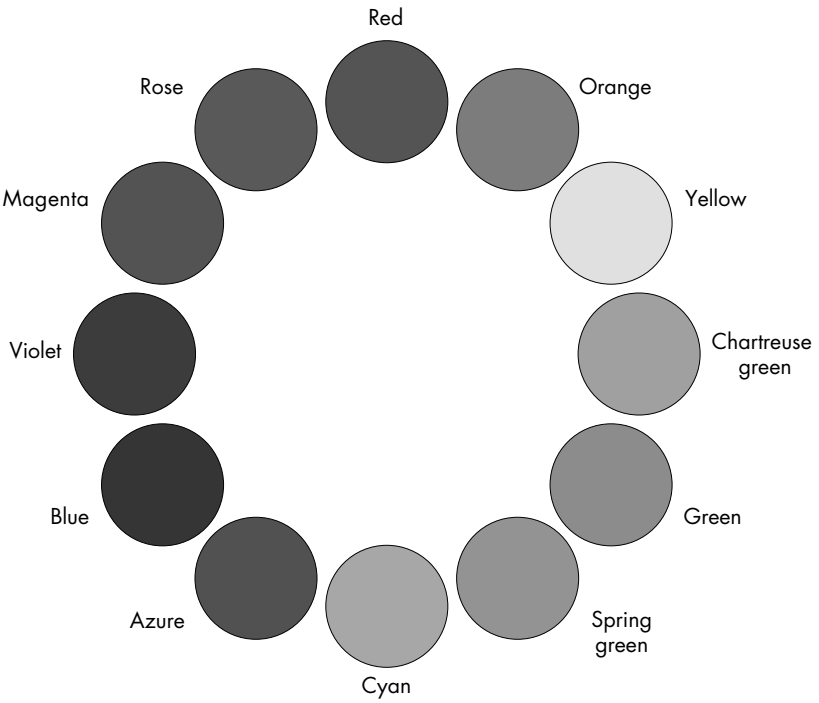


Figure 16-3: Color wheel

The palettes are set at initialization. Nothing displays to the LCD, but we can observe the effects of the sample through BGB's VRAM viewer. Figure 16-4 shows a screenshot of the viewer's window.

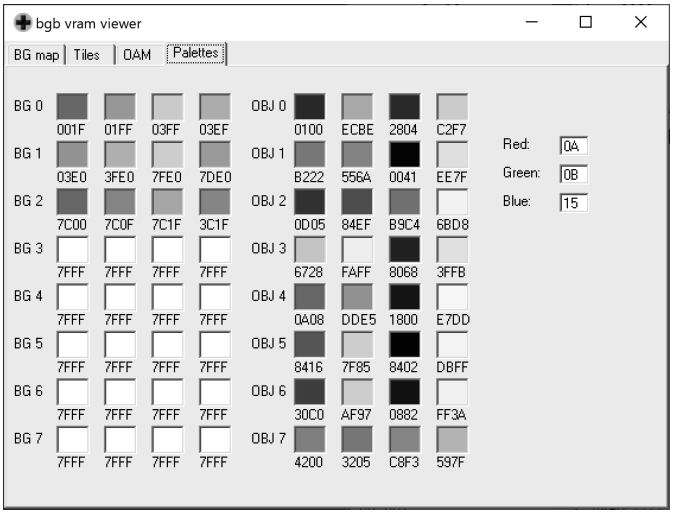


Figure 16-4: CGB palettes

The colors in the background palettes match those in the color wheel. The sprite palettes on the right (`OBJ 0` to `OBJ 7`) may look different when you

run the sample; because we haven't set these palettes yet, they have random values.

To compile the sample, you need to change two of the build commands. First, remove the `--dmg` parameter from the `rgblink` command, because it is meant to disable any feature not available on the DMG (for example, all of the CGB features). Second, add the `--color-only` parameter to the `rgbfix` command. This marks the ROM as *CGB-only*, which means it works only on the CGB. The two modified commands should look like the following:

```
> rgblink --tiny --map sample.map --sym sample.sym -o sample.gbc main.o sample.o
> rgbfix --title game --color-only --pad-value 0 --validate sample.gbc
```

Note that the ROM extension is now *.gbc*, instead of *.gb*. Most emulators expect the *.gbc* extension for CGB ROMs, so it's best to stick to that naming convention.

The `rgbfix` command has additional parameters to control the level of compatibility of a ROM with DMG and CGB hardware. I'll cover these in more detail in "Compatibility Mode" on page 300.

Leveraging the VRAM Banks

Now that we know how to load colored palettes, let's see how the VRAM is organized on the CGB and the improvements brought to the tiles and tilemaps.

The DMG has 8KiB of VRAM, which holds up to 384 tiles and 2 tilemaps. The CGB adds another 8KiB of VRAM, for a total of 16KiB. But because the memory map has only 8KiB dedicated to VRAM, the VRAM is split into two banks of 8KiB. To switch between the two VRAM banks, the CGB features a new register called `rVBK`. Bit 0 is used to select the bank, and the remaining bits are unused. The first bank is organized exactly like the VRAM in the DMG (see Table 5-3). The second is organized similarly, as shown in Table 16-2.

Table 16-2: Organization of VRAM Bank 2

Area	Size	Sub-area	Start	End
Tiles	6KiB	Block 0	$8000	$87FF
		Block 1	$8800	$8FFF
		Block 2	$9000	$97FF
Tilemaps	2KiB	Tilemap 0 parameters	$9800	$9BFF
		Tilemap 1 parameters	$9C00	$9FFF

Like in the first VRAM bank, there are 384 tiles, organized into three blocks. This increases the total number of tiles to 768. However, there are no extra tilemaps. There are still only two of them, just like on the DMG, and both are in the first bank. The second VRAM bank is dedicated to additional parameters, such as tile flips, applying to the tilemaps in the first bank.

For each tile index in the first bank's tilemaps, there is a byte at the same address in the second bank that holds the parameters applying to that tile. Table 16-3 shows how a parameter byte is organized.

Table 16-3: The Tilemap Parameter Byte

Bits	Purpose	Description
0–2	Palette number	Selects one of the eight background palettes
3	VRAM bank number	Selects the tile VRAM bank
4	—	Unused
5	Horizontal flip	Flips the tile horizontally when the bit is set
6	Vertical flip	Flips the tile vertically when the bit is set
7	Priority mode	Selects the priority mode (0 = OAM priority mode, 1 = background priority mode)

Bits 0 to 2 select a background palette for the tile. This is not necessary on the DMG, because there is only one palette. Bit 3 is the number of the bank from which the tile data is pulled when the PPU renders the graphics to the LCD. This unlocks access to the full 768 tiles in VRAM.

Bits 5 and 6 work like the sprite flipping bits from Table 7-4. They can be leveraged to dramatically increase the tiles' variation. Finally, bit 7 controls the priority of the background tile over the sprites. When this bit is set, the background always renders over the sprites, no matter how bit 7 of the OAM control flags is set (for more on this, see "Object Attribute Memory" on page 107). By using both the OAM bit 7 flag and the tiles' priority mode, discussed further in "Managing Priority" on page 287, you can create the illusion of a sprite moving between background elements.

The *cgb_bg* sample shows a colored background representing a ship deck. The original PNG asset was converted using a custom tool called *cgbconv*. This tool and its full source code are available in the repository. Like gfxconv, I developed cgbconv to meet the needs of this book, and it's not meant to be a generic solution.

Similar to gfxconv, the cgbconv tool extracts tilesets and tilemaps from PNG files. However, it generates graphics data that takes advantage of new features available on the CGB, such as palettes, background tile flips, and VRAM banks. A conversion results in four binary files: a PAL file, a CHR file, an IDX file, and a PRM file. I'll describe each of these next.

PAL files contain eight palettes. Empty palettes are added if the original PNG asset does not contain enough colors to fill eight distinct palettes. The PAL file for the sample looks like Figure 16-5.

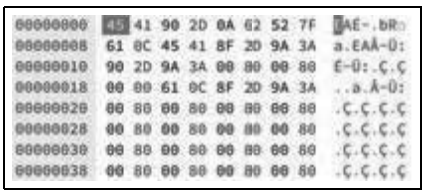


Figure 16-5: The CGB palette binary file

Each line represents a palette, consisting of four colors of 2 bytes each. The sample uses four palettes extracted from the PNG. The remaining palettes are filled with `$8000` (notice how the bytes are reversed). This value is not a valid BGR555 color because the 16th bit is set to 1, while the format uses only 15 bits. Using an invalid color makes the unused color easy to spot even in binary format.

Similar to their counterparts from gfxconv, CHR files contain up to 512 tiles, equivalent to four tile blocks in VRAM (two blocks per VRAM bank). The first 256 tiles are meant to be copied into VRAM bank 0 and the remaining ones into VRAM bank 1. The CHR file in the sample has far fewer tiles (about 38), so a single copy to VRAM bank 0 will be enough.

IDX files are equivalent to the TLM files produced by gfxconv. They contain tile indices that point to tiles in the CHR file. Their content is meant to be copied to the tilemap area of the first VRAM bank.

Finally, PRM files contain tile parameters. The tool identifies an appropriate palette, assigns the tile's VRAM bank number, and sets the correct flips. A PRM file's content is meant to be copied to the tilemap area of the second VRAM bank.

Listing 16-3 shows the code to bring the four binary files into the ROM.

```
section "graphics", rom0

palette_data:
incbin "ship.pal"
palette_data_end:

tileset:
incbin "ship.chr"
tileset_end:

tilemap_indices:
incbin "ship.idx"
tilemap_indices_end:

tilemap_parameters:
incbin "ship.prm"
tilemap_parameters_end:
```

Listing 16-3: Including binary data

We declare a section to store the files' binary data, then use `incbin` repeatedly to include the content of each file as is.

During the initialization of the sample, we copy the binary data from the files to the VRAM, starting with the palettes, as shown in Listing 16-4.

```
copy [rBCPS], 0 | BCPSF_AUTOINC
ld hl, palette_data
ld c, palette_data_end - palette_data
.palette_copy
    copy [rBCPD], [hli]
    dec c
    jr nz, .palette_copy
```

Listing 16-4: Copying the palettes

This code copies the PAL file using the auto increment feature. This is very efficient, because the code in the loop only needs to copy the palette data from the ROM, starting at the `palette_data` address, into the `rBCPD` register repeatedly.

Figure 16-6 shows the palettes in BGB.

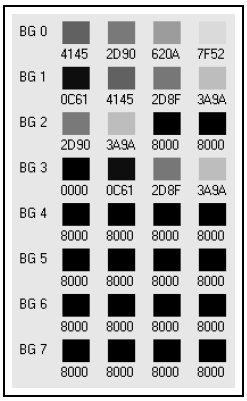


Figure 16-6: The copied palettes of the sample

The unused palette colors are all set to `$8000`, an invalid color that makes the unused colors easy to identify in BGB.

Next, we copy the tile data from the tileset (CHR file) in the first block of the first VRAM bank. The code is similar to what would be used on a DMG, so I won't replicate it here. The sample uses only 38 tiles, which are all located in block 0 of VRAM bank 0, as shown in Figure 16-7.

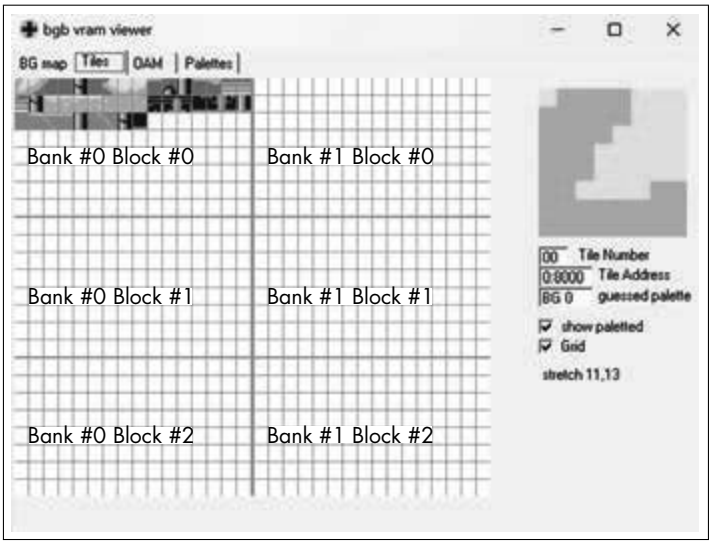


Figure 16-7: The sample tiles and the VRAM blocks

The display in BGB's VRAM viewer is split into six areas. The three blocks of bank 0 are on the left, and the three blocks of bank 1 are on the right. Thanks to this organization, all the tiles are always visible in the VRAM viewer.

Finally, the tilemap indices and parameters are copied to the first tilemap (address `$9800`). For each tilemap, the indices must reside in bank 0, while the parameters must be at the same addresses in bank 1. The indices are copied to bank 0 using the code in Listing 16-5.

```
copy [rVBK], 0
ld de, _SCRN0
ld hl, tilemap_indices
.tilemap_indices_copy
    copy [de], [hli]
    inc de

    ld a, l
    cp a, low(tilemap_indices_end)
    jr nz, .tilemap_indices_copy
    ld a, h
    cp a, high(tilemap_indices_end)
    jr nz, .tilemap_indices_copy
```

Listing 16-5: Copying the tilemap indices

The parameter copy is basically the same, but the parameters must reside in bank 1. We switch the active bank before the copy takes place, using `copy [rVBK], 1`.

Once built and run, the sample should look like Figure 16-8.

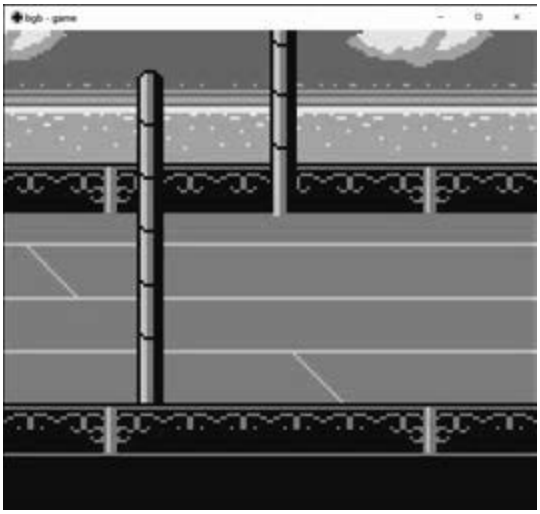


Figure 16-8: The ship deck

The deck itself is brown and the sky is blue. The graphics are a significant improvement over the DMG.

To end this section, let's talk about debugging with VRAM banks in BGB. It's not possible to see all the VRAM banks at once in the debugger's data view, because we can map only a single bank at any one time. For example, right after the sample has started, the values from VRAM bank 1 are visible at address `$9800` in the debugger's data view. That's because the code switches to that bank at the end of the sample initialization.

When debugging, you can check the values of the VRAM bank that is not mapped, thanks to BGB's IO map. From the debugger, the IO map (shown in Figure 16-9) can be accessed via the Window menu or by pressing F10.

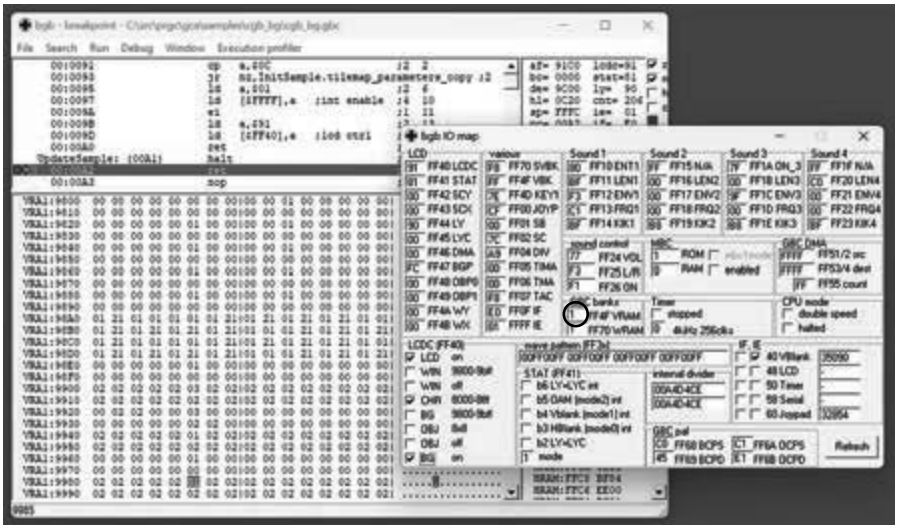


Figure 16-9: The IO map for bank 1

Notice the VRAM label in the CGB Banks section. You can edit this value by changing it and pressing Enter to apply the change.

For example, in the sample, if you change the value to 0 the data view switches to the content of bank 0, as shown in Figure 16-10.

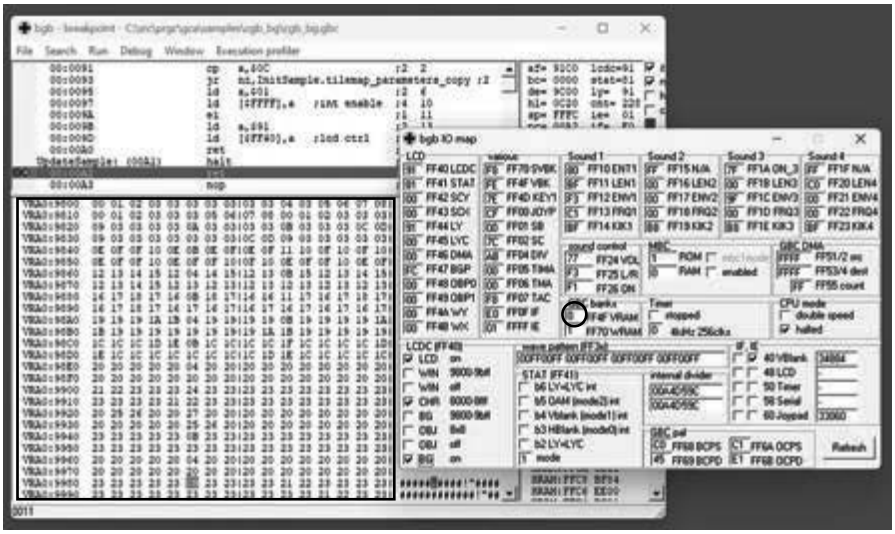


Figure 16-10: The IO map for bank 0

You might have to scroll the memory map in the data view to find the relevant section. You can also use CTRL-G to go directly to an address you specify.

You can experiment with the effects of various tilemap parameters by modifying the values in VRAM. For example, the original value of the first tilemap parameter at address `$9800` is `$00`. This means the tile uses the first palette and has no flip applied to it. You can edit this value in the data view by right-clicking on the byte at `$9800` and choosing the Modify Code/Data option). Figure 16-11 shows examples of what happens when you change the first tilemap parameter.

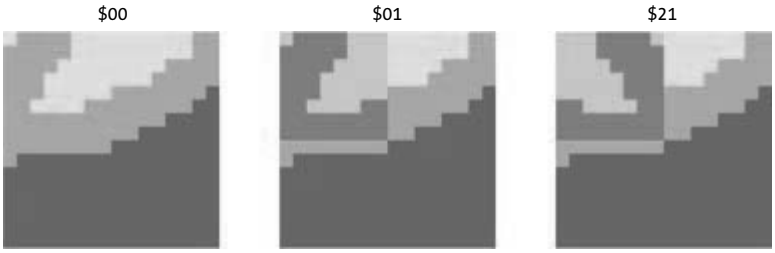


Figure 16-11: Modifying the tilemap parameter

The tilemap parameter for the top-left tile is originally equal to `$00`. The image on the left shows what this looks like. In the middle, I've changed the parameter value to `$01`, which changes the palette of the tile. On the right, I've changed the value to `$21`, which adds a horizontal flip in addition to changing the palette.

Refer to Table 16-3 when trying out different values.

Using the Sprite Functionalities

Like the tilemaps, the sprites can leverage the new colored palettes and the extra VRAM bank available on the CGB. Let's see how to do this using the sprite OAM flags.

The OAM flags (the last byte of the OAM attributes, as discussed in "Object Attribute Memory" on page 107) are a bit different on the CGB, to reflect its enhanced graphics capabilities. Table 16-4 summarizes the organization of the flags on the CGB.

Table 16-4: CGB OAM Flags

Bits	Description
0–2	Sprite palette number
3	VRAM bank number
4	Unused in the CGB
5	Flips the sprite horizontally when the bit is set
6	Flips the sprite vertically when the bit is set
7	Draws the background colors 1 to 3 over the sprite when the bit is set

The palette number is now set through 3 bits (bits 0 to 2) in the flags so that we can use any of the eight palettes. Bit 3 is dedicated to choosing the VRAM bank from which to pull the tile data for the sprite. Bit 4 is unused, because it is the palette number on the DMG. Bit 5 to 7 are unchanged.

The *cgb_sprites* sample used in this section adds two rotating coins to the previous sample to demonstrate the use of the new flags. It displays the coins using sprites referring to the same tile graphics; however, the coins are different colors and rotate in opposite directions. This is achieved by setting the OAM flags to the appropriate values.

At the start of the initialization, the coins' tile data is loaded into VRAM bank 1, as shown in Listing 16-6.

```
    copy [rVBK], 1
    ld de, _VRAM8000
    ld hl, coins
    .coins_copy
        copy [de], [hli]
        inc de

        ld a, l
```

```
    cp a, low(coins_end)
    jr nz, .coins_copy
    ld a, h
    cp a, high(coins_end)
    jr nz, .coins_copy
```

Listing 16-6: Sprite tile data

The sample uses 8×16 sprites (enabled through the `LCDCF_OBJ16` flag of the `rLCDC` register). Because coins are 16×16, they use two sprites each: one for the left part of the coin and one for the right part. There are six frames of rotating animation, with four 8×8 tiles representing each frame, for a total of 24 tiles for the whole coin rotation.

Once the sample is done copying the tile data, it sets two sprite palettes, one for each coin, as shown in Listing 16-7.

```
copy [rOCPS], 0 | BCPSF_AUTOINC

    ; black
    copy [rOCPD], $00
    copy [rOCPD], $00
    ; orange
    copy [rOCPD], $7F
    copy [rOCPD], $02
    ; yellow
    copy [rOCPD], $BF
    copy [rOCPD], $1F
    ; white
    copy [rOCPD], $FF
    copy [rOCPD], $FF

    ; black
    copy [rOCPD], $00
    copy [rOCPD], $00
    ; red
    copy [rOCPD], $5C
    copy [rOCPD], $01
    ; orange
    copy [rOCPD], $FE
    copy [rOCPD], $09
    ; white
    copy [rOCPD], $FF
    copy [rOCPD], $FF
```

Listing 16-7: Sprite palettes

The first color of each palette (black, here) is *transparent*. Sprite pixels in this transparent color don't render to the LCD, which means the background will display through those pixels.

In the sample, we clear the remaining six palettes with a dummy color. This is not a mandatory step, since the sample uses only two palettes, but when debugging, it might help to set those palettes to a flashy color (say, magenta); this makes it easier to quickly spot sprites that use a noninitialized palette. You can then fix the issue either by loading the missing palettes or by making sure the sprites use already loaded palettes.

At the end of the initialization function, the sample clears the OAM via the `InitOAM` macro and initializes the sprites used for the two coins. Listing 16-8 shows the code to set the X position, Y position, and flags parameters of each sprite.

```
InitOAM

; set the coin parameters that are fixed
copy [COIN_0_SPRITE_LEFT + OAMA_Y], COINS_SPRITE_Y
copy [COIN_0_SPRITE_LEFT + OAMA_X], COIN_0_SPRITE_X
copy [COIN_0_SPRITE_LEFT + OAMA_FLAGS], COIN_PALETTE_0 | OAMF_BANK1
copy [COIN_0_SPRITE_RIGHT + OAMA_Y], COINS_SPRITE_Y
copy [COIN_0_SPRITE_RIGHT + OAMA_X], COIN_0_SPRITE_X + 8
copy [COIN_0_SPRITE_RIGHT + OAMA_FLAGS], COIN_PALETTE_0 | OAMF_BANK1

copy [COIN_1_SPRITE_LEFT + OAMA_Y], COINS_SPRITE_Y
copy [COIN_1_SPRITE_LEFT + OAMA_X], COIN_1_SPRITE_X
copy [COIN_1_SPRITE_LEFT + OAMA_FLAGS], \
     COIN_PALETTE_1 | OAMF_BANK1 | OAMF_XFLIP
copy [COIN_1_SPRITE_RIGHT + OAMA_Y], COINS_SPRITE_Y
copy [COIN_1_SPRITE_RIGHT + OAMA_X], COIN_1_SPRITE_X + 8
copy [COIN_1_SPRITE_RIGHT + OAMA_FLAGS], \
     COIN_PALETTE_1 | OAMF_BANK1 | OAMF_XFLIP
```

Listing 16-8: Sprite initialization

These OAM parameters never change during the sample execution. The X and Y positions of the sprites are set using constants defined in the program.

The flags are more relevant to this chapter. First, the code assigns different palette numbers to the sprites for each coin so that they display in different colors. All the sprites reference tiles in VRAM bank 1, because this is where the tile data was copied to. Finally, the code flips the sprites for the second coin horizontally to make that coin rotate in the opposite direction of the first one.

The sample animates the sprites by changing their tile IDs over time in the update function. We'll skip the code here, but you can find it in the GitHub repository. Figure 16-12 shows a screenshot of the running sample.

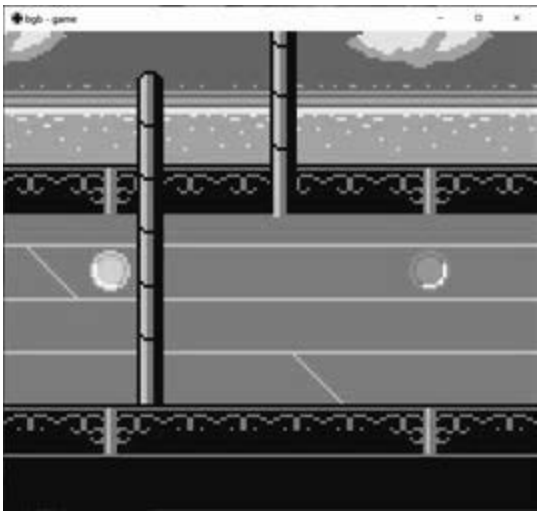


Figure 16-12: The CGB sprites sample

You'll have to run it yourself to appreciate the differences in the coins. They should appear in different colors and rotate in opposite directions.

Managing Priority

On the DMG, sprites can display either over or behind the background. The behavior for each sprite is controlled through bit 7 of the OAM flags. The CGB offers three parameters you can use to control the sprites and background rendering priority.

The first parameter is the *master priority* flag, which is bit 0 of the LCD control register (`rLCDC`). On the DMG, this bit controls whether the background is enabled or not, but it has been repurposed on the CGB: When bit 0 is cleared, the background always renders behind the sprites, and the other parameters related to priority are ignored. When the flag is set, the two remaining parameters determine the priority.

NOTE *There is very little incentive to disable the background in a DMG program, because the graphics would automatically become extremely simplistic. That's most likely why bit 0 of the LCD control register was repurposed on the CGB.*

The second parameter is bit 7 in the OAM flags, which works like on the DMG (see Table 16-4 in the previous section). When the bit is set, the background renders over the sprites so that the sprites become partially or fully hidden.

The last parameter is bit 7 of the tilemap parameters, described in Table 16-3. When this is set, the background always renders over the sprites.

Figure 16-13 summarizes how priority works on the CGB.

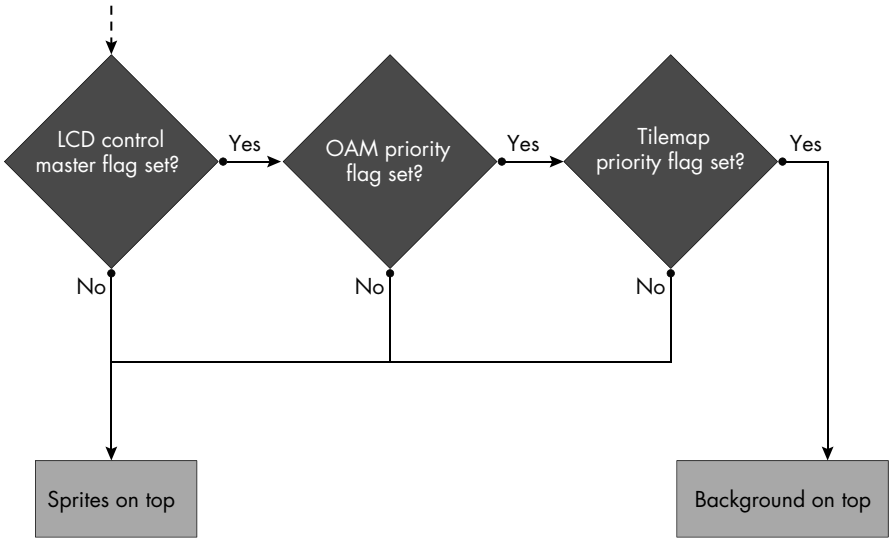


Figure 16-13: Determining background priority on the CGB

This section's sample, *cgb_priority*, displays a coin moving horizontally near the left mast of the ship deck. Figure 16-14 shows snapshots of the coin's movement.

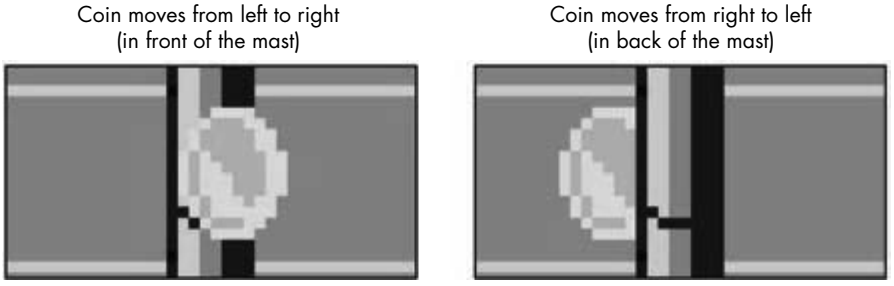


Figure 16-14: Displaying the coin over and behind the mast

When the coin moves from left to right, it displays in front of the mast. When the coin moves from right to left, it displays behind the mast. This makes it look like the coin is moving around the mast.

To achieve this effect, the sample leverages the tilemap parameter priority flag (see Figure 16-13). The coin moves over two tiles of the mast. The sample toggles the priority flag of those two tiles using the code shown in Listing 16-9.

```
    ld a, [WRAM_TOGGLE_PRIORITY]
    dec a
    jr nz, .toggle_background_parameters
        copy [WRAM_TOGGLE_PRIORITY], 0

        copy [rVBK], 1
```

```
        ld hl, _SCRN0 + 8 * SCRN_VX_B + 5
        ld a, [hl]
        xor a, %10000000
        ld [hl], a

        ld de, 32
        add hl, de
        ld a, [hl]
        xor a, %10000000
        ld [hl], a
    .toggle_background_parameters
```

Listing 16-9: Toggling priority

The `WRAM_TOGGLE_PRIORITY` flag has a value of either zero or one. When the flag is cleared, the tile parameters remain untouched. When it's set, the code toggles the priority flags of the two tiles. Toggling the flags works with simple `xor` operations so that bits other than the priority one remain unchanged. The code switches to VRAM bank 1 because that's where the tile parameters are stored.

The `WRAM_TOGGLE_PRIORITY` flag is set every time the coin reaches its leftmost or rightmost position. At those times, the coin graphics do not overlap with the mast, so switching the mast tile flags does not cause any visual glitches. The code responsible for raising `WRAM_TOGGLE_PRIORITY` is part of the coin animation in the update function of the sample, which is not shown here.

Be sure to toggle tile parameters from inside the vblank, because these parameters reside in VRAM. Also, keep in mind that color 0 of a background tile is transparent when displaying on top of sprites, which means the palette of such a tile holds only three active colors instead of the usual four. This limits your artistic choices when designing the tile graphics.

Copying Data to VRAM with DMA

The last new graphics feature of the CGB is VRAM DMA, an additional piece of DMA functionality that can copy data very quickly from ROM, ERAM, or WRAM to VRAM. Using VRAM DMA is optional, but it's good to know how to leverage it in case you need the extra speed.

In Chapter 12 you learned about the OAM DMA functionality of the DMG, which quickly copies sprite attributes to OAM. This functionality is also available on the CGB and is twice as fast when the CPU is in double speed mode, which I'll cover in the next section. VRAM DMA shares the same purpose of copying data quickly from one place to another, but it's easier to use and more versatile than OAM DMA.

NOTE *The emulation of the DMA feature in BGB is not accurate. It is far more permissive than on real hardware. When using DMA, make sure to test your programs on real hardware.*

VRAM DMA has five dedicated registers: two for the source address, two for the destination address, and one for the copy size and control flags.

We specify the source address through `rHDMA1` (the high byte) and `rHDMA2` (the low byte). We ignore the lower 4 bits of the low byte, which are always considered to be zero, so the address format is `$XXX0`. There are two valid address ranges: `$0000` to `$7FF0` (in the ROM) and `$A000` to `$DFF0` (in the ERAM or WRAM). For example, for the address `$6270`, `rHDMA1` would be `$62` and `rHDMA2` would be `$70`. Because the lower 4 bits of the low byte are ignored, using any low byte between `$71` and `$7F` in `rHDMA2` would yield the same source address of `$6270`. Addresses in VRAM canot be used as source addresses. The result of such a DMA copy would be trash data in VRAM.

We specify the destination address through `rHDMA3` (the high byte) and `rHDMA4` (the low byte). Like with `rHDMA2`, we ignore the lower 4 bits in `rHDMA4`. We also ignore the higher 3 bits of the high byte, always considering them to be equal to `%100`. This makes sense, because all valid VRAM addresses have their top 3 bits equal to `%100`. The valid address range is `$8000` to `$9FF0`, covering all of VRAM. That means we can copy tile and tilemap data using DMA, which is a lot faster than using load instructions.

The last register, `rHDMA5`, has several purposes, including starting the DMA copy, specifying the size of the source data, setting the transfer mode, and monitoring the transfer status. The DMA copy starts as soon as the register is written, so make sure registers `rHDMA1` to `rHDMA4` contain the source and destination addresses prior to writing any value into `rHDMA5`.

The lower 7 bits of the register specify the size of the data to copy, which is *not* expressed in bytes. Instead, it is a value from 0 to 127 that follows the formula

```
value = (byte_size / 16) - 1
```

where:

```
byte_size = (value + 1) * 16
```

When copying tiles, you can think of the value as the number of tiles to copy minus one, because a tile is 16 bytes. That means up to 128 tiles (2,048 bytes) can be copied at once. That's equivalent to a tile block, as shown in Chapter 5. When copying data into the tilemaps, think of the value as the number of 16-indice groups minus one. Such a group is half a line of tiles in the tilemap. Up to 2,048 indices can be copied at once, which is equivalent to two tilemaps. The DMA copy is not limited to tile block and tilemap boundaries. It's perfectly valid to copy to any range inside the VRAM, so organize your data and copy it with DMA in the way that's most convenient for your program.

The high bit of `rHDMA5` has two roles: to select one of the two transfer modes and to monitor the transfer status. The two available transfer modes are the general-purpose mode and the hblank mode.

The *general-purpose* mode processes the data copy in a single shot. The CPU blocks execution until the copy is done. It takes around 7.6 microseconds to transfer each group of 16 bytes, or approximately 977 microseconds for the maximum of 2,048 bytes per copy. Unlike OAM DMA copies, the duration of a VRAM DMA copy is not affected by the CPU speed mode; it takes the same amount of time in normal and double speed modes. The DMA copy should not spill over into the LCD refresh, because it could corrupt the graphics. As the programmer, you need to make sure the copy happens during the vblank or when the LCD is disabled.

The *hblank* mode processes the data copy during the hblank periods of the LCD refresh, from lines 0 to 143. At the start of each hblank, the CPU stops executing any instructions for a duration of 7.6 microseconds and copies 16 bytes' worth of data to VRAM instead. Once a DMA copy is initiated (by writing to `rHDMA5`), the CGB starts copying data at the next hblank, and the process continues over as many hblanks as necessary to copy all the data into VRAM. For example, when a DMA copy of 1,600 bytes (100×16 bytes) initiates during the vblank, the copy occurs from lines 0 to 99, with 16 bytes copied during each line's hblank. The largest possible copy size is 2,048 bytes (128×16 bytes). This means that whenever a DMA copy is initiated during a vblank it's guaranteed to end before the next vblank, because the copy occurs over 128 hblanks out of the total of 144 hblanks in an LCD refresh.

Note that no transfer takes place during the vblank, because hblanks occur only during the LCD refresh. A transfer might get processed over two LCD refreshes. For example, if a DMA copy of 800 bytes (50×16 bytes) is initiated on line 104, only 40 groups of 16 bytes will be copied before the vblank because there are only 40 hblanks left in the LCD refresh. The copy will then pause during the vblank, and the 10 remaining groups of 16 bytes will be copied during the hblanks of lines 0 to 9 of the next LCD refresh. When using hblank mode, always make sure there are enough hblanks between the start of the DMA transfer and the time at which the data must be in VRAM for rendering. Otherwise, the graphics may not display as expected for a frame or so.

Once a DMA transfer starts, you can use the high bit of `rHDMA5` to monitor whether the transfer has completed. When the bit is zero, the transfer is in progress. When the bit is set, the transfer is complete.

There are two pitfalls to avoid when using the hblank mode. First, the hblank interrupt must not be enabled, because it would interfere with the hblank DMA transfer. You can have any other LCD interrupt, but not the hblank one. Second, don't use the `halt` instruction when the transfer is still in progress, because this will pause the transfer. This is where the monitoring flag comes in handy. After starting a transfer, you can use the flag to delay any calls to `halt`.

Table 16-5 summarizes the DMA registers.

Table 16-5: DMA Registers

Register	Bits	Description
rHDMA1	All	Source address high byte
rHDMA2	All	Source address low byte (lower 4 bits always considered zero)
rHDMA3	All	Destination address high byte (higher 3 bits always considered %100)
rHDMA4	All	Destination address low byte (lower 4 bits always considered zero)
rHDMA5	0–6	Size of the transfer (byte size / 16 – 1)
rHDMA5	7	Transfer mode (0 = general purpose, 1 = hblank) and monitoring (0 = in progress, 1 = complete)

It's possible to initiate a DMA copy to either of the VRAM banks, but you'll need to switch to the proper destination VRAM bank first, using the rVBK register as explained previously in this chapter.

The first sample we'll use in this section, *cgb_dma_gp*, looks exactly like the one from the previous section. However, during initialization, it transfers the ROM data using the general-purpose DMA functionality.

Listing 16-10 shows a macro dedicated to starting DMA transfers.

```
; start a DMA copy
; \1 = destination address
; \2 = source address
; \3 = size
; \4 = mode (HDMA5F_MODE_GP or HDMA5F_MODE_HBL)
macro DmaCopy
    assert (\3) >= 16, "The DMA copy size must be at least 16 bytes"
    assert (\3) % 16 == 0, "The DMA copy size must be a multiple of 16 bytes"
    copy [rHDMA1], high(\2)
    copy [rHDMA2], low(\2)
    copy [rHDMA3], high(\1)
    copy [rHDMA4], low(\1)
    copy [rHDMA5], (\4) | (((\3) / 16) - 1)
endm
```

Listing 16-10: DMA generic macro

This macro sets the registers based on four parameters: the destination address, the source address, the byte size of the copy, and the transfer mode. It also performs a couple of checks on the copy size.

Listing 16-11 shows two convenience macros that are mode-specific specializations of the generic macro from Listing 16-10.

```
; start a general-purpose DMA copy
; \1 = destination address
; \2 = source address
; \3 = size
macro DmaCopy_General
```

```
    DmaCopy \1, \2, \3, HDMA5F_MODE_GP
endm

; start an hblank DMA copy
; \1 = destination address
; \2 = source address
; \3 = size
macro DmaCopy_Hblank
    DmaCopy \1, \2, \3, HDMA5F_MODE_HBL
endm
```

Listing 16-11: DMA mode-specific macros

One triggers a general-purpose DMA copy and the other an hblank DMA copy. Listing 16-12 shows the sample initialization code that uses the macros.

```
    ; VRAM 0
    copy [rVBK], 0
        ; tileset (background)
        DmaCopy_General _VRAM8000, tileset, tileset_end - tileset
        ; tile indices
        DmaCopy_General _SCRN0, \
                        tilemap_indices, \
                        tilemap_indices_end - tilemap_indices

    ; VRAM 1
    copy [rVBK], 1
        ; tileset (coin)
        DmaCopy_General _VRAM8000, coins, coins_end - coins
        ; tile parameters
        DmaCopy_General _SCRN0, \
                        tilemap_parameters, \
                        tilemap_parameters_end - tilemap_parameters
```

Listing 16-12: General-purpose DMA usage

DMA copies are faster than regular loop copies. During initialization, this won't matter much because the LCD is disabled and there are no time constraints for accessing the VRAM. But during the vblank, which lasts only 1.08 ms, DMA might help you push more data into VRAM over a single frame. Another advantage of using DMA is that the DMA macros change only register `a`. With a loop macro, the `hl` and `de` registers would also change, meaning their contents would be lost if not saved beforehand.

Let's take a look at another sample in the GitHub repository, *cgb_dma_hb*, which does not transfer the tiles' indices to VRAM during initialization. Instead, it copies them when you press the A button, using an hblank DMA copy. The `DmaCopy_Hblank` function inside the sample's update function triggers the transfer.

There are two important aspects to this sample. First, it enables the LCD interrupt, as shown in Listing 16-13.

```
copy [rSTAT], STATF_MODE10
copy [rIE], IEF_VBLANK | IEF_LCDC
ei
```

Listing 16-13: Enabling the LCD interrupt

This will allow us to observe the behavior in BGB (more details on this soon). Notice that the interrupt is set to trigger on entering mode 2 (the OAM scan), because triggering it on mode 0 (the hblank) would interfere with the DMA copy on real hardware.

Second, the update function waits for the transfer to complete before returning, as shown in Listing 16-14.

```
.wait_dma
    ld a, [rHDMA5]
    and a, HDMA5F_BUSY
    jr z, .wait_dma

ret
```

Listing 16-14: Waiting for transfer completion

This step is absolutely necessary on real hardware, because otherwise `halt` will be called prematurely, stopping the DMA transfer.

To observe the hblank DMA in action in BGB, you'll need a combination of LCD interrupts and breakpoints. In the sample code, two labels assist with placing breakpoints: `breakpoint_1_transfer` and `breakpoint_2_lcd`. The first breakpoint helps catch the start of the transfer after you press the A button. The second breakpoint is in the hblank LCD interrupt and lets you track each transfer, one at a time. Here are the steps to observe the hblank transfer:

1. In the debugger, place a breakpoint on `breakpoint_1_transfer`.
2. Go back to the main window and press **A** to trigger the hblank DMA transfer.
3. Once the debugger stops on the first breakpoint, place a second breakpoint on `breakpoint_2_lcd`.
4. Press **F5** to open the VRAM viewer and select **BG Map** to see the tilemap.
5. Press **F9** to resume execution in the debugger window. The tilemap should get filled 16 bytes at a time each time you press F9.

The tiles in the tilemap get set 16 at a time, which means that 16 tile indices are transferred per hblank. The whole tilemap is set after 64 hblanks (1,024 indices / 16). For reference, Figure 16-15 shows the state of the background tilemap at various points along the way.

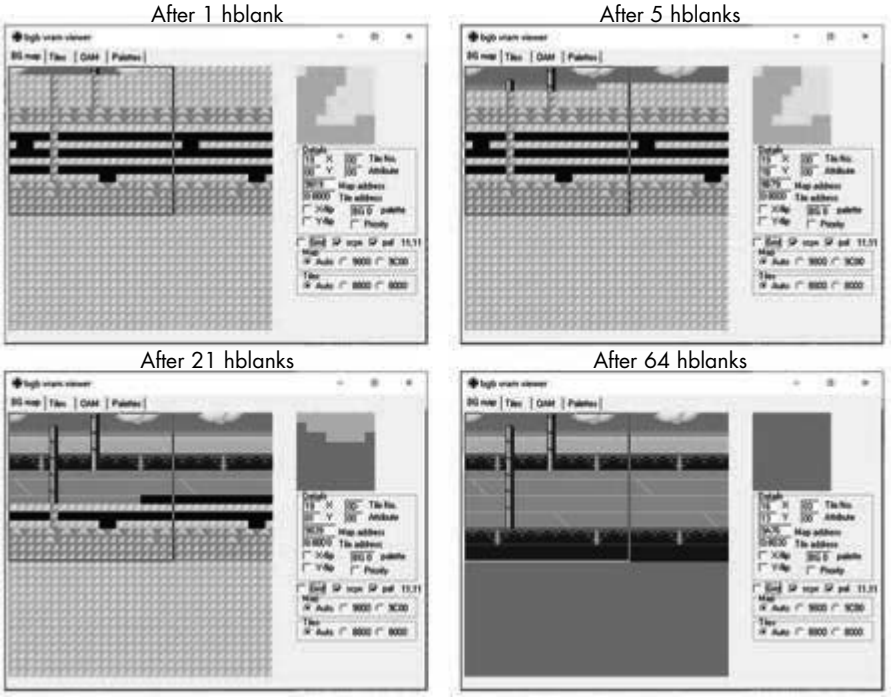


Figure 16-15: Hblank DMA at four different stages of progression

You can see how the tilemap gradually fills as more and more hblanks occur.

This ends our coverage of VRAM DMA. Remember that using this feature is optional; you may not need it for every project, but it's good to know the option is there when you want the extra speed.

CPU Double Speed Mode

While most of the upgrades on the CGB relate to graphics, the CPU also got a significant boost in performance. In this section, you'll learn about *double speed mode*, which allows the CPU to execute instructions twice as fast as in the normal mode. You'll also see what impact this extra speed has on some of features of the CGB.

The CGB CPU has a frequency of up to 8.388608 MHz (2^{23} Hz), twice the speed of the DMG CPU. When you turn on the CGB, its CPU runs at 4.194304 MHz, just like the DMG's. However, it is possible to switch back and forth between this normal mode and double speed mode, where the CPU runs at 8.388608 MHz.

Because the CPU executes instructions faster in double speed mode, you can implement more complex algorithms and generally get more work done inside a single frame. OAM DMA copies, serial transfers, the divider, and the timer also run at double speed. An OAM DMA copy completes in 76 microseconds instead of 152 microseconds. A serial transfer takes 0.488 ms instead of 0.977 ms, for a transfer rate of up to 2,048 bytes per second.

The divider updates twice as fast, so the timer triggers twice as many interrupts. Double speed doesn't affect everything, however. For example, the frame rate is still 59.73, the LCD refresh and the vblank still have the same duration, and, as mentioned previously, VRAM DMA copies take the same amount of time.

The main drawback of double speed mode is that it drains the battery faster. It's good practice to switch back to normal speed mode when you don't need the extra speed, like when displaying menus. Also, switching speed is not instantaneous. It usually takes around one to two frames, so you'll need to control the timing of the CPU speed change to avoid any visually unpleasant frame dropping. Good times to switch CPU speeds include right after a fade out or during a menu transition.

The CGB uses two bits of the `rKEY1` register to control the speed mode: bit 7 and bit 0. Bit 7 is a read-only bit that indicates the current speed mode. When the CGB is in normal speed mode, the bit is cleared. When it's in double speed mode, the bit is set. Bit 0 is used to request a speed mode change. The other bits are unused.

Let's explore how to switch the CPU speed in code using the *cgb_cpu_speed* sample, which toggles the CPU speed each time we press the A button.

Listing 16-15 shows the specific set of instructions you need to change the speed mode.

```
; request a CPU speed change
ld hl, rKEY1
set 0, [hl]

; disable input polling
copy [rP1], P1F_GET_NONE

; save the interrupt flags into (b)
copy b, [rIE]

; disable all interrupts and clear pending interrupts
xor a
ld [rIE], a
ld [rIF], a

; trigger the speed change
stop

; restore the interrupt flags
copy [rIE], b
```

Listing 16-15: Switching CPU speed

First, bit 0 of `rKEY1` is raised, telling the CGB to prepare to switch to a different speed. Next, the code disables input polling, preventing an input interrupt from occurring during the speed change. You can skip this step if

input polling is disabled already (for example, if you're using the technique from Chapter 8).

Then, the code saves the interrupt flags in `rIE` into register `b`, which will be restored at the end of the routine. Next, it resets `rIE` and `rIF` to disable interrupts and clear any pending interrupts. This prevents interrupts from triggering during the speed change.

A `stop` instruction initiates the speed change process. This is not the typical usage of `stop`. On the CGB, this instruction acts differently when bit 0 of `rKEY1` is raised; it was repurposed to trigger a speed change by blocking the CGB for one to two frames and making sure no instructions execute. After the `stop` instruction completes, the CPU runs at the new speed. Finally, the interrupt flags are restored using the value in register `b`.

When bit 0 of `rKEY1` is cleared, the `stop` instruction works just as described in Chapter 2.

The sample's update function has a loop that waits for the vblank. Usually it's better to use `halt` to wait for the vblank, but the sample keeps count of the number of iterations of that loop. It then displays that number in hexadecimal on the LCD to visually show the speed change, as in Figure 16-16.

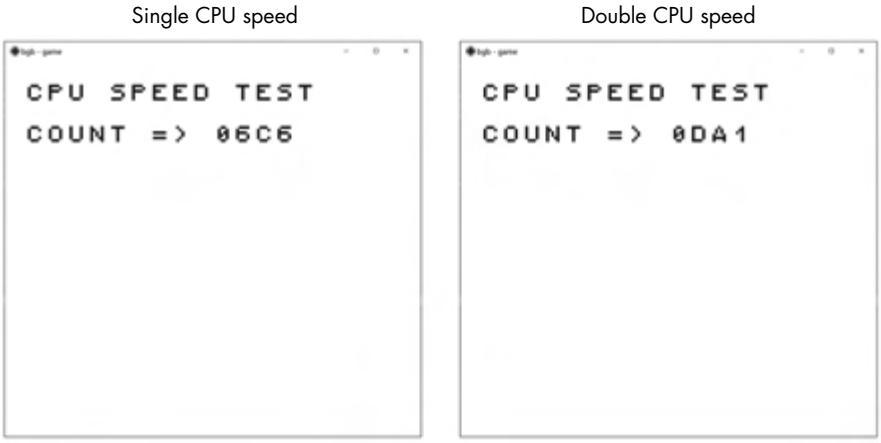


Figure 16-16: Changing the CPU speed on the CGB

In normal speed mode, there are 1,734 iterations (`$6C6`). In double speed mode, there are 3,489 (`$DA1`), roughly twice as many, which is what we expect.

Additional Upgrades

In this section, you'll learn about the remaining upgrades of the CGB: WRAM banks, faster serial transfer rates, and infrared communication. I'll keep the coverage of these features short and without samples, because their usage is either straightforward (in the case of WRAM banks and serial transfer) or impractical (in the case of infrared communication).

WRAM Banks

The CGB has four times more WRAM than the DMG, with 32KiB spread over eight banks of 4KiB each. The WRAM memory map range is split into two spaces of 4KiB each. Space #0 spans addresses from `$C000` to `$CFFF`, and space #1 spans from `$D000` to `$DFFF`. Space #0 always maps to bank 0. You can map space #1 to any bank other than 0 by writing to the `rSVBK` register. Writing a 0 to the register has the same effect as writing a 1 and causes space #1 to map to bank 1.

Only the lower 3 bits of the register are relevant, because the maximum bank number is 7. The other bits are unused and ignored.

NOTE *If you move the stack to WRAM, which you learned how to do in Chapter 15, it's safer to keep it in the address range of space #0 (from `$C000` to `$CFFF`). Because that space cannot map to any bank but bank 0, the stack will always be accessible. If you put the stack in the address range of space #1, the bank containing your stack might get unmapped, so you'll need to make sure no stack operations happen when the bank containing the stack isn't mapped. Otherwise, `push` and `pop` instructions will operate on the wrong data, which will systematically cause bugs.*

Serial Transfer Rates

The CGB also offers options to increase the serial transfer rate, which lets programs transfer significantly more data per frame. While the DMG is stuck with a single transfer clock speed of 8,192 Hz, the CGB has four speeds. First, there's a new flag in bit 1 of the serial control register, `rSC` (see Chapter 11 for a refresher on this register), that influences the transfer rate. When the bit is cleared, the rate does not change. When the bit is set, the rate is multiplied by 32, resulting in a huge leap in transfer rate. In addition, the two normal speed clock frequencies are doubled when the CPU is in double speed mode.

Table 16-6 summarizes all the possible transfer rates and the values of the associated parameters.

Table 16-6: Serial Transfer Rates

CPU speed	`rSC` bit 1	Clock frequency (Hz)	KiB per second	Bytes per frame
Normal	0	8,192	1	17
Double	0	16,384	2	34
Normal	1	262,144	32	548
Double	1	524,288	64	1,097

The last two columns are theoretical maxima, not necessarily realistic targets, as discussed in Chapter 11.

Interestingly, these faster clock speeds are compatible with the DMG. Although the DMG has only a single internal clock speed of 8,192 Hz, during a serial transfer in child mode, it can borrow any external clock from a CGB. As long as the CGB acts as the parent, you can get very fast transfer rates

between that device and a DMG, and you don't have to write specific code using a slower transfer rate.

Infrared

The final addition to the CGB is an infrared (IR) port that enables it to exchange data with another CGB (and, theoretically, other IR devices, such as TV remote controls). The advantage of IR communication is that, unlike serial communication, it does not require a cable.

However, there are several drawbacks to using IR. The IR ports of the two devices need to be physically aligned, or one of the devices might not detect the signal properly. Interference can also occur during the data exchange, making error detection a strong requirement. And, as in serial communication, there are no established data transfer protocols. There is not even a clock assigned to IR transfers, so you'll need to manage transfer timing yourself. To make things even more complicated, IR exchanges data one bit at a time. These disadvantages make IR communication suitable for only simple data exchanges, which is probably why few commercial games (less than 20) used it.

A single register is associated with IR: the *reception port* register, `rRP`. It has 4 bits to control the IR behavior. Bits 6 and 7 set the mode of the IR port to either receive or send. When both bits are 0, the port is in *receive* mode. When both bits are 1, the port is in *send* mode. Other values are meaningless, and you should not use them.

When in receive mode, bit 1 of `rRP` represents the status of the IR signal on the port. This is not intuitive, but when the bit is cleared, it means a signal is being received. Conversely, when the bit is set, no signal is being received.

When in send mode, bit 0 controls whether the IR LED is on or off. When the bit is set, the LED is on, and when it's cleared, the LED is off. You should turn off the LED before switching back to receive mode. Otherwise, the IR receptor might catch its infrared light and mistakenly interpret it as a signal from another device.

Table 16-7 summarizes how the bits of the `rRP` register are used.

Table 16-7: The `rRP` Register

Bits	Access	Purpose
0	Read/write	LED control (0 = off, 1 = on)
1	Read-only	Received signal (0 = signal received, 1 = no signal)
6–7	Read/write	Mode setting (`%00` = receive, `%11` = send)

To avoid unnecessary battery consumption, reset the `rRP` register to zero once the program is done using IR.

BGB does not emulate the IR functionality, so you'll need to run IR programs on real hardware. This can be very inconvenient when developing, especially since communication over IR can be tricky to get right. Serial communication is a better way to communicate reliably with another DMG or CGB.

Compatibility Mode

All the samples we've explored so far in this chapter were CGB-only programs, which do not run on a DMG. This is due to the `--color-only` parameter of the `rgbfix` command, which sets the CGB flag in the cartridge header at address `$0143` to `$C0`. Table 16-8 shows what the possible values of this flag indicate.

Table 16-8: CGB Flag Values

CGB flag	Mode	`rgbfix` parameter	DMG compatible?	CGB compatible?
`$80`	DMG/CGB	`--color-compatible`	Yes	Yes
`$C0`	CGB	`--color-only`	No	Yes
Any other	DMG	—	Yes	Yes (DMG mode)

This section focuses on *compatibility mode*, which is when the CGB flag is set to `$80`. In this mode, a ROM runs on the DMG just like any DMG ROM would, but when it's run on a CGB, the ROM has full access to all of the CGB's additional features. In other words, this mode allows you to create a ROM that takes full advantage of the CGB but still runs on a DMG.

To leverage compatibility mode, you need to learn how to detect the hardware model and how to test the model-dependent behaviors in BGB. Let's take a look at what's involved in each of those steps. Then, we'll explore a sample that detects the type of hardware model it's running on.

Detecting the Hardware Model

In compatibility mode, the values of the `a` and `b` registers at the very start of the program indicate the hardware model the ROM is running on. Figure 16-17 illustrates the process for determining this.

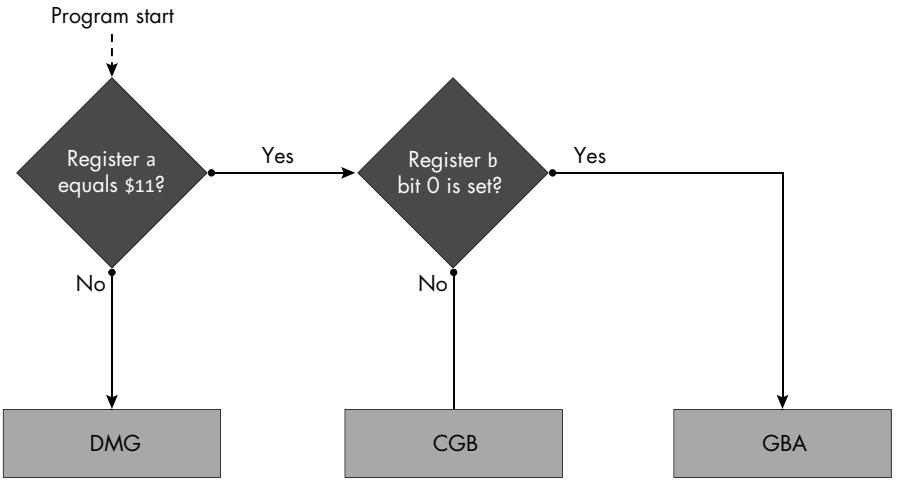


Figure 16-17: Hardware model detection

If the value of the `a` register is *not* `$11` at the start of the program, then the program is running on a DMG. Otherwise, we need to check bit 0 of the `b` register. If the bit is clear, then the program is running on a CGB. Otherwise, it is running on a Game Boy Advance (GBA). The GBA's hardware is very different from the DMG's and CGB's, and I don't cover it in this book. However, it can play DMG and CGB ROMs.

One issue that may arise if you attempt this is that palette colors may appear considerably darker on the GBA. Detecting the GBA hardware gives the ROM the opportunity to use modified palette colors that look better on a GBA. Assuming 5-bit BGR color channels (values from 0 to 31), the following formula shows a good systematic channel correction that will brighten the colors and improve their visibility on the GBA LCD:

```
gba_color = cgb_color * 3 / 4 + 8
```

For example, assuming the original red channel equals 24, this would correct it to 26 (24 * 3 / 4 + 8) on the GBA. This correction must be applied to each channel (blue, green, and red) separately.

Testing in BGB

BGB can emulate both the DMG and the CGB. By default, it chooses the most appropriate hardware to run a ROM, but you can force it to use a specific model. This comes in handy for testing ROMs that are compatible with both devices, because BGB usually automatically switches to CGB mode for such ROMs.

To force BGB to run in DMG mode, you must set the `SystemMode` parameter to `0`, as in the following command:

```
bgb -set SystemMode=0 rom.gbc
```

To force CGB mode, set the `SystemMode` to `1` instead. You can also change the emulated system through the System tab of BGB's Options menu, as shown in Figure 16-18.

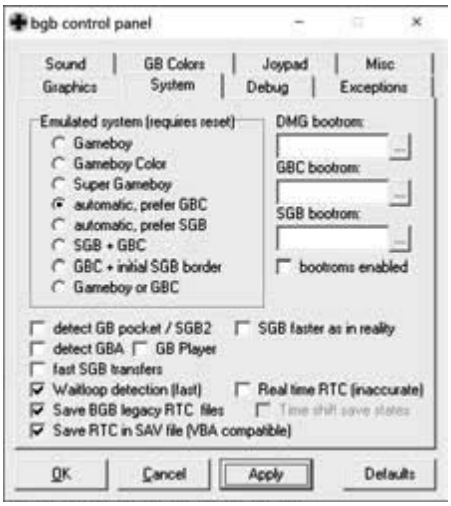


Figure 16-18: Selecting the system to emulate in BGB

Set the **Emulated System** to **Gameboy** to force DMG mode or to **Gameboy Color** to force CGB mode. Unfortunately, there is no way to force GBA mode, because the emulator does not support that hardware.

Running on Different Hardware

In this section, we'll use the *cgb_compatibility* sample to show how to detect the hardware model the program is running on and run code specific to the detected hardware. This sample has customized initialization functions for each hardware model (DMG, CGB, and GBA). The relevant function is called based on the information available at the program's start.

First, the sample figures out the hardware model using the macro in Listing 16-16.

```
macro DetermineHardwareModel
    cp a, $11
    jr nz, .cgb\@
        bit 0, b
        jr z, .gba\@
            ld a, HARDWARE_MODEL_GBA
            jr .done\@
        .gba\@
        ld a, HARDWARE_MODEL_CGB
        jr .done\@
    .cgb\@
```

```
        ld a, HARDWARE_MODEL_DMG
    .done\@
    ; save the hardware model
    ld [HARDWARE_MODEL_REGISTER], a
endm
```

Listing 16-16: Determining the hardware model

The sample uses this macro right at the start of the main function, when the `a` and `b` registers still hold the information about the hardware model. Any other operation might change the values of those registers, causing this information to be lost. The macro checks the registers to determine the model in use, following the steps in Figure 16-17. It then copies the result to a memory location so that it can be referenced later in the program. We can then use the `a` and `b` registers safely for any other purpose.

Listing 16-17 shows the code to define the constants used in the macro.

```
def HARDWARE_MODEL_REGISTER     equ (_HRAM)

def HARDWARE_MODEL_DMG          equ (1)
def HARDWARE_MODEL_CGB          equ (2)
def HARDWARE_MODEL_GBA          equ (3)
```

Listing 16-17: Defining the hardware model constants

The values of these constants facilitate the process of calling the right initialization function, which I'll explain next. A `HARDWARE_MODEL_REGISTER` constant is also defined to specify the memory location holding the hardware model. The first location in HRAM is used here, but that choice is arbitrary; any location in HRAM or WRAM would work.

Listing 16-18 shows the code that calls the relevant initialization function.

```
InitSample:
    ld a, [HARDWARE_MODEL_REGISTER]
    dec a
    jp z, InitSample_DMG
    dec a
    jp z, InitSample_CGB
    dec a
    jp z, InitSample_GBA
```

Listing 16-18: Hardware-dependent initialization

This function retrieves the hardware constant stored in HRAM. Then, it decrements that value until it reaches zero and jumps to the corresponding initialization function. The code works as expected thanks to the values used for the constants defined in Listing 16-17.

Note that the code *jumps* to the right initialization function instead of *calling* it. This is a small optimization technique that saves a few bytes in the ROM and several cycles when running the program. When the chosen

initialization function returns, the code will go back into the main function from which the `InitSample` function was originally called.

NOTE *While it's not possible to emulate the GBA in BGB, you can edit the code to force the GBA initialization function to be run. For the most part, that function is the same as the CGB initialization function. The only difference is that it applies the palette color correction described in "Detecting the Hardware Model" on page 300.*

Figure 16-19 shows screenshots of the same sample running in two different system modes.

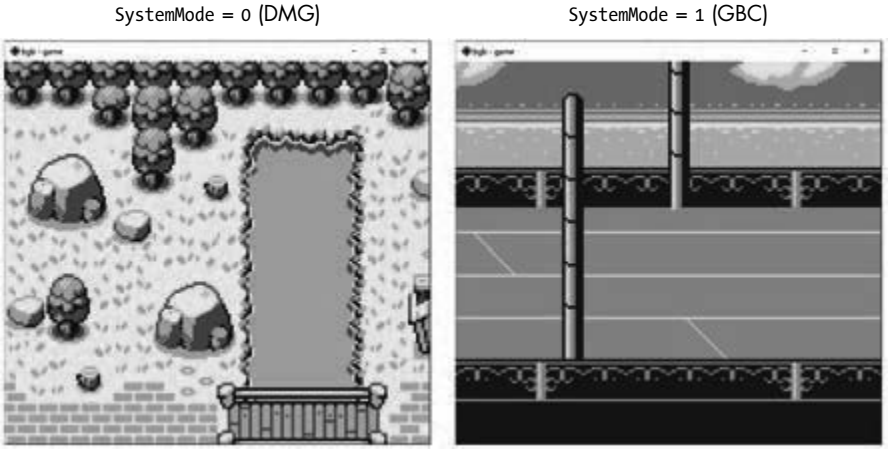


Figure 16-19: The compatibility sample running in DMG mode and CGB mode

A program can behave very differently on different hardware models. It's up to you, as the programmer, to decide how to vary the behavior to suit each device type.

Summary

This chapter introduced all the new graphics features of the CGB, including the colored palettes, the extra VRAM bank, the new background and sprite parameters, and the fast data copy unlocked by VRAM DMA.

You learned how to enable the CPU's double speed mode to make your programs faster, and you learned about additional CGB upgrades such as WRAM banks, fast serial transfer rates, and infrared capabilities. Finally, you explored how to create programs that behave differently on the DMG and CGB so that they can take full advantage of both hardware models.

In the next chapter, we'll take a look at the Game Boy Printer, an accessory that lets programs print graphics on thermal paper.

17

GAME BOY PRINTER

The *Game Boy Printer*, also known as the *Pocket Printer*, is a small portable printer released in 1998. It prints images in four shades of gray on thermal paper. It's compatible with both the DMG and the CGB and leverages serial communication to exchange data.

This chapter introduces the hardware and the protocol used to communicate with the printer. It assumes you're already familiar with serial communication, covered in Chapter 11.

We'll begin with a brief look at the hardware and software you'll need to test your printer programs. Then you'll learn about the communication protocol used to control the printer from your programs. This requires working with several types of packets and printer flags and following a well-defined sequence for sending data.

Finally, we'll go through a sample that demonstrates how the printer prints graphics displayed on the screen. The sample shows how to define the packets to be sent to the printer, implement the protocol over the serial communication interface, and check for errors during execution.

Testing Printer Programs

BGB does not emulate the Game Boy Printer. To test programs that use this peripheral, you need to use real hardware or simulate the printer with a program. You'll learn about both options in this section.

Using Real Hardware

To run programs that use the printer on real hardware, you'll need a physical Game Boy Printer, as shown in Figure 17-1, and a DMG (or CGB).



Figure 17-1: The Game Boy Printer

You'll also need a serial cable to link the printer to the device running the program. The printer features a small port connection; refer to Table 11-1 to make sure you have the right cable and/or adapter.

The printer prints on *thermal paper*, using heat to darken chemicals in the paper instead of using ink. Nintendo used to sell special paper with adhesive backing so that printed images could be used as stickers, but that paper has long been discontinued and finding usable rolls today is almost impossible. Thankfully, other companies still make thermal paper that is compatible with the Game Boy Printer. As long as the paper width is 38 mm (1.5 inches), you should be able to use it with the printer. I used Seiko's S-951 38 mm thermal paper rolls when writing this book. A new paper roll is too large for the printer cover to close, but that shouldn't affect the printing itself, and once enough paper has been consumed the roll diameter will shrink enough to allow you to close the cover.

NOTE *It can be difficult to insert a new roll of paper into the printer. Cutting the end of the paper roll in the shape of a triangle can help it go through the roller more easily.*

One big drawback of using real hardware is the lack of a debugger. To make development smoother, it's probably best to first develop using a printer simulator, covered next, and then check the behavior on real hardware.

Simulating the Printer

From a technical perspective, there is no difference between communicating with a printer and communicating with another DMG, so it's possible to create a ROM that stands in for the printer. The advantage of simulating the printer is that you can test the print functionality by linking the BGB instance running your program with a BGB instance running the printer simulator ROM (see Chapter 11 for a refresher on this). This method is not fully accurate, but it allows you to get 90 percent of the work done before testing the program on real hardware, which is way more time-consuming.

Explaining how to create a printer simulator is outside the scope of this book, but one is available along with its full source code at *https://github.com/mdagois/gca/tree/main/tools/printer*. See the *README* file in that folder for more information.

Serial Protocol

To communicate successfully with the Game Boy Printer, a program must follow a well-defined serial protocol, which we'll cover in this section. During serial communication, the DMG running the program assumes the role of the parent, while the printer becomes the child. The program is responsible for properly driving the data transfer and getting back the printer status.

The data is organized in packets, with each packet consisting of several bytes. There are five types of packets (init, print, data, break, and status) that all share the same generic structure shown in Table 17-1.

Table 17-1: Generic Packet Structure

Field	Size (bytes)	Bytes sent from DMG	Printer response
Magic number	2	$88, $33	$00
Packet type	1	$01 = init, $02 = print, $04 = data, $08 = break, $0F = status	$00
Compression	1	0 = no compression, 1 = compression	$00
Payload size	2	Number of payload bytes	$00
Payload	0–640	Any value	$00
Checksum	2	Sum of all bytes from the command type to the last byte of the payload	$00
Ack	1	$00	$81
Status	1	$00	Printer flags

The *magic number* consists of 2 bytes with fixed values that mark the start of every packet. This helps the printer detect malformed packets and invalid packet boundaries.

The *packet type* has five possible values: init (`$01`), print (`$02`), data (`$04`), break (`$08`), and status (`$0F`). The content of a packet depends on its type, which I'll go over in more detail in the next section.

The *compression* field (CF) is a flag that indicates whether the data is compressed or not. This is relevant only for data packets; for other packets, it should be set to zero.

The *payload size* is the number of bytes contained in the payload and is relevant only for print and data packets. For other packets, there is no payload and thus the size is always zero.

The *payload* is a bunch of bytes whose meaning depends on the packet type. Only print and data packets have a payload.

The *checksum* is a 2-byte value that is the sum of all bytes from the packet type to the last byte of the payload. This sum doesn't include the magic number or any of the fields after the payload. The program must set the checksum properly. The printer then computes the checksum on its side when receiving data. When there is a mismatch between the checksum in the packet and the checksum the printer computed, the printer considers the packet invalid and raises an error. When the checksums match, it considers the packet valid.

All the fields mentioned so far are bytes of data flowing from the DMG running the program to the printer, meant to give information to the printer. The final two fields, *ack* and *status*, are different; they represent the printer's response to the packet sent by the DMG. The DMG sends `$00` for both fields. The response from the printer to the ack field is always `$81`. This is the printer identifier, which acts as an acknowledgment from the printer. The response to the status field consists of the printer flags, which I'll explain a bit later.

It's important to understand that packets are conceptual constructs. When it comes to programming, all the bytes in the packets are sent one by one using serial communication (see Chapter 11). There are two key facts to keep in mind when sending those bytes. First, the printer has a timeout of about 100 ms, meaning that you must send bytes within 100 ms of each other, or else the printer assumes that the connection has been lost and resets its internal memory. This timeout applies for bytes sent in a single packet, as well as between packets. Second, you must not send bytes too fast. The printer needs some time to process the bytes it receives from the DMG, so you must introduce a delay of around 300 microseconds between each byte transfer.

Packet Types

In this section you'll learn about the five packet types that the printer supports: init (`$01`), print (`$02`), data (`$04`), break (`$08`), and status (`$0F`). You need to understand the purpose and composition of each type of packet to control the printer from your programs.

Init

An *init* packet instructs the printer to reset its status flags and clear its internal memory. This packet has no payload, and it always contains exactly 10 bytes with fixed values, as shown in Figure 17-2.

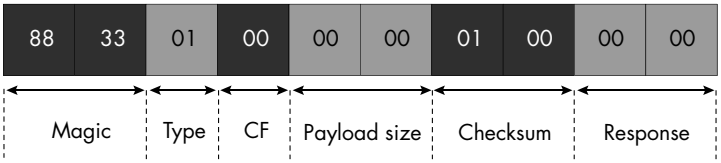


Figure 17-2: Init packet structure

The status received from the printer (the last byte of the response) should be $00.

Print

A *print* packet instructs the printer to print the data stored in its internal memory. The packet always has a payload of 4 bytes, for a total packet size of 14 bytes, as shown in Figure 17-3.

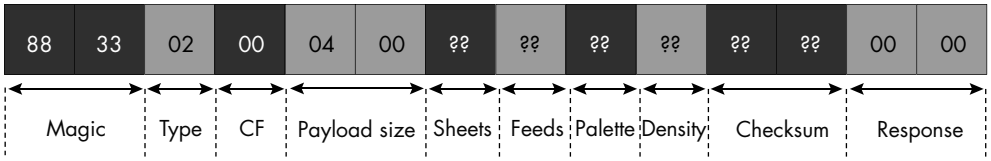


Figure 17-3: Print packet structure

The first byte of the payload is the number of sheets to print. A *sheet* consists of all the data in the printer memory. Setting this to a value higher than $01 is akin to printing the same page several times.

The second byte is the line feed control. A *feed* is a blank space of about 2.64 mm (one-tenth of an inch). The top nibble is the number of feeds before the image is printed (pre-printing feeds), and the low nibble is the number of feeds after (post-printing feeds). For example, if the byte equals $23, then there are two pre-printing feeds and three post-printing feeds. Line feeds keep images printing consecutively apart from each other. They also avoid users accidentally damaging the image when ripping the printed paper off the roll. I recommend a value of $14 for single-sheet prints.

The third byte is the palette used for printing, which works exactly like the palettes on the DMG. See Chapter 5 for a refresher.

The fourth and final byte is the *intensity*, which determines how dark the colors will print. Lower values mean lighter colors, and higher values mean darker colors. A value of $40 is a good start for experimenting. Values larger than $80 have the same effect as $80.

Data

A *data* packet instructs the printer to store image data into its internal memory. The data in this memory is organized just like regular VRAM tiles (see Chapter 5). There is no concept of tilemaps for the printer, so all the data is raw tile data. The printer's internal memory is 5,760 bytes, which is large enough to hold 360 tiles (5,760 / 16). That's 18 rows of 20 tiles, which is equivalent to a 160×144 image (the resolution of the DMG LCD).

Each data packet always contains exactly two rows of tiles, which is 640 bytes of data (20 tiles × 2 rows × 16 bytes). To fill the entire internal memory of the printer, you'll need nine data packets. Figure 17-4 shows the structure of a data packet.

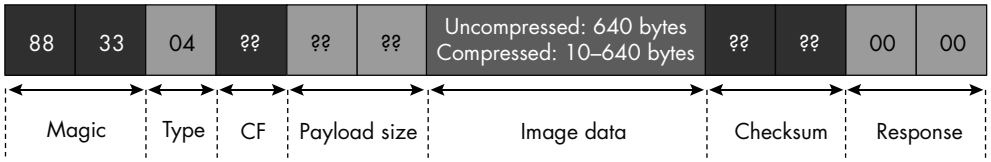


Figure 17-4: Data packet structure

A data packet may have a size of zero. In that case, the packet acts as a *sentinel* value to mark the end of the image data before printing.

You can compress the payload of a data packet to shorten the time it takes to send the data to the printer. Compressed data also takes less space when stored in the ROM. The algorithm for compressing the payload splits the data into raw blocks and repeat blocks.

A *raw block* consists of a control byte followed by one or more data bytes. The control byte's bit 7 is always zero, which identifies the block as raw. The remaining bits of the control byte (bits 0 to 6) specify the number of data bytes in the block minus one. That means the minimum number of data bytes is 1 and the maximum is 128. The data bytes that follow the control byte are raw data, which is meant to be copied as is into the printer's internal memory. You can think of a raw block as a bunch of data that you cannot compress. Raw blocks actually make compressed data bigger by 1 byte (the control byte) compared to uncompressed data.

A *repeat block* consists of a control byte followed by a single data byte, making it always 2 bytes long. The control byte's bit 7 is always one, which identifies the block as a repeat block. The remaining bits of the control byte specify the number of times the data byte must be repeated, minus two. In this case, the minimum number of repeated bytes is 2 and the maximum is 129. Repeat blocks are the only compression mechanism used in compressed data. To achieve a high compression ratio (that is, produce a smaller payload that is faster to send), there must be as many repeat blocks as possible in the compressed data.

Let's look at a few examples to get a better idea of what the two kinds of blocks look like. Suppose you have a block like the following in the compressed payload of a data packet:

```
07 30 A4 67 8B CD F9 14 0D
```

The control byte is `$07`, so it's a raw block (bit 7 is zero) of size 8 (7 + 1). Once uncompressed, the data would be just the 8 bytes that follow the control byte, as shown here:

```
30 A4 67 8B CD F9 14 0D
```

Now, consider this block:

```
88 CD
```

The control byte is `$88`. This time, it's a repeat block (bit 7 is one) of size 10 (8 + 2). The uncompressed data would be 10 bytes long, as shown here:

```
CD CD CD CD CD CD CD CD CD CD
```

Raw blocks and repeat blocks can be interleaved in the compressed payload of a data packet. The uncompressed data size of such a packet must always be 640 bytes. If there are too many bytes after decompression, the printer discards the extra bytes. If there are not enough bytes, it raises an error.

Unfortunately, due to the tile data organization, repeat blocks are not very common in compressed data. You can turn bytes into a repeat block only when all 8 pixels of a tile row have the same color. Additionally, those pixels must have color index 0 or 3. The compression algorithm cannot compress pixels of color index 1 or 2 because of the way the color index bits for a row are split over 2 bytes (as explained in "Tiles" on page 74. Using color index 1 or 2 for a pixel automatically indicates nonrepeating bytes, meaning repeat blocks cannot be used.

In general, it's quite difficult to achieve good compression ratios with the available algorithm. Most games use uncompressed data, with the *Pokémon Trading Card Game* being the most notable (and maybe only) exception. This game contains a lot of printable cards, so compression most likely saved a nonnegligible amount of ROM space.

Break

A *break* packet instructs the printer to cancel any ongoing printing. The user of the program typically triggers this operation. Figure 17-5 shows the content of the break packet.

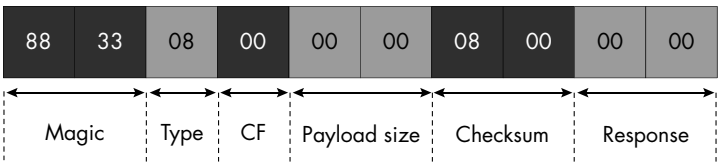


Figure 17-5: Break packet structure

This packet has no payload and always contains exactly 10 bytes with fixed values.

Status

A *status* packet, also known as a *NUL* packet, checks whether the printer is printing or an error has occurred. Its structure is shown in Figure 17-6.

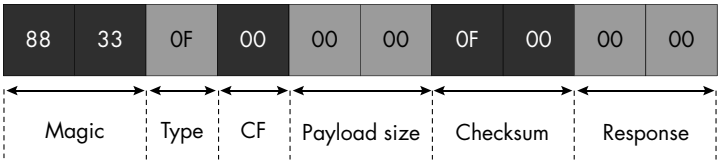


Figure 17-6: Status packet structure

A status packet has no payload and always contains exactly 10 bytes with fixed values. We use this packet to get the printer flags, covered next; it has no effect per se.

Printer Flags

The *printer flags* are bits containing information about the status of the printer. There are eight flags that report three statuses and five errors, listed in Table 17-2. Sending packets causes the printer flags to change, and you'll need to use a status packet to check the flags.

Table 17-2: Printer Flags

Bit	Name	Type	Description
0	Checksum error	Error	A checksum mismatch occurred.
1	Printing	Status	The printer is printing.
2	Memory full	Status	The printer's internal memory (5,760 bytes) is full.
3	Ready to print	Status	The printer has received some data.
4	Packet error	Error	Something is wrong in the packet data.
5	Paper jam	Error	Paper is stuck in the printer.
6	Other error	Error	An error not covered by the other flags has occurred.
7	Low battery	Error	The printer's batteries need to be changed.

Commercial games use specific *error codes* to report errors. These are listed in Table 17-3.

Table 17-3: Error Codes

Code	Trigger	Description
01	Status bit 7 is raised.	The printer's batteries need to be changed.
02	Both ack and status equal $FF.	The printer is not connected properly.
03	Status bit 5 is raised.	Paper is stuck in the printer.
04	Status bit 6 is raised.	An error not covered by the other codes has occurred.

The error codes are defined in the Game Boy Printer's manual, and users can refer to that manual to understand any issues games report. This is convenient for game developers too, because it means they don't have to include complex error messages in the ROM. For example, Figure 17-7 is the error screen on the Game Boy Camera when the printer's battery is low.



Figure 17-7: Printer error #01

The error screen reports only the error number #01. There is no need to mention that the battery is low, because that's what the error number implies.

There are no dedicated error codes for checksum and packet errors, because these are programming errors that should not occur in released games. However, in any program, when you detect these kinds of errors, you should report them with code #04.

Print Sequence

Now that you understand the basics of the communication protocol, let's go over how to use a series of packets to control the printer. The following steps are necessary to get the printer to print an image:

1. The program sends an init packet to reset the hardware. The printer status becomes `$00`, and it clears its internal memory. I recommend doing an early error check at this point, at least to make sure a printer is actually connected.
2. The program sends up to nine data packets to fill the printer's internal memory with tile data. After receiving the first packet, the printer raises its *ready to print* flag. After receiving nine packets, it raises its *memory full* flag. It is valid to not send any data packets, such as when outputting line feeds only.
3. The program sends an empty data packet (the sentinel) to mark the end of the tile data. This packet, also called the *data end* packet, must have a payload size of zero. It is mandatory, even if no data packets were sent before it.
4. The program sends a print packet to start the printing process. The printer raises its *printing* flag.
5. The program sends status packets until the *printing* flag is cleared. Once the *printing* flag clears, the printing is done. Perform error checks for each status packet to make sure no issues (like a low battery or a paper jam) interrupted the printing process.

Using this sequence, you can print an image of up to 160×144 pixels. To print more images, repeat the sequence as many times as necessary. Specifying line feeds with values of `$00` in the print packets allows you to print a continuous image of a height greater than 144 pixels. I recommend setting a post-printing line feed of at least four lines in the last print packet, to make sure you can tear the paper away from the printer without damaging the printed image.

Sending an Image to the Printer

In this section we'll explore the *printer* sample in the GitHub repository that sends an image to the printer, following the sequence of packets explained in the previous section.

This sample sends the image shown in Figure 17-8.



Figure 17-8: Print data

The image is 160×144 pixels, so it takes nine data packets.

Figure 17-9 shows a screenshot of the sample alongside the printer simulator.

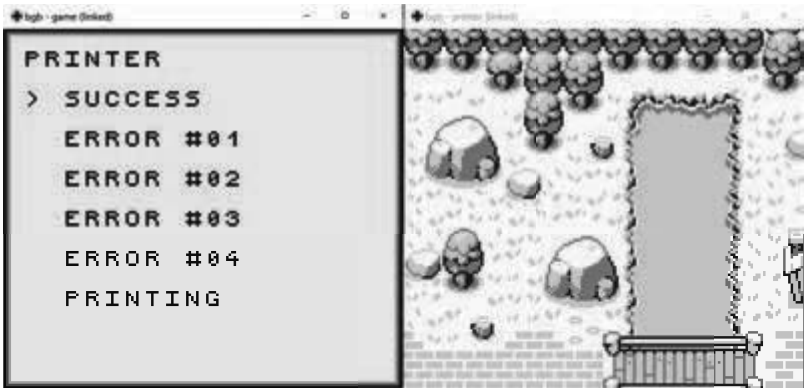


Figure 17-9: The printer simulator (left) and the printer sample (right)

Pressing the A button starts the printing sequence, and the program displays the result of the operation at the end. The result will be *success* if the printing process completed successfully, or one of the four possible error codes if an unexpected event occurred.

Figure 17-10 is a physical printout of the image, produced using a real DMG and a Game Boy Printer.

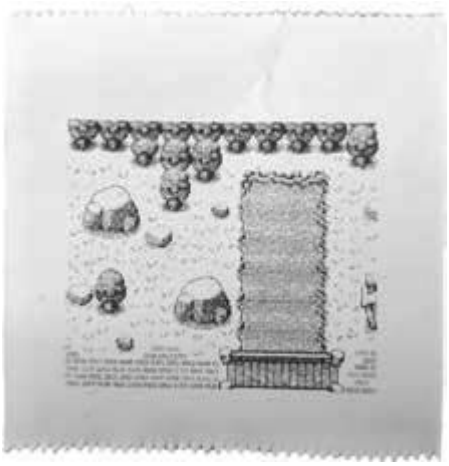


Figure 17-10: A physical printout

The thermal printer creates the image line by line, using a series of tiny dots. A bit of texture builds up in regions of the screen with plain colors, like the water area above the bridge. Sending an image to the printer involves four steps: defining the packets, monitoring the printer state, transferring the packets, and checking for errors. Let's take a look at how each of these are handled in the *printer* sample.

Packet Definition

The first thing we need to do to send an image to the printer is define the packets we'll send. To keep things simple, all packets are embedded in the ROM. The init, data sentinel, and status packets differ only by their command type (and implicitly their checksum), so the `FixedPacket` macro defines all of them, as shown in Listing 17-1.

```
macro FixedPacket
db PKT_MAGIC_0, PKT_MAGIC_1, \1, PKT_COMPRESSION_OFF
db $00, $00, \1, $00, $00, $00
endm

packet_init:
    FixedPacket PKT_TYPE_INIT
packet_init_end:

packet_data_sentinel:
    FixedPacket PKT_TYPE_DATA
packet_data_sentinel_end:

packet_status:
    FixedPacket PKT_TYPE_STATUS
packet_status_end:
```

Listing 17-1: Defining the fixed packets

The print packet has a payload with parameters such as the line feeds and palette. To keep things simple, the sample sets the parameters to hard-coded values, as shown in Listing 17-2.

```
def PKT_PRINT_PAYLOAD_SIZE equ ($0004)
def PKT_PRINT_SHEET_COUNT  equ ($01)
def PKT_PRINT_FEEDS        equ ($14)
def PKT_PRINT_PALETTE      equ ($1B)
def PKT_PRINT_DENSITY      equ ($40)
def PKT_PRINT_CHECKSUM     equ (PKT_TYPE_PRINT + PKT_PRINT_PAYLOAD_SIZE + \
                               PKT_PRINT_SHEET_COUNT + PKT_PRINT_FEEDS + \
                               PKT_PRINT_PALETTE + PKT_PRINT_DENSITY)

packet_print:
db PKT_MAGIC_0, PKT_MAGIC_1
db PKT_TYPE_PRINT, PKT_COMPRESSION_OFF
dw PKT_PRINT_PAYLOAD_SIZE
db PKT_PRINT_SHEET_COUNT
db PKT_PRINT_FEEDS
db PKT_PRINT_PALETTE
db PKT_PRINT_DENSITY
dw PKT_PRINT_CHECKSUM
db $00, $00
packet_print_end:
```

Listing 17-2: Defining the print packet

The data packets are handled differently, because their payload depends on the image sent to the printer. For the sample, we first convert the image from Figure 17-8 to tiles (using the cgbconv tool, introduced in the prevous chapter), then turn it into a series of nine uncompressed data packets using a custom tool called *packetize*. This tool takes a CHR file and generates a PKT file from it. A PKT file is a list of data packets that contain the CHR file's tile data in their payloads. The full source code of packetize is available in the GitHub repository at *https://github.com/mdagois/gca/blob/main/tools/conv/packetize.cpp*.

In the sample, we use an `incbin` directive to import the PKT file, as shown in Listing 17-3.

```
packet_data:
incbin "print_data.pkt"
packet_data_end:
```

Listing 17-3: Importing the data packets

The `incbin` directive is surrounded by two labels referenced later in the code.

Now that we've imported the data packets into the ROM, let's see how they are sent to the printer.

State Tracking

The sample maintains some state to drive the transfer of the packets and their bytes. Listing 17-4 shows this state, which consists of the address of the current state function, some data pointers, the packet responses, and the printing result.

```
rsreset
def PRINT_STATE_FUNCTION_ADDRESS        rb 2

def PRINT_STATE_DATA_POINTER_NEXT_BYTE  rb 2
def PRINT_STATE_DATA_POINTER_LAST_BYTE  rb 2

def PRINT_STATE_RESPONSE_ACK            rb 1
def PRINT_STATE_RESPONSE_STATUS         rb 1

def PRINT_STATE_TRANSFER_RESULT         rb 1

def sizeof_PRINT_STATE                  rb 0
```

Listing 17-4: Tracking the state

The state function determines the current behavior of the program relative to transferring data to the printer. Each packet has an associated function. There are also functions to check for errors, which we'll discuss shortly.

The data pointers keep track of the *next byte* and *last byte* to send in the current packet. The next byte pointer is incremented each time a byte is sent. Once that pointer equals the last byte pointer, the packet has been fully sent.

The packet responses, consisting of the ack and status received from the printer, are used during error checking and result determination. They are only valid and relevant after a full packet has been sent and before another packet transfer starts.

The transfer result is set at the end of a transfer, based on the responses from the printer. It specifies whether the printing operation succeeded or not. During transfer, it has a value that indicates the printing operation is in progress.

Packet Transfer

With the packet definitions in the ROM and the print state in WRAM, the sample is ready to start executing the printing job. The sample code follows the sequence explained in "Print Sequence" on page 314. The flow of the program relies heavily on timer and serial interrupts, discussed in Chapters 10 and 11, respectively.

A timer is first set to trigger after roughly 300 microseconds (the desired inter-byte delay). After the timer interrupt occurs, the timer stops and the serial transfer of a byte is initiated by the interrupt function. We use the

serial interrupt to wait for the transfer to complete, which takes close to 1 ms. Inside the serial interrupt function, we start the timer again to schedule the next byte transfer 300 microseconds later. The timer interrupt then triggers again, and the cycle goes on until there are no more bytes to transfer.

Listing 17-5 shows the code that starts the printing sequence.

```
    copy [WRAM_PRINT_STATE + PRINT_STATE_TRANSFER_RESULT], \
        RESULT_IN_PROGRESS

    ; start the packet transfer sequence
    PrepareNextPacket SendPacketByte_Init, \
                      packet_init, packet_init_end
    StartTimer
```

Listing 17-5: Starting the printing sequence

The result is updated to indicate that the printing operation is in progress. The first packet is set to init, and the timer starts.

The timer is set up via a macro shown in Listing 17-6.

```
macro StartTimer
    ld a, 256 - 5
    ld [rTMA], a
    ld [rTIMA], a
    ld [rDIV], a
    copy [rTAC], TACF_START | TACF_16KHZ
endm
```

Listing 17-6: Starting the timer

The parameters are set according to the procedure explained in Chapter 10. The frequency is 16,384 Hz, and the timer module is 251. This corresponds to roughly 3,276 interrupts per second (16,384 / (256 – 251)), leading to an interrupt every 305 microseconds or so. This is a good setup because the interrupt triggers at a pace slightly above the desired inter-byte delay of 300 microseconds. A pace of less than 300 microseconds would not be safe, because the printer might miss some of the bytes the program sends to it.

Listing 17-7 shows the timer interrupt code.

```
TimerInterrupt:
    PushAllRegisters

    StopTimer

    load16 hl, WRAM_PRINT_STATE + PRINT_STATE_FUNCTION_ADDRESS
    CallHL

    PopAllRegisters
    reti
```

Listing 17-7: The timer interrupt

This code stops the timer, because only one trigger at a time is necessary. Then, it calls the state function that performs the necessary tasks for transferring data to the printer.

Listing 17-8 is an example of a state function.

```
SendPacketByte_Print:
    SendPacketByte_Common
    PrepareNextPacket SendPacketByte_Status, packet_status, packet_status_end
    ret
```

Listing 17-8: The print packet state function

This function handles the print packet. Other packets' state functions have a very similar structure.

The `SendPacketByte_Common` macro, shown in Listing 17-9, handles instructions common to all packet state functions.

```
macro SendPacketByte_Common
    ; send a byte
    load16 hl, WRAM_PRINT_STATE + PRINT_STATE_DATA_POINTER_NEXT_BYTE
    copy [rSB], [hl]
    copy [rSC], SCF_START | SCF_SOURCE

    ; update the data pointer to point to the next byte to send
    inc hl
    store16 WRAM_PRINT_STATE + PRINT_STATE_DATA_POINTER_NEXT_BYTE, hl

    ; check whether all the data for the packet has been sent
    ; and return if that is not the case
    load16 de, WRAM_PRINT_STATE + PRINT_STATE_DATA_POINTER_LAST_BYTE
    ld a, e
    cp a, l
    ret nz
    ld a, h
    cp a, d
    ret nz

    ; all the packet data has been sent
endm
```

Listing 17-9: Sending a byte

This macro sends a byte and increments the data pointer pointing to the next byte to send. It also checks whether the packet has been fully sent, in which case it does not return early and the state function executes the instructions of `PrepareNextPacket`, shown in Listing 17-10.

```
macro PrepareNextPacket
    ld hl, \1
    store16 WRAM_PRINT_STATE + PRINT_STATE_FUNCTION_ADDRESS, hl
    ld hl, \2
    store16 WRAM_PRINT_STATE + PRINT_STATE_DATA_POINTER_NEXT_BYTE, hl
    ld hl, \3
    store16 WRAM_PRINT_STATE + PRINT_STATE_DATA_POINTER_LAST_BYTE, hl
endm
```

Listing 17-10: Preparing the next packet

The `PrepareNextPacket` macro specifies the next packet to send, setting the state function address and the data pointers. The data pointers are the addresses of the start and the end of the next packet to send.

The final piece of the system is the serial interrupt, whose code is shown in Listing 17-11.

```
SerialInterrupt:
    PushAllRegisters

    copy [WRAM_PRINT_STATE + PRINT_STATE_RESPONSE_ACK], \
         [WRAM_PRINT_STATE + PRINT_STATE_RESPONSE_STATUS]
    copy [WRAM_PRINT_STATE + PRINT_STATE_RESPONSE_STATUS], [rSB]

    StartTimer

    PopAllRegisters
    reti
```

Listing 17-11: The serial interrupt

This saves the last byte received into `WRAM_PRINT_STATE + PRINT_STATE_RESPONSE_STATUS` and the second-to-last byte into `WRAM_PRINT_STATE + PRINT_STATE_RESPONSE_ACK`. The values at these memory locations are relevant only between packets, at which point they contain the ack and status bytes of the packet that was just sent. The rest of the time, you should ignore their values and consider them garbage. The reason for this design choice is that detecting whether a packet completes in the serial interrupt is complicated, because the serial interrupt only cares about single bytes, not packets. To ease the implementation of the interrupt, the code just saves the bytes each time.

At the end of the interrupt function, the timer restarts so that the code sends a new byte after 300 microseconds. After all the bytes of a packet are sent, the code will send the next packet. After the print packet is sent, the printer will start printing the image, and the program will keep sending status packets until the print operation finishes.

Error Check

There are a couple of state functions intended to check for errors. They both use the same macro; one is called right after the init packet has been sent and the other after every status packet. The first function is shown in Listing 17-12.

```
SendPacketByte_CheckConnection:
    CheckForErrors

    ; no error
    PrepareNextPacket SendPacketByte_Data, packet_data, packet_data_end
    StartTimer
    ret
```

Listing 17-12: Connection check

These functions do not send any bytes, so they need to restart the timer themselves (as no serial interrupt occurs).

The `CheckForErrors` macro performs the actual error checks, shown in Listing 17-13.

```
macro CheckForErrors
    .check_not_connected
        ld a, [WRAM_PRINT_STATE + PRINT_STATE_RESPONSE_ACK]
        cp a, $FF
        jr nz, .connected
        ld a, [WRAM_PRINT_STATE + PRINT_STATE_RESPONSE_STATUS]
        cp a, $FF
        jr nz, .connected

        ; not connected
        copy [WRAM_PRINT_STATE + PRINT_STATE_TRANSFER_RESULT], RESULT_ERROR_02
        ret

    .connected

    ld a, [WRAM_PRINT_STATE + PRINT_STATE_RESPONSE_STATUS]

    .check_low_battery
        bit PRINTER_STATUS_BIT_LOW_BATTERY, a
        jr z, .low_battery
            ; low battery
            copy [WRAM_PRINT_STATE + PRINT_STATE_TRANSFER_RESULT], \
                 RESULT_ERROR_01
            ret
        .low_battery

    .check_paper_jam
        bit PRINTER_STATUS_BIT_PAPER_JAM, a
```

```
        jr z, .paper_jam
            ; paper jam
            copy [WRAM_PRINT_STATE + PRINT_STATE_TRANSFER_RESULT], \
                 RESULT_ERROR_03
            ret
        .paper_jam

    .check_other_error ; catch all
        and a, (1 << PRINTER_STATUS_BIT_CHECKSUM_ERROR) | \
               (1 << PRINTER_STATUS_BIT_PKT_ERROR) | \
               (1 << PRINTER_STATUS_BIT_OTHER_ERROR)
        jr z, .other_error
            ; other error
            copy [WRAM_PRINT_STATE + PRINT_STATE_TRANSFER_RESULT], \
                 RESULT_ERROR_04
            ret
        .other_error
endm
```

Listing 17-13: Checking for errors

This macro checks for all the possible errors. When an error occurs, it sets the `PRINT_STATE_TRANSFER_RESULT` variable and returns early. This causes the whole transfer to abort because the timer doesn't restart, and the program can check the final result. If the result is `RESULT_SUCCESS`, then the image has printed successfully. If the result equals any of the error codes, such as `RESULT_ERROR_01`, then the image has only partially printed and it should be sent for printing again after the cause of the error (for example, a paper jam) has been fixed.

In a typical printing sequence, the program transfers the data one packet at a time, and the packets themselves are sent one byte at a time. It preserves a delay of 300 or more microseconds between each byte to give the printer enough time to process the data. Check out the complete code in the repository for more details.

This chapter's sample uses packets with static content, where the data packets' payload is always the same and the print parameters are hardcoded. To offer more flexibility to the user, you'll need to use packets with dynamic content. For example, this could allow the user to set the number of feeds or the intensity in the print packet. One way to achieve this is to generate the packets in WRAM before sending them to the printer. When doing this, be sure to get the checksum right, as otherwise the printer will generate an error.

Summary

In this chapter, you learned about the communication protocol used to control the Game Boy Printer, including the structure and purpose of the

different packet types used to communicate with the printer. You should now have a strong grasp of the packet sequence required to print an image.

This chapter's sample gave you all the practical knowledge necessary to produce physical prints with your programs. Leveraging the printer requires mastery of the serial communication capability of the console, so make sure to review Chapter 11 before attempting your own implementation.

In the next chapter, you'll learn about the Super Game Boy, a peripheral you can use to play DMG games on a TV and to add a new set of interesting enhancements to your programs.

18

SUPER GAME BOY

The Super Game Boy (SGB), released in 1994, is a Super Nintendo Entertainment System (SNES) peripheral that allows you to play DMG games on a TV. In this chapter, you'll learn about the additional SGB functionalities that enhance DMG programs.

This chapter may be one of the most challenging parts of the book, because the SGB features are somewhat tacked on to the original DMG hardware. The hardware engineers clearly went above and beyond to include as many pieces of functionality as they could, and as a result, it's a deceptively complex piece of equipment with a charm of its own.

We'll start with an overview of the SGB's capabilities, including color graphics, screen borders, predefined sounds, and support for multiple joypads. Next, I'll provide a theoretical explanation of the commands that control and send data to the SGB and an introduction to graphics-related concepts on this device.

We'll then move on to more practical concerns, such as enabling the SGB features, testing programs, and detecting the hardware model a program is running on. Finally, building on your newfound knowledge of the SGB, you'll explore several samples that demonstrate its capabilities.

SGB Overview

The SGB takes the form of a SNES cartridge. It contains most of the DMG's circuitry, lacking mainly the serial port and the unnecessary LCD component. DMG cartridges are plugged into the SGB, which runs the games and handles the necessary data conversions to display graphics and play sound on a TV through the SNES.

A Japan-exclusive upgrade, the Super Game Boy 2 (SGB2), was released in 1998. From a programmer's perspective, the main difference is the addition of a small port for serial communication. Besides that, the SGB2 runs at the same speed as the DMG, while the SGB1 has a slightly faster clock (around 2 percent). In practice, the clock difference doesn't matter much when writing programs, so I'll ignore it in this book.

To give you an idea of what you can do with the SGB, here is an overview of its functionalities.

First, the SGB can leverage several color palettes to create colored game screens. The palette system is quite different from the one on the CGB, and it comes with more restrictions, as I'll explain later. The SGB can also display a customized border around the game screen. The SNES displays a 256×224-pixel image on the TV, significantly bigger than the 160×144-pixel image on the DMG. The SGB allows programs to fill the extra space with a custom border. In this chapter, I'll use the term *screen* to refer to the 160×144-pixel area that displays the LCD image at the very center of the TV display. The *border* refers to the area around the screen.

NOTE *The SGB has several functionalities related to color graphics. Many of the images in this chapter won't be as effective when viewed in black and white. Full-color versions of the images are available in the GitHub repository at* https://github.com/mdagois/gca/tree/main/images.

The SGB has enhanced audio capabilities as well. It can play predefined sounds that take advantage of the SNES hardware and thus are of higher quality than the sound on the DMG.

Next, the SGB allows programs to access up to four joypads at the same time, a convenient way to add multiplayer modes with a single cartridge. One game that leverages this functionality is *Bomberman GB*, also known as *Wario Blast: Featuring Bomberman!* in North America and Europe.

Finally, you can execute code using the SNES CPU and audio processing unit (APU). *Space Invaders* is the best example of a game leveraging this functionality. It has a game mode that runs entirely on the SNES, but this functionality requires SNES knowledge, so it's not covered in this book.

Commands

You can access SGB functionalities by sending commands from the program to the device. Commands follow a well-defined format and are sent to the SGB using a serial protocol. The SGB handles the communication with the SNES, so programs never have to communicate directly with that console at any time.

The SGB is a very peculiar device, and there is a lot of information to digest in this section. Don't worry if you find some of it confusing at first; I'll walk you through these concepts using samples later in the chapter.

Format

All the SGB commands follow the same structure, which is important to understand before writing SGB programs. Figure 18-1 shows the format of a command.

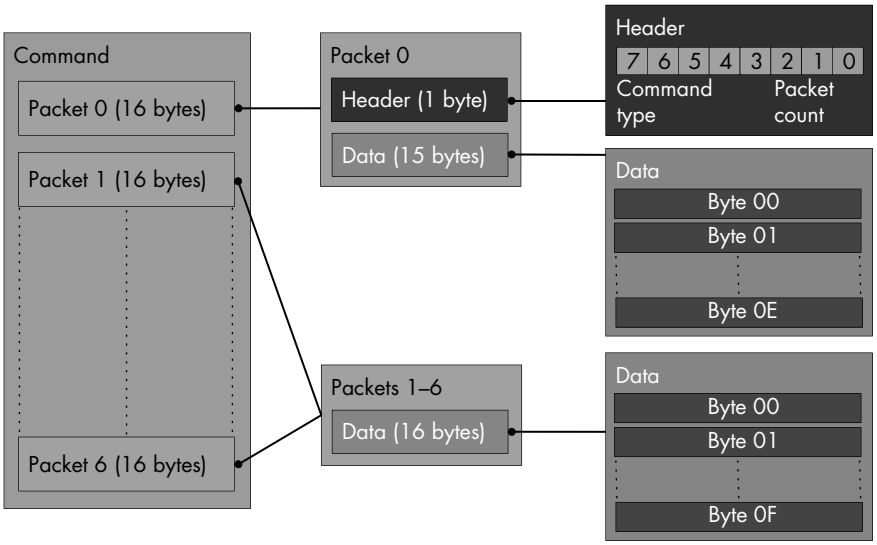


Figure 18-1: The SGB command format

Commands consist of up to 7 packets, each of which is 16 bytes (128 bits) long. The first byte of the first packet of a command is the *command header*. Its lower 3 bits hold the number of packets in the command, and its higher 5 bits represent the command type. The remaining bytes of the command's packets are the command data, known as the *payload*.

Figure 18-2 shows the contents of an actual command.

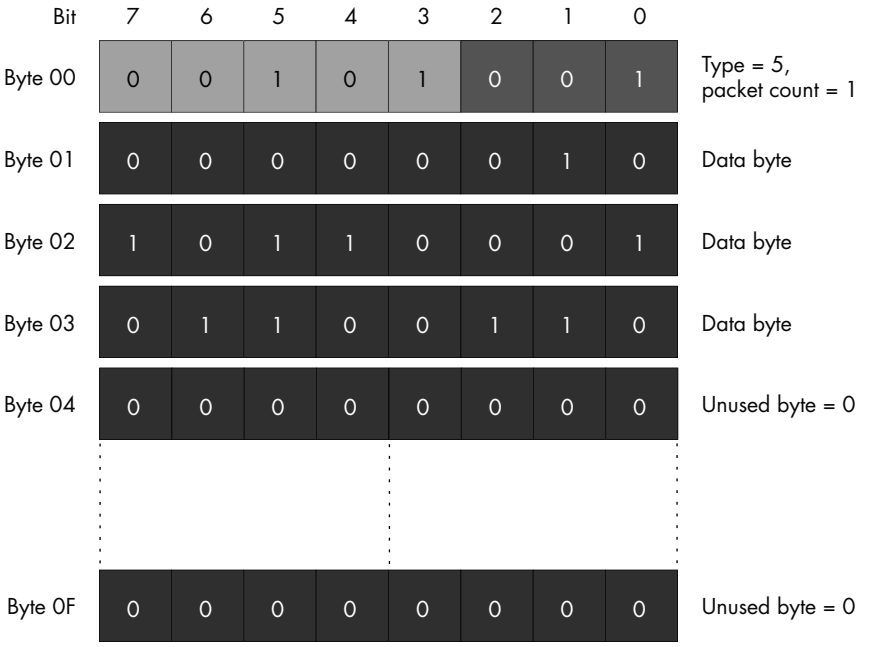


Figure 18-2: An example of a command and its contents

What you should note at this point is that the value of byte `00`, the command header, tells us that the command type is `$05` and that the command consists of a single packet.

Payload

There are two types of payload available to commands: packet and LCD. Most of the commands transfer data through the *packet payload*, which means all the data resides inside their packets. Because commands can consist of up to 7 packets, with 16 bytes per packet and 1 byte reserved for the header in the first packet, the maximum packet payload of a command is 111 bytes ($7 \times 16 - 1$).

Commands with higher payload requirements can use the *LCD payload*, an alternative data transfer method that supports up to 4,096 bytes (4KiB) of data. A program puts tile data in the VRAM, and the SGB converts the pixel colors sent to the LCD to binary data. Each pixel color is a 2-bit value. For example, white is `%00`, light gray is `%01`, dark gray is `%10`, and black is `%11`. A group of four pixels forms a byte. The binary data is constructed one 8×8 screen tile at a time, with each tile representing 16 bytes' worth of data. Figure 18-3 shows an example screen tile and the resulting data bytes.

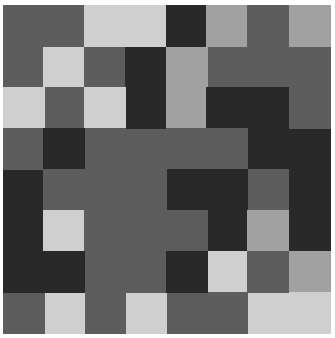


Figure 18-3: An example tile payload

Tiles are read from left to right and from top to bottom. Figure 18-4 shows all the tiles displayed on the LCD.

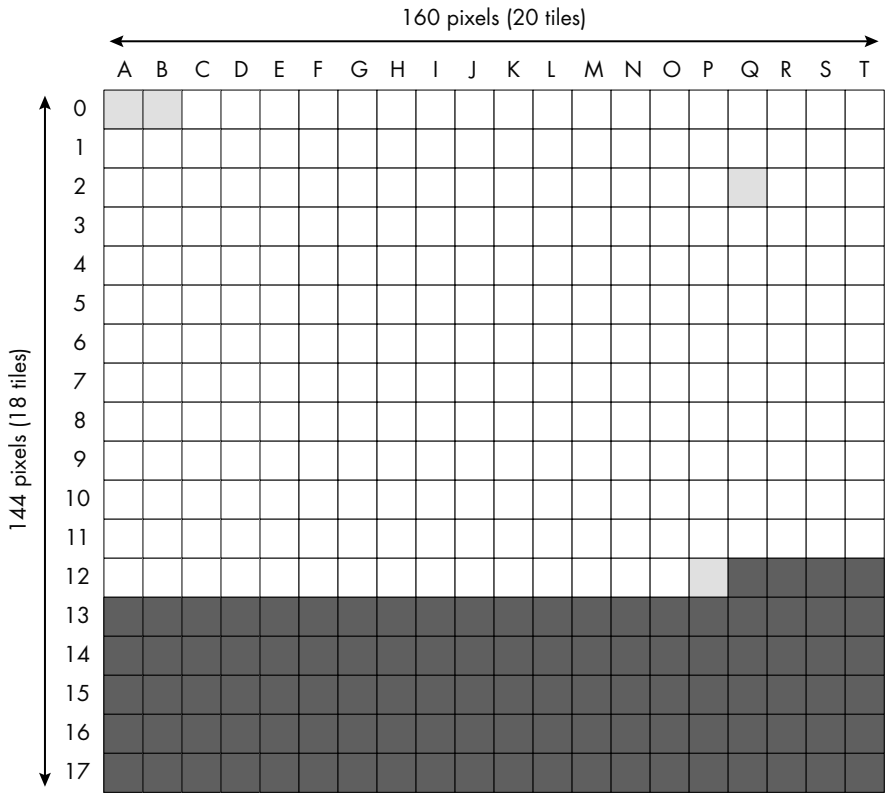


Figure 18-4: The LCD tile order

Tile A0 is the first to be read, tile B0 the second, tile Q2 the 57th, and tile P12 the 256th (these four tiles are highlighted in the figure so it's easy for you to pick them out; they don't have any special significance other than to illustrate the order in which tiles are read). The SGB stops after reading the 256th tile because an LCD payload can contain at most 256 tiles. Since

each tile is 16 bytes, that sums up to 4,096 bytes. The remaining tiles, from Q12 to T17, are ignored, and their values do not matter.

In essence, the data payload must be *drawn* to the LCD to be transferred. This is not a typical way to transfer data, and the SGB's hardware engineers probably resorted to this transfer method only because they didn't have any other option.

The following sequence is one way programs can achieve an LCD payload transfer:

1. Mask the screen. This is equivalent to freezing the image currently on the screen. As the LCD data path is used as a data transfer bus, masking is necessary to prevent any garbage from displaying on the screen.
2. Ensure the LCD and background are enabled.
3. Set the background scrolling to zero in both directions.
4. Hide (or disable) the window.
5. Hide (or disable) the sprites.
6. Copy a payload of up to 4KiB to the start of the VRAM (`$8000` to `$8FFF`).
7. Set the tile addressing mode to $8000.
8. Set the active tilemap tile indices so that all the data in VRAM gets drawn to the upper part of the LCD. (See the following discussion for more details on the tilemap organization.)
9. Set the background palette to `%11100100` (`$E4`). With this palette, the colors on the LCD are reinterpreted to the exact same bytes that were copied to VRAM.
10. Send the command that leverages the LCD payload. This triggers the data transfer to an appropriate memory location (like the SNES WRAM).
11. Wait for five full frames for the transfer to be complete. The frame in which you sent the command does not count.
12. Set the VRAM and LCD parameters to display the next screen.
13. Unmask the screen.

Figure 18-5 shows the tilemap organization mentioned in step 8.

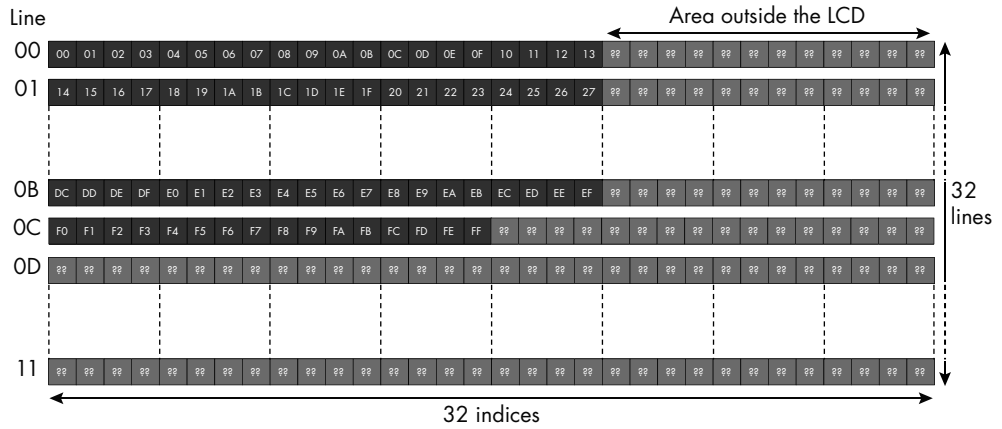


Figure 18-5: The typical LCD payload tilemap organization

This transfers 4,096 bytes, or 256 tiles (4,096 / 16 = 256). The indices of the tilemap are set so that all those tiles display, in order, from the top left of the LCD. The indices span almost 13 lines (256 / 20 = 12.8 lines). They do not cover the whole LCD area, which consists of 20×18 tiles. The values of the other indices (marked as ??) are irrelevant because they do not correspond to transferred data.

LCD payloads may seem convoluted, but again, there was most likely no other data path available for the SGB to transfer large amounts of data via single commands. Keep in mind that the sequence described here is just one way to achieve this; it's possible to use a different tile addressing mode, tilemap indices, and even palette, as some of this chapter's samples demonstrate. As long as the image in the top 20×13-tile area of the LCD corresponds to the data flow meant for the SGB, the behavior won't change from the typical algorithm described in this section. For example, if all the bits of the data in VRAM were inverted, using palette `$1B` (`%00011011`) instead of `$E4` (`%11100100`) would yield the same result.

Serial Protocol

Packets are sent to the SGB using a serial protocol. A transfer occurs bit by bit, from the least significant bit of the first byte to the most significant bit of the last byte. Listing 18-1 shows the bits of the command from Figure 18-2 in the order they would transfer to the SGB.

```
1, 0, 0, 1, 0, 1, 0, 0    ; byte 00
0, 1, 0, 0, 0, 0, 0, 0    ; byte 01
1, 0, 0, 0, 1, 1, 0, 1    ; byte 02
0, 1, 1, 0, 0, 1, 1, 0    ; byte 03
0, 0, 0, 0, 0, 0, 0, 0    ; byte 04
...                       ; byte 05 to 0E
0, 0, 0, 0, 0, 0, 0, 0    ; byte 0F
```

Listing 18-1: An example of the bit send order in a command

Programs send packets to the SGB through bits 4 and 5 of the joypad status register, `rP1`. The values of those two bits change over time to create a digital signal that encodes the packet data. The serial protocol adds a little overhead by marking the boundaries of each packet. It prepends a *packet start marker* to the signal before each packet transfer. Likewise, it appends a sentinel to the signal after each packet transfer.

The protocol also introduces rest periods in the signal to clearly separate the pulses carrying information and give the SGB time to process the packets. For example, it would be difficult to distinguish a single 1 from two consecutive 1s in the signal without any rest period in between. There are short rest periods between the packet bits and longer rest periods between the packets themselves. Table 18-1 lists the possible combinations of bits 4 and 5 for signal generation.

Table 18-1: Signal Generation Using the `rP1` Register

Bit 4	Bit 5	Description
0	0	Packet start marker
0	1	0 or sentinel
1	0	1
1	1	Rest period between bits or packets

The program must avoid reading joypad input while generating a signal using the `rP1` register. This is because the two processes would interfere with each other, causing the input to be read incorrectly and generating an erroneous signal. The programmer must ensure this does not happen.

Here are the steps to follow to create a signal for a packet:

1. Set the signal to the packet start marker.
2. Maintain a short rest period (a few cycles).
3. For each of the 128 bits of the packet, set the signal to either 0 or 1 depending on the bit value, then maintain a short rest period (a few cycles).
4. Set the signal to the sentinel.
5. Maintain a long rest period (about four frames). This step is only mandatory if there is another packet to send.

Figure 18-6 shows the signal to send the first byte of the packet in Figure 18-2.

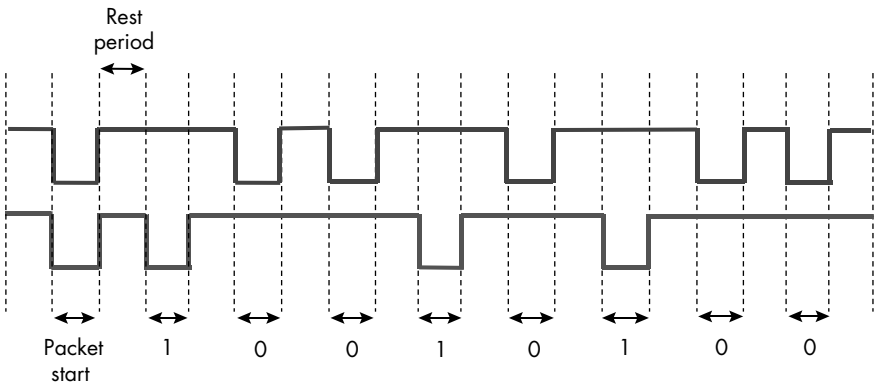


Figure 18-6: An example of a generated signal

The top line corresponds to bit 4 of `rP1` and the bottom line to bit 5. The signal begins with the packet start marker, followed by bits of data interleaved with rest periods.

Types

There are 24 commands, organized into several categories, that allow programs to control the SGB. Table 18-2 lists the command types, which I'll explain in detail as we encounter them in this chapter's samples.

Table 18-2: SGB Commands

Command	Mnemonic	Code	Packets	4KiB	Category	Meaning
`PAL01`	Palette 01	`00`	1	—	Color	Set colors for palettes 0 and 1.
`PAL23`	Palette 23	`01`	1	—	Color	Set colors for palettes 2 and 3.
`PAL03`	Palette 03	`02`	1	—	Color	Set colors for palettes 0 and 3.
`PAL12`	Palette 12	`03`	1	—	Color	Set colors for palettes 1 and 2.
`ATTR_BLK`	Attribute block	`04`	1–*7*	—	Color	Set screen tile attributes using rectangles.
`ATTR_LIN`	Attribute line	`05`	1–*7*	—	Color	Set screen tile attributes using lines.
`ATTR_DIV`	Attribute division	`06`	1	—	Color	Set screen tile attributes by dividing the screen into two regions.
`ATTR_CHR`	Attribute character	`07`	1–6	—	Color	Set screen tile attributes using a list.

Table 18-2: SGB Commands *(continued)*

Command	Mnemonic	Code	Packets	4KiB	Category	Meaning
SOUND	Sound	08	1	—	Sound	Play/stop predefined sounds.
SOU_TRN	Sound transfer	09	1	Yes	Data	Transfer SNES sound data.
PAL_SET	Palette set	0A	1	—	Color	Set palettes transferred through PAL_TRN.
PAL_TRN	Palette transfer	0B	1	Yes	Color	Transfer up to 512 palettes for later use.
ATRC_EN	Attraction enable	0C	1	—	Settings	Enable/disable the attraction mode (screensaver).
ICON_EN	Icon enable	0E	1	—	Settings	Enable/disable the SGB menu options.
DATA_SND	Data send	0F	1	—	Data	Transfer data to SNES WRAM.
DATA_TRN	Data transfer	10	1	Yes	Data	Transfer data to SNES WRAM.
MLT_REQ	Multiplayer request	11	1	—	Multiplayer	Activate multiple joypads.
JUMP	Jump	12	1	—	Data	Set the program counter and vblank interrupt of the SNES.
CHR_TRN	Character transfer	13	1	Yes	Border	Transfer the tile data used by the border.
PCT_TRN	Picture transfer	14	1	Yes	Border	Transfer the tilemap used by the border.
ATTR_TRN	Attribute transfer	15	1	Yes	Color	Transfer 45 screens' worth of screen tile attributes for later use.
ATTR_SET	Attribute set	16	1	—	Color	Set screen tile attributes transferred through ATTR_TRN.
MASK_EN	Mask enable	17	1	—	Display	Control the refresh of the display.
PAL_PRI	Palette priority	19	1	—	Settings	Control the program/user palette priority.

Commands in the Data category require knowledge of the SNES, which is beyond the scope of this book, but I'll cover the commands from the other categories in detail using samples.

Graphics

This section introduces several concepts, such as color palettes, screen tiles, and borders, that are related to the SGB's graphics functionalities. This information is relevant for commands in the Color and Border categories.

Color Palettes

The SGB has access to seven BGR555 color palettes. As you may recall from Chapter 16, BGR555 is a 15-bit color format that represents colors as three components: blue, green, and red. Components take 5 bits each, and they are organized over 2 bytes, as shown in Figure 18-7.

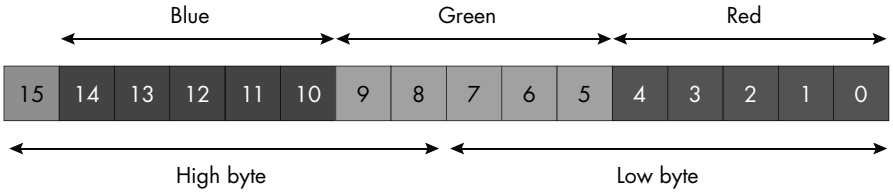


Figure 18-7: The BGR555 color format

The highest bit is unused. The same color format is used by the CGB palettes.

The LCD screen area uses the first four palettes (indices 0 to 3), each consisting of four colors. However, the first color of those palettes is shared, so they are more like three-color palettes with a common extra color. This means that instead of 16 colors (4 palettes × 4 colors), you can specify a total of 13 colors (4 palettes × 3 colors + 1 color).

The graphics of the border use the other three palettes (indices 4 to 6), each containing 16 colors. The first color of each palette is a transparent color that does not display.

Figure 18-8 sums up the palette information graphically (the colors are randomly chosen).

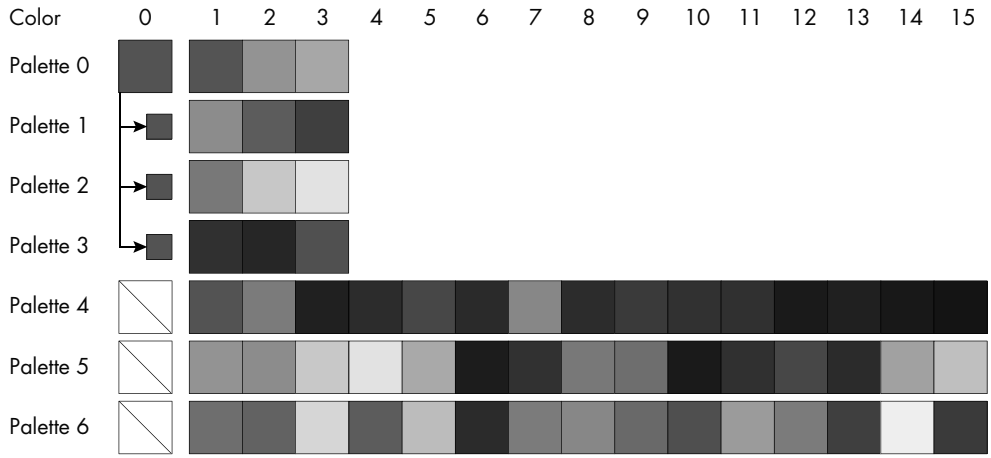


Figure 18-8: SGB color palettes

Notice the shared color and transparent colors at index 0 of palettes 0 to 3 and 4 to 6, respectively. This is important, because it limits the number of colors you have available per palette when creating assets.

Screen Tiles

Screen tiles are 8×8-pixel areas on the LCD. Because the LCD is 160×144 pixels, there are 20×18 screen tiles. The first tile is at the top left and the last tile at the bottom right. You can assign any of the first four palettes to each screen tile, which defines its colors. The palettes are referred to by 2-bit indices called *attributes* in SGB jargon, a term commonly used in color commands.

Because palettes are applied to screen tiles, you must be careful when dealing with anything other than static screens. Scrolling moves the tilemap graphics across the screen, but the screen tiles and their assigned colors stay fixed. The screen tile palettes also affect sprites. For example, if a sprite overlaps two tiles with different palettes, its two halves will use different palettes.

Border Graphics

The SNES generates a 256×224-pixel image and displays it on the TV. The DMG's 160×144-pixel LCD area displays right in the middle of the TV image, leaving a border of 48 pixels on both the left and right and 40 pixels on the top and bottom of the TV display. The SGB lets programs display custom graphics inside that border.

The border's graphics are built in the SNES graphics format. It can leverage the three extended 16-color palettes, with the first color of each palette being transparent. A border is really just a 256×224-pixel image with a transparent hole in the middle to keep the program area visible. For example, Figure 18-9 shows a PNG asset in one of the border samples.

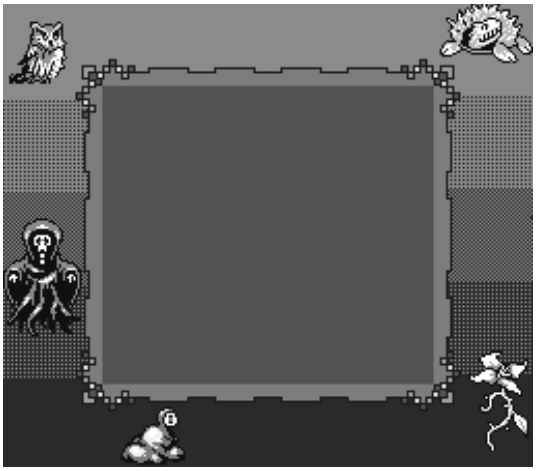


Figure 18-9: An example of a border image

The samples use magenta as the transparent color, setting it as the first color in each of the 16-color palettes.

The SNES graphics format is different from the format on the DMG, but the general concepts remain similar, with SNES graphics still consisting of tiles and tilemaps. In Chapter 5, you learned about the encoding of the 2-bit color indices inside each 8×8 tile on the DMG. On the SNES, tiles are still 8×8 pixels, but they use 16-color palettes, so the color indices are 4 bits long. This doubles the number of bytes necessary to store a tile, from 16 bytes to 32 bytes.

However, the 4 bits of the color indices are not fully interleaved inside a tile's bytes. Instead, bits 0 and 1 of the color indices are interleaved inside the first 16 bytes, while bits 2 and 3 are interleaved inside the last 16 bytes. This tile organization is convenient because it means you can view a single SNES tile as two consecutive DMG tiles.

Figure 18-10 shows how the 4 bits of the very first color index of a tile would be stored in the tile data.

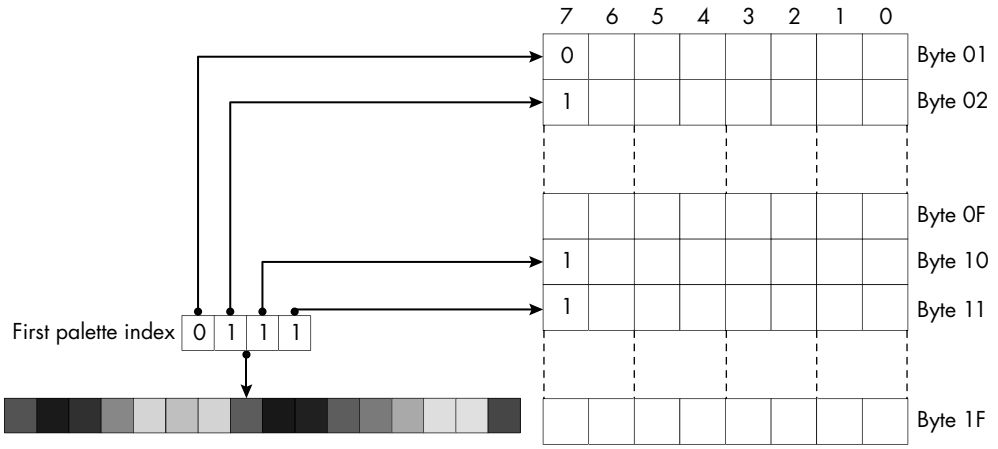


Figure 18-10: Color encoding in a SNES tile

Bit 0 of the index goes into bit 7 of byte 0, and bit 1 goes into bit 7 of byte 1. Then, bit 2 of the index goes into bit 7 of byte 16 and bit 3 into bit 7 of byte 17. Other color indices are stored the same way over the two groups of 16 bytes.

In the context of the SGB, a SNES tilemap consists of 32×28 entries, enough to cover the entire 256×224-pixel area displayed on the TV. Each tilemap entry uses 2 bytes to store the information shown in Table 18-3.

Table 18-3: Border Tilemap Format

Byte	Description	Value
00–07	Tile index	0–255
08–09	Unused	Must be 0
0A–0C	Palette index	4–6
0D	Background priority	Must be 0
0E	Horizontal flip	0 = no flip, 1 = flip
0F	Vertical flip	0 = no flip, 1 = flip

The tile index and flips work similarly to with DMG sprites, as covered in Chapter 7. The palette index should be 4 to 6, meaning it must point to one of the 16-color palettes.

Asset Converters

We'll use two custom tools, sgbconv and sfcconv, to convert the PNG assets for this chapter's samples. These tools are dedicated to producing SGB-compatible data.

The first tool, *sgbconv*, converts color graphics meant for the LCD screen area. It turns a PNG asset into a set of four binary files in the CHR, TLM, PAL, and ATR formats. The CHR and TLM files contain the tile data and the tilemap, respectively. The format is the same as on the DMG, as

described in Chapter 6. The PAL files contain up to four palettes, with each palette taking 8 bytes (four BGR555 colors). The shared color is exported for each palette in the PAL file, instead of exporting it only once (this is an arbitrary design choice). The ATR files contain screen tile attributes, which are really just palette indices.

The second tool, *sfcconv*, converts assets for the custom border. This tool makes some arbitrary decisions, like using magenta as the transparent color. It outputs three types of files: PAL, CHR, and TLM. The PAL files contain up to three 16-color palettes. The CHR files contain SNES tile data, with each tile being 32 bytes. Finally, the TLM files contain SNES tilemaps.

Both sgbconv and sfcconv are available, along with their source code, in the book's GitHub repository. They are very similar to tools introduced in previous chapters, such as gfxconv (DMG) and cgbconv (CGB). I've kept the tools separate as they each focus on one particular type of hardware, but you could merge all their functionalities into a single program.

Building and Testing SGB Programs

In this section you'll learn the necessary steps to build and test SGB programs. It lays the groundwork for studying this chapter's samples.

Enabling Enhanced Features

For your ROM to run on the SGB, you must enable its enhanced features via two fields in the cartridge header. Added especially for the SGB, the *SGB flag* at address `$0146` must be set to `$03`. You also need to set the *old licensee code* at address `$014B` to `$33`. This field specifies the game publisher for every game released before the SGB. For example, a value of `$01` specifies Nintendo, and a value of `$08` specifies Capcom. When the SGB was released, it was decided that new games should set this field to `$33` to avoid incompatibilities with older games when run on the SGB. The *new licensee code* field specifies the game publisher, which is encoded in the two bytes at addresses `$0144` and `$0145`.

If you use any values other than `$03` for the SGB flag and `$33` for the old licensee code, the SGB disables the enhanced features and ignores any SGB commands in your program. The `rgbfix` command has parameters to set these two fields from the command line, as shown here:

```
rgbfix --sgb-compatible --old-licensee 0x33 ...
```

Note that the SGB1 and SGB2 predate the CGB and cannot run CGB-only programs. However, by enabling both the SGB compatibility mode described in this section and the CGB compatibility mode discussed in Chapter 16, you can write programs that take full advantage of the DMG, CGB, and SGB.

Testing on BGB

Now that you know how to enable the SGB's enhanced features in your programs, let's see how to test them in BGB. BGB can emulate the SGB, and by default it starts any SGB-compatible ROM in SGB mode. However, during development, it's better to switch between DMG mode and SGB mode to properly test both versions. BGB has a command line parameter called `SystemMode` to do just that.

To force BGB to run in SGB mode, set `SystemMode` to `2`, as in the following command:

```
bgb -set SystemMode=2 rom.gb
```

To force it to run in DMG mode, set `SystemMode` to `0` instead. You can also force BGB to emulate either of the two SGB hardware models using the `DetectBGP` parameter, as shown in the following command:

```
bgb -set SystemMode=2 -set DetectGBP=1 rom.gb
```

Setting `DetectBGP` to `0` forces BGB to launch in SGB1 mode, while a value of `1` corresponds to the SGB2.

All these options are also available through the System tab of BGB's Options menu, shown in Figure 18-11.

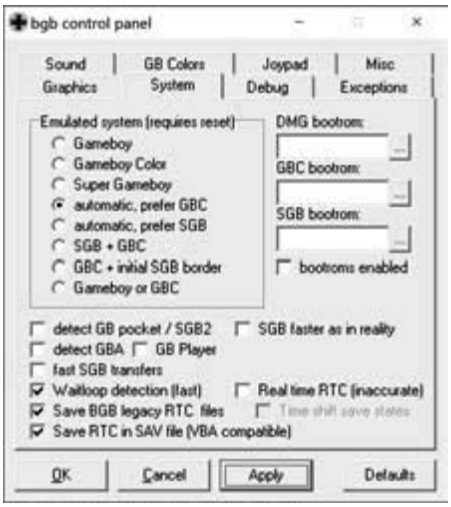


Figure 18-11: SGB options

The Emulated System radio button corresponds to the `SystemMode` command line parameter. The Detect GB pocket / SGB2 checkbox corresponds to the `DetectGBP` command line parameter.

Note that BGB does not support all SGB functionalities. Commands from the Data, Settings, and Sound categories have no effect. To test those functionalities, you'll need to use real hardware.

Testing on Real Hardware

To test on real hardware, you need both a SNES and an SGB. Additionally, you need a Super Multitap and up to four SNES controllers to test the multi-joypad functionality.

For owners of a Super Famicom, the Japanese version of the SNES, both SGB1 and SGB2 are available. The SGB2 is harder to find, significantly more expensive, and barely different from the SGB1, so an SGB1 is probably the better choice.

I used a Super Famicom with four controllers, an SGB1, an SGB2, and a HORI Multitap HSM-07 to test the samples for this book.

Hardware Detection

In this section, we'll use the *sgb_detection* sample in the GitHub repository to illustrate how a program can detect whether it's running on a DMG, SGB1, or SGB2. Technically, there is no such thing as an SGB-only program. Any program that can run on the SGB should run properly on a DMG as well. Thus, programs must verify whether they're running on an SGB before sending SGB commands. The verification process relies on the initial values of registers a and c when the program starts, as shown in Figure 18-12.

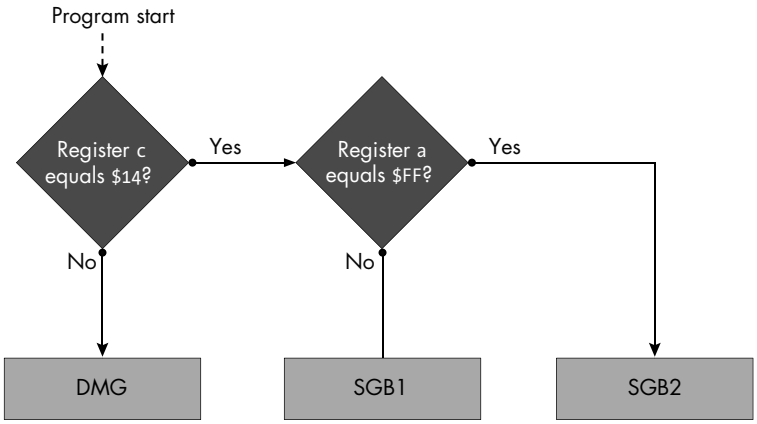


Figure 18-12: SGB hardware model detection

This sample features a macro that detects the hardware model the program is running on, shown in Listing 18-2.

```
macro DetermineHardwareModel
    ld b, a
    ld a, c
    cp a, $14
    jr nz, .sgb\@
        ld a, b
        inc a
        jr nz, .sgb2\@
```

```
            ld a, HARDWARE_MODEL_SGB2
            jr .done\@
        .sgb2\@
        ld a, HARDWARE_MODEL_SGB1
        jr .done\@
    .sgb\@
        ld a, HARDWARE_MODEL_DMG
    .done\@
    ; save the hardware model
    ld [HARDWARE_MODEL_REGISTER], a
endm
```

Listing 18-2: Detecting the hardware model

This is a relatively straightforward implementation of the process depicted in Figure 18-12. The code in the macro is meant to execute at the very start of the program, because it relies on the values of `a` and `c` at boot time. Once the hardware model is determined, the code stores that information in `HARDWARE_MODEL_REGISTER`, which is just a location in HRAM. Listing 18-3 shows the definitions of the constants used in the macro.

```
def HARDWARE_MODEL_REGISTER     equ (_HRAM)

def HARDWARE_MODEL_DMG          equ (1)
def HARDWARE_MODEL_SGB1         equ (2)
def HARDWARE_MODEL_SGB2         equ (3)
```

Listing 18-3: Defining the hardware model constants

The sample contains an initialization function for each hardware model. Those functions are mostly identical for now, but they do display a different number to the LCD (0 for DMG, 1 for SGB1, and 2 for SGB2), as shown in Listing 18-4.

```
def MODEL_NUMBER_TILE_ADDRESS   equ (_SCRN0 + 3 * SCRN_VX_B + 15)
def TILE_ZERO_INDEX             equ (80)

InitSample_DMG:
    copy [rBGP], %11100100
    CopyMemory _VRAM, graphics_data, graphics_data_end - graphics_data
    copy [MODEL_NUMBER_TILE_ADDRESS], TILE_ZERO_INDEX + 0

    copy [rIE], IEF_VBLANK
    ei
    copy [rLCDC], LCDCF_ON | LCDCF_BG8800 | LCDCF_BG9800 | LCDCF_BGON
    ret

InitSample_SGB1:
    copy [rBGP], %11100100
    CopyMemory _VRAM, graphics_data, graphics_data_end - graphics_data
```

```
    copy [MODEL_NUMBER_TILE_ADDRESS], TILE_ZERO_INDEX + 1

    copy [rIE], IEF_VBLANK
    ei
    copy [rLCDC], LCDCF_ON | LCDCF_BG8800 | LCDCF_BG9800 | LCDCF_BGON
    ret

InitSample_SGB2:
    copy [rBGP], %11100100
    CopyMemory _VRAM, graphics_data, graphics_data_end - graphics_data
    copy [MODEL_NUMBER_TILE_ADDRESS], TILE_ZERO_INDEX + 2

    copy [rIE], IEF_VBLANK
    ei
    copy [rLCDC], LCDCF_ON | LCDCF_BG8800 | LCDCF_BG9800 | LCDCF_BGON
    ret

InitSample:
    ld a, [HARDWARE_MODEL_REGISTER]
    dec a
    jp z, InitSample_DMG
    dec a
    jp z, InitSample_SGB1
    dec a
    jp z, InitSample_SGB2
```

Listing 18-4: Hardware-dependent initialization

The `InitSample` function checks the value of `HARDWARE_MODEL_REGISTER`. Depending on that value, which reflects the hardware model the program is currently running on, the appropriate hardware-specific initialization function executes.

The sample looks like Figure 18-13 when run in BGB in SGB1 mode.



Figure 18-13: Running the detection sample in SGB1 mode

The screen shows a 1, meaning the hardware was properly detected as an SGB1. If we were to force BGB to run in DMG mode, the sample would display a 0, and if we were to force it to run in SGB2 mode, it would display a 2.

NOTE *BGB actually displays a border around the screen in SGB mode, but because we haven't sent any border data yet, the border is just black. I've removed it from this and all other SGB sample screenshots with black borders in this book, but they will appear if you run the samples yourself.*

Because the SGB1 and SGB2 are mostly identical, I'll treat them as the same device in this chapter's samples and handle both devices with one initialization function.

Sample Foundations

All the remaining samples of this chapter share several pieces of code: the definition of the command headers, the wait mechanics for adding delays, and a function to send commands to the SGB. In this section, we'll go over each of these shared pieces of code so that we can focus on the commands themselves in the rest of the chapter. The code in this section is from the *sgb_init* sample, which I'll cover in more detail later.

Defining the Command Headers

As you saw in Figure 18-1, the header byte of a command consists of the command type and the number of packets in the command. In the samples, the command types are defined as the constants shown in Listing 18-5.

```
def COMMAND_PAL01       equ ($00)
def COMMAND_PAL23       equ ($01)
def COMMAND_PAL03       equ ($02)
def COMMAND_PAL12       equ ($03)
def COMMAND_ATTR_BLK    equ ($04)
def COMMAND_ATTR_LIN    equ ($05)
def COMMAND_ATTR_DIV    equ ($06)
def COMMAND_ATTR_CHR    equ ($07)
def COMMAND_SOUND       equ ($08)
def COMMAND_SOU_TRN     equ ($09)
def COMMAND_PAL_SET     equ ($0A)
def COMMAND_PAL_TRN     equ ($0B)
def COMMAND_ATRC_EN     equ ($0C)
def COMMAND_ICON_EN     equ ($0E)
def COMMAND_DATA_SND    equ ($0F)
def COMMAND_DATA_TRN    equ ($10)
def COMMAND_MLT_REQ     equ ($11)
def COMMAND_JUMP        equ ($12)
def COMMAND_CHR_TRN     equ ($13)
```

```
def COMMAND_PCT_TRN     equ ($14)
def COMMAND_ATTR_TRN    equ ($15)
def COMMAND_ATTR_SET    equ ($16)
def COMMAND_MASK_EN     equ ($17)
def COMMAND_PAL_PRI     equ ($19)
```

Listing 18-5: Command types

The values of the command types come straight from Table 18-2. Most commands only ever have one packet, with a few exceptions that can have six or seven, so it's possible to define a constant for every possible command header. These constants are defined in Listing 18-6.

```
def PAL01               equ ((COMMAND_PAL01 << 3) | 1)
def PAL23               equ ((COMMAND_PAL23 << 3) | 1)
def PAL03               equ ((COMMAND_PAL03 << 3) | 1)
def PAL12               equ ((COMMAND_PAL12 << 3) | 1)
def ATTR_BLK_1          equ ((COMMAND_ATTR_BLK << 3) | 1)
def ATTR_BLK_2          equ ((COMMAND_ATTR_BLK << 3) | 2)
def ATTR_BLK_3          equ ((COMMAND_ATTR_BLK << 3) | 3)
def ATTR_BLK_4          equ ((COMMAND_ATTR_BLK << 3) | 4)
def ATTR_BLK_5          equ ((COMMAND_ATTR_BLK << 3) | 5)
def ATTR_BLK_6          equ ((COMMAND_ATTR_BLK << 3) | 6)
def ATTR_BLK_7          equ ((COMMAND_ATTR_BLK << 3) | 7)
def ATTR_LIN_1          equ ((COMMAND_ATTR_LIN << 3) | 1)
def ATTR_LIN_2          equ ((COMMAND_ATTR_LIN << 3) | 2)
def ATTR_LIN_3          equ ((COMMAND_ATTR_LIN << 3) | 3)
def ATTR_LIN_4          equ ((COMMAND_ATTR_LIN << 3) | 4)
def ATTR_LIN_5          equ ((COMMAND_ATTR_LIN << 3) | 5)
def ATTR_LIN_6          equ ((COMMAND_ATTR_LIN << 3) | 6)
def ATTR_LIN_7          equ ((COMMAND_ATTR_LIN << 3) | 7)
def ATTR_DIV            equ ((COMMAND_ATTR_DIV << 3) | 1)
def ATTR_CHR_1          equ ((COMMAND_ATTR_CHR << 3) | 1)
def ATTR_CHR_2          equ ((COMMAND_ATTR_CHR << 3) | 2)
def ATTR_CHR_3          equ ((COMMAND_ATTR_CHR << 3) | 3)
def ATTR_CHR_4          equ ((COMMAND_ATTR_CHR << 3) | 4)
def ATTR_CHR_5          equ ((COMMAND_ATTR_CHR << 3) | 5)
def ATTR_CHR_6          equ ((COMMAND_ATTR_CHR << 3) | 6)
def SOUND               equ ((COMMAND_SOUND << 3) | 1)
def SOU_TRN             equ ((COMMAND_SOU_TRN << 3) | 1)
def PAL_SET             equ ((COMMAND_PAL_SET << 3) | 1)
def PAL_TRN             equ ((COMMAND_PAL_TRN << 3) | 1)
def ATRC_EN             equ ((COMMAND_ATRC_EN << 3) | 1)
def ICON_EN             equ ((COMMAND_ICON_EN << 3) | 1)
def DATA_SND            equ ((COMMAND_DATA_SND << 3) | 1)
def DATA_TRN            equ ((COMMAND_DATA_TRN << 3) | 1)
def MLT_REQ             equ ((COMMAND_MLT_REQ << 3) | 1)
def JUMP                equ ((COMMAND_JUMP << 3) | 1)
def CHR_TRN             equ ((COMMAND_CHR_TRN << 3) | 1)
```

```
def PCT_TRN             equ ((COMMAND_PCT_TRN << 3) | 1)
def ATTR_TRN            equ ((COMMAND_ATTR_TRN << 3) | 1)
def ATTR_SET            equ ((COMMAND_ATTR_SET << 3) | 1)
def MASK_EN             equ ((COMMAND_MASK_EN << 3) | 1)
def PAL_PRI             equ ((COMMAND_PAL_PRI << 3) | 1)
```

Listing 18-6: Command headers

Multiple constants are defined for the commands that can have different numbers of packets, such as `ATTR_BLK` and `ATTR_LIN`, to accommodate the different possible headers for these commands. These constants are convenient when defining command packets, and we'll use them in all of this chapter's samples.

Implementing the Wait Mechanics

It's quite common to have to wait for several frames when programming the SGB, so the samples include a convenient macro to do just that. Listing 18-7 shows this macro, which takes the number of frames to wait as a parameter.

```
; \1 = number of frames to wait
macro WaitFrames
    push bc
    push de
    ld c, \1
    call _WaitFrames
    pop de
    pop bc
endm
```

Listing 18-7: The `WaitFrames` macro

The macro pushes the `bc` and `de` registers and copies the number of frames to wait into the `c` register. It then calls the `_WaitFrames` function, shown in Listing 18-8, which implements the actual wait mechanics.

```
_WaitFrames:
    ; (c) holds the number of frames to wait
    .all_frames
        ld de, 1756
        .single_frame
            nop
            nop
            nop
            dec de
            ld a, d
            or a, e
            jr nz, .single_frame
```

```
        dec c
        jr nz, .all_frames
    ret
```

Listing 18-8: The `_WaitFrames` function

This function uses `.single_frame`, an inner loop of 10 cycles that iterates 1,756 times, for a total of 17,559 cycles (the last `jr` is only 2 cycles). As the calculation in Listing 5-1 told us, a frame is 17,556 cycles long, so this inner loop is covering a tad more than a frame.

Then, an outer loop, `.all_frames`, iterates `c` times to produce the desired wait time, as `c` holds the number of frames to wait. Because of the extra cycles the function waits slightly longer than is strictly required, but in the context of SGB programming, its precision is good enough.

The wait function implements a *busy wait*, meaning it wastes cycles doing nothing. While you could use `halt` to wait for the vblank and count frames, it's not a generic enough solution here. When there is a risk of a long interrupt, it's better to disable interrupts using `di` before sending commands to the SGB, to avoid any potential timing issues in the generated packet signal. Because the `halt` method relies on the vblank interrupt (at least when it comes to counting frames), we cannot use it when interrupts are disabled; hence the use of a busy wait instead.

Also, note the way the wait function checks the `de` register for a zero value. Instead of checking whether both `d` and `e` equal zero, which would require loading them into `a` and using `cp`, the code or's both registers into `a`. This saves 2 cycles and at least 2 bytes.

Sending Commands

Sending a command is the most fundamental operation when programming the SGB. This chapter's samples use a handy macro, shown in Listing 18-9, to hide all the details of sending a command, such as generating a signal and waiting for the command to be processed.

```
macro SendCommand
    ld hl, \1
    call _SendCommand
endm
```

Listing 18-9: The `SendCommand` macro

This macro takes a single parameter, the address of the command data, and sets the `hl` register to that address. It then calls an underlying function, `_SendCommand` (shown in Listing 18-10), that does the heavy lifting.

```
_SendCommand:
    ; get the command header
    ld a, [hl]
    push af
```

```
    ; retrieve the number of packets in the command
    and a, $07

    ; check that there is at least one packet
    ret z

    ; send the packets of the command
    ld c, a
    .send_packet
        call _SendPacket
        dec c
        jr nz, .send_packet

    ; retrieve the command type
    pop af
    srl a
    srl a
    srl a

    ; check whether the command is using a 4KiB payload
    cp a, COMMAND_SOU_TRN
    jr z, .4KB_command
    cp a, COMMAND_PAL_TRN
    jr z, .4KB_command
    cp a, COMMAND_DATA_TRN
    jr z, .4KB_command
    cp a, COMMAND_CHR_TRN
    jr z, .4KB_command
    cp a, COMMAND_PCT_TRN
    jr z, .4KB_command
    cp a, COMMAND_ATTR_TRN
    jr z, .4KB_command

    ; not a 4KiB command
    ret

    .4KB_command
    ; wait extra frames for completion of 4KiB commands
    WaitFrames 2

    ret
```

Listing 18-10: The _SendCommand function

This function manages the command as a whole by extracting the number of packets in the command and sending each packet one by one via a dedicated function called `_SendPacket`. The `_SendCommand` function returns early when there are no packets, but that's just a safety measure because technically this should never happen.

After all the packets are sent, the `_SendCommand` function extracts the command type and adds an extra delay for commands with an LCD payload. Only two frames of extra delay are necessary here, because the `_SendPacket` function (shown in Listing 18-11) already adds four frames of inter-packet delay.

```
_SendPacket:
    ; packet start marker
    xor a
    ldh [rP1], a
    ld a, P1F_GET_NONE
    ldh [rP1], a

    ; send the bytes of the packet
    ld b, SGB_BYTES_PER_PACKET
    .send_byte
        ld e, [hl]
        inc hl

        ; send the bits of the byte
        ld d, 8
        .send_bit
            rrc e
            ld a, P1F_4     ; one
            jr c, .pulse
                ld a, P1F_5 ; zero
            .pulse
            ldh [rP1], a
            ld a, P1F_GET_NONE
            ldh [rP1], a
            dec d
            jr nz, .send_bit

        dec b
        jr nz, .send_byte

    ; sentinel
    ld a, P1F_5
    ldh [rP1], a
    ld a, P1F_GET_NONE
    ldh [rP1], a
```

```
    ; wait for about 60 ms
    WaitFrames 4
    ret
```

Listing 18-11: The _SendPacket function

This function creates the signal using the `rP1` register, following the algorithm described in "Serial Protocol" on page 331. The code sends the packet marker and then enters an outer loop, `.send_byte`, to send bytes one by one. Each byte is sent bit by bit in an inner loop, `.send_bit`. By the time that loop is entered, `e` holds the byte to send. The `rrc` instruction rotates `e` and kicks the next bit to send into the carry. The code in the loop then sets the signal to 0 or 1 according to the carry value and finishes with a rest period. Note that the code increments the `hl` register with `inc hl` each time a byte is processed. This happens for each of the 16 bytes in the packet. By the end of the function, the `hl` register points to the next packet in the command. After sending the sentinel, the function ends by waiting four frames, the necessary rest period between packets.

We now have a high-level mechanism to send commands. The samples we'll explore in the reminder of this chapter will use the command header definitions, the `WaitFrames` macro, and the `SendCommand` macro to send different commands to the SGB to demonstrate the use of its features.

Initialization Procedure

Once a program knows it's running on an SGB, it must properly initialize the device using a specific sequence of steps. This section will use the *sgb_init* sample to demonstrate how to do this. The initialization involves the following steps:

1. Wait for 60 frames to let the SGB set its internal state.
2. Mask the screen. This is particularly relevant when using 4KiB payload commands.
3. Send the initialization commands to set the SNES WRAM. These are `DATA_SND` commands with fixed content. Eight of them are mandatory, and five are required only if using `SOU_TRN` commands later in the program.
4. Optionally, send custom commands. This is program-specific. Most programs will at least set a border or load palettes.
5. Wait for one frame.
6. Unmask the screen. This ends the initialization sequence.

This sequence was the original Nintendo protocol to initialize the SGB, but many games don't follow it exactly. For example, instead of masking the screen, they might disable the LCD or make it all black. All these methods achieve the goal of preventing garbage from being displayed on the LCD while payloads are being sent to the SGB, so feel free to use whichever

one best suits your program. Later in the chapter, we'll use one of these alternative methods to work around a glitch related to borders.

The sample we'll go over in this section implements the device initialization sequence using a function called `InitializeSgbDevice`. This function is called from the main initialization function for the SGB, `InitSample_SGB`, shown in Listing 18-12.

```
InitSample_SGB:
    copy [rBGP], %11100100
    CopyMemory _VRAM, graphics_data, graphics_data_end - graphics_data
    copy [MODEL_NUMBER_TILE_ADDRESS], TILE_ZERO_INDEX + 1

    copy [rIE], IEF_VBLANK
    ei
    copy [rLCDC], LCDCF_ON | LCDCF_BG8800 | LCDCF_BG9800 | LCDCF_BGON

    call InitializeSgbDevice

    ret
```

Listing 18-12: The SGB initialization function

The `InitSample_SGB` function first sets the background palette, then copies all the graphics data from the ROM to the VRAM. Next, it sets the number displayed on the screen, which is `1` in this case because we are running on an SGB. Finally, the function sets the interrupts, enables the LCD, and calls `InitializeSgbDevice`.

Listing 18-13 shows the code for the `InitializeSgbDevice` function, which implements the device initialization steps.

```
InitializeSgbDevice:
    WaitFrames 60
    SendCommand SgbPacket_MaskEn_Freeze

def COMMAND_INDEX = 0
rept ((SgbPackets_CoreInit_End - SgbPackets_CoreInit) / SGB_BYTES_PER_PACKET)
    SendCommand SgbPackets_CoreInit + COMMAND_INDEX
def COMMAND_INDEX += SGB_BYTES_PER_PACKET
endr

    call PerformCustomInitialization

    WaitFrames 1
    SendCommand SgbPacket_MaskEn_Cancel
    ret
```

Listing 18-13: Performing device initialization

The first step is to wait 60 frames using the `WaitFrames` macro. The second step is to mask the screen, or stop refreshing the SNES VRAM, causing

the TV to display the same image until the mask is canceled. The `MASK_EN` command, which consists of a single packet whose details are in Table 18-4, controls masking.

Table 18-4: The `MASK_EN` Command

Byte	Description	Value
00	Header	$B9
01	Mask action	0 = cancel, 1 = freeze, 2 = black, 3 = color 0
02–0F	Unused	0

The *mask action* is the only variable part, with four possible actions: cancel, freeze, black, and color 0. *Cancel* disables masking, refreshing the screen in each frame as usual. The other three actions enable masking in different ways. *Freeze* preserves the currently displayed image, in effect freezing the screen. *Black* simply displays a black screen. *Color 0* displays a blank screen using the first color of the palettes (the shared color). Masking affects only the screen, not the border.

In code, we use a macro to define the four possible mask commands, as shown in Listing 18-14.

```
; \1 = mask type
macro SgbPacket_MaskEn
db MASK_EN, \1
ds 14, PADDING
endm

SgbPacket_MaskEn_Cancel:
    SgbPacket_MaskEn MASK_EN_CANCEL

SgbPacket_MaskEn_Freeze:
    SgbPacket_MaskEn MASK_EN_FREEZE

SgbPacket_MaskEn_Black:
    SgbPacket_MaskEn MASK_EN_BLACK

SgbPacket_MaskEn_Color0:
    SgbPacket_MaskEn MASK_EN_COLOR0
```

Listing 18-14: Defining the mask commands

In this macro, the header byte is `MASK_EN`, and the mask action is passed as a parameter so that it is easy to define all four possible mask commands. The last line of the macro simply fills the unused bytes of the command with zeros (`PADDING`).

The next step from Listing 18-13 is to send all the device initialization commands. Instead of typing 13 commands, we use a repeat block, which allows us to change the number of commands by just editing the command data. For example, we can delete the optional commands for `SOU_TRN` without needing to change the repeat block code. Listing 18-15 shows the contents of each initialization command. Again, these are all `DATA_SND` commands whose content is fixed.

```
SgbPackets_CoreInit:
; mandatory initial commands
db DATA_SND, $5D, $08, $00, $0B, $8C, $D0, $F4
db      $60, $00, $00, $00, $00, $00, $00, $00
db DATA_SND, $52, $08, $00, $0B, $A9, $E7, $9F
db      $01, $C0, $7E, $E8, $E8, $E8, $E8, $E0
db DATA_SND, $47, $08, $00, $0B, $C4, $D0, $16
db      $A5, $CB, $C9, $05, $D0, $10, $A2, $28
db DATA_SND, $3C, $08, $00, $0B, $F0, $12, $A5
db      $C9, $C9, $C8, $D0, $1C, $A5, $CA, $C9
db DATA_SND, $31, $08, $00, $0B, $0C, $A5, $CA
db      $C9, $7E, $D0, $06, $A5, $CB, $C9, $7E
db DATA_SND, $26, $08, $00, $0B, $39, $CD, $48
db      $0C, $D0, $34, $A5, $C9, $C9, $80, $D0
db DATA_SND, $1B, $08, $00, $0B, $EA, $EA, $EA
db      $EA, $EA, $A9, $01, $CD, $4F, $0C, $D0
db DATA_SND, $10, $08, $00, $0B, $4C, $20, $08
db      $EA, $EA, $EA, $EA, $EA, $60, $EA, $EA
; optional initial commands for SOU_TRN
db DATA_SND, $00, $09, $00, $0B, $AD, $C2, $02
db      $C9, $09, $D0, $1A, $A9, $01, $8D, $00
db DATA_SND, $0B, $09, $00, $0B, $42, $AF, $DB
db      $FF, $00, $F0, $05, $20, $73, $C5, $80
db DATA_SND, $16, $09, $00, $0B, $03, $20, $76
db      $C5, $A9, $31, $8D, $00, $42, $68, $68
db DATA_SND, $21, $09, $00, $01, $60, $00, $00
db      $00, $00, $00, $00, $00, $00, $00, $00
db DATA_SND, $00, $08, $00, $03, $4C, $00, $09
db      $00, $00, $00, $00, $00, $00, $00, $00
SgbPackets_CoreInit_End:
```

Listing 18-15: The initialization packets

The `DATA_SND` command sends data to the SNES WRAM. It consists of a single packet, whose contents are detailed in Table 18-5.

Table 18-5: The `DATA_SND` Command

Byte	Description	Value
00	Header	$79
01	Destination address (low byte)	$1800–$1FFF for bank $00 $B000–$BFFF for bank $7E $0000–$FFFF for bank $7F Any other address requires consideration before use.
02	Destination address (high byte)	Same as above
03	Bank number	Same as above
04	Number of bytes of data	1–11
05–0F	Data bytes	Any (0 if unused)

The `DATA_SND` command defines the destination address in the SNES WRAM over 3 bytes: 2 for the address itself and 1 for the bank number. Then comes a byte specifying the number of bytes of data to send to the SNES WRAM in the packet, followed by the data bytes themselves.

In the device initialization commands in Listing 18-15, the first eight commands (the mandatory ones) write a total of 88 bytes from `$0810` to `$0867` in bank 0. The optional commands write 34 bytes from `$0900` to `$0921` and then another 3 bytes from `$0800` to `$0802`, also all in bank 0. The bytes are written in address ranges programs cannot safely access. While the purpose of this data is unclear, commercial games send those commands, and we can assume the SGB uses this data to perform some kind of internal state tracking.

The next step from Listing 18-13 is to call the custom initialization function shown in Listing 18-16.

```
PerformCustomInitialization:
    ret
```

Listing 18-16: The function to perform custom initialization

This function does nothing for now, but it will include some code in subsequent samples. The two final steps from Listing 18-13 are to wait for a single frame and cancel the mask. After the initialization function returns, the program is free to run and send additional commands. In the following sections you'll get experience with some of those commands, starting with adding color.

Adding Color to the Screen

SGB programs add color to the LCD screen area using two types of color commands. The first type sets the color of the SGB palettes, and the second type sets the attributes of the screen tiles. This section covers all the commands of both types.

Keep in mind that the screen can only access the first four palettes of the SGB (the other three are for the border). When I talk about palettes in this section, I'm referring specifically to these four palettes.

The Base Color Sample

The first sample of this section is the *sgb_color_base* sample, which displays a color image when run on the SGB. Figure 18-14 shows how it compares with the DMG.

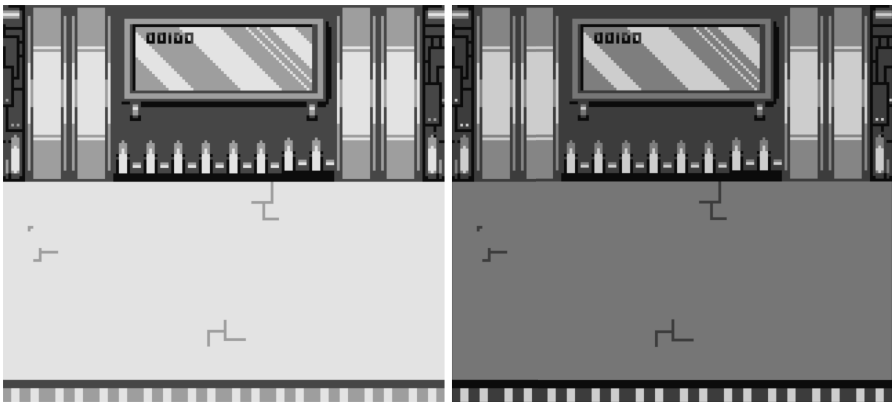


Figure 18-14: The base color command sample (DMG vs. SGB)

The SGB version uses four palettes to create a nice color variation of the DMG graphics.

The sample uses the `PAL01` and `PAL23` commands to set the color palettes. Table 18-6 shows the details of `PAL01`.

Table 18-6: The `PAL01` Command

Byte	Description	Value
00	Header	$01
01–02	Palette 0 color 0	Any valid color
03–04	Palette 0 color 1	Any valid color
05–06	Palette 0 color 2	Any valid color
07–08	Palette 0 color 3	Any valid color
09–0A	Palette 1 color 1	Any valid color
0B–0C	Palette 1 color 2	Any valid color
0D–0E	Palette 1 color 3	Any valid color
0F	Unused	0

The colors of palette 0 are right after the header, followed by the colors of palette 1, excluding the redundant shared color. The `PAL23`, `PAL03`, and `PAL12` commands are organized in the same way, affecting different palettes.

In code, the commands are built by including the PAL file from sgbconv, as shown in Listing 18-17.

```
SgbPacket_Palette01:
db PAL01
incbin "tech_sgb.pal", 0, 8
incbin "tech_sgb.pal", 10, 6
db PADDING

SgbPacket_Palette23:
db PAL23
incbin "tech_sgb.pal", 16, 8
incbin "tech_sgb.pal", 26, 6
db PADDING
```

Listing 18-17: The palette commands

Each palette of the PAL file has a copy of the shared color, and that color must be omitted for palettes 1 and 3 when constructing the commands. Fortunately, `incbin` has two optional parameters, a *start position* and a *length*, both expressed in bytes, that you can leverage to include just the necessary parts of the PAL file: all four colors from palettes 0 and 2, and three colors from palettes 1 and 3. In this case, the code skips color 0 from palette 1 (bytes 8 and 9 of the PAL file).

BGB displays the SGB palettes in its VRAM viewer, which is quite convenient when debugging color issues. Figure 18-15 shows the palettes for the *sgb_color_base* sample.

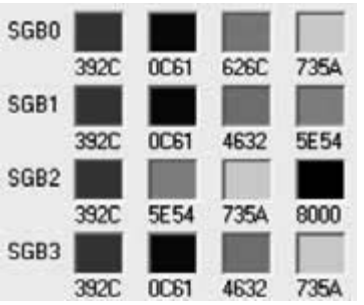


Figure 18-15: The SGB palettes in BGB

This sample uses the `ATTR_CHR` command to set the attributes of all the screen tiles. This command contains a list of attributes assigned to screen tiles, in order, from a given starting tile. Table 18-7 shows its first packet.

This packet contains a few parameters before the actual list of attributes. First, there is the header, with the command type and the number of packets in the command. Then come the X and Y positions of the starting tile. For a full screen, both would be set to zero to start from the top-left tile. Next, we have the number of attributes in the command. For a full screen's worth of attributes, this would be 360. The first packet can hold up to 40 attributes, and each subsequent packet can be filled with up to 64 attributes. As the `ATTR_CHR` command can contain up to six packets, the maximum

number of attributes is indeed 360 (40 + 5 × 64). Finally, it is possible to specify whether attributes are to be assigned by rows (horizontally) or by columns (vertically).

Table 18-7: The ATTR_CHR Command (First Packet)

Byte	Description	Value
00	Header	$38 + packet count (up to 6)
01	Starting X coordinate	0–19
02	Starting Y coordinate	0–17
03–04	Number of attributes	1–360
05	Write direction	0 = horizontal, 1 = vertical
06	Attributes 0–3	Any
07	Attributes 4–7	Any
…	…	…
0F	Attributes 36–39	Any

The sample sets all the screen tiles with a single ATTR_CHR command. Listing 18-18 shows the packets that do this.

```
SgbPacket_AttrChr:
db ATTR_CHR_6
db $00, $00 ; coordinates
dw $0168    ; number of indices
db ATTR_CHR_WRITE_HORIZONTAL
; palette indices
def ROW_BYTE_OFFSET = 0
rept 18
incbin "tech_sgb.atr", ROW_BYTE_OFFSET, 5
def ROW_BYTE_OFFSET += 8
endr
```

Listing 18-18: The ATTR_CHR packets

The six packets set the starting X and Y positions to zero and provide a full set of 360 attributes organized into rows, as this is how sgbconv exports them. However, the ATR file exported from sgbconv actually contains more than 360 attributes (20×18 tiles). The original PNG asset is 256×144 pixels, so it has 576 attributes (32×18) instead. The sample combines a repeat block with incbin to include only the parts of the ATR file that are necessary. The attributes for the last 12 tiles of each row are skipped, which amounts to 3 ignored bytes per row, as there are 4 attributes per byte. We end up with a complete ATTR_CHR command starting at label SgbPacket_AttrChr.

The PAL01, PAL23, and ATTR_CHR commands are sent to the SGB via the custom initialization function, shown in Listing 18-19.

```
PerformCustomInitialization:
    SendCommand SgbPacket_Palette01
    SendCommand SgbPacket_Palette23
    SendCommand SgbPacket_AttrChr

    ld hl, rLCDC
    res LCDCB_ON, [hl]
    CopyMemory _VRAM + (2 * TILES_PER_BLOCK * BYTES_PER_TILE), \
               graphics_tileset_sgb, \
               graphics_tileset_sgb_end - graphics_tileset_sgb
    CopyMemory _SCRN0, \
               graphics_tilemap_sgb, \
               graphics_tilemap_sgb_end - graphics_tilemap_sgb
    ld hl, rLCDC
    set LCDCB_ON, [hl]

    ret
```

Listing 18-19: The custom initialization function for the base color sample

This function also takes care of copying the tile data and tilemap to VRAM, because if this data were copied over before the custom initialization the screen would be frozen (via `MASK_EN`) before the palettes and attributes loaded. This would cause the image to display with the DMG palette for roughly a second before suddenly switching to color (after the mask is canceled). Because that effect is unpleasant, it's better to freeze on a blank screen (that is, with no image at all). We accomplish that by moving all data copies to VRAM inside the custom initialization function.

Note that we must disable the LCD before these data copies occur, as otherwise the VRAM might not be accessible. We do this by clearing bit 7 in the `rLCDC` register. The LCD-breaking bug mentioned in Chapter 6 does not apply here, so there is no need to wait for the vblank to disable the LCD.

Alternative Attribute Commands

The `ATTR_CHR` command is not the only way to set screen tile attributes. The `ATTR_BLK`, `ATTR_LIN`, and `ATTR_DIV` commands can also assign attributes, but they are more situational. We'll use the *sgb_color_index* sample to illustrate these three commands. In this sample, you can use the following keys to send different commands to the SGB:

- The RIGHT key sends an `ATTR_BLK` command.
- The LEFT key sends an `ATTR_LIN` command.
- The UP and DOWN keys send horizontal and vertical `ATTR_DIV` commands, respectively.
- The B button sends an `ATTR_CHR` command to clear the screen back to the default plain color.

The sample uses very bare-bones palettes with single colors, as shown in Listing 18-20.

```
SgbPacket_Palette01:
db PAL01
dw $0000, $FF7F, $0000, $0000 ; white
dw        $007C, $0000, $0000 ; blue
db PADDING

SgbPacket_Palette23:
db PAL23
dw $0000, $1F00, $0000, $0000 ; red
dw        $E003, $0000, $0000 ; green
db PADDING
```

Listing 18-20: The palettes for the sgb_color_index *sample*

This makes it easier to visualize the effects of the commands introduced in this section.

Let's check out the ATTR_BLK command, which is sent when we press the RIGHT key. Table 18-8 details the first packet of that command.

Table 18-8: The ATTR_BLK Command (First Packet)

Byte	Description	Value
00	Header	$20 + packet count (up to 7)
01	Number of blocks	1–18
02	Block 0 control	Bit 0: enable inside Bit 1: enable border Bit 2: enable outside Other: unused
03	Block 0 palettes	Bits 0–1: inside palette index Bits 2–3: border palette index Bits 4–5: outside palette index Other: unused
04	Block 0 left	1–19
05	Block 0 top	1–17
06	Block 0 right	(Value of left)–19
07	Block 0 bottom	(Value of top)–17
08–0D	Block 1	Same organization as block 0
0E–0F	Block 2	Same organization as block 0 (first 2 bytes only)

The ATTR_BLK command defines up to 18 rectangles, called *blocks*, to which palettes are assigned. Each block consists of three parts: a one-tile-wide border, an inside, and an outside. Each part can optionally be assigned a palette. A block is defined by 6 bytes. First, a control byte specifies which parts of the block get assigned a palette. Any combination of the three parts

is possible; for example, you can assign palettes to the border and inside of a block, but not to its outside. The second byte of a block holds the palette indices, which become the attributes of the screen tiles included in each part of the block. We only need to specify the palette indices corresponding to the parts enabled in the control byte. The last 4 bytes define the block's rectangular boundary by its top-left and bottom-right tile coordinates. Subsequent blocks follow the same format, sometimes going over packet boundaries; block 2, for example, starts in the first packet of the command and ends in the second one.

The right and bottom coordinates of a block should be superior to the left and top coordinates, respectively. Otherwise, the command will give unexpected results.

There are two additional behaviors to take into account for block borders. First, when only the inside or only the outside is enabled in the control byte, the border gets assigned the palette of that enabled part. Second, when both the inside and the outside are enabled but not the border, the command doesn't affect the border palette.

Listing 18-21 shows the `ATTR_BLK` command used in the sample.

```
SgbPacket_AttrBlk:
db ATTR_BLK_2
db 3 ; number of blocks
db ATTR_BLK_CONTROL_ALL
db ATTR_BLK_OUTSIDE_PALETTE_1 | ATTR_BLK_BORDER_PALETTE_2 | \
   ATTR_BLK_INSIDE_PALETTE_3
db 1, 1, 9, 9 ; block coordinates
db ATTR_BLK_CONTROL_INSIDE
db ATTR_BLK_INSIDE_PALETTE_3
db 2, 11, 8, 16 ; block coordinates
db ATTR_BLK_CONTROL_BORDER
db ATTR_BLK_BORDER_PALETTE_3
db 11, 1, 18, 16 ; block coordinates
ds 12, PADDING
```

Listing 18-21: The block command

This command defines three blocks spanning over two packets. It uses only 4 bytes of the second packet (for the last 4 bytes of the third block), so it fills the remaining 12 bytes with zero padding.

After sending the command, the sample screen looks like Figure 18-16.

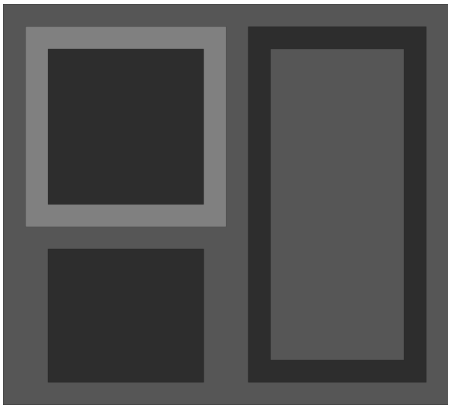


Figure 18-16: The sample showing the result of the ATTR_BLK command

This shows the three different blocks defined in Listing 18-21.

The sample also demonstrates the ATTR_LIN command, whose details are shown in Table 18-9.

Table 18-9: The ATTR_LIN Command (First Packet)

Byte	Description	Value
00	Header	$28 + packet count (up to 7)
01	Number of lines	1–110
02	Line 0	Bits 0–4: line number Bits 5–6: palette Bit 7: vertical line (0) or horizontal line (1)
03–0F	Lines 1–13	Same organization as line 0

This command assigns palettes per screen tile line. You can assign palettes to up to 110 lines in a single command, though it's unusual to need that many. For each line entry, the command specifies a palette index, a direction (vertical or horizontal), and a number. This number is a row number if the line is horizontal and a column number otherwise.

The sample uses a command with 22 lines, shown in Listing 18-22.

```
SgbPacket_AttrLin:
db ATTR_LIN_2
db 22 ; number of lines
; lines data
def ATTR_LINE_NUM = 0
rept 4
db ATTR_LIN_HORIZONTAL | ATTR_LIN_PALETTE_0 | ATTR_LINE_NUM + 0
db ATTR_LIN_HORIZONTAL | ATTR_LIN_PALETTE_1 | ATTR_LINE_NUM + 1
db ATTR_LIN_HORIZONTAL | ATTR_LIN_PALETTE_2 | ATTR_LINE_NUM + 2
db ATTR_LIN_HORIZONTAL | ATTR_LIN_PALETTE_3 | ATTR_LINE_NUM + 3
def ATTR_LINE_NUM += 4
```

```
endr
db ATTR_LIN_HORIZONTAL | ATTR_LIN_PALETTE_0 | ATTR_LINE_NUM + 0
db ATTR_LIN_HORIZONTAL | ATTR_LIN_PALETTE_1 | ATTR_LINE_NUM + 1
db ATTR_LIN_VERTICAL | ATTR_LIN_PALETTE_0 | 16
db ATTR_LIN_VERTICAL | ATTR_LIN_PALETTE_1 | 17
db ATTR_LIN_VERTICAL | ATTR_LIN_PALETTE_2 | 18
db ATTR_LIN_VERTICAL | ATTR_LIN_PALETTE_3 | 19
ds 4, PADDING
```

Listing 18-22: The line command

This command first assigns palettes to 18 tile rows, then sets up the rightmost 4 tile columns.

Figure 18-17 shows the result of the `ATTR_LIN` command.

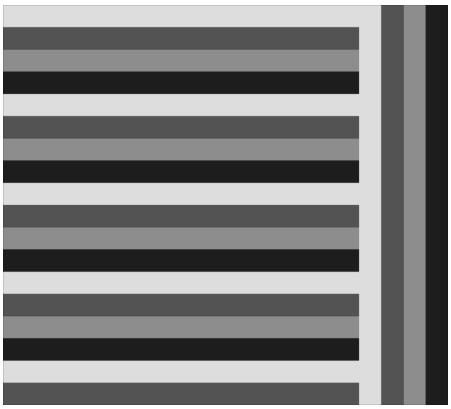


Figure 18-17: The sample showing the result of the `ATTR_LIN` command

The vertical lines, defined last in the command packets, override the effect of the horizontal lines that come before them.

The last alternative attribute command, `ATTR_DIV`, uses a single line to divide the screen into three parts. When the line is vertical, it defines a one-tile-wide border, a left region, and a right region. When the line is horizontal, it defines a one-tile-wide border, a top region, and a bottom region. The three parts are assigned palettes independently. Table 18-10 shows the details of the command.

Table 18-10: The `ATTR_DIV` Command

Byte	Description	Value
00	Header	$30
01	Control	Bits 0–1: palette index for the right/bottom region Bits 2–3: palette index for the left/top region Bits 4–5: palette index for the border Bit 6: vertical division (0) or horizontal division (1)
02	Line number	0–17 for a horizontal line 0–19 for a vertical line
03–0F	Unused	0

A control byte specifies the palette of each part and the direction of the line (horizontal or vertical). Then, the last byte holds the line number: a row number for a horizontal division and a column number for a vertical division.

The sample can send `ATTR_DIV` commands with both horizontal and vertical lines, as shown in Listing 18-23.

```
SgbPacket_AttrDiv_Vertical:
db ATTR_DIV
db ATTR_DIV_VERTICAL | ATTR_DIV_TOP_LEFT_PALETTE_1 | \
   ATTR_DIV_BORDER_PALETTE_2 | ATTR_DIV_BOTTOM_RIGHT_PALETTE_3
db 14 ; dividing column
ds 13, PADDING

SgbPacket_AttrDiv_Horizontal:
db ATTR_DIV
db ATTR_DIV_HORIZONTAL | ATTR_DIV_TOP_LEFT_PALETTE_1 | \
   ATTR_DIV_BORDER_PALETTE_2 | ATTR_DIV_BOTTOM_RIGHT_PALETTE_3
db 10 ; dividing row
ds 13, PADDING
```

Listing 18-23: The divide commands

They're basically the same command with the direction flipped and a different line number.

Figure 18-18 shows two screenshots of the sample, with the results of each `ATTR_DIV` command.



Figure 18-18: The sample showing the results of the ATTR_DIV commands

Pressing the DOWN key produces the result on the left and the UP key produces the result on the right.

As mentioned previously, usage of the alternatives to ATTR_CHR will be quite situational, as they require the screen tiles to be organized in a particular way. They can save some ROM space when the conditions are right, but the generic ATTR_CHR will apply to more situations.

Palette Commands with LCD Payload

Two commands, PAL_TRN and PAL_SET, can be used instead of PAL01, PAL23, PAL03, and PAL12. The PAL_TRN command first loads a bunch of palettes into the SNES WRAM using an LCD payload. Then, it activates four of these palettes with the PAL_SET command.

This section uses the *sgb_color_transfer_pal* sample to demonstrate how to use the PAL_TRN and PAL_SET commands. The PAL_TRN command consists of just a header, as shown in Table 18-11.

Table 18-11: The PAL_TRN Command

Byte	Description	Value
00	Header	$59
01–0F	Unused	0

This command leverages the LCD payload to transfer up to 512 palettes. Because each palette is 8 bytes, 512 palettes fill the whole available space of 4,096 bytes.

Listing 18-24 shows how the sample defines the command.

```
SgbPacket_PalTrn:
db PAL_TRN
ds 15, PADDING
```

Listing 18-24: The palette transfer command

Before sending the command, we must fill the VRAM with palettes and set a tilemap to make sure the palette data is sent to the LCD, as explained in “Payload” on page 328. Listing 18-25 shows this setup.

```
CopyMemory _VRAM, palettes_sgb, palettes_sgb_end - palettes_sgb
ld hl, _SCRN1
ld [hl], 0
inc hl
ld [hl], 1
```

Listing 18-25: Setting up the LCD payload for `PAL_TRN`

This code first copies the palettes to the base address of the VRAM (`$8000`), then sets up the tilemap. The sample uses the second tilemap (`_SCRN1`), because the first tilemap is already reserved for the sample's actual graphics. Note that only two indices are written in the tilemap. The sample has only four palettes, which add up to 32 bytes (4 palettes × 4 colors × 2 bytes per color). This means that all the palette data required for the sample fits inside two tiles (each tile being 16 bytes). We don't need to care about the values of the other indices in the tilemap because they don't correspond to any data the sample uses. However, note that the `PAL_TRN` command will still transfer 4KiB of data, as this is completely independent of the way the tilemap is set.

The palette data copied to VRAM is the PAL file itself, which is added to the ROM as shown in Listing 18-26.

```
palettes_sgb:
incbin "tech_sgb.pal"
palettes_sgb_end:
```

Listing 18-26: Including the palette data

There is no need to skip the shared color this time, like we did in Listing 18-17. Palettes are all considered independent of each other in this case, because the SGB allows programs to combine them in any order, as explained next.

Once the `PAL_TRN` command is processed, it stores the palettes in the SNES WRAM. However, the SGB can't access the palettes from there. You still need to explicitly set the SGB's four palettes via the `PAL_SET` command, whose details are shown in Table 18-12.

Table 18-12: The `PAL_SET` Command

Byte	Description	Value
00	Header	$51
01–02	Index of palette 0	0–511
03–04	Index of palette 1	0–511
05–06	Index of palette 2	0–511
07–08	Index of palette 3	0–511
09	Options	Bits 0–5: attribute file number (0–44) Bit 6: cancel mask if set Bit 7: use attribute screen number if set
0A–0F	Unused	0

The 8 bytes after the header are the indices of the 4 palettes (from among the up to 512 palettes in the SNES WRAM) used as SGB palettes. Byte `09` holds a bunch of interesting optimization options. Using bits 0 to 5 as well as bit 7, we can set an *attribute file* from the `PAL_SET` command. This is related to the `ATTR_TRN` command, which I'll explain shortly. Then, bit 6 can replace the cancel mask command (`MASK_EN` with action 0), typically sent after all SGB commands are completed. In other words, these options reduce the number of commands sent to the SGB.

Listing 18-27 shows the `PAL_SET` command used in the sample.

```
SgbPacket_PalSet:
db PAL_SET
dw $0000, $0001, $0002, $0003 ; palette indices
db PAL_SET_FLAG_NONE
ds 6, PADDING
```

Listing 18-27: The palette setting command

This just sets all four palettes from the SNES WRAM. In a more complex program, you can leverage the large number of palettes to easily switch between different color variations.

Finally, Listing 18-28 shows how the commands are sent in the sample.

```
    ld hl, rLCDC
    set LCDCB_BG9C00, [hl]
    set LCDCB_BG8000, [hl]
    SendCommand SgbPacket_PalTrn
    ld hl, rLCDC
    res LCDCB_BG9C00, [hl]
    res LCDCB_BG8000, [hl]
    SendCommand SgbPacket_PalSet
    SendCommand SgbPacket_AttrChr
```

Listing 18-28: Sending commands

Before sending the PAL_TRN command, the code enables the second tilemap, because that's where the tile indices meant for the LCD payload transfer are. It also switches the addressing mode to $8000, as all the palette data resides at the start of the VRAM, which is accessible only in that mode. Then, the PAL_SET command is sent to load the palettes into the SGB's memory. Finally, an ATTR_CHR command sets the attributes of the screen tiles.

In the sample, the two tiles containing the palette data sent via PAL_TRN are visible in BGB's VRAM viewer, as shown in Figure 18-19.

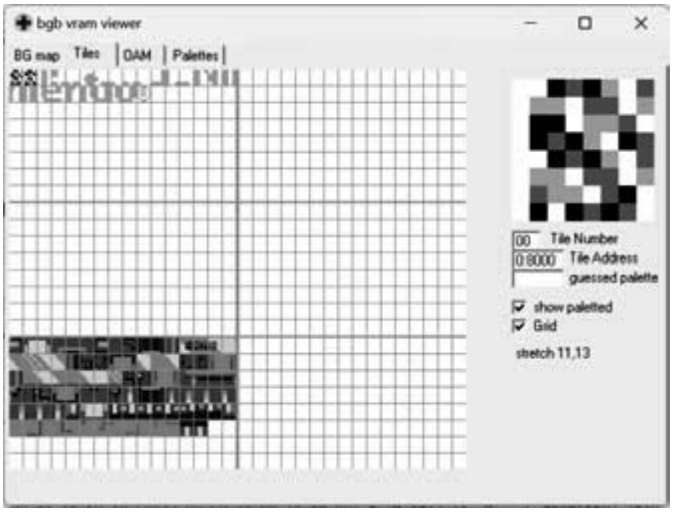


Figure 18-19: The PAL_TRN data tiles

The two tiles at the top left look like garbage data. That's the LCD payload from the PAL_TRN command. Because the sample does not use that part of VRAM for any other purpose, the data is still there when the sample is running (alongside part of the Nintendo logo). Though it's not necessary, you can, as an exercise, reconstruct the value of the original data from the color of those tiles.

Attribute Commands with LCD Payload

The ATTR_TRN and ATTR_SET commands are an alternative to using the ATTR_CHR command to set screen tile attributes. Attribute files are first loaded into the SNES WRAM with the ATTR_TRN command, using an LCD payload. Then, the ATTR_SET command selects one of the files.

In this section, we'll use the *sgb_color_transfer_index* sample to explore how to use the ATTR_TRN and ATTR_SET commands.

NOTE *The term* attribute file *does not refer to an actual file on disk. It really just means a bunch of attributes.*

The process is very similar to the PAL_TRN and PAL_SET method described in the previous section. The ATTR_TRN command leverages the LCD payload

to bring up to 45 attribute files into the SNES WRAM. An attribute file contains the attributes for 20×18 tiles, for a total size of 90 bytes (20 × 18 tiles / 4 attributes per byte). A full payload is 4,050 bytes (90 × 45), and the remaining 46 bytes are unused. After loading the attribute files into the SNES WRAM with the `ATTR_TRN` command, you can use a single `ATTR_SET` command to replace the LCD screen tile attributes with the attributes defined in any of those files.

Table 18-13 shows the details of the `ATTR_TRN` command.

Table 18-13: The `ATTR_TRN` Command

Byte	Description	Value
00	Header	$A9
01–0F	Unused	0

Like the `PAL_TRN` command, it consists of just a header with no packet data, because all the data associated with the command is in its LCD payload.

Listing 18-29 shows the command defined in the *sgb_color_transfer_index* sample.

```
SgbPacket_AttrTrn:
db ATTR_TRN
ds 15, PADDING
```

Listing 18-29: The attribute transfer command

The data meant for the LCD payload is included in the ROM as shown in Listing 18-30.

```
graphics_sgb_attr_chr:
def ROW_BYTE_OFFSET = 0
rept 18
incbin "tech_sgb.atr", ROW_BYTE_OFFSET, 5
def ROW_BYTE_OFFSET += 8
endr
graphics_sgb_attr_chr_end:
```

Listing 18-30: Including the attribute data

The code includes the LCD payload data from the ATR file (*tech_sgb.atr*) and processes it just like the `ATTR_CHR` command in Listing 18-18 (to remove the extra unnecessary attributes).

Listing 18-31 shows how to copy the attribute data to VRAM to form the LCD payload.

```
    CopyMemory _VRAM, \
               graphics_sgb_attr_chr, \
               graphics_sgb_attr_chr_end - graphics_sgb_attr_chr
    xor a
    ld hl, _SCRN1
    ld c, 6
    .scrn1_tilemap
        ld [hli], a
        inc a
        dec c
        jr nz, .scrn1_tilemap
```

Listing 18-31: Creating the LCD payload for ATTR_TRN

Like for PAL_TRN, the code puts the data at the base of the VRAM and sets up the second tilemap for the transfer. The sample has only one attribute file to transfer (90 bytes of data). With 16 bytes per tiles, 6 indices in the tilemap are enough to cover all the data ($16 \times 6 >= 90$).

After the ATTR_TRN command is processed, you can choose and apply any of the transferred attribute files with the ATTR_SET command. Table 18-14 shows the details of this command.

Table 18-14: The ATTR_SET Command

Byte	Description	Value
00	Header	$B1
01	Options	Bits 0–5: attribute file number Bit 6: cancel mask if set
02–0F	Unused	0

There is only a single byte used besides the header. It holds the attribute file number as well as a bit to cancel the mask. That bit works exactly like in PAL_SET and is meant as an optimization reducing the number of commands sent to the SGB.

Listing 18-32 shows the ATTR_SET command used in the sample.

```
SgbPacket_AttrSet:
db ATTR_SET
db ATTR_SET_FLAG_NONE | 0
ds 14, PADDING
```

Listing 18-32: The attribute setting command

This command sets the attribute file number to 0, because there is only one in the sample, and it does not leverage the cancel mask flag. Listing 18-33 shows how the sample sends the commands.

```
    SendCommand SgbPacket_Palette01
    SendCommand SgbPacket_Palette23
    ld hl, rLCDC
    set LCDCB_BG9C00, [hl]
    set LCDCB_BG8000, [hl]
    SendCommand SgbPacket_AttrTrn
    ld hl, rLCDC
    res LCDCB_BG9C00, [hl]
    res LCDCB_BG8000, [hl]
    SendCommand SgbPacket_AttrSet
```

Listing 18-33: Sending commands

The code enables the second tilemap and switches the addressing mode to $8000 before sending the `ATTR_TRN` command. This is very similar to the way the *sgb_color_transfer_pal* sample set up the `PAL_TRN` command and ensured the LCD payload was properly transferred.

Like for the *sgb_color_transfer_pal* sample, the LCD payload in the sample is visible in BGB's VRAM viewer, as shown in Figure 18-20.

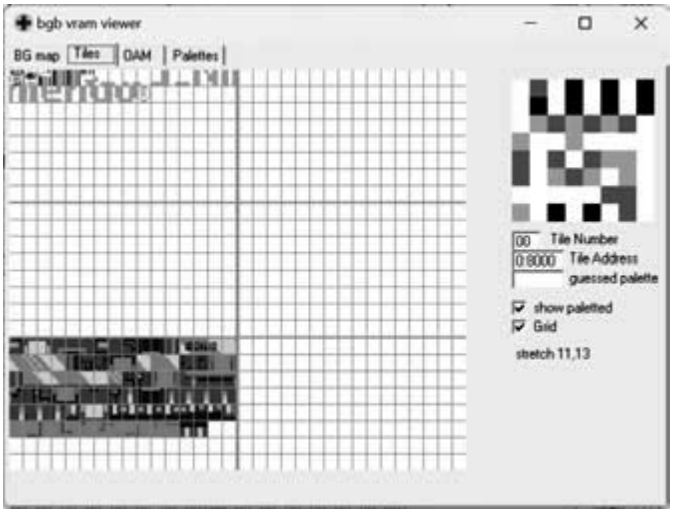


Figure 18-20: The `ATTR_TRN` data tiles

The payload covers six tiles, as expected.

Combination of LCD Payload Commands

Now that you know how to use the `PAL_TRN` and `ATTR_TRN` commands to transfer palettes and attributes into the SNES WRAM, let's see how we can set both the active palettes and the screen tile attributes with a single `PAL_SET` command. This will save us from having to use an extra `ATTR_SET` command to set the screen tile attributes. The *sgb_color_transfer_combined* sample demonstrates this technique.

Listing 18-34 shows how to set up the payload of the PAL_TRN and ATTR_TRN commands.

```
CopyMemory _VRAM, palettes_sgb, palettes_sgb_end - palettes_sgb
CopyMemory _VRAM + BYTES_PER_ATTRIBUTE_SCREEN, \
           graphics_sgb_attr_chr, \
           graphics_sgb_attr_chr_end - graphics_sgb_attr_chr
xor a
ld hl, _SCRN1
ld c, 12
.scrn1_tilemap
    ld [hli], a
    inc a
    dec c
    jr nz, .scrn1_tilemap
```

Listing 18-34: The combined payload

Both commands will use the same payload, and that requires a special organization of the data. First, the palettes are copied to the very start of the VRAM. From the PAL_SET command's perspective, the palette indices are 0 to 3, as expected. Then, the attribute file is copied with a 90-byte offset. From the ATTR_SET command's perspective, the number of that file is 1, because we skip file number 0 to give room for the palettes. Finally, we set up the tilemap indices. This time, the data extends up to the 180th byte in VRAM, so we must transfer 12 tiles (180 / 16 = 11.25).

Listing 18-35 shows the commands the sample uses.

```
SgbPacket_PalTrn:
db PAL_TRN
ds 15, PADDING

SgbPacket_AttrTrn:
db ATTR_TRN
ds 15, PADDING

SgbPacket_PalSet:
db PAL_SET
dw $0000, $0001, $0002, $0003
db PAL_SET_FLAG_SET_ATTR | 1
ds 6, PADDING
```

Listing 18-35: The combined commands

These commands are very similar to the ones we used in the two previous samples. However, this time, the sample does leverage the PAL_SET command's attribute file capability, mentioned in Table 18-12. That means this single PAL_SET command selects both palettes and screen tile attributes, and we don't need an ATTR_SET command in the sample. Listing 18-36 shows the transfer of the combined commands.

```
    ld hl, rLCDC
    set LCDCB_BG9C00, [hl]
    set LCDCB_BG8000, [hl]
    SendCommand SgbPacket_PalTrn
    SendCommand SgbPacket_AttrTrn
    ld hl, rLCDC
    res LCDCB_BG9C00, [hl]
    res LCDCB_BG8000, [hl]
    SendCommand SgbPacket_PalSet
```

Listing 18-36: Sending the combined commands

While both `PAL_TRN` and `ATTR_TRN` use the same payload in VRAM, this still transfers that payload twice, because they are two distinct commands with different destination addresses in the SNES WRAM for their respective payloads. In other words, the commands transfer the same payload to two different locations in the SNES WRAM, but because neither of the commands needs the whole 4KiB of data, the sample can conveniently set the VRAM only once.

The main takeaway of this sample is that there is a lot of flexibility around the LCD payload organization. There is also room to optimize the number of copies to VRAM and the number of commands sent to the SGB.

The main limitation of the SGB color palettes is that they apply only to screen tiles. They're great for displaying static images, like splash screens, with up to 13 distinct colors, but for anything with sprites or scrolling, you'll need to work around the restrictions.

Setting a Custom Border

Two commands, `CHR_TRN` and `PCT_TRN`, are dedicated to setting custom borders in SGB programs. They use LCD payloads to transfer all the data related to the border graphics. This section covers both commands in detail.

Foundation

The `CHR_TRN` command transfers the tile data used for the border graphics. Table 18-15 shows the details of this command.

Table 18-15: The `CHR_TRN` Command

Byte	Description	Value
00	Header	$99
01	Block number	0 or 1
02–0F	Unused	0

Besides the header, it is necessary to specify the tile block number. There are two blocks in the SNES VRAM, each containing up to 128 tiles, for a maximum of 256 tiles available for the border graphics. However, because

each tile is 32 bytes, an LCD payload can only hold up to 128 tiles (4,096 / 32). To take advantage of all 256 tiles, you need two `CHR_TRN` commands: one with the block number set to 0 (to transfer tiles `$00` to `$7F`) and one with the block number set to 1 (to transfer tiles `$80` to `$FF`).

The `PCT_TRN` command, shown in Table 18-16, transfers the tilemap and color palettes for the border.

Table 18-16: The `PCT_TRN` Command

Byte	Description	Value
00	Header	$A1
01–0F	Unused	0

You must organize the LCD payload of `PCT_TRN` in a specific way, as shown in Table 18-17.

Table 18-17: The `PCT_TRN` Payload

Bytes	Size (bytes)	Description	Value
000–6FF	1,792	Tilemap	32×28 tilemap entries
700–7FF	256	Unused	Any
800–819	32	Palette 4	16 BGR colors
820–839	32	Palette 5	16 BGR colors
840–859	32	Palette 6	16 BGR colors
860–FFF	1,952	Unused	Any

The tilemap is at the very start of the payload. Because a tilemap consists of 32×28 entries of 2 bytes each, it takes a total of 1,792 bytes. Then, the three 16-color palettes are at offset `$800` in the payload. The unused space between `$700` and `$7FF` corresponds to 4×28 unused tilemap entries. The SNES tilemaps are actually 32×32 entries, but for the border, only 32×28 entries are necessary, so the unused ones are ignored.

To demonstrate their use, we'll go over the *sgb_border_base* sample, which uses the three commands shown in Listing 18-37.

```
SgbPacket_ChrTrn_Block0:
db CHR_TRN, 0
ds 14, PADDING

SgbPacket_ChrTrn_Block1:
db CHR_TRN, 1
ds 14, PADDING

SgbPacket_PctTrn:
db PCT_TRN
ds 15, PADDING
```

Listing 18-37: The border commands

Two CHR_TRN commands transfer up to 256 (2 × 128) tiles to the SNES VRAM. One PCT_TRN command transfers the tilemap and palettes.

Listing 18-38 shows the setup of the payloads in VRAM and how the commands are sent.

```
; send the first block of border tile data
ld hl, rLCDC
res LCDCB_ON, [hl]
CopyMemory _VRAM, graphics_border_tileset, SNES_TILE_BLOCK_SIZE
ld hl, rLCDC
set LCDCB_ON, [hl]
SendCommand SgbPacket_ChrTrn_Block0

; send the second block of border tile data
ld hl, rLCDC
res LCDCB_ON, [hl]
CopyMemory _VRAM, \
           graphics_border_tileset + SNES_TILE_BLOCK_SIZE, \
           SNES_TILE_BLOCK_SIZE
ld hl, rLCDC
set LCDCB_ON, [hl]
SendCommand SgbPacket_ChrTrn_Block1

; send the border tilemap
ld hl, rLCDC
res LCDCB_ON, [hl]
CopyMemory _VRAM, \
           graphics_border_tilemap, \
           graphics_border_tilemap_end - graphics_border_tilemap
CopyMemory _VRAM + SNES_TILEMAP_SIZE, \
           graphics_border_pal, \
           graphics_border_pal_end - graphics_border_pal
ld hl, rLCDC
set LCDCB_ON, [hl]
SendCommand SgbPacket_PctTrn
```

Listing 18-38: Interleaved commands

Because all three commands use large LCD payloads, it is not possible to fit the data for all the commands in VRAM at the same time. As a result, the sample must send the commands sequentially. For each command, it turns off the LCD, loads its payload into VRAM, turns the LCD back on, and finally sends the command. This sequence repeats until all three commands are sent.

An important part of the sample is that the tilemap used for data transfer must be set for full 4KiB payloads. The sample has a function to do just that, shown in Listing 18-39.

```
_FillTransferTilemap:
    ld hl, rLCDC
    res LCDCB_ON, [hl]

    xor a
    ld hl, _SCRN1
    ld de, 12 ; columns to skip
    ld c, 13 ; rows
    .scrn1_tilemap
        ld b, 20 ; columns
        .scrn1_line
            ld [hli], a
            inc a
            dec b
            jr nz, .scrn1_line
        add hl, de
        dec c
        jr nz, .scrn1_tilemap

    ld hl, rLCDC
    set LCDCB_ON, [hl]
    ret
```

Listing 18-39: Setting the tilemap to send 4KiB worth of data

This is a direct implementation of the typical transfer tilemap organization from Figure 18-5.

Figure 18-21 shows a screenshot of the sample.



Figure 18-21: The base border sample

This looks the same as the original asset shown in Figure 18-9, but with the actual DMG program running in the LCD area.

The sample runs well enough in BGB, but on real hardware, there is a glitch. It doesn't happen in BGB because BGB does not fully emulate the way the border works on real hardware. In the emulator, the border appears suddenly, over the course of a single frame. On real hardware, a default border, which looks like the case of a DMG, displays first. After the border commands are sent, that default border fades out to black, and then the custom border fades in. The glitch occurs when the border is fully black. For about four frames, the LCD flickers into a black screen, even when properly masked. Next, I'll explain how you can work around that issue.

Glitch Workaround

It's not possible to fix the glitch, so programs must work around it. The *sgb_border_base_fixed* sample demonstrates a simple trick that uses the palettes and tile attributes to temporarily turn the entire LCD area black, making the flicker unnoticeable. Several commercial games, such as *Donkey Kong* and *Mole Mania*, use a similar approach.

Listing 18-40 shows the color commands of the *sgb_border_base_fixed* sample.

```
SgbPacket_Palette01_Black:
db PAL01
dw $0000, $0000, $0000, $0000
dw        $0000, $0000, $0000
db PADDING

SgbPacket_Palette01:
db PAL01
dw $6BFC, $3B11, $29A6, $1061
dw        $3B11, $29A6, $1061
db PADDING

SgbPacket_AttrBlk:
db ATTR_BLK_1
db 1 ; number of blocks
db ATTR_BLK_CONTROL_ALL
db ATTR_BLK_OUTSIDE_PALETTE_0 | ATTR_BLK_BORDER_PALETTE_0 | \
   ATTR_BLK_INSIDE_PALETTE_0
db 0, 0, 19, 17 ; block coordinates
ds 8, PADDING
```

Listing 18-40: The color commands

The first command sets palettes 0 and 1 to full black (although palette 1 doesn't really matter, because the sample uses only one palette). The next command sets those same palettes to colors the program uses. The last command sets all the screen tile attributes to palette 0.

Listing 18-41 shows the new SGB initialization code.

```
InitializeSgbDevice:
    WaitFrames 60

    SendCommand SgbPacket_Palette01_Black
    SendCommand SgbPacket_AttrBlk

    copy [rLCDC], LCDCF_ON | LCDCF_BG8800 | LCDCF_BG9800 | LCDCF_BGON
    WaitFrames 2

    SendCommand SgbPacket_MaskEn_Freeze

def COMMAND_INDEX = 0
rept ((SgbPackets_CoreInit_End - SgbPackets_CoreInit) / SGB_BYTES_PER_PACKET)
    SendCommand SgbPackets_CoreInit + COMMAND_INDEX
def COMMAND_INDEX += SGB_BYTES_PER_PACKET
endr

    call PerformCustomInitialization

    WaitFrames 1
    SendCommand SgbPacket_MaskEn_Cancel

    WaitFrames 128

    SendCommand SgbPacket_Palette01

    ret
```

Listing 18-41: The fixed SGB initialization

After the mandatory 60-frame wait, this sends the black palette and tile attribute commands, making the whole LCD screen area black. Then, it turns on the LCD (this line was moved from `InitSample_SGB`) and waits two frames to make sure the SNES VRAM properly updates before freezing. As soon as the mask cancels, the SNES starts the fade animation of the border. The sample waits 128 frames, roughly the length of that animation. Finally, it sends the program palette command.

The two mask commands aren't really necessary in this sample. Because the screen tile attributes are set to use palette 0, which is fully black, when the graphics are displayed only black appears on the screen (even though the sample still writes four colors to the LCD).

The main takeaway here is that there are various ways to perform the SGB initialization. Many games use significantly different variations of what Nintendo originally recommended. The only truly necessary parts are to

wait 60 frames at the start and to send the mandatory `DATA_SND` commands. Make sure to always test on real hardware when using modified initialization sequences, or stick to the steps outlined in this chapter if you don't have access to the hardware.

More Border Samples

Here, I'll use the *sgb_border_overlap* and *sgb_border_fullscreen* samples to demonstrate a few simple techniques involving the border.

First, *sgb_border_overlap* shows that the border graphics can overlap the LCD area. As explained earlier, a border is just a 256×244-pixel image with a hole in the middle. It's possible to fill the hole with non-transparent pixels, partially hiding the LCD area.

The second sample, *sgb_border_fullscreen*, shows that you can easily create a single full-screen image by combining the LCD and the border graphics. As long as the palette colors match, it's possible to create nice splash screens this way.

Figure 18-22 shows screenshots of both samples.



Figure 18-22: The overlapping border and full screen samples

The code for these two samples isn't too different from the *sgb_border_base* sample, so I won't replicate it here. As always, you can find the complete code in the GitHub repository.

Unlocking Multiplayer Options

SGB programs can leverage the `MLT_REQ` command to access the input of up to four SNES joypads, enabling multiplayer options. Table 18-18 shows the details of this command.

Table 18-18: The `MLT_REQ` Command

Byte	Description	Value
00	Header	$89
01	Control	0 = one joypad 1= two joypads 3 = four joypads
02–0F	Unused	0

A control byte specifies how many joypads to enable. By default, only one joypad is enabled, but it's possible to enable two or four joypads.

No matter how many joypads we enable, they must all be read through the `rP1` register, the same way the DMG joypad is read. The SGB selects the next joypad to be read each time input polling is disabled, which happens when `P1F_GET_NONE` is written to `rP1`. While input polling is disabled, the value of the low nibble of `rP1` indicates which joypad is going to be read next. For joypads 0, 1, 2, and 3, the nibble will equal `$F`, `$E`, `$D`, and `$C`, respectively. However, because the SGB cycles through the enabled joypads in order, it's usually simpler to make sure that all the enabled joypads are read exactly once each frame. This avoids the need to check the low nibble of `rP1` at all.

The *sgb_multi* sample we'll go over in this section cycles between one, two, and four joypads when you press the SELECT key of the first joypad. The sample sends an appropriate `MLT_REQ` command depending on the number of joypads you want to enable, as shown in Listing 18-42.

```
macro SgbPacket_MltReq
db MLT_REQ, \1
ds 14, PADDING
endm

SgbPacket_OneJoypad:
    SgbPacket_MltReq MLT_REQ_ONE_JOYPAD

SgbPacket_TwoJoypads:
    SgbPacket_MltReq MLT_REQ_TWO_JOYPADS

SgbPacket_FourJoypads:
    SgbPacket_MltReq MLT_REQ_FOUR_JOYPADS
```

Listing 18-42: The `MLT_REQ` commands

The code defines three commands, one for each number of enabled joypads.

Listing 18-43 shows the `SwitchJoypadsCount` function, which sends the `MLT_REQ` commands from Listing 18-42.

```
SwitchJoypadsCount:
    ld a, [WRAM_JOYPADS_COUNT]

    cp a, 1
```

```
    jr nz, .switch_to_2_joypads
        copy [WRAM_JOYPADS_COUNT], 2
        SendCommand SgbPacket_TwoJoypads
        ret
    .switch_to_2_joypads

    cp a, 2
    jr nz, .switch_to_4_joypads
        copy [WRAM_JOYPADS_COUNT], 4
        SendCommand SgbPacket_FourJoypads
        ret
    .switch_to_4_joypads

    copy [WRAM_JOYPADS_COUNT], 1
    SendCommand SgbPacket_OneJoypad
    ret
```

Listing 18-43: Switching the number of enabled joypads

The command sent when the `SwitchJoypadsCount` function is called depends on a variable tracking the number of currently enabled joypads. For example, when two joypads are enabled, the function sends the command to enable four joypads.

Listing 18-44 shows the `UpdatePad_All` function, which reads the joypads' input.

```
UpdatePad_All:
    ld a, [WRAM_JOYPADS_COUNT]
    cp a, 1
    jr z, .one_joypad
    cp a, 2
    jr z, .two_joypads

    call UpdatePad_0
    call UpdatePad_1
    call UpdatePad_2
    call UpdatePad_3
    ret

    .two_joypads
    call UpdatePad_0
    call UpdatePad_1
    ret

    .one_joypad
    call UpdatePad_0
    ret
```

Listing 18-44: Reading the input from the joypads

The `UpdatePad_All` function also relies on the variable tracking the number of enabled joypads. It reads all the enabled joypads to ensure that the SGB cycles through all of them within the frame and that it always selects the first joypad at the start of any given frame.

Again, you could check the low nibble of the `rP1` register when input polling is disabled to see which joypad will be read next, but that method is usually quite inconvenient. Even in the case where a program needs only three joypads, it should always read the fourth joypad (because it is not possible to enable just three) to complete a full cycle in each frame.

Figure 18-23 shows what the sample looks like with four joypads enabled and various keys pressed.



Figure 18-23: The multi-joypad sample

You can configure keyboard keys to emulate all four joypads in the Joypad tab of BGB's control panel, which is very convenient for testing multi-joypad code. Each joypad requires 8 keys, for a total of 32 keys for all four joypads.

Playing Preloaded Sounds

The SGB has preloaded sound effects that leverage the SNES's sound capabilities. It can play up to two of those sound effects at the same time. The only limitation is that each must come from a different one of the two sound effect lists, named A and B.

List A, shown in Table 18-19, contains 48 fire-and-forget sound effects that play once and stop, like a character jump sound.

Table 18-19: Sound Effect List A

Code	Description	Frequency	Code	Description	Frequency
01	Nintendo	3	19	Rocket firing	2
02	Game over	3	1A	Rocket landing	2
03	Falling	3	1B	Cup breaking	2
04	OK 0	3	1C	Glass breaking	3
05	OK 1	3	1D	Level up	2
06	Select 0	3	1E	Air injection	3
07	Select 1	3	1F	Sword wielding	3
08	Select 2	2	20	Falling in water	2
09	Error	2	21	Fire	3
0A	Item catch	2	22	Breaking wall	3
0B	Door squeak	2	23	Cancellation	2
0C	Explosion small	3	24	Stepping	3
0D	Explosion medium	3	25	Block hitting	3
0E	Explosion large	3	26	Picture floating	3
0F	Defeat 0	3	27	Screen fade-in	2
10	Defeat 1	3	28	Screen fade-out	2
11	Strike 0	2	29	Window opening	3
12	Strike 1	2	2A	Window closing	2
13	Air sucking	2	2B	Large laser	3
14	Rocket launcher 0	3	2C	Stone door	2
15	Rocket launcher 1	3	2D	Teleportation	3
16	Bubbling	2	2E	Thunder	2
17	Jump	3	2F	Earthquake	2
18	Fast jump	3	30	Small laser	2

List B, shown in Table 18-20, includes 25 sound effects. These are mostly loops, though there are a few fire-and-forget effects as well.

Table 18-20: Sound Effect List B

Code	Description	Loop	Frequency
01	Applause small	Y	2
02	Applause medium	Y	2
03	Applause large	Y	2
04	Wind	Y	1
05	Rain	Y	1
06	Storm	Y	1
07	Hurricane	Y	2
08	Thunder	Y	0
09	Earthquake	Y	0
0A	Lava flow	Y	0
0B	Wave	Y	0
0C	River	Y	3
0D	Waterfall	Y	2
0E	Character running	Y	3
0F	Horse galloping	Y	3
10	Warning sound	Y	1
11	Futuristic car	Y	0
12	Jet flying	Y	1
13	UFO flying	Y	2
14	Electromagnetic waves	Y	0
15	Score up	Y	3
16	Fire	Y	2
17	Camera shutter	N	0
18	Writing	N	0
19	Erasing	N	0

Table 18-21 shows details of the SOUND command used to control the sound effects.

Table 18-21: The SOUND Command

Byte	Description	Value
00	Header	$41
01	Port 1 (effect A)	$00: retrigger effect $01–$30: set and play effect $80: stop effect
02	Port 2 (effect B)	$00: retrigger effect $01–$19: set and play effect $80: stop effect
03	Parameters	Bit 0–1: port 1 frequency scale Bits 2–3: port 1 volume level (3 = mute) Bits 4–5: port 2 frequency scale Bits 6–7: port 2 volume level
04	BGM port	$00 or $80 when unused
05–0F	Unused	0

Two bytes are dedicated to *ports*, which schedule sound effects on underlying audio channels. Port 1 lets programs play effects from List A, while port 2 is for effects from List B. You can use both ports at the same time. You can set a port to any sound effect code from the corresponding list, which triggers the playback of the sound effect.

The next byte holds the sound effect parameters for the ports. There are two parameters for each port: a frequency scale, which controls the pitch of the sound, and a volume level. The frequency scales range from 0 (low frequency) to 3 (high frequency). As you saw in Tables 18-19 and 18-20, each sound effect has a frequency associated with it by default, which is most likely the originally intended frequency. The volume level ranges from 0 (high) to 2 (low).

The last byte of the command is the background music (BGM) port. Using the BGM port requires some SNES audio programming knowledge, so I won't cover it in detail.

Two special values are available for each port. First, you can use a value of $80 instead of a list code to immediately stop a sound effect. This value is also convenient for unused ports. Second, a value of $00 *retriggers* the currently playing sound effect, which allows you to change the playback parameters of that sound effect. For example, assuming a loop is playing on port 2 with frequency scale 3 and volume 2, it is possible to retrigger it to play at frequency scale 1 and volume 0. The sound effect doesn't restart when it's retriggered; the new parameters simply apply to the sound while it keeps playing.

The volume level for port 1 also has a special value that you should be aware of. Setting the volume level for this port to 3, or *mute*, allows you to do

fade-ins and fade-outs. Even though it's set through port 1's volume level, mute affects all three ports. When you send a command with mute enabled (that is, with port 1's volume level set to 3), any sound playing fades out. The SGB then enters mute mode, and ports 1 and 2 do not output any sound until port 1's volume level is set back to any value other than 3 by another command. When the SGB is in mute mode, the next sound fades in as the playback starts, and the SGB exits mute mode. It starts up in mute mode by default, and and ports 1 and 2 don't produce any sound until you send a command that takes it out of that mode.

To fade in a sound, send a command with all ports at `$80` and port 1's volume level at 3 (to ensure you're in mute mode), then send the actual command to play sound with port 1's volume level set to any value other than 3. To fade out a sound, send a command to start a sound, then send a command with port 1's volume level at 3 and all ports set to `$00` (retrigger).

Before moving on to the next sample, keep in mind that BGB does *not* emulate the `SOUND` command. Sending this command has no effect when the program runs in the emulator, which won't play any sound at all. You'll need real hardware to try out the `SOUND` command.

The *sgb_sound_test* sample is a sound test program that can play any of the SGB's preloaded sound effects. This is a good way to experiment with those sound effects, and it's especially handy for understanding how retrigger and mute work so that you can implement on-the-fly parameter changes and fades properly in your programs. When run in BGB, the sample looks like Figure 18-24.



Figure 18-24: The sound test sample

The A button sends a `SOUND` command with the parameters chosen via the program's UI. The `$00` and `$80` values mean retrigger and stop, respectively.

The sample uses a dynamically built `SOUND` command, because there are way too many possible `SOUND` commands to keep them all in the ROM. Listing 18-45 shows how the WRAM reserves the necessary space for the command.

```
def WRAM_SOUND_COMMAND  rb SGB_BYTES_PER_PACKET
```

Listing 18-45: Reserving space for the SOUND command

As the SOUND command only ever has a single packet, the size of the reserved space is 16 bytes (SGB_BYTES_PER_PACKET).

Listing 18-46 shows the function that builds and sends the SOUND command each time you press the A button.

```
PlaySound:
    ld hl, WRAM_SOUND_COMMAND
    copy [hli], SOUND
    copy [hli], [WRAM_EFFECT_A_ID]
    copy [hli], [WRAM_EFFECT_B_ID]

    ld a, [WRAM_EFFECT_B_VOLUME]
    and a, $03
    sla a
    sla a
    swap a
    ld b, a

    ld a, [WRAM_EFFECT_B_PITCH]
    and a, $03
    swap a
    or a, b
    ld b, a

    ld a, [WRAM_EFFECT_A_VOLUME]
    and a, $03
    sla a
    sla a
    or a, b
    ld b, a

    ld a, [WRAM_EFFECT_A_PITCH]
    and a, $03
    or a, b
    ld [hli], a

    copy [hli], [WRAM_BGM_ID]

    SendCommand WRAM_SOUND_COMMAND
    ret
```

Listing 18-46: Playing a preloaded sound

The PlaySound function uses variables stored in WRAM to build the SOUND command dynamically. Those variables are the port, pitch, and volume of

sound effects A and B. There is also one additional variable for the BGM port. Once the command is built, the `PlaySound` function sends it to the SGB.

Controlling the SGB Settings

The SGB has several pieces of functionality beyond what it exposes to programs through commands. The Settings commands interact with two of these: the built-in menu and the attraction mode.

The *built-in menu* allows users to change the color palette used for the LCD area, remap the joypad buttons, and use predefined borders, among other things. Users can open the built-in menu at any time by pressing both shoulder buttons on the SNES joypad (L and R) simultaneously. Figure 18-25 shows that menu (located at the bottom of the screen).



Figure 18-25: The SGB's built-in menu

The *attraction mode* is the equivalent of a screensaver, meant to avoid screen burn-in on CRT monitors. The SGB enters attraction mode after about 7 minutes of joypad inactivity. The user can also trigger this mode manually by entering L, L, L, L, and R with the shoulder buttons. (Note that it's not available with all borders; for example, it won't trigger when the Game Boy border or the black border is used.)

The `ICON_EN` command, shown in Table 18-22, lets programs disable three of the SGB's functionalities in the built-in menu: the palette menus, the joypad menu, and the commands themselves.

Table 18-22: The `ICON_EN` Command

Byte	Description	Value
00	Header	$71
01	Flags	Bit 0: disable palette menus
		Bit 1: disable joypad menu
		Bit 2: disable commands
02–0F	Unused	0

Disabling the palette menus might be important if the program relies on some elements being of a particular color. For example, a user should prob-

ably not be allowed to change the palette if red objects have a specific meaning in a game. Likewise, a program that instructs the user to press specific buttons might want to block the joypad menu to avoid any button remapping. Those cases are likely to be rare, though, because the program will still need to be able to run on a regular DMG with fixed colors and buttons. You can enable both menus again by sending another `ICON_EN` command with the flags cleared.

Figure 18-26 shows what the built-in menu looks like after disabling the palette menus (left) or the joypad menu (right).

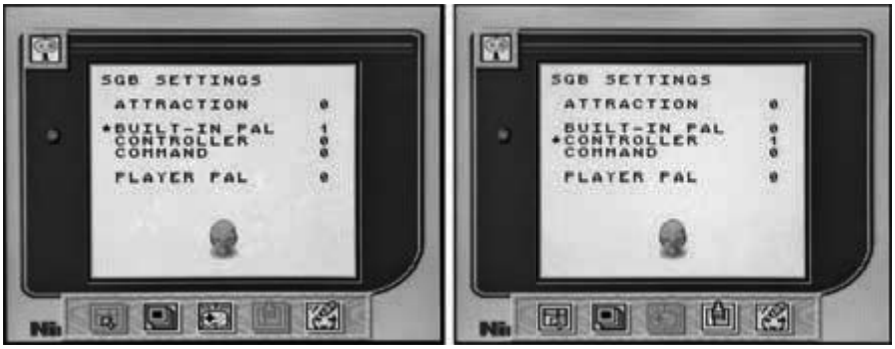


Figure 18-26: The SGB menu with palette menus and the joypad menu disabled

The disabled menus are faded out to clearly distinguish them from active menus.

Disabling commands is an irreversible action. The SGB ignores all subsequent commands, and the SNES must be reset before you can send commands again. It's not exactly clear what the benefits are of doing this rather than simply not sending any more commands, but if a program doesn't need to send any additional commands this option might make sense.

The `ATRC_EN` command, shown in Table 18-23, controls enabling of the attraction mode.

Table 18-23: The `ATRC_EN` Command

Byte	Description	Value
00	Header	$61
01	Enable flag	0 = enable, 1 = disable
02–0F	Unused	0

When the attraction mode is disabled, the SGB never enters that mode. The attraction mode is unlikely to be problematic in most cases, but it might cause glitches when a program uses custom SNES code, in which case the `ATRC_EN` command comes in handy.

Table 18-24 shows the last command in the Settings category, `PAL_PRI`.

Table 18-24: The `PAL_PRI` Command

Byte	Description	Value
00	Header	$C9
01	Priority	0 = user choice, 1 = program
02–0F	Unused	0

This command specifies whether the palette commands the program sends should override the palette the user defines through the SGB's built-in menu. When the priority is set to 0 (the default), the SGB ignores all palette-related commands after the user has set a custom palette.

As with the `SOUND` command, BGB does not emulate the commands described in this section. This makes sense, because these commands influence functionalities that run on the SNES. You'll need real hardware to see the commands in action.

The *sgb_settings* sample is dedicated to the Settings commands. Its code is pretty straightforward and does not feature anything worth replicating in this book, but it's still a convenient way to try out these commands before implementing them in a program (especially the `PAL_PRI` command, which can be rather unintuitive).

Sending Data for the SNES

In this section, we'll take a quick look at the commands in the Data category. As mentioned earlier, these commands require SNES programming knowledge, so I won't cover them fully in this book. Their main goal is to allow programs to access the SNES CPU and APU.

The `DATA_SND` command was already presented in Table 18-5, because this command is used for device initialization. It sends data to the SNES WRAM. A sibling command, `DATA_TRN`, has the same effect, but it uses an LCD payload instead of a packet payload. Table 18-25 shows its details.

Table 18-25: The `DATA_TRN` Command

Byte	Description	Value
00	Header	$89
01	Destination address (low byte)	$1800–$1FFF for bank $00 $B000–$BFFF for bank $7E $0000–$FFFF for bank $7F Any other address requires consideration before use.
02	Destination address (high byte)	Same as above
03	Bank number	Same as above
04–0F	Unused	0

The `JUMP` command, shown in Table 18-26, executes programs transferred to the SNES WRAM using `DATA_SND` or `DATA_TRN`.

Table 18-26: The JUMP Command

Byte	Description	Value
00	Header	$91
01	Code address (low byte)	A sensible address in WRAM
02	Code address (high byte)	A sensible address in WRAM
03	Bank number	A sensible bank number
04	NMI address (low byte)	$0000 in bank 0 Otherwise, a sensible address in WRAM for the default behavior
05	NMI address (high byte)	—
06	Bank number	—
07–0F	Unused	0

In the JUMP command, it's possible to set the program counter of the SNES as well as the non-maskable interrupt (NMI), which is the vblank interrupt in SNES jargon. The details of the command will only make sense if you understand SNES programming; otherwise, you can safely ignore it.

The SOU_TRN command sends data to the APU of the SNES. It can also run sound programs leveraging the SNES hardware. Table 18-27 shows its format.

Table 18-27: The SOU_TRN Command

Byte	Description	Value
00	Header	$49
01–0F	Unused	0

The LCD payload of the SOU_TRN command has a well-defined format with a variable number of data bytes, as shown in Table 18-28.

Table 18-28: The SOU_TRN Payload

Byte	Description	Value
000–001	Number of data bytes	1–4088
002–003	Destination address	$0400–$2AFF for code $2B00–$4AFF for music data $4DB0–$EEFF for sampling data
004–NNN	Data bytes	Any
NNN+1–NNN+2	Sentinel	$0000
NNN+3–NNN+4	Jump address	$0400
NNN+5–FFF	Unused	0

Like the JUMP command, the SOU_TRN command requires SNES programming skills. Unless you know about SNES audio programming, you can safely ignore that command.

Summary

In this chapter, you learned about all the features of the SGB, such as its color graphics, preloaded sounds, and multiplayer options. You should now have enough understanding of each available command to control the SGB and leverage its functionalities. You also learned the practical aspects of sending those commands from your programs, from defining the commands in your code to implementing the low-level serial protocol to communicate with the SGB.

The SGB is a bit peculiar, and debugging can be challenging because BGB offers only limited support for it. But once you have routines in place to send commands, enhancing programs is relatively straightforward.

Game Boy Coding Adventure (Early Access) © 2025 by Maximilien Dagois

19

GOING FURTHER

Congratulations! You've reached the end of the book, which covered programming the original Game Boy released in 1989 as well as the Game Boy Color, the Game Boy Printer, and the Super Game Boy.

You learned about all the hardware features and registers of the Game Boy and its revisions. You should now have all the necessary knowledge to create your own Game Boy programs, and you can use the many code samples in the book and GitHub repository as references when implementing your own ideas.

This is only the beginning of your journey! Check out the following online documentation to discover hardware, assembly language, or toolchain options that were not covered in this book:

- Pan Docs, a comprehensive Game Boy programming reference at *https://gbdev.io/pandocs/*
- The documentation of the RGBDS toolchain at *https://rgbds.gbdev.io/docs/master/*
- The manual of the BGB emulator at *https://bgb.bircd.org/manual.html*

I highly recommend that you try creating your own programs. The best way to start is to implement a simple puzzle game, like *Sokoban*, or an early

arcade game, such as *Pong*. Programming games will strengthen your assembly skills, and you can then move on to writing more ambitious programs.

To close our study of the Game Boy, Table 19-1 links locations in the memory map to chapters of the book. You can use this table to quickly jump to the relevant chapter when you need a refresher on any given topic.

Table 19-1: Memory Map to Chapter Reference

Start	End	Size (bytes)	Usage	Section
$0000	$003F	64	Restart vectors	Chapter 2
$0040	$0047	8	Vblank interrupt function	Chapter 7
$0048	$004F	8	LCD interrupt function	Chapter 12
$0050	$0057	8	Timer interrupt function	Chapter 10
$0058	$005F	8	Serial interrupt function	Chapter 11
$0060	$0067	8	Joypad interrupt function	Chapter 8
$0068	$00FF	152	Free ROM space	—
$0100	$014F	80	Cartridge header	Chapter 3
$0150	$7FFF	32,432	Free ROM space	—
$8000	$97FF	6,144	6KiB VRAM (384 tiles)	Chapter 5
$9800	$9BFF	1,024	1KiB VRAM (tilemap #0)	Chapter 5
$9C00	$9FFF	1,024	1KiB VRAM (tilemap #1)	Chapter 5
$A000	$BFFF	8,192	8KiB ERAM	Chapter 13
$C000	$DFFF	8,192	8KiB WRAM	Chapter 1
$E000	$FDFF	7,680	—	—
$FE00	$FE9F	160	OAM (40 sprites)	Chapter 7 Chapter 16
$FEA0	$FEFF	96	—	—
$FF00	$FF00	1	Joypad status (rP1)	Chapter 8
$FF01	$FF01	1	Serial buffer (rSB)	Chapter 11
$FF02	$FF02	1	Serial control (rSC)	Chapter 11
$FF03	$FF03	1	—	—
$FF04	$FF04	1	Divider (rDIV)	Chapter 10
$FF05	$FF05	1	Timer counter (rTIMA)	Chapter 10
$FF06	$FF06	1	Timer modulo (rTMA)	Chapter 10
$FF07	$FF07	1	Timer control (rTAC)	Chapter 10
$FF08	$FF09	2	—	—
$FF10	$FF14	5	Channel 1 registers (rNR10–rNR14)	Chapter 9
$FF15	$FF15	1	—	—
$FF16	$FF19	4	Channel 2 registers (rNR21–rNR14)	Chapter 9
$FF1A	$FF1E	5	Channel 3 registers (rNR30–rNR34)	Chapter 9
$FF1F	$FF1F	1	—	—

$FF20	$FF23	4	Channel 3 registers (rNR41–rNR44)	Chapter 9
$FF24	$FF26	3	Audio control registers (rNR50–rNR52)	Chapter 9
$FF27	$FF2F	9	—	—
$FF30	$FF3F	16	Wave RAM	Chapter 9
$FF40	$FF40	1	LCD control (rLCDC)	Chapter 6 Chapter 16
$FF41	$FF41	1	LCD status (rSTAT)	Chapter 12
$FF42	$FF43	2	Background scrolling Y/X (rSCY/rSCX)	Chapter 7
$FF44	$FF44	1	Line number (rLY)	Chapter 12
$FF45	$FF45	1	LY compare (rLYC)	Chapter 12
$FF46	$FF46	1	DMA (rDMA)	Chapter 12
$FF47	$FF47	1	Background palette (rBGP)	Chapter 7
$FF48	$FF49	2	Sprite palettes (rOBP0 and rOBP1)	Chapter 7
$FF4A	$FF4B	2	Window scrolling Y/X (rWY/rWX)	Chapter 7
$FF4C	$FF4C	1	—	—
$FF4D	$FF4D	1	CPU speed mode control (rKEY1)	Chapter 16
$FF4E	$FF4E	1	—	—
$FF4F	$FF4F	1	VRAM bank (rVBK)	Chapter 16
$FF50	$FF50	1	Boot ROM disable flag	Chapter 3
$FF51	$FF55	5	VRAM DMA (rHDMA1 to rHDMA5)	Chapter 16
$FF56	$FF56	1	Infrared reception port (rRP)	Chapter 16
$FF57	$FF67	16	—	—
$FF68	$FF68	1	Background color palette specification (rBCPS)	Chapter 16
$FF69	$FF69	1	Background color palette data (rBCPD)	Chapter 16
$FF6A	$FF6A	1	Object color palette specification (rOCPS)	Chapter 16
$FF6B	$FF6B	1	Object color palette data (rOCPD)	Chapter 16
$FF6C	$FF6F	4	—	—
$FF70	$FF70	1	WRAM bank (rSVBK)	Chapter 16
$FF71	$FF7F	15	—	—
$FF80	$FFFE	127	HRAM	Chapter 1
$FFFF	$FFFF	1	Interrupt enable register (rIE)	Chapter 7

Game Boy Coding Adventure (Early Access) © 2025 by Maximilien Dagois

A

INSTRUCTIONS

This appendix provides a list of assembly instructions used in this book for quick reference (Table A-3). Table A-1 summarizes the parameter nomenclature of the instructions described in that table.

Table A-1: Legend for Instruction Table

Operand	Description
`n8`	An 8-bit integer constant
`n16`	A 16-bit integer constant
`s8`	An 8-bit signed integer (–128 to 127)
`r8`	Any of the 8-bit registers: `a`, `b`, `c`, `d`, `e`, `h`, and `l`
`r16`	Any of the general-purpose 16-bit registers: `bc`, `de`, and `hl`
`u3`	A 3-bit unsigned integer constant (0 to 7)
`v8`	Any of the restart vectors: `$00`, `$08`, `$10`, `$18`, `$20`, `$28`, `$30`, and `$38`
`cnd`	Any of the conditions: `z`, `nz`, `c`, and `nc`
`high(`*nnn*`)`	The most significant byte of *nnn* (a 16-bit register or address)
`low(`*nnn*`)`	The least significant byte of *nnn* (a 16-bit register or address)

Table A-2 is a reminder of what the conditions mean in relation to the flags in the `f` register.

Table A-2: Meaning of the Conditions

Condition	f flag test
z	Z == 1
nz	Z == 0
c	C == 1
nc	C == 0

Table A-3 lists the instructions available on the DMG, CGB, and SGB. Cycle costs are expressed in M-cycles, or T-states divided by 4. For instructions with a condition (cnd), there are two cycle costs: one for when the condition is met (for example, the jump occurs) and one for when the condition is not met (the jump is skipped). These are expressed in the format *X*/*Y*.

The Flags column indicates how each of the ZNHC flags are influenced by the instruction. A dash (-) means the flag is untouched. A 0 or a 1 means the flag is set to that value. A Z, N, H, or C means the flag is modified based on the result of the operation, as explained in "Flags" on page 14.

For 16-bit operations, flags H and C are set according to the most significant byte.

C^{-1} means the value of the carry flag before the instruction is executed.

Table A-3: DMG CPU Instructions

Instruction	Mnemonic	Bytes	Cycles	Flags	Description
adc a,r8	**ad**d with **c**arry	1	1	ZOHC	a = a + r8 + C
adc a,[hl]	″	1	2	ZOHC	a = a + [hl] + C
adc a,n8	″	2	2	ZOHC	a = a + n8 + C
add a,r8	**add**	1	1	ZOHC	a = a + r8
add a,[hl]	″	1	2	ZOHC	a = a + [hl]
add a,n8	″	2	2	ZOHC	a = a + n8
add hl,r16	″	1	2	-OHC	hl = hl + r16
add hl,sp	″	1	2	-OHC	hl = hl + sp
add sp,s8	″	2	4	OOHC	sp = sp + s8
and a,r8	**and**	1	1	ZO1O	a = a and r8
and a,[hl]	″	1	2	ZO1O	a = a and [hl]
and a,n8	″	2	2	ZO1O	a = a and n8
bit u3,r8	test **bit**	2	2	ZO1-	Z = 1 if bit u3 = 0 in r8
bit u3,[hl]	″	2	3	ZO1-	Z = 1 if bit u3 = 0 in [hl]
call n16	**call** function	3	6	—	sp = sp – 2, [sp + 1] = high(pc), [sp] = low(pc), pc = n16
call cnd,n16	″	3	6/3	—	if cnd then call n16
ccf	**c**omplement **c**arry **f**lag	1	1	-00C	C = C xor 1

Table A-3: DMG CPU Instructions *(continued)*

Instruction	Mnemonic	Bytes	Cycles	Flags	Description
`cp a,r8`	**c**om**p**are	1	1	`Z1HC`	a – r8
`cp a,[hl]`	″	1	2	`Z1HC`	a – [hl]
`cp a,n8`	″	2	2	`Z1HC`	a – n8
`cpl`	**c**om**pl**ement	1	1	`-11-`	a = a xor $FF
`daa`	**d**ecimal **a**djust (**a**)	1	1	`Z-0C`	Turns a to BCD format
`dec r8`	**dec**rement	1	1	`Z1H-`	r8 = r8 – 1
`dec [hl]`	″	1	3	`Z1H-`	[hl] = [hl] – 1
`dec r16`	″	1	2	—	r16 = r16 – 1
`dec sp`	″	1	2	—	sp = sp – 1
`di`	**d**isable **i**nterrupts	1	1	—	IME = 0
`ei`	**e**nable **i**nterrupts	1	1	—	IME = 1
`halt`	**halt**	1	—	—	Enters low power mode
`inc r8`	**inc**rement	1	1	`Z0H-`	r8 = r8 + 1
`inc [hl]`	″	1	3	`Z0H-`	[hl] = [hl] + 1
`inc r16`	″	1	2	—	r16 = r16 + 1
`inc sp`	″	1	2	—	sp = sp + 1
`jp n16`	**j**um**p**	3	4	—	pc = n16
`jp cnd,n16`	″	3	4/3	—	if cnd then `jp n16`
`jp hl`	″	1	1	—	pc = hl
`jr n16`	**j**ump **r**elative	2	3	—	pc = n16
`jr cnd,n16`	″	2	3/2	—	if cnd then `jr n16`
`ld r8,r8`	**l**oa**d**	1	1	—	left r8 = right r8
`ld r8,n8`	″	2	2	—	r8 = n8
`ld r16,n16`	″	3	3	—	r16 = n16
`ld [hl],r8`	″	1	2	—	[hl] = r8
`ld [hl],n8`	″	2	3	—	[hl] = n8
`ld r8,[hl]`	″	1	2	—	r8 = [hl]
`ld [r16],a`	″	1	2	—	[r16] = a
`ld [n16],a`	″	3	4	—	[n16] = a
`ld a,[r16]`	″	1	2	—	a = [r16]
`ld a,[n16]`	″	3	4	—	a = [n16]
`ld [hld],a`	″	1	2	—	[hl] = a, [hl] = [hl] – 1
`ld [hli],a`	″	1	2	—	[hl] = a, [hl] = [hl] + 1
`ld a,[hld]`	″	1	2	—	a = [hl], [hl] = [hl] – 1
`ld a,[hli]`	″	1	2	—	a = [hl], [hl] = [hl] + 1
`ld sp,n16`	″	3	3	—	sp = n16

Table A-3: DMG CPU Instructions *(continued)*

Instruction	Mnemonic	Bytes	Cycles	Flags	Description
`ld [n16],sp`	″	3	5	—	`[n16] = low(sp), [n16 + 1] = high(sp)`
`ld hl,sp+s8`	″	2	3	`00HC`	`hl = sp + s8`
`ld sp,hl`	″	1	2	—	`sp = hl`
`ldh [n16],a`	**l**oa**d** **h**igh	2	3	—	`[n16] = a` with `n16` in `[$FF00–$FFFF]`
`ldh [c],a`	″	1	2	—	`[$FF00 + c] = a`
`ldh a,[n16]`	″	2	3	—	`a = [n16]` with `n16` in `[$FF00–$FFFF]`
`ldh a,[c]`	″	1	2	—	`a = [$FF00 + c]`
`nop`	**no** **o**peration	1	1	—	Nothing
`or a,r8`	**or**	1	1	`Z000`	`a = a or r8`
`or a,[hl]`	″	1	2	`Z000`	`a = a or [hl]`
`or a,n8`	″	2	2	`Z000`	`a = a or n8`
`pop af`	**pop** from the stack	1	3	`ZNHC`	`f = [sp], a = [sp + 1], sp = sp + 2`
`pop r16`	″	1	3	—	`low(r16) = [sp], high(r16) = [sp + 1], sp = sp + 2`
`push af`	**push** onto the stack	1	4	—	`sp = sp – 2, [sp + 1] = a, [sp] = f`
`push r16`	″	1	4	—	`sp = sp – 2, [sp + 1] = high(r16), [sp] = low(r16)`
`res u3,r8`	**res**et bit	2	2	—	Bit `u3` = 0 in `r8`
`res u3,[hl]`	″	2	4	—	Bit `u3` = 0 in `[hl]`
`ret`	**ret**urn	1	4	—	`low(pc) = [sp], high(pc) = [sp + 1], sp = sp + 2`
`ret cnd`	″	1	5/2	—	if `cnd` then `ret`
`reti`	**ret**urn and enable **i**nterrupts	1	4	—	`ret`, IME = 1
`rl r8`	**r**otate **l**eft through carry	2	2	`Z00C`	`r8`: C <- [7 <- 0] <- C^{-1}
`rl [hl]`	″	2	4	`Z00C`	`[hl]`: C <- [7 <- 0] <- C^{-1}
`rla`	**r**otate **l**eft through carry **a**	1	1	`000C`	a: C <- [7 <- 0] <- C^{-1}
`rlc r8`	**r**otate **l**eft no **c**arry	2	2	`Z00C`	`r8`: C <- [7 <- 0] <- [7]
`rlc [hl]`	″	2	4	`Z00C`	`[hl]`: C <- [7 <- 0] <- [7]
`rlca`	**r**otate **l**eft no **c**arry **a**	1	1	`000C`	a: C <- [7 <- 0] <- [7]

Table A-3: DMG CPU Instructions *(continued)*

Instruction	Mnemonic	Bytes	Cycles	Flags	Description
`rr r8`	**r**otate **r**ight through carry	2	2	`Z00C`	`r8`: C^{-1} -> [7 -> 0] -> `C`
`rr [hl]`	″	2	4	`Z00C`	`[hl]`: C^{-1} -> [7 -> 0] -> `C`
`rra`	**r**otate **r**ight through carry **a**	1	1	`000C`	a: C^{-1} -> [7 -> 0] -> `C`
`rrc r8`	**r**otate **r**ight no **c**arry	2	2	`Z00C`	`r8`: [0] -> [7 -> 0] -> `C`
`rrc [hl]`	″	2	4	`Z00C`	`[hl]`: [0] -> [7 -> 0] -> `C`
`rrca`	**r**otate **r**ight no **c**arry **a**	1	1	`000C`	a: [0] -> [7 -> 0] -> `C`
`rst v8`	**rst**art (fast call)	1	4	—	`high(pc)` = `$00`, `low(pc)` = `v8`
`sbc a,r8`	**s**u**b**tract with **c**arry	1	1	`Z1HC`	a = a – `r8` – `C`
`sbc a,[hl]`	″	1	2	`Z1HC`	a = a – `[hl]` – `C`
`sbc a,n8`	″	2	2	`Z1HC`	a = a – `n8` – `C`
`scf`	**s**et **c**arry **f**lag	1	1	`-001`	`C` = 1
`set u3,r8`	**set** bit	2	2	—	Bit `u3` = 1 in `r8`
`set u3,[hl]`	″	2	4	—	Bit `u3` = 1 in `[hl]`
`sla r8`	**s**hift **l**eft **a**rithmetic	2	2	`Z00C`	`r8`: `C` <- [7 <- 0] <- 0
`sla [hl]`	″	2	4	`Z00C`	`[hl]`: `C` <- [7 <- 0] <- 0
`sra r8`	**s**hift **r**ight **a**rithmetic	2	2	`Z00C`	`r8`: [7] -> [7 -> 0] -> `C`
`sra [hl]`	″	2	4	`Z00C`	`[hl]`: [7] -> [7 -> 0] -> `C`
`srl r8`	**s**hift **r**ight **l**ogic	2	2	`Z00C`	`r8`: 0 -> [7 -> 0] -> `C`
`srl [hl]`	″	2	4	`Z00C`	`[hl]`: 0 -> [7 -> 0] -> `C`
`stop`	**stop**	2	—	—	Enters very low power mode
`sub a,r8`	**sub**tract	1	1	`Z1HC`	a = a – `r8`
`sub a,[hl]`	″	1	2	`Z1HC`	a = a – `[hl]`
`sub a,n8`	″	2	2	`Z1HC`	a = a – `n8`
`swap r8`	**swap** nibble	2	2	`Z000`	Swaps the nibbles in `r8`
`swap [hl]`	″	2	4	`Z000`	Swaps the nibbles in `[hl]`
`xor a,r8`	e**x**clusive **or**	1	1	`Z000`	a = a xor `r8`
`xor a,[hl]`	″	1	2	`Z000`	a = a xor `[hl]`
`xor a,n8`	″	2	2	`Z000`	a = a xor `n8`

Game Boy Coding Adventure (Early Access) © 2025 by Maximilien Dagois

B

OPCODES

The DMG CPU has 500 operation codes, or opcodes, each of which corresponds to a single action that the CPU can perform. Table B-1 contains a few examples, just to give you an idea of what they look like.

Table B-1: Opcode Examples

Opcode	Operation
`$80`	Add register `b` to register `a`.
`$81`	Add register `c` to register `a`.
`$CBC7`	Set bit 0 in register `a`.
`$CBCF`	Set bit 1 in register `a`.
`$CBD7`	Set bit 2 in register `a`.

The opcodes are very granular. There is no generic opcode to add any register to register `a`. Instead, there is one opcode to add `a` to `a`, one opcode to add `b` to `a`, one opcode to add `c` to `a`, and so on. The purpose of this granularity is to save ROM space. When the DMG was first released, memory was pretty expensive and any space you could save in the ROM was a big win. If we had a generic "add to `a`" opcode, it would take at least 2 bytes of ROM space: 1 byte for the opcode itself and 1 byte for the parameter that indicates which register to add to `a`. With the granular opcodes, adding a register to `a` only ever takes 1 byte of ROM.

Some of the opcodes in Table B-1 are 1 byte and others are 2 bytes. The 1-byte opcodes are mostly inherited from the Intel 8080, which featured 256 opcodes. One byte is enough to represent 256 opcodes, but the DMG CPU has nearly twice that number. To accommodate all of them, the CPU

architects decided to use a second set of 256 opcodes, adding an *opcode prefix* to each one in this set. Each 2-byte opcode starts with the same prefix, `CB`.

Unless you're programming an emulator, which is outside the scope of this book, you usually won't have to care much about opcodes at all. The assembly language abstracts away most of the granularity for you.

Table B-2 summarizes the parameter nomenclature of the opcodes described in Tables B-3 and B-4.

Table B-2: Legend for Opcode Tables

Operand	Description
`n8`	An 8-bit integer constant
`n16`	A 16-bit integer constant
`s8`	An 8-bit signed integer (–128 to 127)

Table B-3 lists the 1-byte opcodes available on the DMG, CGB, and SGB. Note that the instructions might still consume more than 1 byte, because while the opcode is 1 byte long, the instruction might require some operands. For example, the `jp $1234` instruction, which jumps to a 16-bit address, needs 1 byte for the opcode (`C3`) and 2 bytes for the address to jump to (`$1234`).

Cycle costs are expressed in M-cycles, or T-states divided by 4. For instructions with a condition, there are two cycle costs: one for when the condition is met (for example, the jump occurs) and one for when the condition is not met (the jump is skipped). These are expressed in the format *X*/*Y*.

The Flags column indicates how each of the `ZNHC` flags are influenced by the instruction. A dash (-) means the flag is untouched. A `0` or a `1` means the flag is set to that value. A `Z`, `N`, `H`, or `C` means the flag is modified based on the result of the operation, as explained in "Flags" on page 14.

Table B-3: The 1-Byte Opcodes

Opcode	Instruction	Bytes	Cycles	Flags	Opcode	Instruction	Bytes	Cycles	Flags
`00`	`nop`	1	1	—	`01`	`ld bc,n16`	3	3	—
`02`	`ld [bc],a`	1	2	—	`03`	`inc bc`	1	2	—
`04`	`inc b`	1	1	`Z0H-`	`05`	`dec b`	1	1	`Z1H-`
`06`	`ld b,n8`	2	2	—	`07`	`rlca`	1	1	`000C`
`08`	`ld [n16],sp`	3	5	—	`09`	`add hl,bc`	1	2	`-0HC`
`0A`	`ld a,[bc]`	1	2	—	`0B`	`dec bc`	1	2	—
`0C`	`inc c`	1	1	`Z0H-`	`0D`	`dec c`	1	1	`Z1H-`
`0E`	`ld c,n8`	2	2	—	`0F`	`rrca`	1	1	`000C`
`10`	`stop`	2	1	—	`11`	`ld de,n16`	3	3	—
`12`	`ld [de],a`	1	2	—	`13`	`inc de`	1	2	—
`14`	`inc d`	1	1	`Z0H-`	`15`	`dec d`	1	1	`Z1H-`
`16`	`ld d,n8`	2	2	—	`17`	`rla`	1	1	`000C`
`18`	`jr s8`	2	3	—	`19`	`add hl,de`	1	2	`-0HC`
`1A`	`ld a,[de]`	1	2	—	`1B`	`dec de`	1	2	—

Table B-3: The 1-Byte Opcodes *(continued)*

Opcode	Instruction	Bytes	Cycles	Flags	Opcode	Instruction	Bytes	Cycles	Flags
1C	inc e	1	1	Z0H-	1D	dec e	1	1	Z1H-
1E	ld e,n8	2	2	—	1F	rra	1	1	000C
20	jr nz,s8	2	3/2	—	21	ld hl,n16	3	3	—
22	ld [hli],a	1	2	—	23	inc hl	1	2	—
24	inc h	1	1	Z0H-	25	dec h	1	1	Z1H-
26	ld h,n8	2	2	—	27	daa	1	1	Z-0C
28	jr z,s8	2	3/2	—	29	add hl,hl	1	2	-0HC
2A	ld a,[hli]	1	2	—	2B	dec hl	1	2	—
2C	inc l	1	1	Z0H-	2D	dec l	1	1	Z1H-
2E	ld l,n8	2	2	—	2F	cpl	1	1	-11-
30	jr nc,s8	2	3/2	—	31	ld sp,n16	3	3	—
32	ld [hld],a	1	2	—	33	inc sp	1	2	—
34	inc [hl]	1	3	Z0H-	35	dec [hl]	1	3	Z1H-
36	ld [hl],n8	2	3	—	37	scf	1	1	-001
38	jr c,s8	2	3/2	—	39	add hl,sp	1	2	-0HC
3A	ld a,[hld]	1	2	—	3B	dec sp	1	2	—
3C	inc a	1	1	Z0H-	3D	dec a	1	1	Z1H-
3E	ld a,n8	2	2	—	3F	ccf	1	1	-00C
40	ld b,b	1	1	—	41	ld b,c	1	1	—
42	ld b,d	1	1	—	43	ld b,e	1	1	—
44	ld b,h	1	1	—	45	ld b,l	1	1	—
46	ld b,[hl]	1	2	—	47	ld b,a	1	1	—
48	ld c,b	1	1	—	49	ld c,c	1	1	—
4A	ld c,d	1	1	—	4B	ld c,e	1	1	—
4C	ld c,h	1	1	—	4D	ld c,l	1	1	—
4E	ld c,[hl]	1	2	—	4F	ld c,a	1	1	—
50	ld d,b	1	1	—	51	ld d,c	1	1	—
52	ld d,d	1	1	—	53	ld d,e	1	1	—
54	ld d,h	1	1	—	55	ld d,l	1	1	—
56	ld d,[hl]	1	2	—	57	ld d,a	1	1	—
58	ld e,b	1	1	—	59	ld e,c	1	1	—
5A	ld e,d	1	1	—	5B	ld e,e	1	1	—
5C	ld e,h	1	1	—	5D	ld e,l	1	1	—
5E	ld e,[hl]	1	2	—	5F	ld e,a	1	1	—
60	ld h,b	1	1	—	61	ld h,c	1	1	—
62	ld h,d	1	1	—	63	ld h,e	1	1	—
64	ld h,h	1	1	—	65	ld h,l	1	1	—
66	ld h,[hl]	1	2	—	67	ld h,a	1	1	—
68	ld l,b	1	1	—	69	ld l,c	1	1	—
6A	ld l,d	1	1	—	6B	ld l,e	1	1	—

Table B-3: The 1-Byte Opcodes *(continued)*

Opcode	Instruction	Bytes	Cycles	Flags	Opcode	Instruction	Bytes	Cycles	Flags
6C	ld l,h	1	1	—	6D	ld l,l	1	1	—
6E	ld l,[hl]	1	2	—	6F	ld l,a	1	1	—
70	ld [hl],b	1	2	—	71	ld [hl],c	1	2	—
72	ld [hl],d	1	2	—	73	ld [hl],e	1	2	—
74	ld [hl],h	1	2	—	75	ld [hl],l	1	2	—
76	halt	1	1	—	77	ld [hl],a	1	2	—
78	ld a,b	1	1	—	79	ld a,c	1	1	—
7A	ld a,d	1	1	—	7B	ld a,e	1	1	—
7C	ld a,h	1	1	—	7D	ld a,l	1	1	—
7E	ld a,[hl]	1	2	—	7F	ld a,a	1	1	—
80	add a,b	1	1	Z0HC	81	add a,c	1	1	Z0HC
82	add a,d	1	1	Z0HC	83	add a,e	1	1	Z0HC
84	add a,h	1	1	Z0HC	85	add a,l	1	1	Z0HC
86	add a,[hl]	1	2	Z0HC	87	add a,a	1	1	Z0HC
88	adc a,b	1	1	Z0HC	89	adc a,c	1	1	Z0HC
8A	adc a,d	1	1	Z0HC	8B	adc a,e	1	1	Z0HC
8C	adc a,h	1	1	Z0HC	8D	adc a,l	1	1	Z0HC
8E	adc a,[hl]	1	2	Z0HC	8F	adc a,a	1	1	Z0HC
90	sub b	1	1	Z1HC	91	sub c	1	1	Z1HC
92	sub d	1	1	Z1HC	93	sub e	1	1	Z1HC
94	sub h	1	1	Z1HC	95	sub l	1	1	Z1HC
96	sub [hl]	1	2	Z1HC	97	sub a	1	1	Z1HC
98	sbc a,b	1	1	Z1HC	99	sbc a,c	1	1	Z1HC
9A	sbc a,d	1	1	Z1HC	9B	sbc a,e	1	1	Z1HC
9C	sbc a,h	1	1	Z1HC	9D	sbc a,l	1	1	Z1HC
9E	sbc a,[hl]	1	2	Z1HC	9F	sbc a,a	1	1	Z1HC
A0	and b	1	1	Z010	A1	and c	1	1	Z010
A2	and d	1	1	Z010	A3	and e	1	1	Z010
A4	and h	1	1	Z010	A5	and l	1	1	Z010
A6	and [hl]	1	2	Z010	A7	and a	1	1	Z010
A8	xor b	1	1	Z000	A9	xor c	1	1	Z000
AA	xor d	1	1	Z000	AB	xor e	1	1	Z000
AC	xor h	1	1	Z000	AD	xor l	1	1	Z000
AE	xor [hl]	1	2	Z000	AF	xor a	1	1	1000
B0	or b	1	1	Z000	B1	or c	1	1	Z000
B2	or d	1	1	Z000	B3	or e	1	1	Z000
B4	or h	1	1	Z000	B5	or l	1	1	Z000
B6	or [hl]	1	2	Z000	B7	or a	1	1	Z000
B8	cp b	1	1	Z1HC	B9	cp c	1	1	Z1HC
BA	cp d	1	1	Z1HC	BB	cp e	1	1	Z1HC

Table B-3: The 1-Byte Opcodes *(continued)*

Opcode	Instruction	Bytes	Cycles	Flags	Opcode	Instruction	Bytes	Cycles	Flags
BC	cp h	1	1	Z1HC	BD	cp l	1	1	Z1HC
BE	cp [hl]	1	2	Z1HC	BF	cp a	1	1	Z1HC
C0	ret nz	1	5/2	—	C1	pop bc	1	3	—
C2	jp nz,n16	3	4/3	—	C3	jp n16	3	4	—
C4	call nz,n16	3	6/3	—	C5	push bc	1	4	—
C6	add a,n8	2	2	Z0HC	C7	rst 00h	1	4	—
C8	ret z	1	5/2	—	C9	ret	1	4	—
CA	jp z,n16	3	4/3	—	CB	prefix	—	—	—
CC	call z,n16	3	6/3	—	CD	call n16	3	6	—
CE	adc a,n8	2	2	Z0HC	CF	rst 08h	1	4	—
D0	ret nc	1	5/2	—	D1	pop de	1	3	—
D2	jp nc,n16	3	4/3	—	D3	—	—	—	—
D4	call nc,n16	3	6/3	—	D5	push de	1	4	—
D6	sub n8	2	2	Z1HC	D7	rst 10h	1	4	—
D8	ret c	1	5/2	—	D9	reti	1	4	—
DA	jp c,n16	3	4/3	—	DB	—	—	—	—
DC	call c,n16	3	6/3	—	DD	—	—	—	—
DE	sbc a,n8	2	2	Z1HC	DF	rst 18h	1	4	—
E0	ldh [a8],a	2	3	—	E1	pop hl	1	3	—
E2	ld [c],a	1	2	—	E3	—	—	—	—
E4	—	—	—	—	E5	push hl	1	4	—
E6	and n8	2	2	Z010	E7	rst 20h	1	4	—
E8	add sp,s8	2	4	00HC	E9	jp hl	1	1	—
EA	ld [n16],a	3	4	—	EB	—	—	—	—
EC	—	—	—	—	ED	—	—	—	—
EE	xor n8	2	2	Z000	EF	rst 28h	1	4	—
F0	ldh a,[a8]	2	3	—	F1	pop af	1	3	ZNHC
F2	ld a,[c]	1	2	—	F3	di	1	1	—
F4	—	—	—	—	F5	push af	1	4	—
F6	or n8	2	2	Z000	F7	rst 30h	1	4	—
F8	ld hl,sp+s8	2	3	00HC	F9	ld sp,hl	1	2	—
FA	ld a,[n16]	3	4	—	FB	ei	1	1	—
FC	—	—	—	—	FD	—	—	—	—
FE	cp n8	2	2	Z1HC	FF	rst 38h	1	4	—

Table B-4 lists the 2-byte opcodes available on the DMG, CGB, and SGB. These correspond to instructions that never need to save any operands in the ROM, so their byte cost is always 2.

Table B-4: The 2-Byte Opcodes

Opcode	Instruction	Bytes	Cycles	Flags	Opcode	Instruction	Bytes	Cycles	Flags
CB00	rlc b	2	2	Z00C	CB01	rlc c	2	2	Z00C
CB02	rlc d	2	2	Z00C	CB03	rlc e	2	2	Z00C
CB04	rlc h	2	2	Z00C	CB05	rlc l	2	2	Z00C
CB06	rlc [hl]	2	4	Z00C	CB07	rlc a	2	2	Z00C
CB08	rrc b	2	2	Z00C	CB09	rrc c	2	2	Z00C
CB0A	rrc d	2	2	Z00C	CB0B	rrc e	2	2	Z00C
CB0C	rrc h	2	2	Z00C	CB0D	rrc l	2	2	Z00C
CB0E	rrc [hl]	2	4	Z00C	CB0F	rrc a	2	2	Z00C
CB10	rl b	2	2	Z00C	CB11	rl c	2	2	Z00C
CB12	rl d	2	2	Z00C	CB13	rl e	2	2	Z00C
CB14	rl h	2	2	Z00C	CB15	rl l	2	2	Z00C
CB16	rl [hl]	2	4	Z00C	CB17	rl a	2	2	Z00C
CB18	rr b	2	2	Z00C	CB19	rr c	2	2	Z00C
CB1A	rr d	2	2	Z00C	CB1B	rr e	2	2	Z00C
CB1C	rr h	2	2	Z00C	CB1D	rr l	2	2	Z00C
CB1E	rr [hl]	2	4	Z00C	CB1F	rr a	2	2	Z00C
CB20	sla b	2	2	Z00C	CB21	sla c	2	2	Z00C
CB22	sla d	2	2	Z00C	CB23	sla e	2	2	Z00C
CB24	sla h	2	2	Z00C	CB25	sla l	2	2	Z00C
CB26	sla [hl]	2	4	Z00C	CB27	sla a	2	2	Z00C
CB28	sra b	2	2	Z00C	CB29	sra c	2	2	Z00C
CB2A	sra d	2	2	Z00C	CB2B	sra e	2	2	Z00C
CB2C	sra h	2	2	Z00C	CB2D	sra l	2	2	Z00C
CB2E	sra [hl]	2	4	Z00C	CB2F	sra a	2	2	Z00C
CB30	swap b	2	2	Z000	CB31	swap c	2	2	Z000
CB32	swap d	2	2	Z000	CB33	swap e	2	2	Z000
CB34	swap h	2	2	Z000	CB35	swap l	2	2	Z000
CB36	swap [hl]	2	4	Z000	CB37	swap a	2	2	Z000
CB38	srl b	2	2	Z00C	CB39	srl c	2	2	Z00C
CB3A	srl d	2	2	Z00C	CB3B	srl e	2	2	Z00C
CB3C	srl h	2	2	Z00C	CB3D	srl l	2	2	Z00C
CB3E	srl [hl]	2	4	Z00C	CB3F	srl a	2	2	Z00C
CB40	bit 0,b	2	2	Z01-	CB41	bit 0,c	2	2	Z01-
CB42	bit 0,d	2	2	Z01-	CB43	bit 0,e	2	2	Z01-
CB44	bit 0,h	2	2	Z01-	CB45	bit 0,l	2	2	Z01-
CB46	bit 0,[hl]	2	3	Z01-	CB47	bit 0,a	2	2	Z01-
CB48	bit 1,b	2	2	Z01-	CB49	bit 1,c	2	2	Z01-
CB4A	bit 1,d	2	2	Z01-	CB4B	bit 1,e	2	2	Z01-
CB4C	bit 1,h	2	2	Z01-	CB4D	bit 1,l	2	2	Z01-
CB4E	bit 1,[hl]	2	3	Z01-	CB4F	bit 1,a	2	2	Z01-

Table B-4: The 2-Byte Opcodes *(continued)*

Opcode	Instruction	Bytes	Cycles	Flags	Opcode	Instruction	Bytes	Cycles	Flags
CB50	bit 2,b	2	2	Z01-	CB51	bit 2,c	2	2	Z01-
CB52	bit 2,d	2	2	Z01-	CB53	bit 2,e	2	2	Z01-
CB54	bit 2,h	2	2	Z01-	CB55	bit 2,l	2	2	Z01-
CB56	bit 2,[hl]	2	3	Z01-	CB57	bit 2,a	2	2	Z01-
CB58	bit 3,b	2	2	Z01-	CB59	bit 3,c	2	2	Z01-
CB5A	bit 3,d	2	2	Z01-	CB5B	bit 3,e	2	2	Z01-
CB5C	bit 3,h	2	2	Z01-	CB5D	bit 3,l	2	2	Z01-
CB5E	bit 3,[hl]	2	3	Z01-	CB5F	bit 3,a	2	2	Z01-
CB60	bit 4,b	2	2	Z01-	CB61	bit 4,c	2	2	Z01-
CB62	bit 4,d	2	2	Z01-	CB63	bit 4,e	2	2	Z01-
CB64	bit 4,h	2	2	Z01-	CB65	bit 4,l	2	2	Z01-
CB66	bit 4,[hl]	2	3	Z01-	CB67	bit 4,a	2	2	Z01-
CB68	bit 5,b	2	2	Z01-	CB69	bit 5,c	2	2	Z01-
CB6A	bit 5,d	2	2	Z01-	CB6B	bit 5,e	2	2	Z01-
CB6C	bit 5,h	2	2	Z01-	CB6D	bit 5,l	2	2	Z01-
CB6E	bit 5,[hl]	2	3	Z01-	CB6F	bit 5,a	2	2	Z01-
CB70	bit 6,b	2	2	Z01-	CB71	bit 6,c	2	2	Z01-
CB72	bit 6,d	2	2	Z01-	CB73	bit 6,e	2	2	Z01-
CB74	bit 6,h	2	2	Z01-	CB75	bit 6,l	2	2	Z01-
CB76	bit 6,[hl]	2	3	Z01-	CB77	bit 6,a	2	2	Z01-
CB78	bit 7,b	2	2	Z01-	CB79	bit 7,c	2	2	Z01-
CB7A	bit 7,d	2	2	Z01-	CB7B	bit 7,e	2	2	Z01-
CB7C	bit 7,h	2	2	Z01-	CB7D	bit 7,l	2	2	Z01-
CB7E	bit 7,[hl]	2	3	Z01-	CB7F	bit 7,a	2	2	Z01-
CB80	res 0,b	2	2	—	CB81	res 0,c	2	2	—
CB82	res 0,d	2	2	—	CB83	res 0,e	2	2	—
CB84	res 0,h	2	2	—	CB85	res 0,l	2	2	—
CB86	res 0,[hl]	2	4	—	CB87	res 0,a	2	2	—
CB88	res 1,b	2	2	—	CB89	res 1,c	2	2	—
CB8A	res 1,d	2	2	—	CB8B	res 1,e	2	2	—
CB8C	res 1,h	2	2	—	CB8D	res 1,l	2	2	—
CB8E	res 1,[hl]	2	4	—	CB8F	res 1,a	2	2	—
CB90	res 2,b	2	2	—	CB91	res 2,c	2	2	—
CB92	res 2,d	2	2	—	CB93	res 2,e	2	2	—
CB94	res 2,h	2	2	—	CB95	res 2,l	2	2	—
CB96	res 2,[hl]	2	4	—	CB97	res 2,a	2	2	—
CB98	res 3,b	2	2	—	CB99	res 3,c	2	2	—
CB9A	res 3,d	2	2	—	CB9B	res 3,e	2	2	—
CB9C	res 3,h	2	2	—	CB9D	res 3,l	2	2	—
CB9E	res 3,[hl]	2	4	—	CB9F	res 3,a	2	2	—

Table B-4: The 2-Byte Opcodes *(continued)*

Opcode	Instruction	Bytes	Cycles	Flags	Opcode	Instruction	Bytes	Cycles	Flags
CBA0	res 4,b	2	2	—	CBA1	res 4,c	2	2	—
CBA2	res 4,d	2	2	—	CBA3	res 4,e	2	2	—
CBA4	res 4,h	2	2	—	CBA5	res 4,l	2	2	—
CBA6	res 4,[hl]	2	4	—	CBA7	res 4,a	2	2	—
CBA8	res 5,b	2	2	—	CBA9	res 5,c	2	2	—
CBAA	res 5,d	2	2	—	CBAB	res 5,e	2	2	—
CBAC	res 5,h	2	2	—	CBAD	res 5,l	2	2	—
CBAE	res 5,[hl]	2	4	—	CBAF	res 5,a	2	2	—
CBB0	res 6,b	2	2	—	CBB1	res 6,c	2	2	—
CBB2	res 6,d	2	2	—	CBB3	res 6,e	2	2	—
CBB4	res 6,h	2	2	—	CBB5	res 6,l	2	2	—
CBB6	res 6,[hl]	2	4	—	CBB7	res 6,a	2	2	—
CBB8	res 7,b	2	2	—	CBB9	res 7,c	2	2	—
CBBA	res 7,d	2	2	—	CBBB	res 7,e	2	2	—
CBBC	res 7,h	2	2	—	CBBD	res 7,l	2	2	—
CBBE	res 7,[hl]	2	4	—	CBBF	res 7,a	2	2	—
CBC0	set 0,b	2	2	—	CBC1	set 0,c	2	2	—
CBC2	set 0,d	2	2	—	CBC3	set 0,e	2	2	—
CBC4	set 0,h	2	2	—	CBC5	set 0,l	2	2	—
CBC6	set 0,[hl]	2	4	—	CBC7	set 0,a	2	2	—
CBC8	set 1,b	2	2	—	CBC9	set 1,c	2	2	—
CBCA	set 1,d	2	2	—	CBCB	set 1,e	2	2	—
CBCC	set 1,h	2	2	—	CBCD	set 1,l	2	2	—
CBCE	set 1,[hl]	2	4	—	CBCF	set 1,a	2	2	—
CBD0	set 2,b	2	2	—	CBD1	set 2,c	2	2	—
CBD2	set 2,d	2	2	—	CBD3	set 2,e	2	2	—
CBD4	set 2,h	2	2	—	CBD5	set 2,l	2	2	—
CBD6	set 2,[hl]	2	4	—	CBD7	set 2,a	2	2	—
CBD8	set 3,b	2	2	—	CBD9	set 3,c	2	2	—
CBDA	set 3,d	2	2	—	CBDB	set 3,e	2	2	—
CBDC	set 3,h	2	2	—	CBDD	set 3,l	2	2	—
CBDE	set 3,[hl]	2	4	—	CBDF	set 3,a	2	2	—
CBE0	set 4,b	2	2	—	CBE1	set 4,c	2	2	—
CBE2	set 4,d	2	2	—	CBE3	set 4,e	2	2	—
CBE4	set 4,h	2	2	—	CBE5	set 4,l	2	2	—
CBE6	set 4,[hl]	2	4	—	CBE7	set 4,a	2	2	—
CBE8	set 5,b	2	2	—	CBE9	set 5,c	2	2	—
CBEA	set 5,d	2	2	—	CBEB	set 5,e	2	2	—
CBEC	set 5,h	2	2	—	CBED	set 5,l	2	2	—
CBEE	set 5,[hl]	2	4	—	CBEF	set 5,a	2	2	—

Table B-4: The 2-Byte Opcodes *(continued)*

Opcode	Instruction	Bytes	Cycles	Flags	Opcode	Instruction	Bytes	Cycles	Flags
CBF0	set 6,b	2	2	—	CBF1	set 6,c	2	2	—
CBF2	set 6,d	2	2	—	CBF3	set 6,e	2	2	—
CBF4	set 6,h	2	2	—	CBF5	set 6,l	2	2	—
CBF6	set 6,[hl]	2	4	—	CBF7	set 6,a	2	2	—
CBF8	set 7,b	2	2	—	CBF9	set 7,c	2	2	—
CBFA	set 7,d	2	2	—	CBFB	set 7,e	2	2	—
CBFC	set 7,h	2	2	—	CBFD	set 7,l	2	2	—
CBFE	set 7,[hl]	2	4	—	CBFF	set 7,a	2	2	—